

Administrarea sistemelor Linux

de Paul Cobbaut

traducere din limba engleză de I. C.

Linux System Administration

Paul Cobbaut

lt-1.9

Publicat Joi 01 Aug 2013 01:01:19 CEST

Rezumat

Această carte este menită să fie utilizată într-un curs condus de un instructor. Pentru analiză, intenția este ca această carte să fie citită lângă un computer Linux funcțional astfel încât să faceți imediat fiecare subiect, practicând fiecare comandă.

Această carte se adresează administratorilor novici de sistem Linux (și poate fi interesantă și folositoare pentru utilizatorii casnici care vor să știe un pic mai mult despre sistemul lor Linux).

Oricum, această carte nu este făcută să fie o introducere a aplicațiilor desktop Linux ca editoare de text, browsere, clienți e-mail, aplicații multimedia sau office.

Mai multe informații și fișiere gratuite .pdf sînt disponibile la **<http://linux-training.be>**.

Contactați autorul:

- Paul Cobbaut: paul.cobbaut@gmail.com, <http://www.linkedin.com/in/cobbaut>

Contribuitori ai proiectului Linux Training sînt:

- Serge van Ginderachter: serge@ginsys.eu, construirea scripturilor și setarea infrastructurii
- Ywein Van den Brande: ywein@crealaw.eu, licență și secțiuni legale
- Hendrik De Vloed: hendrik.devloed@ugent.be, pentru scriptul buildheader.pl

Dorim de asemeni să mulțumim recenzenților noștri:

- Wouter Verhelst: wo@uter.be, <http://grep.be>
- Geert Goossens: mail.goossens.geert@gmail.com, <http://www.linkedin.com/in/geertgoossens>
- Elie De Brauer: elie@de-brauer.be, <http://www.de-brauer.be>
- Christophe Vandeplas: christophe@vandeplas.com, <http://christophe.vandeplas.com>
- Bert Desmet: bert@devnox.be, <http://blog.bdesmet.be>
- Rich Yonts: richyonts@gmail.com

Copyright 2007-2013 Netsec BVBA, Paul Cobbaut

Permisiunea este acordată de a copia, distribui și/sau modifica acest document în termenii **Licenței GNU Free Documentation**, Versiunea 1.3 sau a oricărei versiuni mai târzii publicată de Free Software Foundation; cu absența Secțiunilor Invariante, fără Texte Copertă Față, și fără Texte Copertă Spate. O copie a licenței este inclusă în secțiunea intitulată 'GNU Free Documentation License'.

CUPRINS

I.	management procese.	.5
1.	introducere la procese.	6
2.	priorități procese.	.16
3.	procese background.	23
II.	management disk.	29
4.	dispozitive disk	30
5.	partiții disk.	41
6.	sisteme fișier.	50
7.	montare	.58
8.	introducere la uuid	.66
9.	introducere la raid	.71
10.	management volume logice.	.79
11.	dispozitive iSCSI.	97
III.	management butare.	.105
12.	încărcător de boot.	106
13.	init și runlevel.	116
IV.	management sistem.	.130
14.	planificare	131
15.	jurnalizare.	.137
16.	management memorie.	147
17.	management pachet.	.153
V.	management rețea.	.172
18.	generalități networking.	173
19.	configurare interfață de rețea	.185
20.	interceptare rețea.	198
21.	legare și cuplare	204
22.	client ssh și server	.212
23.	introducere în nfs	.222
24.	introducere în networking	228
VI.	management kernel.	.239
25.	kernel-ul Linux.	240
26.	management biblioteci.	.256
VII.	management backup.	.260
27.	backup.	261
VIII.	Apendice	270
A.	Licență (în limba română).	271
B.	Licență (în limba engleză).	.278

Lista Tabelelor

4.1.	numire dispozitive ide.	32
4.2.	numire dispozitive scsi.	32
5.1.	partiții primare, extinse și logice.	42
5.2.	numele partiției.	42
10.1.	exemplu de partiționare disk.	80
10.2.	exemplu LVM.	80

Partea I. management procese

Capitolul 1. introducere la procese

Conținut

1.1. terminologie.	7
1.2. management procese de bază.	7
1.3. procese semnal.	.10
1.4. practică: management procese de bază.	.13
1.5. soluție: management procese de bază.	14

1.1. terminologie

proces

Un **proces** este un cod sursă compilat care se execută în mod curent pe un sistem.

PID

Toate procesele au un **proces id** sau **PID**.

PPID

Fiecare proces are un proces părinte (cu un **PPID**). Procesul **child** este deseori început de către procesul **parent**.

init

Procesul **init** are întotdeauna procesul ID 1. Procesul **init** este început de către **kernel** astfel că din punct de vedere tehnic nu are un proces părinte. **init** servește ca **părinte adoptiv** pentru procesele **orfane**.

kill

Cînd un proces încetează să se execute, procesul moare, cînd vreți ca procesul să moară, îl omoriți (**kill**).

daemon

Procesele care încep cînd deschidem sistemul și se execută mereu sînt numiți procese **daemon** sau **daemoni**. Acești **daemons** nu mor niciodată.

zombie

Cînd un proces este omorît, dar totuși mai apare pe sistem, atunci procesul se numește **zombie**. Nu puteți omorî zombii, pentru că sînt deja morți.

1.2. management procese de bază

\$\$ și \$PPID

Unele variabile de mediu shell conțin informații despre procese. Variabila **\$\$** va deține procesul **ID curent**, și **\$PPID** conține **PID-ul părinte**. În realitate **\$\$** este un parametru shell și nu o variabilă, nu-i puteți încredința o valoare.

Mai jos folosim **echo** pentru a afișa valorile **\$\$** și **\$PPID**.

```
[paul@RHEL4b ~]$ echo $$ $PPID  
4224 4223
```

pidof

Puteți găsi toate id-urile proceselor după nume folosind comanda **pidof**.

```
root@debian ~# pidof mingetty  
2819 2798 2797 2796 2795 2794
```

parent și child

Procesele au o relație **parent-child**. Fiecare proces are un proces părinte.

Cînd începeți un nou **bash** puteți folosi **echo** pentru a verifica dacă **pid-ul** de

mai sus este **ppid** a noului shell. Procesul **child** de-deasupra este acum procesul **parent**.

```
[paul@RHEL4b ~]$ bash
[paul@RHEL4b ~]$ echo $$ $PPID
4812 4224
```

Tastînd **exit** va termina procesul curent și ne va aduce înapoi valorile originale pentru **\$\$** și **\$PPID**.

```
[paul@RHEL4b~]$ echo $$ $PPID
4812 4224
[paul@RHEL4b~]$ exit
exit
[paul@RHEL4b~]$ echo $$ $PPID
4224 4223
[paul@RHEL4b~]$
```

fork și exec

Un proces pornește un alt proces în două faze. Prima dată procesul crează un **fork** (furcă) asupra lui, o copie identică. Apoi procesul fork execută un **exec** pentru a înlocui procesul fork cu ținta procesului child.

```
[paul@RHEL4b~]$ echo $$
4224
[paul@RHEL4b~]$ bash
[paul@RHEL4b~]$ echo $$ $PPID
5310 4224
[paul@RHEL4b~]$
```

exec

Cu comanda **exec** puteți executa un proces fără să faceți fork unui nou proces. În următoarea captură de ecran un **shell Korn** (ksh) este pornit și este înlocuit cu un **shell bash** folosind comanda **exec**. **Pidul shell-ului bash** este la fel cu **pidul shell-ului Korn**. Ieșind din **shell-ul bash** child mă va aduce înapoi la **bash-ul** parent, nu în **shell-ul Korn** (care nu mai există deloc).

```
[paul@RHEL4b ~]$ echo $$
4224 # PID of bash
[paul@RHEL4b ~]$ ksh
$ echo $$ $PPID
5343 4224 # PID of ksh and bash
$ exec bash
[paul@RHEL4b ~]$ echo $$ $PPID
5343 4224 # PID of bash and bash
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ echo $$
4224
```

ps

Una din cele mai comune utilitare pe Linux pentru a ne uita la procese este **ps**. Următoarea captură de ecran arată relația parent child între trei procese bash.


```
[paul@RHEL4b~]$ echo $$ $PPID
4224 4223
[paul@RHEL4b~]$ bash
[paul@RHEL4b~]$ echo $$ $PPID
4866 4224
[paul@RHEL4b~]$ bash
[paul@RHEL4b~]$ echo $$ $PPID
4884 4866
[paul@RHEL4b~]$ ps fx
PID TTY          STAT     TIME COMMAND
4223 ?             S        0:01 sshd: paul@pts/0
4224 pts/0        Ss       0:00  \_ -bash
4866 pts/0        S        0:00  \_ bash
4884 pts/0        S        0:00  \_ bash
4902 pts/0        R+       0:00  \_ ps fx
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ ps fx
PID TTY          STAT     TIME COMMAND
4223 ?             S        0:01 sshd: paul@pts/0
4224 pts/0        Ss       0:00  \_ -bash
4866 pts/0        S        0:00  \_ bash
4903 pts/0        R+       0:00  \_ ps fx
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b~]$ ps fx
PID TTY          STAT     TIME COMMAND
4223 ?             S        0:01 sshd: paul@pts/0
4224 pts/0        Ss       0:00  \_ -bash
4904 pts/0        R+       0:00  \_ ps fx
[paul@RHEL4b~]$
```

Pe Linux, **ps fax** este deseori utilizat. Pe Solaris **ps -ef** (care de asemeni funcționează pe Linux) este comun. Aici este o ieșire parțială din **ps fax**.

```
[paul@RHEL4a ~]$ ps fax
PID    TTY          STAT     TIME COMMAND
1 ?             S        0:00 init [5]
...

3713   ?             Ss       0:00 /usr/sbin/sshd
5042   ?             Ss       0:00  \_ sshd: paul [priv]
5044   ?             S        0:00  \_ sshd: paul@pts/1
5045   pts/1        Ss       0:00  \_ -bash
5077   pts/1        R+       0:00  \_ ps fax
```

pgrep

La fel cu **ps -C**, puteți folosi de asemeni **pgrep** pentru a căuta un proces după numele comenzii.

```
[paul@RHEL5 ~]$ sleep 1000 &
[1] 32558
[paul@RHEL5 ~]$ pgrep sleep
32558
[paul@RHEL5 ~]$ ps -C sleep
PID    TTY          TIME CMD
32558 pts/3    00:00:00 sleep
```

Puteți de asemeni lista procesul numelui comenzii cu `pgrep`.

```
paul@laika:~$ pgrep -l sleep
9661 sleep
```

top

Un alt utilitar popular pe Linux este **top**. Utilitarul **top** poate ordona procesele pe care le utilizează **cpu** sau alte proprietăți. Puteți de asemeni **închide** procese din interiorul **top**. Apăsați **h** în interiorul **top** pentru ajutor.

În caz de necesitate, **top** este deseori primul utilitar pe care îl pornim, pentru că ne furnizează de asemeni informații despre memorie și spațiu swap.

1.3. procese semnal

kill

Comanda **kill** va închide un proces. Captura de ecran de mai jos arată cum să folosim un **kill** standard pentru a opri procesul cu **pid** 1942.

```
paul@ubuntu910:~$ kill 1942
paul@ubuntu910:~$
```

Folosind **kill** trimitem un **semnal** procesului.

listă semnale

Procesele care se execută pot primi semnale între ele sau de la utilizatori. Puteți avea o listă a semnalelor tastând **kill -l**, adică litera **l**, nu numărul 1.

```
[paul@RHEL4a ~]$ kill -l
1) SIGHUP          2) SIGINT          3) SIGQUIT         4) SIGILL
5) SIGTRAP         6) SIGABRT        7) SIGBUS          8) SIGFPE
9) SIGKILL         10) SIGUSR1       11) SIGSEGV        12) SIGUSR2
13) SIGPIPE        14) SIGALRM       15) SIGTERM        17) SIGCHLD
18) SIGCONT        19) SIGSTOP       20) SIGTSTP        21) SIGTTIN
22) SIGTTOU        23) SIGURG        24) SIGXCPU        25) SIGXFSZ
26) SIGVTALRM      27) SIGPROF       28) SIGWINCH       29) SIGIO
30) SIGPWR         31) SIGSYS        34) SIGRTMIN       35) SIGRTMIN+1
36) SIGRTMIN+2     37) SIGRTMIN+3    38) SIGRTMIN+4     39) SIGRTMIN+5
40) SIGRTMIN+6     41) SIGRTMIN+7    42) SIGRTMIN+8     43) SIGRTMIN+9
44) SIGRTMIN+10    45) SIGRTMIN+11   46) SIGRTMIN+12    47) SIGRTMIN+13
48) SIGRTMIN+14    49) SIGRTMIN+15   50) SIGRTMAX-14    51) SIGRTMAX-13
52) SIGRTMAX-12    53) SIGRTMAX-11   54) SIGRTMAX-10    55) SIGRTMAX-9
56) SIGRTMAX-8     57) SIGRTMAX-7    58) SIGRTMAX-6     59) SIGRTMAX-5
60) SIGRTMAX-4     61) SIGRTMAX-3    62) SIGRTMAX-2     63) SIGRTMAX-1
64) SIGRTMAX
[paul@RHEL4a ~]$
```

kill -1 (SIGHUP)

Este comun pe Linux să utilizăm primul semnal **SIGHUP** (sau HUP sau 1) pentru a transmite unui proces că ar trebui să recitească fișierul lui de configurare. De aceea, comanda `kill -1 1` forțează procesul `init` (`init` se execută întotdeauna cu `pid 1`) pentru a reciti fișierul lui de configurare.

```
root@deb503:~# kill -1 1
root@deb503:~#
```

Depinde de dezvoltatorul procesului să decidă dacă procesul poate face asta executându-se, sau dacă el trebuie să se oprească și să se pornească. Depinde de utilizator să citească documentația programului.

kill -15 (SIGTERM)

Semnalul **SIGTERM** este de asemeni numit un **kill standard**. Dacă `kill` este executat fără a specifica semnalul, un `kill -15` este asumat. Ambele comenzi din captura de ecran de mai jos sînt identice.

```
paul@ubuntu910:~$ kill 1942
paul@ubuntu910:~$ kill -15 1942
```

kill -9 (SIGKILL)

SIGKILL este diferit de mai multe alte semnale anume că nu este trimis procesului, ci **kernel-ului Linux**. Un `kill -9` este de asemeni numit un **sure kill**. **Kernel-ul** va închide procesul. Ca dezvoltator nu aveți nici un interes de a intercepta un semnal `kill -9`.

```
root@rhel53 ~# kill -9 3342
```

killall

Comanda `killall` de asemeni va face default trimiterea unui **semnal 15** procesului.

Această comandă și contrapartea ei SysV `killall5` poate fi utilizată cînd închidem sistemul. Această captură de ecran arată cum Red Hat Enterprise Linux 5.3 folosește `killall5` cînd închide sistemul.

```
root@rhel53 ~# grep killall /etc/init.d/halt
action $"Sending all processes the TERM signal..." /sbin/killall5 -15
action $"Sending all processes the KILL signal..." /sbin/killall5 -9
```

pkill

Puteți folosi comanda `pkill` pentru a închide un proces după numele comenzii.

```
[paul@RHEL5 ~]$ sleep 1000 &
[1] 30203
[paul@RHEL5 ~]$ pkill sleep
[1]+  Terminated          sleep 1000
[paul@RHEL5 ~]$
```

top

În `top` tasta `k` vă permite să selectați un **semnal** și un **pid** de închis. Mai jos este o captură de ecran parțială a liniei dedesubtul sumarului în `top` după ce apăsăm `k`.

PID to kill: 1932

Kill PID 1932 with signal [15]: 9

SIGSTOP și SIGCONT

Un proces care se execută poate fi **suspendat** cînd primește un semnal **SIGSTOP**. Asta e la fel cu **kill -19** pe Linux, dar poate avea un număr diferit pe alte sisteme Unix.

Un proces **suspendat** nu folosește nici un **ciclu cpu**, ci stă în memorie și poate fi reanimat cu un semnal **SIGCONT** (**kill -18** pe Linux).

Ambele semnale vor fi utilizate în secțiunea despre procese **background**.

1.4. practică: management procese de bază

1. Folosiți **ps** pentru a căuta procesul **init** după nume.
2. Care este **procesul id** al procesului **init**?
3. Folosiți comanda **who am i** pentru a determina numele terminalului.
4. Folosind numele terminalului de mai sus, folosiți **ps** pentru a găsi toate procesele asociate cu terminalul.
5. Care este **procesul id** a shell-ului?
6. Care este **procesul parent id** al shell-ului?
7. Începeți două instanțe **sleep 3342** în background.
8. Localizați **procesul id** al tuturor comenzilor **sleep**.
9. Afișați doar acele două procese **sleep** în **top**. Apoi închideți top.
10. Folosiți un **standard kill** pentru a închide unul din procesele **sleep**.
11. Folosiți o singură comandă pentru a închide toate procesele **sleep**.

1.5. soluție: management procese de bază

1. Folosiți **ps** pentru a căuta procesul **init** după nume.

```
root@rhel53 ~# ps -C init
PID   TTY      TIME CMD
1      ?        00:00:04 init
```

2. Care este **procesul id** al procesului **init**?

1

3. Folosiți comanda **who am i** pentru a determina numele terminalului.

```
root@rhel53 ~# who am i
paul      pts/0      2010-04-12 17:44 (192.168.1.38)
```

4. Folosind numele terminalului de mai sus, folosiți **ps** pentru a găsi toate procesele asociate cu terminalul.

```
root@rhel53 ~# ps fax | grep pts/0
2941 ?          S          0:00      \_ sshd: paul@pts/0
2942 pts/0      Ss         0:00      \_ -bash
2972 pts/0      S          0:00      \_ su -
2973 pts/0      S          0:00      \_ -bash
3808 pts/0      R+         0:00      \_ ps fax
3809 pts/0      R+         0:00      \_ grep pts/0
```

sau de asemeni

```
root@rhel53 ~# ps -ef | grep pts/0
paul      2941 2939  0  17:44 ?          00:00:00 sshd: paul@pts/0
paul      2942 2941  0  17:44 pts/0      00:00:00 -bash
root      2972 2942  0  17:45 pts/0      00:00:00 su -
root      2973 2972  0  17:45 pts/0      00:00:00 -bash
root      3816 2973  0  21:25 pts/0      00:00:00 ps -ef
root      3817 2973  0  21:25 pts/0      00:00:00 grep pts/0
```

5. Care este **procesul id** al shell-ului?

2973 în captura de ecran de mai sus, probabil diferit pentru dvs.

echo \$\$ ar trebui să afișeze același număr cu cel pe care l-ați găsit.

6. Care este **procesul parent id** a shell-ului?

2972 în captura de ecran de mai sus, probabil diferit pentru dvs..

În acest exemplu PPID este din comanda **su -** , dar când sîntem în interiorul gnome atunci de exemplu gnome-terminal poate fi procesul parent.

7. Începeți două instanțe a **sleep 3342** în background.

```
sleep 3342 &
sleep 3342 &
```

8. Localizați **procesul id** al tuturor comenzilor **sleep**.

```
pidof sleep
```

9. Afişaţi doar acele două procese **sleep** în **top**. Apoi închideţi top.

`top -p pidx, pidy` (înlocuiţi `pidx pidy` cu numerele respective)

10. Folosiţi un **standard kill** pentru a închide unul din procesele **sleep**.

`kill pidx`

11. Folosiţi o singură comandă pentru a închide toate procesele **sleep**.

`pkill sleep`

Capitolul 2. priorități procese

Conținut

2.1. prioritate și valori nice.	17
2.2. practică: priorități procese.	20
2.3. soluție: priorități procese.	21

2.1. prioritate și valori nice

introducere

Toate procesele au o **prioritate** și o valoare **nice**. O prioritate mai mare a proceselor va lua mai mult **timp cpu** decât procesele cu prioritate joasă. Puteți influența asta cu comenzile **nice** și **renice**.

conducte (mkfifo)

Procesele pot comunica unul cu altul prin intermediul **conductelor (pipe)**. Aceste **conducte** pot fi create cu comanda **mkfifo**.

Această captură de ecran arată crearea a patru conduite distincte (într-un nou director)

```
paul@ubuntu910:~$ mkdir procs
paul@ubuntu910:~$ cd procs/
paul@ubuntu910:~/procs$ mkfifo pipe33a pipe33b pipe42a pipe42b
paul@ubuntu910:~/procs$ ls -l
total 0
prw-r--r-- 1 paul paul 0 2010-04-12 13:21 pipe33a
prw-r--r-- 1 paul paul 0 2010-04-12 13:21 pipe33b
prw-r--r-- 1 paul paul 0 2010-04-12 13:21 pipe42a
prw-r--r-- 1 paul paul 0 2010-04-12 13:21 pipe42b
paul@ubuntu910:~/procs$
```

amuzament cu cat

Pentru a demonstra folosirea comenzilor **top** și **renice** vom face comanda **cat** să utilizeze **conductele** anterior create pentru a genera o încărcare deplină pe **cpu**.

Comanda **cat** este copiată cu un nume distinct în directorul curent. (Asta ne permite să recunoaștem ușor procesele din interiorul **top**. Puteți face același exercițiu fără a copia comanda **cat**, dar utilizând utilizatori diferiți. Sau puteți să vă uitați doar la **pid-ul** fiecărui proces.)

```
paul@ubuntu910:~/procs$ cp /bin/cat proj33
paul@ubuntu910:~/procs$ cp /bin/cat proj42
paul@ubuntu910:~/procs$ echo -n x | ./proj33 - pipe33a > pipe33b &
[1] 1670
paul@ubuntu910:~/procs$ ./proj33 <pipe33b >pipe33a &
[2] 1671
paul@ubuntu910:~/procs$ echo -n z | ./proj42 - pipe42a > pipe42b &
[3] 1673
paul@ubuntu910:~/procs$ ./proj42 <pipe42b >pipe42a &
[4] 1674
```

Comenzile pe care le vedeți mai sus vor crea două procese **proj33** care utilizează **cat** pentru a ricoșa caracterul x între **pipe33a** și **pipe33b**. Și idem pentru caracterul z și **proj42**.

top

Doar rulînd **top** fără opțiuni sau argumente va afișa toate procesele și o descriere a informațiilor. Partea de sus a ecranului **top** ar putea arăta la fel ca aceasta.

```
top - 13:59:29 up 48 min, 4 users, load average: 1.06, 0.25, 0.14
Tasks: 139 total, 3 running, 136 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.3%us, 99.7%sy, 0.0%ni, 0.0%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 509352k total, 460040k used, 49312k free, 66752k buffers
Swap: 746980k total, 0k used, 746980k free, 247324k cached
```

Observați că timpul **cpu idle (0.0%id)** este zero. Asta se întâmplă deoarece procesele noastre **cat** consumă întregul **cpu**. Rezultatele pot varia pe sistemele cu patru sau mai multe **nuclee cpu**.

top -p

Captura de ecran a lui **top -p 1670,1671,1673,1674** de mai jos arată patru procese, toate dintre ele folosind aproximativ 25 procente din **cpu**.

```
paul@ubuntu910:~$ top -p 1670,1671,1673,1674
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1674	paul	20	0	2972	616	524	S	26.6	0.1	0:11.92	proj42
1670	paul	20	0	2972	616	524	R	25.0	0.1	0:23.16	proj33
1671	paul	20	0	2972	616	524	S	24.6	0.1	0:23.07	proj33
1673	paul	20	0	2972	620	524	R	23.0	0.1	0:11.48	proj42

Toate cele patru procese au o **prioritate egală (PR)**, și se bat pentru **timp cpu**. Pe unele sisteme **kernel-ul Linux** poate atribui **valori de prioritate** ușor variabile, dar rezultatul totuși vor fi patru procese luptându-se pentru **timp cpu**.

renice

De vreme ce procesele deja rulează, trebuie să folosim comanda **renice** pentru a le schimba **valorile nice (NI)**.

Captura de ecran de mai jos arată cum să utilizăm **renice** pentru ambele procese **proj33**.

```
paul@ubuntu910:~$ renice +8 1670
1670: old priority 0, new priority 8
paul@ubuntu910:~$ renice +8 1671
1671: old priority 0, new priority 8
```

Utilizatorii normali pot atribui o **valoare nice** de la zero la 20 asupra proceselor pe care ei le dețin. Doar utilizatorul **root** poate folosi valori negative nice. Fiți foarte atenți cu valori nice negative, pentru că ele pot face imposibil de utilizat tastatura sau ssh-ul într-un sistem.

impactul valorilor nice

Impactul unei valori nice pe procese care se execută poate varia. Captura de ecran de mai jos arată rezultatul comenzii noastre **renice +8**. Priviți la valorile **%CPU**.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1674	paul	20	0	2972	616	524	S	46.6	0.1	0:22.37	proj42
1673	paul	20	0	2972	620	524	R	42.6	0.1	0:21.65	proj42
1671	paul	28	8	2972	616	524	S	5.7	0.1	0:29.65	proj33
1670	paul	28	8	2972	616	524	R	4.7	0.1	0:29.82	proj33

Este important să amintim ca întotdeauna să faceți procesele nice mai puțin importante la procese mai importante. Folosirea **valorilor nice negative** poate avea un impact sever asupra utilizării sistemului.

nice

nice lucrează identic cu **renice** dar este folosit când începem o comandă. Captura de ecran arată cum să pornim un script cu o valoare **nice** de cinci.

```
paul@ubuntu910:~$ nice -5 ./backup.sh
```

2.2. practică: priorități procese

1. Creați un nou director și creați șase **conducte** în acel director.
2. Ricoșați un caracter între două **conducte**.
3. Folosiți **top** și **ps** pentru a afișa informație (pid, ppid, prioritate, valoare nice, ...) despre aceste două procese **cat**.
4. Ricoșați un alt caracter între celelalte două conduite, dar de această dată porniți comenzile **nice**. Verificați dacă toate procesele **cat** se bat pentru cpu. (Simțiți-vă liber să începeți încă două comenzi cat cu conductele care au rămas).
5. Folosiți **ps** pentru a verifica dacă cele două noi procese **cat** au o valoare **nice**. Folosiți opțiunile -o și -C a comenzii **ps** pentru asta.
6. Folosiți **renice** pentru a crește valoarea nice de la 10 la 15. Observați diferența cu comenzile uzuale.

2.3. soluție: priorități procese

1. Creați un nou director și creați șase **conducte** în acel director.

```
[paul@rhel53~]$ mkdir pipes ; cd pipes
[paul@rhel53 pipes]$ mkfifo p1 p2 p3 p4 p5 p6
[paul@rhel53 pipes]$ ls -l
total 0
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p1
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p2
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p3
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p4
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p5
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p6
```

2. Ricoșăți un caracter între două **conducte**.

```
[paul@rhel53 pipes]$ echo -n x | cat - p1 > p2 &
[1] 4013
[paul@rhel53 pipes]$ cat <p2 >p1 &
[2] 4016
```

3. Folosiți **top** și **ps** pentru a afișa informație (pid, ppid, prioritate, valoare nice, ...) despre aceste două procese cat.

top (probabil primele două linii top)

```
[paul@rhel53 pipes]$ ps -C cat
PID      TTY      TIME    CMD
4013     pts/0    00:03:38 cat
4016     pts/0    00:01:07 cat
```

```
[paul@rhel53 pipes]$ ps fax | grep cat
4013     pts/0    R        4:00    |      \_ cat - p1
4016     pts/0    S        1:13    |      \_ cat
4044     pts/0    S+       0:00    |      \_ grep cat
```

4. Ricoșăți un alt caracter între celelalte două conduite, dar de această dată porniți comenzile **nice**. Verificați dacă toate procesele **cat** se bat pentru cpu. (Simțiți-vă liber să începeți încă două comenzi cat cu conductele care au rămas).

```
echo -n y | nice cat -p3 > p4 &
nice cat <p4 >p3 &
```

5. Folosiți **ps** pentru a verifica dacă cele două noi procese **cat** au o valoare **nice**. Folosiți opțiunile -o și -C a comenzii **ps** pentru asta.

```
[paul@rhel53 pipes]$ ps -C cat -o pid,ppid,pri,ni,comm
PID PPID PRI NI COMMAND
4013 3947 14  0 cat
4016 3947 21  0 cat
4025 3947 13 10 cat
4026 3947 13 10 cat
```

6. Folosiți **renice** pentru a crește valoarea nice de la 10 la 15. Observați diferența cu comenzile uzuale.

```
[paul@rhel53 pipes]$ renice +15 4025
4025: old priority 10, new priority 15
[paul@rhel53 pipes]$ renice +15 4026
4026: old priority 10, new priority 15

[paul@rhel53 pipes]$ ps -C cat -o pid,ppid,pri,ni,comm
PID PPID PRI NI COMMAND
4013 3947 14  0 cat
4016 3947 21  0 cat
4025 3947  9 15 cat
4026 3947  8 15 cat
```

Capitolul 3. procese background

Conținut

3.1. procese background.24

3.2. practică: procese background.26

3.3. soluție: procese background. 27

3.1. procese background

jobs

Lucrurile care se execută în background în shell-ul curent pot fi afișate cu comanda **jobs**. Prin default nu veți avea nici o **sarcină (jobs)** rulând în background.

```
root@rhel53 ~# jobs
root@rhel53 ~#
```

Această comandă **jobs** va fi folosită de mai multe ori în această secțiune.

control-Z

Unele procese pot fi **suspendate** cu combinația de taste **Ctrl-Z**. Asta trimite un semnal **SIGSTOP** kernel-ului **Linux**, în mod efectiv înghețând operația procesului.

Cînd facem asta în **vi(m)**, atunci **vi(m)** se duce în background. **vi(m)** din background poate fi văzut cu comanda **jobs**.

```
[paul@RHEL4a ~]$ vi procdemo.txt

[5]+ Stopped                  vim procdemo.txt
[paul@RHEL4a ~]$ jobs
[5]+ Stopped                  -vim procdemo.txt
```

& și

Procesele care sînt pornite în background folosind caracterul **&** la sfîrșitul liniei de comandă sînt de asemeni vizibile cu comanda **jobs**.

```
[paul@RHEL4a ~]$ find / > allfiles.txt 2> /dev/null &
[6] 5230
[paul@RHEL4a ~]$ jobs
[5]+ Stopped                  vim procdemo.txt
[6]- Running                  find / >allfiles.txt 2>/dev/null &
[paul@RHEL4a ~]$
```

jobs -p

O opțiune interesantă este **jobs -p** pentru a vedea **procesul id** a proceselor background.

```
[paul@RHEL4b ~]$ sleep 500 &
[1] 4902
[paul@RHEL4b~]$ sleep 400 &
[2] 4903
[paul@RHEL4b~]$ jobs -p
4902
4903
[paul@RHEL4b~]$ ps `jobs -p`
PID  TTY      STAT   TIME  COMMAND
4902 pts/0    S      0:00  sleep 500
4903 pts/0    S      0:00  sleep 400
[paul@RHEL4b~]$
```


fg

Rulînd comanda **fg** va aduce un job din background în foreground. Numărul job-ului din background pentru a-l aduce în foreground este parametrul lui **fg**.

```
[paul@RHEL5 ~]$ jobs
[1]  Running          sleep 1000 &
[2]- Running          sleep 1000 &
[3]+ Running          sleep 2000 &
[paul@RHEL5 ~]$ fg 3
sleep 2000
```

bg

Job-urile care sînt **suspendate** în background pot fi începute în background cu **bg**. Comanda **bg** va trimite un semnal **SIGCONT**.

Mai jos este un exemplu a comenzii sleep (suspendate cu **Ctrl-Z**) fiind reactivată în background cu **bg**.

```
[paul@RHEL5 ~]$ jobs
[paul@RHEL5 ~]$ sleep 5000 &
[1] 6702
[paul@RHEL5 ~]$ sleep 3000

[2]+ Stopped          sleep 3000
[paul@RHEL5 ~]$ jobs
[1]- Running          sleep 5000 &
[2]+ Stopped          sleep 3000
[paul@RHEL5 ~]$ bg 2
[2]+ sleep 3000
[paul@RHEL5 ~]$ jobs
[1]- Running          sleep 5000 &
[2]+ Running          sleep 3000 &
[paul@RHEL5 ~]$
```

3.2. practică: procese background

1. Folosiți comanda **jobs** pentru a verifica dacă aveți un proces care rulează în background.
2. Folosiți **vi** pentru a crea un mic fișier text. Suspendați **vi** în background.
3. Verificați cu **jobs** dacă **vi** este suspendat în background.
4. Începeți **find / > allfiles.txt 2>/dev/null** în foreground. Suspendați-o în background înainte ca ea să se termine.
5. Începeți două procese **sleep** lungi în background.
6. Afișați toate **job-urile** din background.
7. Folosiți comanda **kill** pentru a suspenda ultimul proces **sleep**.
8. Continuați procesul **find** în background (asigurați-vă că rulează din nou).
9. Puneți una dintre comenzile **sleep** înapoi în foreground.
10. (dacă timpul permite, o întrebare generală de recapitulare...) Explicați în detaliu de unde vin numerele din următoarea captură de ecran. Când sînt variabilele înlocuite de valoarea lor? De către care shell?

```
[paul@RHEL4b ~]$ echo $$ $PPID
4224 4223
[paul@RHEL4b ~]$ bash -c "echo $$ $PPID"
4224 4223
[paul@RHEL4b ~]$ bash -c 'echo $$ $PPID'
5059 4224
[paul@RHEL4b ~]$ bash -c `echo $$ $PPID`
4223: 4224: command not found
```

3.3. soluție: procese background

1. Folosiți comanda **jobs** pentru a verifica dacă aveți un proces care rulează în background.

jobs (amuzamentul cu cat încă rulează?)

2. Folosiți **vi** pentru a crea un mic fișier text. Suspendați **vi** în background.

vi text.txt
(înăuntrul vi apăsați ctrl-z)

3. Verificați cu **jobs** dacă **vi** este suspendat în background.

```
[paul@rhel53 ~]$ jobs
[1]+  Stopped          vim text.txt
```

4. Începeți **find / > allfiles.txt 2>/dev/null** în foreground. Suspendați-o în background înainte ca ea să se termine.

```
[paul@rhel53 ~]$ find / > allfiles.txt 2>/dev/null
(apăsați ctrl-z)
[2]+  Stopped          find / > allfiles.txt 2> /dev/null
```

5. Începeți două procese **sleep** lungi în background.

sleep 4000 & ; sleep 5000 &

6. Afișați toate **job-urile** din background.

```
[paul@rhel53 ~]$ jobs
[1]-  Stopped          vim text.txt
[2]+  Stopped          find / > allfiles.txt 2> /dev/null
[3]   Running          sleep 4000 &
[4]   Running          sleep 5000 &
```

7. Folosiți comanda **kill** pentru a suspenda ultimul proces **sleep**.

```
[paul@rhel53 ~]$ kill -SIGSTOP 4519
[paul@rhel53 ~]$ jobs
[1]   Stopped          vim text.txt
[2]-  Stopped          find / > allfiles.txt 2> /dev/null
[3]   Running          sleep 4000 &
[4]+  Stopped          sleep 5000
```

8. Continuați procesul **find** în background. (asigurați-vă că merge din nou).

bg 2 (verificați job-id în lista de job-uri)

9. Puneți una dintre comenzile **sleep** înapoi în foreground.

fg 3 (din nou verificați job-id-ul)

10. (dacă timpul permite, o întrebare generală de recapitulare...) Explicați în detaliu de unde vin numerele din următoarea captură de ecran. Când sînt variabilele înlocuite de valoarea lor? De către care shell?

```
[paul@RHEL4b ~]$ echo $$ $PPID
4224 4223
[paul@RHEL4b ~]$ bash -c "echo $$ $PPID"
4224 4223
[paul@RHEL4b ~]$ bash -c 'echo $$ $PPID'
5059 4224
[paul@RHEL4b ~]$ bash -c `echo $$ $PPID`
4223: 4224: command not found
```

Shell-ul curent bash va înlocui \$\$ și \$PPID în timp ce scanează linia, și înainte de a executa comanda echo.

```
[paul@RHEL4b ~]$ echo $$ $PPID
4224 4223
```

Variabilele au acum ghilimele duble, dar bashul curent va înlocui \$\$ și \$PPID în timp ce scanează linia, și înainte de a executa comanda bash -c.

```
[paul@RHEL4b ~]$ bash -c "echo $$ $PPID"
4224 4223
```

Variabilele au acum ghilimele simple. Bash-ul curent **nu** va înlocui \$\$ și \$PPID. Comanda bash -c va fi executată înainte ca variabilele să fie înlocuite cu valorile lor. Acest bash de acum este cel care înlocuiește \$\$ și \$PPID cu valorile lor.

```
[paul@RHEL4b ~]$ bash -c 'echo $$ $PPID'
5059 4224
```

Cu apostrofuri shell-ul va înlocui ambele variabile înainte ca echo înglobat să fie executat. Rezultatul acestui echo sînt două procese id. Acestea sînt date ca și comenzi lui bash -c. Dar cele două numere nu sînt comenzi!

```
[paul@RHEL4b ~]$ bash -c `echo $$ $PPID`
4223: 4224: command not found
```

Partea II. management disk

Capitolul 4. dispozitive disk

Conținut

4.1. terminologie.	.31
4.2. nume dispozitive.	.32
4.3. descoperire dispozitive disk.	.32
4.4. ștergerea unui hard-disk.	.36
4.5. setări avansate hard-disk.	37
4.6. practică: dispozitive hard-disk.	38
4.7. soluție: dispozitive hard-disk.	.39

Acest capitol vă învață cum să localizați și să recunoașteți **dispozitive hard-disk**. Acesta vă pregătește pentru următorul capitol, unde punem **partiții** pe aceste dispozitive.

4.1. terminologie

platan, cap, pistă, cilindru, sector

Datele sînt în mod obișnuit stocate pe disk-uri magnetice sau **platane disk**. Plataneele sînt rotate (la viteze mari). Datele sînt citite de **capuri**, care sînt foarte aproape de suprafața platanului, fără să-l atingă! Capurile sînt montate pe un braț (numit uneori o perie sau o furcă).

Datele sînt scrise pe cercuri concentrice numite **piste**. Pista zero este (de obicei) pe exterior. Timpul care e necesar pentru poziționarea capului pe o anumită pistă este numit **timp de căutare**. Deseori platanurile sînt puse unul pe deasupra celuilalt, de aici setul de piste accesibile la o anumită poziție a periei care formează un **cilindru**. Pistele sînt divizate în **sectoare** de 512 biți, cu mai mult spațiu neutilizat (**roster**) între sectoarele de pe exteriorul platanului.

Cînd spargeți **timpul de acces** avertizat al unui hard-disk, veți observa că cel mai mult din acel timp este luat de mișcarea capurilor (aproximativ 65%) și de **latența rotațională** (aproximativ 30%).

dispozitiv bloc

Dispozitivele hard-disk cu acces aleatoriu au un înveliș de abstracție numit **dispozitiv bloc** pentru a permite formatarea blocurilor în mărime fixă (de obicei 512 biți). Blocurile pot fi accesate independent de accesul altor blocuri. Un dispozitiv bloc are litera b pentru a denota tipul de fișier în ieșirea **ls -l**.

```
[root@RHEL4b ~]# ls -l /dev/sda*
brw-rw---- 1 root disk 8, 0 Aug 4 22:55 /dev/sda
brw-rw---- 1 root disk 8, 1 Aug 4 22:55 /dev/sda1
brw-rw---- 1 root disk 8, 2 Aug 4 22:55 /dev/sda2
[root@RHEL4b ~]#
```

Notăți că un **dispozitiv consolă** este un curent constant de caractere, fiind denotat cu un c în **ls -l**.

Notăți de asemenea că standardul **ISO 9660** pentru cdrom folosește o mărime bloc de **2048 biți** bloc.

Hard-disk-urile vechi (și floppy-disk-urile) folosesc **cilindru-cap-sector** cînd adresează accesul unui sector pe disk. Cele mai multe disk-uri folosesc **LBA (Adresarea Block-urilor Logice)**.

ide sau scsi

În realitate, titlul ar trebui să fie **ata** sau **scsi**, de vreme ce ide este un dispozitiv compatibil ata. Cele mai multe desktop-uri folosesc **dispozitive ata**, cele mai multe servere folosesc **scsi**.

ata

Un **controlator ata** permite două dispozitive per bus, un **master** și un **slave**. Doar dacă controlatorul și dispozitivele suportă **selectarea cablurilor**, trebuie să le setați manual cu jumperi.

Odată cu introducerea **sata** (serial ata), ata original a fost redenumit în **paralel ata**. Disk-urile optice deseori folosesc **atapi**, care este o interfață ATA folosind protocolul de comunicare SCSI.

scsi

Un **controlator scsi** permite mai mult de două dispozitive. Când se folosește **SCSI (Small Computer System Interface)**, fiecare dispozitiv primește un **id scsi** unic. **Controlatorul scsi** de asemeni are nevoie de un **id scsi**, nu folosiți acest id pentru dispozitive atașate-scsi.

SCSI-urile mai vechi pe 8 biți sînt numite acum **învechite**, pentru că cele de 16-biți sînt larg răspîndite. Când viteza bus a fost dublată la 10 Mhz, acest lucru a fost cunoscut ca **SCSI rapid**. Dublînd viteza la 20Mhz l-a făcut **ultra SCSI**. Citiți <http://en.wikipedia.org/wiki/SCSI> pentru mai multe standarde SCSI.

4.2. nume dispozitive

numire dispozitive ata (ide)

Toate dispozitivele **ata** pe sistem vor începe cu **/dev/hd** urmat de un element literă. Hdd-ul master de pe primul **controlator ata** este **/dev/hda**, slave este **/dev/hdb**. Pentru al doilea controlator, numele dispozitivelor sînt **/dev/hdc** și **/dev/hdd**.

Tabelul 4.1. numire dispozitive ide

controlator	conexiune	nume dispozitiv
ide0	master	/dev/hda
	slave	/dev/hdb
ide1	master	/dev/hdc
	slave	/dev/hdd

Este posibil să aveți doar **/dev/hda** și **/dev/hdd**. Primul este un singur hard-disk ata, al doilea este cdrom-ul (prin default configurat ca slave).

denumire dispozitive scsi

Dispozitivele **scsi** urmează o schemă similară, dar toate încep cu **/dev/sd**. Când rămîneți fără litere (după **/dev/sdz**), puteți continua cu **/dev/sdaa** și **/dev/sdab** și așa mai departe. (Vom vedea mai tîrziu că volumele **lvm** sînt în mod comun văzute ca **/dev/md0**, **/dev/md1** etc.)

Mai jos este un **exemplu** anume cum pot fi numite dispozitivele scsi pe Linux. Adăugînd un disk **scsi** sau un controlator raid cu o adresă joasă scsi va schimba schema numelor (schimbînd adresele scsi mai sus cu o literă mai departe în alfabet).

Tabelul 4.2. numire dispozitive scsi

dispozitiv	scsi id	numele dispozitivului
disk 0	0	/dev/sda
disk 1	1	/dev/sdb
controlator raid 0	5	/dev/sdc
controlator raid 1	6	/dev/sdd

4.3. descoperire dispozitive disk

/sbin/fdisk

Puteți începe utilizînd **/sbin/fdisk** pentru a afla ce tip de disk-uri sînt văzute de kernel. Mai jos este un exemplu pe Debian, cu două **disk-uri ata-ide** prezente.


```
root@barry:~# fdisk -l | grep Disk
Disk /dev/hda: 60.0 GB, 60022480896 bytes
Disk /dev/hdb: 81.9 GB, 81964302336 bytes
```

Și aici un exemplu de **disk-uri sata** pe un laptop cu Ubuntu. Țineți minte că disk-urile **sata** sînt prezentate cu notarea **scsi** /dev/sdx.

```
root@laika:~# fdisk -l | grep Disk
Disk /dev/sda: 100.0 GB, 100030242816 bytes
Disk /dev/sdb: 100.0 GB, 100030242816 bytes
```

Aici este o previzualizare a disk-urilor pe un server RHEL4u3 cu două **disk-uri scsi** de 72GB. Acest server este atașat la un **NAS** cu patru **disk-uri NAS** din jumătate de terrabiți. Pe disk-urile NAS, patru dispozitive LVM (/dev/mdx) software RAID sînt configurate.

```
[root@tsvt11 ~]# fdisk -l | grep Disk
Disk /dev/sda: 73.4 GB, 73407488000 bytes
Disk /dev/sdb: 73.4 GB, 73407488000 bytes
Disk /dev/sdc: 499.0 GB, 499036192768 bytes
Disk /dev/sdd: 499.0 GB, 499036192768 bytes
Disk /dev/sde: 499.0 GB, 499036192768 bytes
Disk /dev/sdf: 499.0 GB, 499036192768 bytes
Disk /dev/md0: 271 MB, 271319040 bytes
Disk /dev/md2: 21.4 GB, 21476081664 bytes
Disk /dev/md3: 21.4 GB, 21467889664 bytes
Disk /dev/md1: 21.4 GB, 21476081664 bytes
```

Puteți de asemeni utiliza **fdisk** pentru a obține informații despre un anume dispozitiv hard-disk.

```
[root@rhel4 ~]# fdisk -l /dev/sda
```

```
Disk /dev/sda: 12.8 GB, 12884901888 bytes
255 heads, 63 sectors/track, 1566 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
```

Device	Boot	Start	End	Blocks	Id	System
/dev/sda1	*	1	13	104391	83	Linux
/dev/sda2		14	1566	12474472+	8e	Linux LVM

Mai tîrziu vom folosi **fdisk** pentru a face lucruri periculoase ca ștergerea și crearea partițiilor.

/bin/dmesg

Mesajele de boot ale kernel-ului pot fi văzute după boot cu **dmesg**. Pentru că dispozitivele hard-disk sînt detectate de kernel în timpul butării, puteți de asemeni folosi **dmesg** pentru a afla informații despre dispozitivele disk.

```

root@barry:~# dmesg | grep "[hs]d[a-z]"
Kernel command line: root=/dev/hda1 ro
    ide0: BM-DMA at 0xfc00-0xfc07, BIOS settings: hda:DMA, hdb:DMA
    ide1: BM-DMA at 0xfc08-0xfc0f, BIOS settings: hdc:DMA, hdd:DMA
hda: ST360021A, ATA DISK drive
hdb: Maxtor 6Y080L0, ATA DISK drive
hdc: SONY DVD RW DRU-510A, ATAPI CD/DVD-ROM drive
hdd: SONY DVD RW DRU-810A, ATAPI CD/DVD-ROM drive
hda: max request size: 128KiB
hda: 117231408 sectors (60022 MB) w/2048KiB Cache, CHS=65535/16/63, UDMA
    hda: hda1 hda2
hdb: max request size: 128KiB
hdb: 160086528 sectors (81964 MB) w/2048KiB Cache, CHS=65535/16/63, UDMA
    hdb: hdb1 hdb2
hdc: ATAPI 32X DVD-ROM DVD-R CD-R/RW drive, 8192kB Cache, UDMA(33)
hdd: ATAPI 40X DVD-ROM DVD-R CD-R/RW drive, 2048kB Cache, UDMA(33)
...

```

Aici este un alt exemplu **dmesg** (de pe același computer de mai sus, dar cu un disk mai mare cu 200GB acum).

```

paul@barry:~$ dmesg | grep -i "ata disk"
[ 2.624149] hda: ST360021A, ATA DISK drive
[ 2.904150] hdb: Maxtor 6Y080L0, ATA DISK drive
[ 3.472148] hdd: WDC WD2000BB-98DWA0, ATA DISK drive

```

Al treilea și ultimul exemplu **dmesg** rulînd pe RHEL5.3.

```

root@rhel53~# dmesg | grep -i "scsi disk"
sd 0:0:2:0: Attached scsi disk sda
sd 0:0:3:0: Attached scsi disk sdb
sd 0:0:6:0: Attached scsi disk sdc

```

/sbin/lshw

Utilitarul **lshw** va **lista hardware**. Cu opțiunile corecte **lshw** poate arăta multe informații despre disk-uri (și partiții).

Mai jos este o captură de ecran truncată de pe Debian 5:

```

root@debian5:~# aptitude search lshw
p   lshw      - information about hardware configuration

p   lshw-gtk   - information about hardware configuration
root@debian5:~# aptitude install lshw
. . .
root@debian5:~# lshw -class volume
*-volume:0
description: EXT3 volume
vendor: Linux
physical id: 1
bus info: ide@0.0,1
logical name: /dev/hda1
logical name: /
version: 1.0
serial: f327ca8a-8187-48c5-b760-956ec79d414b
size: 19GiB
capacity: 19GiB
capabilities: primary bootable journaled extended_attributes lar\
ge_files huge_files recover ext3 ext2 initialized
configuration: created=2009-10-28 12:02:35 filesystem=ext3 ...
. . .

```

Mai jos este o captură de ecran a **lshw** rulînd pe Ubuntu 10.10 pe un Macbook Pro:

```

root@ubu1010:~# lshw -class volume
*-volume:0 UNCLAIMED
description: EFI GPT partition
physical id: 1
bus info: scsi@0:0.0.0,1
capacity: 2047KiB
capabilities: primary nofs
*-volume:1
description: EXT4 volume
vendor: Linux
physical id: 2
bus info: scsi@0:0.0.0,2
logical name: /dev/sda2
logical name: /
version: 1.0
serial: 101eb20f-3e25-4900-b988-4622c0ee4ff5
size: 142GiB
capacity: 142GiB
. . .

```

/sbin/lsscsi

/sbin/lsscsi vă va arăta o ieșire frumoasă a tuturor dispozitivelor scsi (și a dispozitivelor scsi emulate). Această primă captură de ecran arată lsscsi pe un sistem SPARC.

```

root@shaka:~# lsscsi
[0:0:0:0]    disk    Adaptek      RAID5          v1.0    /dev/sda
[1:0:0:0]    disk    SEAGATE      ST336605FSUN36G 0438     /dev/sdb
root@shaka:~#

```

Aici este aceeași comandă, dar rulînd pe un laptop cu dvdwriter emulat scsi și usb emulat scsi.

```
paul@laika:~$ lsscsi
[0:0:0:0]      disk      ATA          HTS721010G9SA00    MCZ0    /dev/sda
[1:0:0:0]      disk      ATA          HTS721010G9SA00    MCZ0    /dev/sdb
[3:0:0:0]      cd/dvd    _NEC        DVD_RW ND-7551A    1-02    /dev/scd0
[4:0:0:0]      disk      GENERIC     USB Storage-CFC     019A    /dev/sdc
[4:0:0:1]      disk      GENERIC     USB Storage-SDC     019A    /dev/sdd
[4:0:0:2]      disk      GENERIC     USB Storage-SMC     019A    /dev/sde
[4:0:0:3]      disk      GENERIC     USB Storage-MSD     019A    /dev/sdf
```

/proc/scsi/scsi

O altă modalitate de a localiza dispozitivele **scsi** este prin fișierul **/proc/scsi/scsi**.

```
root@shaka:~# cat /proc/scsi/scsi
Attached devices:
Host: scsi0 Channel: 00 Id: 00 Lun: 00
  Vendor: Adaptec Model: RAID5          Rev: V1.0
  Type:   Direct-Access                 ANSI SCSI revision: 02
Host: scsi1 Channel: 00 Id: 00 Lun: 00
  Vendor: SEAGATE Model: ST336605FSUN36G Rev: 0438
  Type:   Direct-Access                 ANSI SCSI revision: 03
root@shaka:~#
```

/sbin/scsi_info și /sbin/scsiinfo

Există de asemeni o comandă **scsi_info**, dar ea nu este întotdeauna instalată prin default.

```
root@shaka:~# scsi_info /dev/sdb
SCSI_ID="0,0,0"
HOST="1"
MODEL="SEAGATE ST336605FSUN36G"
FW_REV="0438"
root@shaka:~#
```

Un alt utilitar simplu este **scsiinfo** care este parte a **scsitools** (de asemeni neinstalat prin default).

```
root@debian5:~# scsiinfo -l
/dev/sda /dev/sdb /dev/sdc
```

4.4. ștergerea unui hard-disk

Înainte de a vinde vechiul hard-disk pe internet, poate fi o idee bună să îl ștergeți. Repartiționîndu-l, sau folosind formatarea Microsoft Windows, sau chiar după o comandă **mkfs**, unii oameni vor fi încă capabili să citească cea mai mare parte de pe disk.

Deși din punct de vedere tehnic utilitarul **/sbin/badbblocks** este făcut pentru a căuta blocuri *bad*, puteți să-l folosiți pentru a șterge în mod complet toate datele de pe un disk. Pentru că asta este într-adevăr scrierea în fiecare sector al disk-ului, poate lua mult timp!

```
root@RHELv4u2:~# badblocks -ws /dev/sdb
Testing with pattern 0xaa: done
Reading and comparing: done
Testing with pattern 0x55: done
Reading and comparing: done
Testing with pattern 0xff: done
Reading and comparing: done
Testing with pattern 0x00: done
Reading and comparing: done
```

4.5. setări avansate hard-disk

Păcălirea setărilor hard-disk (dma, gap, ...) nu sînt discutate în acest curs. Există cîteva utilitare, și **hdparm** și **sdparm** sînt două dintre ele.

/sbin/hdparm poate fi folosit pentru a afișa sau a seta informații și parametri despre un dispozitiv hard-disk ATA (sau SATA). Opțiunile **-i** și **-I** vă vor da și mai multe informații despre proprietățile fizice ale dispozitivului.

```
root@laika:~# hdparm /dev/sdb

/dev/sdb:
IO_support = 0 (default 16-bit)
readonly   = 0 (off)
readahead  = 256 (on)
geometry   = 12161/255/63, sectors = 195371568, start = 0
```

Mai jos sînt informații **hdparm** despre un disk de 200GB IDE.

```
root@barry:~# hdparm /dev/hdd

/dev/hdd:
multcount   = 0 (off)
IO_support  = 0 (default)
unmaskirq   = 0 (off)
using_dma   = 1 (on)
keepsettings = 0 (off)
readonly    = 0 (off)
readahead   = 256 (on)
geometry    = 24321/255/63, sectors = 390721968, start = 0
```

Aici o captură de ecran a **sdparm** pe Ubuntu 10.10.

```
root@ubu1010:~# aptitude install sdparm
...
root@ubu1010:~# sdparm /dev/sda | head -1
/dev/sda: ATA FUJITSU MJA2160B 0081
root@ubu1010:~# man sdparm
```

Folosiți **hdparm** și **sdparm** cu grijă.

4.6. practică: dispozitive hard-disk

Despre acest laborator: pentru a face practică lucrînd cu hard-disk-uri, veți avea nevoie de cîteva hard-disk-uri. Cînd nu este disponibil nici un hard-disk, puteți folosi disk-urile virtuale în **VMware** sau **VirtualBox**. Profesorul vă va ajuta să atașați cîteva disk-uri ATA și/sau disk-uri SCSI la o mașină virtuală. Rezultatele acestui laborator pot fi folosite în următoarele trei laboratoare (partiții, sisteme de fișiere, montări). Vă sfătuim să atașați cel puțin un dispozitiv **ide** și trei disk-uri **scsi** egale în mărime la mașina virtuală.

1. Folosiți **dmesg** pentru a face o listă a dispozitivelor hard-disk detectate la boot-up.
2. Folosiți **fdisk** pentru a afla mărimea totală a tuturor dispozitivelor hard-disk de pe sistem.
3. Închideți o mașină virtuală, adăugați trei dispozitive hard disk **scsi** de 1 GB și un dispozitiv hard-disk virtual **ide** de 400 de megabiți. Dacă este posibil, adăugați de asemeni un alt disk virtual **ide** de 400 de megabiți.
4. Folosiți **dmesg** pentru a verifica ca toate noile disk-uri să fie detectate corespunzător la boot-up.
5. Verificați dacă puteți vedea dispozitivele disk în **/dev**.
6. Folosiți **fdisk** (cu **grep** și **/dev/null**) pentru a afișa mărimea totală a noilor disk-uri.
7. Folosiți **badblocks** pentru a șterge în mod complet unul dintre cele mai mici hard-disk-uri.
8. Priviți la **/proc/scsi/scsi**.
9. Dacă este posibil, instalați **lsscsi**, **lshw** și folosiți-le pentru a lista disk-urile.

4.7. soluție: dispozitive hard-disk

1. Folosiți **dmesg** pentru a face o listă a dispozitivelor hard-disk detectate la boot-up.

Unele posibile răspunsuri...

```
dmesg | grep -i disk
```

Căutarea disk-urilor ATA: `dmesg | grep hd[abcd]`

Căutarea disk-urilor ATA: `dmesg | grep -i "ata disk"`

Căutarea disk-urilor SCSI: `dmesg | grep sd[a-f]`

Căutarea disk-urilor SCSI: `dmesg | grep -i "scsi disk"`

2. Folosiți **fdisk** pentru a afla mărimea totală a tuturor dispozitivelor hard-disk de pe sistem.

```
fdisk -l
```

3. Închideți o mașină virtuală, adăugați trei dispozitive hard disk **scsi** de 1 GB și un dispozitiv hard-disk virtual **ide** de 400 de megabiți. Dacă este posibil, adăugați de asemeni un alt disk virtual **ide** de 400 de megabiți.

Acest exercițiu se petrece în setările VMware sau VirtualBox.

4. Folosiți **dmesg** pentru a verifica ca toate noile disk-uri să fie detectate corespunzător la boot-up.

Vezi răspunsul de la punctul 1.

5. Verificați dacă puteți vedea dispozitivele disk în **/dev**.

```
SCSI+SATA: ls -l /dev/sd*
```

```
ATA: ls -l /dev/hd*
```

6. Folosiți **fdisk** (cu **grep** și **/dev/null**) pentru a afișa mărimea totală a noilor disk-uri.

```
root@rhel53 ~# fdisk -l 2>/dev/null | grep [MGT]B
Disk /dev/hda: 21.4 GB, 21474836480 bytes
Disk /dev/hdb: 1073 MB, 1073741824 bytes
Disk /dev/sda: 2147 MB, 2147483648 bytes
Disk /dev/sdb: 2147 MB, 2147483648 bytes
Disk /dev/sdc: 2147 MB, 2147483648 bytes
```

7. Folosiți **badblocks** pentru a șterge în mod complet unul dintre cele mai mici hard-disk-uri.

#Verificați dispozitivul (/dev/sdc??) pe care vreți să-l ștergeți înainte de a tasta asta.

```

root@rhel53 ~# badblocks -ws /dev/sdc
Testing with pattern 0xaa: done
Reading and comparing: done
Testing with pattern 0x55: done
Reading and comparing: done
Testing with pattern 0xff: done
Testing with pattern 0x00: done
Reading and comparing: done

```

8. Priviți la **/proc/scsi/scsi**.

```

root@rhel53 ~# cat /proc/scsi/scsi
Attached devices:
Host: scsi0 Channel: 00 Id: 02 Lun: 00
  Vendor: VBOX    Model: HARDDISK          Rev: 1.0
  Type:   Direct-Access                    ANSI SCSI revision: 05
Host: scsi0 Channel: 00 Id: 03 Lun: 00
  Vendor: VBOX    Model: HARDDISK          Rev: 1.0
  Type:   Direct-Access                    ANSI SCSI revision: 05
Host: scsi0 Channel: 00 Id: 06 Lun: 00
  Vendor: VBOX    Model: HARDDISK          Rev: 1.0
  Type:   Direct-Access                    ANSI SCSI revision: 05

```

9. Dacă este posibil, instalați **lsscsi**, **lshw** și folosiți-le pentru a lista disk-urile.

Debian, Ubuntu: `aptitude install lsscsi lshw`

Fedora: `yum install lsscsi lshw`

```

root@rhel53 ~# lsscsi
[0:0:2:0]    disk      VBOX      HARDDISK      1.0        /dev/sda
[0:0:3:0]    disk      VBOX      HARDDISK      1.0        /dev/sdb
[0:0:6:0]    disk      VBOX      HARDDISK      1.0        /dev/sdc

```


Capitolul 5. partiții disk

Conținut

5.1. despre partiții.	42
5.2. descoperirea partițiilor.	42
5.3. partiționarea noilor disk-uri.	43
5.4. despre tabelul partițiilor.	45
5.5. practică: partiții.	47
5.6. soluție: partiții.	48

Acest capitol continuă explicarea **dispozitivelor hard-disk** din capitolul anterior. Aici vom pune partiții pe acele dispozitive.

Acest capitol vă pregătește pentru capitolul următor, unde vom pune **sisteme de fișiere** pe partiții.

5.1. despre partiții

primare, extinse și logice

Linux solicită să creați una sau mai multe **partiții**. Următorul paragraf va explica cum să creăm și să utilizăm partiții.

Geometria și mărimea unei partiții este de obicei definită de cilindrul de start și sfârșit (uneori de sector). Partițiile pot fi de tip **primar** (maxim patru), **extinse** (maxim unu) sau **logice** (conținute în interiorul partiției extinse). Fiecare partiție are un **cîmp tip** care conține un cod. Acesta determină sistemul de operare a computerelor sau partițiile sistemului de fișier.

Tabelul 5.1. partiții primare, extinse și logice

tip de partiție	denumire
Primară (max 4)	1-4
Exitinsă (max 1)	1-4
Logică	5-

numele partiției

Am văzut mai înainte că dispozitivele hard-disk sînt numite /dev/hdx sau /dev/sdx x depinzînd de configurația hardware. Alături este numărul partiției, începînd numărarea de la numărul 1. De aici cele patru (posibile) partiții primare sînt numărate de la 1 la 4. Numărarea partiției logice începe întotdeauna cu 5. Astfel /dev/hda2 este a doua partiție de pe primul dispozitiv hard-disk ATA, și /dev/hdb5 este prima partiție logică pe al doilea dispozitiv hard-disk ATA. La fel este pentru SCSI, /dev/sdb3 este a treia partiție pe al doilea disk SCSI.

Tabelul 5.2. numele partiției

partiție	dispozitiv
/dev/hda1	prima partiție primară pe /dev/hda
/dev/hda2	a doua partiție primară sau extinsă pe /dev/hda
/dev/sda5	primul dispozitiv logic pe /dev/sda
/dev/sdb6	a doua partiție logică pe /dev/sdb

5.2. descoperirea partițiilor

fdisk -l

În exemplul **fdisk -l** de mai jos puteți vedea că există două partiții pe **/dev/sdb**. Prima partiție se întinde pe 31 de cilindri și conține o partiție linux swap. A doua partiție este mult mai mare.

```
root@laika:~# fdisk -l /dev/sdb
```

```
Disk /dev/sdb: 100.0 GB, 100030242816 bytes
255 heads, 63 sectors/track, 12161 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
```

Device	Boot	Start	End	Blocks	Id	System
/dev/sdb1		1	31	248976	82	Linux swap / Solaris
/dev/sdb2		32	12161	97434225	83	Linux

```
root@laika:~#
```

/proc/partitions

Fișierul **/proc/partitions** conține un tabel cu numerele majore și minore a dispozitivelor partiționate, numerele lor de blocuri și numele dispozitiv în **/dev**. Verificați cu **/proc/devices** legătura numerelor majore la dispozitivul corespunzător.

```
paul@RHELv4u4:~$ cat /proc/partitions
```

major	minor	#blocks	name
3	0	524288	hda
3	64	734003	hdb
8	0	8388608	sda
8	1	104391	sda1
8	2	8281507	sda2
8	16	1048576	sdb
8	32	1048576	sdc
8	48	1048576	sdd
253	0	7176192	dm-0
253	1	1048576	dm-1

Numărul **major** corespunde cu tipul dispozitivului (sau a driverului) care poate fi găsit în **/proc/devices**. În acest caz 3 corespunde cu **ide** și 8 cu **sd**. Numărul **major** determină **driverul dispozitivului** pentru a fi utilizat cu acest dispozitiv.

Numărul **minor** este o identificare unică a unei instanțe a acestui tip de dispozitiv. Fișierul **devices.txt** în arborele kernel conține o listă deplină cu numere majore și minore.

alte utilitare

Ați putea fi interesat de alternative la **fdisk** ca **parted**, **cfdisk**, **sfdisk** și **gparted**. Acest curs utilizează în principal **fdisk** pentru a partiționa hard-disk-uri.

5.3. partiționarea noilor disk-uri

În exemplul de mai jos, am cumpărat un nou disk pentru sistemul nostru. După ce noul hardware este bine pus, puteți folosi **fdisk** și **parted** pentru a crea partițiile necesare. Acest exemplu folosește **fdisk**, dar nu e nimic greșit dacă folosiți **parted**.

recunoașterea disk-ului

Mai întâi, căutăm cu **fdisk -l** dacă Linux poate vedea noul disk. Da, îl vede, noul disk este văzut ca **/dev/sdb**, dar el nu are nici o partiție încă.

```
root@RHELv4u2:~# fdisk -l
```

```
Disk /dev/sda: 12.8 GB, 12884901888 bytes
255 heads, 63 sectors/track, 1566 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
```

Device	Boot	Start	End	Blocks	Id	System
/dev/sda1	*	1	13	104391	83	Linux
/dev/sda2		14	1566	12474472+	8e	Linux LVM

```
Disk /dev/sdb: 1073 MB, 1073741824 bytes
255 heads, 63 sectors/track, 130 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk /dev/sdb doesn't contain a valid partition table
```

deschiderea disk-ului cu fdisk

Apoi creăm o partiție cu fdisk pe /dev/sdb. Mai întâi începem utilitarul fdisk cu /dev/sdb ca argument. Fiți foarte atenți să nu partiționați disk-ul greșit!!

```
root@RHELv4u2:~# fdisk /dev/sdb
Device contains neither a valid DOS partition table, nor Sun, SGI...
Building a new DOS disklabel. Changes will remain in memory only,
until you decide to write them. After that, of course, the previous
content won't be recoverable.
```

```
Warning: invalid flag 0x0000 of partition table 4 will be corrected...
```

partiție goală tabel

Înăuntrul utilitarului fdisk, putem tasta comanda **p** pentru a vedea disk-urile curente tabelului partițiilor.

```
Command (m for help): p
```

```
Disk /dev/sdb: 1073 MB, 1073741824 bytes
255 heads, 63 sectors/track, 130 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
```

Device	Boot	Start	End	Blocks	Id	System
--------	------	-------	-----	--------	----	--------

creați o nouă partiție

Nici o partiție nu există încă, așa că tastăm **n** pentru a crea o nouă partiție. Alegem **p** pentru primar, **1** pentru numărul partiției, **1** pentru cilindrul de start și **14** pentru cilindrul de sfârșit.

```
Command (m for help): n
Command action
e    extended
p    primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-130, default 1):
Using default value 1
Last cylinder or +size or +sizeM or +sizeK (1-130, default 130): 14
```

Putem lansa acum din nou **p** pentru a verifica schimbările pe care le-am făcut, dar ele nu sînt scrise încă pe disk. Asta înseamnă că încă putem să anulăm această operațiune! Dar arată bine, așa că vom folosi **w** pentru a scrie schimbările pe disk, și apoi să închidem utilitarul **fdisk**.

Command (m for help): **p**

Disk /dev/sdb: 1073 MB, 1073741824 bytes
255 heads, 63 sectors/track, 130 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

Device	Boot	Start	End	Blocks	Id	System
/dev/sdb1		1	14	112423+	83	Linux

Command (m for help): **w**

The partition table has been altered!

Calling ioctl() to re-read partition table.

Syncing disks.

root@RHELv4u2:~#

afișarea noii partiții

Să verificăm din nou cu **fdisk -l** să fim siguri că se potrivește dorințelor noastre. Într-adevăr, captura de ecran de mai jos arată acum o partiție pe /dev/sdb.

root@RHELv4u2:~# **fdisk -l**

Disk /dev/sda: 12.8 GB, 12884901888 bytes
255 heads, 63 sectors/track, 1566 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

Device	Boot	Start	End	Blocks	Id	System
/dev/sda1	*	1	13	104391	83	Linux
/dev/sda2		14	1566	12474472+	8e	Linux LVM

Disk /dev/sdb: 1073 MB, 1073741824 bytes
255 heads, 63 sectors/track, 130 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

Device	Boot	Start	End	Blocks	Id	System
/dev/sdb1		1	14	112423+	83	Linux

root@RHELv4u2:~#

5.4. despre tabelul partiției

master boot record

Informația despre **tabelul partiției** (partiții primare și extinse) este scrisă în **master boot record** sau **mbr**. Puteți folosi **dd** pentru a copia mbr într-un fișier.

Acest exemplu copie master boot record de pe primul hard-disk SCSI.

dd if=/dev/sda of=/SCSIdisk.mbr bs=512 count=1

Același utilitar poate fi folosit de asemeni pentru a șterge toată informația despre partiții pe un disk. Acest exemplu scrie zerouri deasupra master boot record.

```
dd if=/dev/zero of=/dev/sda bs=512 count=1
```

Sau să șteargă toată partiția sau disk-ul.

```
dd if=/dev/zero of=/dev/sda
```

partprobe

Nu uitați că după restaurarea unui **master boot record** cu **dd**, trebuie să forțați kernel-ul să re-citească tabelul partițiilor cu **partprobe**. După ce rulați **partprobe**, partițiile pot fi folosite din nou.

```
[root@RHEL5 ~]# partprobe  
[root@RHEL5 ~]#
```

disk-uri logice

Tabelul partițiilor nu conține informații despre **disk-uri logice**.

Așa că backup-ul **dd** al **mbr** funcționează doar pentru partiții primare și extinse. Pentru a face backup tabelului partițiilor incluzând disk-urile logice, puteți folosi **sfdisk**.

Acest exemplu arată cum să facem backup tuturor informațiilor despre partiții și disk-uri logice într-un fișier.

```
sfdisk -d /dev/sda > parttable.sda.sfdisk
```

Următorul exemplu copie informație **mbr** și toate **disk-urile logice** din /dev/sda în /dev/sdb.

```
sfdisk -d /dev/sda | sfdisk /dev/sdb
```

5.5. practică: partiții

1. Folosiți **fdisk -l** pentru a afișa partițiile existente și mărimea lor.
2. Folosiți **df -h** pentru a afișa partiții existente și mărimea lor.
3. Comparați ieșirea cu **fdisk** și **df**.
4. Creați o partiție primară de 200 MB pe un disk mic.
5. Creați o partiție primară de 400 MB și două partiții logice de 300 MB pe un disk mare.
6. Folosiți **df -h** și **fdisk** pentru a vă verifica.
7. Comparați ieșirea din nou cu **fdisk** și **df**. Ambele comenzi afișează noile partiții?
8. Creați un backup cu **dd** a **mbr** care conține partiția primară de 200 MB.
9. Faceți un backup a **tabelului partiției** conținând partițiile primară de 400 MB și cea logică de 300 MB. Asigurați-vă că disk-urile logice sînt în backup.
- 10 (opțional) Ștergeți toate partițiile cu **fdisk**. Apoi restaurați backup-urile.

5.6. soluție: partiții

1. Folosiți **fdisk -l** pentru a afișa partițiile existente și mărimea lor.

```
ca root: # fdisk -l
```

2. Folosiți **df -h** pentru a afișa partiții existente și mărimea lor.

```
df -h
```

3. Comparați ieșirea cu **fdisk** și **df**.

Unele partiții vor fi listate în ambele ieșiri (poate /dev/sda1 sau /dev/hda1).

4. Creați o partiție primară de 200 MB pe un disk mic.

Alegeți unul dintre disk-urile pe care le-ați adăugat (acest exemplu folosește /dev/sdc).

```
root@rhel53 ~# fdisk /dev/sdc
...
Command (m for help): n
Command action
  e   extended
  p   primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-261, default 1): 1
Last cylinder or +size or +sizeM or +sizeK (1-261, default 261): +200m
Command (m for help): w
The partition table has been altered!
Calling ioctl() to re-read partition table.
Syncing disks.
```

5. Creați o partiție primară de 400 MB și două partiții logice de 300 MB pe un disk mare.

Alegeți unul dintre disk-urile pe care le-ați adăugat (acest exemplu folosește /dev/sdb).

```
fdisk /dev/sdb
```

```
înăuntrul fdisk : n p 1 +400m enter --- n e 2 enter enter --- n l +300m (de două ori)
```

6. Folosiți **df -h** și **fdisk -l** pentru a vă verifica.

```
fdisk -l ; df -h
```

7. Comparați ieșirea din nou cu **fdisk** și **df**. Ambele comenzi afișează noile partiții?

Noile partiții create sînt vizibile cu fdisk.

Dar ele nu sînt afișate cu df.

8. Creați un backup cu **dd** a **mbr** care conține partiția primară de 200 MB.

```
dd if=/dev/sdc of=bootsector.sdc.dd count=1 bs=512
```


9. Faceți un backup a **tabelului partiției** conținând partițiile primară de 400 MB și cea logică de 300 MB. Asigurați-vă că disk-urile logice sînt în backup.

```
sfdisk -d /dev/sdb > parttable.sdb.sfdisk
```

Capitolul 6. sisteme fișier

Conținut

6.1. despre sisteme fișier.	51
6.2. sisteme fișier comune.	51
6.3. punerea unui sistem fișier pe o partiție.	53
6.4. ajustarea unui sistem fișier.	54
6.5. verificarea unui sistem fișier.	54
6.6. practică: sisteme fișier.	56
6.7. soluție: sisteme fișier.	57

Când ați terminat de partiționat hard-disk-ul, puteți pune un **sistem fișier** pe fiecare partiție.

Acest capitol se bazează pe **partițiile** din capitolul precedent, și vă pregătește pentru următorul unde vom **monta** sistemele de fișier.

6.1. despre sisteme fișier

Un sistem fișier este o modalitate de a organiza fișiere pe partiție. În afară de stocarea-pe-bază-de-fișier, sistemele fișier de obicei includ **directoare** și **control de acces**, și conțin meta informații despre fișiere ca timpul de acces, timpul modificării și proprietarul fișierului.

Proprietățile (lungime, setul de caractere, ...) a numelor fișierelor sînt determinate de sistemul de fișier pe care îl alegeți. Directoarele sînt de obicei implementate ca fișiere, va trebui să învățați cum este acest lucru implementat! Controlul de acces în sisteme fișier este urmărit de către proprietarul utilizator (și proprietarul grup - și totalitatea membrilor) în combinație cu una sau mai multe liste de control acces.

Pagina de manual despre filesystems(5) este de obicei accesată tastînd **man fs**. Puteți de asemeni să vă uitați la **/proc/filesystems** pentru a vedea driverele sistemului de fișier încărcat curent.

```
root@rhel53 ~# cat /proc/filesystems | grep -v nodev
ext2
iso9660
ext3
```

6.2. sisteme fișier comune

ext2 și ext3

Cel mai comun sistem fișier Linux a fost odată **ext2** (al doilea extins). Un dezavantaj constituie căutările sistemului de fișier pe ext2 care poate lua mult timp. Veți vedea că ext2 va fi înlocuit de **ext3** pe cele mai multe mașini Linux. Ele sînt în mod esențial la fel, cu excepția **jurnalizării** care este prezentă doar în ext3.

Jurnalizarea înseamnă că schimbările mai întîi sînt scrise într-un fișier-jurnal pe disk. Fișierul-jurnal este curățat cu regularitate, scriind schimbările în sistemul de fișier. Jurnalizarea ține sistemul de fișier într-un status consistent, așa că nu aveți nevoie de o verificare a sistemului de fișier după o închidere necorespunzătoare sau o pană de curent.

Puteți crea aceste sisteme de fișier cu comenzile **/sbin/mkfs** sau **/sbin/mke2fs**. Folosiți **mke2fs -j** pentru a crea un sistem fișier ext3. Puteți converti un ext2 în ext3 cu **tune2fs -j**. Puteți monta sistemul de fișier ext3 ca ext2, dar pierdeți jurnalizarea. Nu uitați să executați **mkinitrd** dacă butați de pe acest dispozitiv.

ext4

Din 2008 noua încarnare a sistemului de fișier ext este **ext4**, disponibil în kernel-ul Linux. **ext4** suportă fișiere mari (pînă la 16 terabiți) și un sistem fișier mai mare decît **ext3** (și multe alte îmbunătățiri).

vfat

Sistemul de fișier **vfat** există sub mai multe forme: **fat12** pentru diskete floppy, **fat16** pe ms-dos, și **fat32** pentru disk-uri mai mari. Implementarea **vfat** pe Linux suportă toate acestea, dar lui vfat îi lipsesc multe îmbunătățiri în ceea ce privește securitatea și link-urile. Disk-urile **fat** pot fi citite de orice sistem de operare, și sînt folosite mult pentru aparate de fotografiat, stick-uri **usb** și pentru a schimba date între diferite sisteme de operare pe un computer a unui utilizator obișnuit.

iso 9660

iso 9660 este formatul standard pentru cdrom-uri. Există șanse să întâlnești acest sistem fișier de asemeni pe hard-disk sub formă de imagini de cdrom-uri (deseori cu extensia .iso). Standardul **iso 9660** limitează numele de fișier la formatul 8.3. Lumea Unix nu i-a plăcut asta, și astfel a adăugat extensiile **rock ridge**, care permit nume de fișier până la 255 caractere și fișiere modale-tip Unix, proprietari și link-uri simbolice. O altă extensie a **iso 9660** este **joliet**, care adaugă 64 de caractere unicode la numele de fișier. Standardul **el torito** extinde **iso 9660** pentru a fi capabil să buteze de pe CD-ROM.

udf

Cele mai multe medii optice astăzi (incluzînd cd-uri și dvd-uri) folosesc **udf**, Universal Disk Format.

swap

Luînd în vedere toate lucrurile, swap nu este un sistem fișier. Dar pentru a utiliza o partiție ca **partiție swap** ea trebuie formatată și montată ca spațiu swap.

altele...

Ați putea întâlni **reiserfs** pe sisteme Linux vechi. Poate veți vedea **zfs** de la Sun sau **btrfs** cu sursă deschisă. Ultimul cere un capitol în sine.

/proc/filesystems

Fișierul **/proc/filesystems** afișează o listă a sistemelor de fișier suportate. Cînd montați un sistem fișier fără a-l defini în mod explicit, atunci mount mai întîi va încerca să probeze **/etc/filesystems** și apoi să probeze **/proc/filesystems** pentru toate sistemele de fișiere fără eticheta **nodev**. Dacă **/etc/filesystems** se termină cu o linie conținînd doar un asterix (*) atunci ambele fișiere sînt probate.

```
paul@RHELv4u4:~$ cat /proc/filesystems
nodev    sysfs
nodev    rootfs
nodev    bdev
nodev    proc
nodev    sockfs
nodev    binfmt_misc
nodev    usbfs
nodev    usbdevfs
nodev    fut exfs
nodev    tmpfs
nodev    pipefs
nodev    eventpollfs
nodev    devpts
nodev    ext2
nodev    ramfs
nodev    hugetlbfs
nodev    iso9660
nodev    relayfs
nodev    mqueue
nodev    selinuxfs
nodev    ext3
nodev    rpc_pipefs
nodev    vmware-hgfs
nodev    autofs
paul@RHELv4u4:~$
```

6.3. punerea unui sistem fișier pe o partiție

Avem acum o partiție nouă. Binarele de sistem pentru a face sisteme fișier pot fi găsite cu ls.

```
[root@RHEL4b ~]# ls -ls /sbin/mk*
-rwxr-xr-x 3 root root 34832 Apr 24 2006 /sbin/mke2fs
-rwxr-xr-x 3 root root 34832 Apr 24 2006 /sbin/mkfs.ext2
-rwxr-xr-x 3 root root 34832 Apr 24 2006 /sbin/mkfs.ext3
-rwxr-xr-x 3 root root 28484 Oct 13 2004 /sbin/mkdosfs
-rwxr-xr-x 3 root root 28484 Oct 13 2004 /sbin/mkfs.msdos
-rwxr-xr-x 3 root root 28484 Oct 13 2004 /sbin/mkfs.vfat
-rwxr-xr-x 1 root root 20313 Apr 10 2006 /sbin/mkinitrd
-rwxr-x--- 1 root root 15444 Oct 5 2004 /sbin/mkzonedb
-rwxr-xr-x 1 root root 15300 May 24 2006 /sbin/mkfs.cramfs
-rwxr-xr-x 1 root root 13036 May 24 2006 /sbin/mkswap
-rwxr-xr-x 1 root root 6192 May 24 2006 /sbin/mkfs
-rwxr-xr-x 1 root root 5905 Aug 3 2004 /sbin/mkbootdisk
[root@RHEL4b ~]#
```

Este timpul să citiți paginile de manual a **mkfs** și **mke2fs**. În exemplul de mai jos, vedeți crearea unui **sistem fișier ext2** pe /dev/sdb1. În viața reală, ați putea dori să folosiți opțiuni ca -m0 și -j.

```

root@RHELv4u2:~# mke2fs /dev/sdb1
mke2fs 1.35 (28-Feb-2004)
Filesystem label=
OS type: Linux
Block size=1024 (log=0)
Fragment size=1024 (log=0)
28112 inodes, 112420 blocks
5621 blocks (5.00%) reserved for the super user
First data block=1
Maximum filesystem blocks=67371008
14 block groups
8192 blocks per group, 8192 fragments per group
2008 inodes per group
Superblock backups stored on blocks:
8193, 24577, 40961, 57345, 73729

```

```

Writing inode tables: done
Writing superblocks and filesystem accounting information: done

```

This filesystem will be automatically checked every 37 mounts or 180 days, whichever comes first. Use tune2fs -c or -i to override.

6.4. ajustarea unui sistem fișier

Puteți folosi **tune2fs** pentru a lista și pune setările sistemului fișier. Prima captură de ecran listează spațiul rezervat pentru root (care este setat la 5 %).

```

[root@rhel4 ~]# tune2fs -l /dev/sda1 | grep -i "block count"
Block count:          104388
Reserved block count: 5219
[root@rhel4 ~]#

```

Acest exemplu schimbă această valoare la zece la sută. Puteți folosi **tune2fs** în timp ce sistemul fișier este activ, chiar dacă este sistemul de fișier root (ca în acest exemplu).

```

[root@rhel4 ~]# tune2fs -m10 /dev/sda1
tune2fs 1.35 (28-Feb-2004)
Setting reserved blocks percentage to 10 (10430 blocks)
[root@rhel4 ~]# tune2fs -l /dev/sda1 | grep -i "block count"
Block count:          104388
Reserved block count: 10430
[root@rhel4 ~]#

```

6.5. verificarea unui sistem fișier

Comanda **fsck** este un utilitar front-end pentru a verifica un sistem fișier să nu aibă erori.

```

[root@RHEL4b ~]# ls /sbin/*fsck*
/sbin/dosfsck  /sbin/fsck          /sbin/fsck.ext2  /sbin/fsck.msdos
/sbin/e2fsck   /sbin/fsck.cramfs   /sbin/fsck.ext3  /sbin/fsck.vfat
[root@RHEL4b ~]#

```

Ultima coloană în **/etc/fstab** este folosită pentru a determina dacă un sistem fișier ar trebui să fie controlat la boot-up.

```
[paul@RHEL4b ~]$ grep ext /etc/fstab
/dev/VolGroup00/LogVol00 / ext3 defaults 1 1
LABEL=/boot /boot ext3 defaults 1 2
[paul@RHEL4b ~]$
```

Verificarea manuală a unui sistem fișier montat rezultă într-un avertisment de la fsck.

```
[root@RHEL4b ~]# fsck /boot
fsck 1.35 (28-Feb-2004)
e2fsck 1.35 (28-Feb-2004)
/dev/sda1 is mounted.
```

WARNING!!! Running e2fsck on a mounted filesystem may cause SEVERE filesystem damage.

Do you really want to continue (y/n)? no

check aborted.

Dar după demontare fsck și **e2fsck** pot fi folosite să verifice un sistem fișier ext2.

```
[root@RHEL4b ~]# fsck /boot
fsck 1.35 (28-Feb-2004)
e2fsck 1.35 (28-Feb-2004)
/boot: clean, 44/26104 files, 17598/104388 blocks
[root@RHEL4b ~]# fsck -p /boot
fsck 1.35 (28-Feb-2004)
/boot: clean, 44/26104 files, 17598/104388 blocks
[root@RHEL4b ~]# e2fsck -p /dev/sda1
/boot: clean, 44/26104 files, 17598/104388 blocks
```

6.6. practică: sisteme fișier

1. Listați sistemele fișier care sînt cunoscute de către sistem.
2. Creați un sistem fișier **ext2** pe partiția de 200 MB.
3. Creați un sistem fișier **ext3** pe partiția de 400 MB și una de 300 MB pe disk-urile logice.
4. Setați spațiul rezervat pentru root pe partiția logică la 0 la sută.
5. Verificați munca cu **fdisk** și **df**.

6.7. soluție: sisteme fișier

1. Listați sistemele fișier care sînt cunoscute de către sistem.

```
man fs
```

```
cat /proc/filesystems
```

```
cat /etc/filesystems (nu pe toate distribuțiile Linux)
```

2. Creați un sistem fișier **ext2** pe partiția de 200 MB.

```
mke2fs /dev/sdc1 (înlocuiți sdc1 cu partiția corectă)
```

3. Creați un sistem fișier **ext3** pe partiția de 400 MB și una de 300 MB pe disk-urile logice.

```
mke2fs -j /dev/sdb1 (înlocuiți sdb1 cu partiția corectă)
```

```
mke2fs -j /dev/sdb5 (înlocuiți sdb5 cu partiția corectă)
```

4. Setati spațiul rezervat pentru root pe partiția logică la 0 la sută.

```
tune2fs -m 0 /dev/sdb5
```

5. Verificați munca cu **fdisk** și **df**.

mkfs (mke2fs) nu face nici o diferență în ieșirea acestor comenzi.

Marea schimbare este în capitolul următor: montarea.

Capitolul 7. montare

Conținut

7.1. montare sisteme fișier locale.	59
7.2. afișarea sistemelor fișier montate.	60
7.3. montări permanente.	61
7.4. securizarea montărilor.	61
7.5. practică: montare sisteme fișier.	63
7.6. soluție: montare sisteme fișier.	64

Odată ce ați pus un sistem fișier pe o partiție, îl puteți **monta**. Montarea unui sistem fișier îl face disponibil pentru utilizare, de obicei ca un director. Numim **montarea unui sistem fișier** în loc de montarea unei partiții pentru că vom vedea mai târziu că putem de asemeni monta sisteme fișier care nu există pe partiții.

7.1. montare sisteme fișier locale

Pe toate sistemele **Unix**, fiecare fișier și fiecare director este parte a unui mare arbore de fișier. Pentru a accesa un fișier, trebuie să știți calea directă începînd din directorul root. Cînd adăugați un **sistem fișier** la computer, trebuie să-l faceți disponibil undeva în arborele fișier. Directorul unde faceți disponibil un sistem fișier este numit un **punct montare**.

/bin/mkdir

Acest exemplu arată cum să creați un nou **punct montare** cu **mkdir**.

```
root@RHELv4u2:~# mkdir /home/project55
```

/bin/mount

Cînd **punctul de montare** este creat, și un **sistem fișier** este prezent pe o partiție, atunci **mount** poate **monta sistemul fișier** pe **directorul punct montare**.

```
root@RHELv4u2:~# mount -t ext2 /dev/sdb1 /home/project55/
```

Odată montat, noul sistem fișier este accesibil utilizatorilor.

/etc/filesystems

În realitate opțiunea explicită **-t ext2** pentru a seta un sistem fișier nu este întotdeauna necesară. Comanda **mount** este capabilă să detecteze automat mult din sistemele fișier.

Cînd montăm un sistem fișier fără să specificăm explicit sistemul fișier, atunci **mount** va proba mai întîi **/etc/filesystems**. **mount** va sări liniile cu directiva **nodev**.

```
paul@RHELv4u4:~$ cat /etc/filesystems
ext3
ext2
nodev proc
nodev devpts
iso9660
vfat
hfs
paul@RHELv4u4:~$
```

/proc/filesystems

Cînd **/etc/filesystems** nu există, sau se termină cu un singur ***** pe ultima linie, atunci **mount** va citi **/proc/filesystems**.

```
[root@RHEL52 ~]# cat /proc/filesystems | grep -v ^nodev
ext2
iso9660
ext3
```

/bin/umount

Puteți **demonta** un sistem fișier folosind comanda **umount**.

```
root@pasha:~# umount /home/reet
```

7.2. afișarea sistemelor fișier montate

Pentru a afișa toate sistemele fișier montate, tastați comanda **mount**. Sau priviți la fișierele **/proc/mounts** și **/etc/mtab**.

/bin/mount

Cea mai comună și mai utilizată cale de a vedea toate montările este de a tasta comanda **mount** fără nici un argument.

```
root@RHELv4u2:~# mount | grep /dev/sdb
/dev/sdb1 on /home/project55 type ext2 (rw)
```

/proc/mounts

Kernel-ul asigură informația în **/proc/mounts** în format fișier, dar **/proc/mounts** nu există ca un fișier pe nici un hard-disk. Uitându-ne la **/proc/mounts** este ca și cum ne-am uita la informație care vine direct din kernel.

```
root@RHELv4u2:~# cat /proc/mounts | grep /dev/sdb
/dev/sdb1 /home/project55 ext2 rw 0 0
```

/etc/mtab

Fișierul **/etc/mtab** nu este actualizat de către kernel, dar este menținut de comanda **mount**. Nu editați **/etc/mtab** manual.

```
root@RHELv4u2:~# cat /etc/mtab | grep /dev/sdb
/dev/sdb1 /home/project55 ext2 rw 0 0
```

/bin/df

O cale mai prietenoasă de a ne uita la sisteme fișier montate este **df**. Comanda **df (diskfree)** are beneficiul adăugat de a ne arăta spațiul liber pe fiecare disk montat. Ca multe dintre comenzile Linux, **df** suportă întrerupătorul **-h** pentru a face ieșirea **mai ușor de citit**.

```
root@RHELv4u2:~# df
Filesystem            1K-blocks    Used    Available    Use%    Mounted on
/dev/mapper/VolGroup00-LogVol00
11707972    6366996    4746240    58% /
/dev/sda1            101086      9300      86567       10%    /boot
none              127988        0      127988       0%    /dev/shm
/dev/sdb1            108865      1550      101694       2%    /home/project55
root@RHELv4u2:~# df -h
Filesystem            Size        Used        Available    Use%    Mounted on
/dev/mapper/VolGroup00-LogVol00
12G 6.1G 4.6G 58% /
/dev/sda1              99M         9.1M         85M         10%    /boot
none                  125M          0         125M          0%    /dev/shm
/dev/sdb1             107M         1.6M        100M          2%    /home/project55
```

În exemplul **df -h** de mai jos puteți vedea mărimea, spațiul liber, gigabiții utilizați, procentajul și punctul montare a unei partiții.

```
root@laika:~# df -h | egrep -e "(sdb2|File)"
Filesystem            Size Used Avail Use% Mounted on
/dev/sdb2             92G   83G   8.6G   91% /media/sdb2
root@laika:~#
```

/bin/du

Comanda **du** poate să facă un sumar a **utilizării disk-ului** pentru fișiere și directoare. Prevenind **du** să intre în subdirectoare cu opțiunea **-s** vă va arăta un total pentru acel director. Această opțiune este deseori utilizată împreună cu **-h**, astfel că **du -sh** pe un punct montare vă afișează suma totală utilizată în acea partiție.

```
root@pasha:~# du -sh /home/reet
881G      /home/reet
```

7.3. montări permanente

Pînă acum, am făcut toate montările manual. Asta merge bine, pînă la următorul reboot. Din fericire există o cale de a transmite computerului să monteze automat anumite sisteme fișier în timpul butării.

/etc/fstab

Asta se face folosind tabelul sistem fișier localizat în fișierul **/etc/fstab**. Mai jos este un exemplu din fișierul **/etc/fstab**.

```
root@RHELv4u2:~# cat /etc/fstab
/dev/VolGroup00/LogVol00 /                ext3          defaults      1 1
LABEL=/boot              /boot          ext3          defaults      1 2
none                      /dev/pts       devpts        gid=5,mode=620 0 0
none                      /dev/shm       tmpfs         defaults      0 0
none                      /proc          proc          defaults      0 0
none                      /sys           sysfs         defaults      0 0
/dev/VolGroup00/LogVol01 swap                swap          defaults      0 0
```

Adăugînd următoarea linie, putem automatiza montarea unui sistem fișier.

```
/dev/sdb1                /home/project55 ext2          defaults      0 0
```

mount /mountpoint

Adăugînd o intrare în **/etc/fstab** are avantajul că puteți simplifica comanda **mount**. Comanda din captura de ecran de mai jos forțează **mount** să caute informații despre partiții în **/etc/fstab**.

```
# mount /home/project55
```

7.4. securizarea montărilor

Sistemele fișier pot fi securizate cu câteva **opțiuni mount**. Aici sînt câteva exemple.

ro

Opțiunea **ro** va monta un sistem fișier cu permisiuni doar de citire, prevenind orice scriere.

```
root@rhel53 ~# mount -t ext2 -o ro /dev/hdb1 /home/project42
root@rhel53 ~# touch /home/project42/testwrite
touch: cannot touch `/home/project42/testwrite': Read-only file system
```

noexec

Opțiunea **noexec** va preveni executarea binarelor și a scripturilor pe sisteme fișier montate.

```
root@rhel53 ~# mount -t ext2 -o noexec /dev/hdb1 /home/project42
root@rhel53 ~# cp /bin/cat /home/project42
root@rhel53 ~# /home/project42/cat /etc/hosts
-bash: /home/project42/cat: Permission denied
root@rhel53 ~# echo echo hello > /home/project42/helloscript
root@rhel53 ~# chmod +x /home/project42/helloscript
root@rhel53 ~# /home/project42/helloscript
-bash: /home/project42/helloscript: Permission denied
```

nosuid

Opțiunea **nosuid** va ignora bitul **setuid** setat executabilelor pe sistem fișier montat.

Notăți că încă se poate seta bitul **setuid** pe fișiere.

```
root@rhel53 ~# mount -o nosuid /dev/hdb1 /home/project42
root@rhel53 ~# cp /bin/sleep /home/project42/
root@rhel53 ~# chmod 4555 /home/project42/sleep
root@rhel53 ~# ls -l /home/project42/sleep
-r-sr-xr-x 1 root root 19564 Jun 24 17:57 /home/project42/sleep
```

Dar utilizatorii nu pot exploata **setuid**.

```
root@rhel53 ~# su - paul
[paul@rhel53 ~]$ /home/project42/sleep 500 &
[1] 2876
[paul@rhel53 ~]$ ps -f 2876
UID    PID    PPID    C  STIME   TTY   STAT   TIME    CMD
paul   2876   2853    0  17:58   pts/0  S      0:00    /home/project42/sleep 500
[paul@rhel53 ~]$
```

noacl

Pentru a preveni dezordinea permisiunilor cu **acl**, folosiți opțiunea **noacl**.

```
root@rhel53 ~# mount -o noacl /dev/hdb1 /home/project42
```

Mai multe **opțiuni mount** pot fi găsite în pagina de manual **mount**.

7.5. practică: montare sisteme fișier

1. Montați mica partiție de 200 MB pe /home/project22.
2. Montați partiția primară mare de 400 MB pe /mnt, apoi copiați niște fișiere pe ea (totul în /etc). Apoi demontați, și montați sistemul fișier ca avînd drepturi doar de citire pe /srv/nfs/salesnumbers. Unde sînt fișierele pe care le-ați copiat?
3. Verificați ce ați făcut cu **fdisk**, **df** și **mount**. Vedeți de asemeni **/etc/mtab** și **/proc/mounts**.
4. Faceți ambele montări permanente, verificați dacă funcționează.
5. Ce se întîmplă cînd montați un sistem de fișier într-un director care conține niște fișiere?
6. Ce se întîmplă cînd montați două sisteme fișier pe același punct de montare?
7. (opțional) Descrieți diferența între aceste comenzi de căutare fișier: **find**, **locate**, **updatedb**, **whereis**, **apropos** și **which**.
8. (opțional) Faceți o verificare a sistemului fișier pe partiția montată la /srv/nfs/salesnumbers.

7.6. soluție: montare sisteme fișier

1. Montați mica partiție de 200 MB pe /home/project22.

```
mkdir /home/project22
```

```
mount /dev/sdc1 /home/project22
```

2. Montați partiția primară mare de 400 MB pe /mnt, apoi copiați niște fișiere pe ea (totul în /etc). Apoi demontați, și montați sistemul fișier ca avînd drepturi doar de citire pe /srv/nfs/salesnumbers. Unde sînt fișierele pe care le-ați copiat?

```
mount /dev/sdb1 /mnt
```

```
cp -r /etc /mnt
```

```
ls -l /mnt
```

```
umount /mnt
```

```
ls -l /mnt
```

```
mkdir -p /srv/nfs/salesnumbers
```

```
mount /dev/sdb1 /srv/nfs/salesnumbers
```

Vedeți acum fișierele în /srv/nfs/salesnumbers...

Dar în mod fizic ele sînt pe ext3 pe partiția /dev/sdb1.

3. Verificați ce ați făcut cu **fdisk**, **df** și **mount**. Vedeți de asemeni **/etc/mtab** și **/proc/mounts**.

```
fdisk -l
```

```
df -h
```

```
mount
```

Toate cele trei comenzi de deasupra ar trebui să arate partițiile montate.

```
grep project22 /etc/mtab
```

```
grep project22 /proc/mounts
```

4. Faceți ambele montări permanente, verificați dacă funcționează.

adăugați următoarele linii în /etc/fstab

```
/dev/sdc1 /home/project22 auto defaults 0 0
```

```
/dev/sdb1 /srv/nfs/salesnumbers auto defaults 0 0
```

5. Ce se întîmplă cînd montați un sistem de fișier într-un director care conține niște fișiere?

Fișierele sînt ascunse pînă la umount.

6. Ce se întîmplă cînd montați două sisteme fișier pe același punct de montare?

Doar ultimul sistem fișier montat este vizibil.

7. (opțional) Descrieți diferența între aceste comenzi de căutare fișier: find, locate, updatedb, whereis, apropos și which.

manualul este prietenul

8. (opțional) Faceți o verificare a sistemului fișier pe partiția montată la /srv/nfs/salesnumbers.

Cel mai bine este să demontați tot înainte de a scrie comanda

```
# fsck /dev/sdb1
```

Capitolul 8. introducere la uuid

Conținut

8.1. despre obiecte unice.	.67
8.2. uuid în /etc/fstab.	.67
8.3. uuid în menu.lst.	.68
8.4. practică: uuid și sisteme fișier.	.69
8.5. soluție: uuid și sisteme fișier.	70

8.1. despre obiecte unice

Un **uuid** sau **universally unique identifier (identificator unic universal)** este folosit pentru a identifica obiecte unice. Acest standard 128 bit permite oricui să creeze un **uuid** unic.

/sbin/vol_id

Mai jos folosim utilitarul **vol_id** pentru a afișa **uuid** a unui sistem fișier **ext3**.

```
root@laika:~# vol_id --uuid /dev/sda1
825d4b79-ec40-4390-8a71-9261df8d4c82
```

/lib/udev/vol_id

Red Hat Enterprise Linux 5 pune **vol_id** în **/lib/udev/vol_id**, care nu este în **\$PATH**. Sintaxa este un pic diferită la Debian/Ubuntu.

```
root@rhel53 ~# /lib/udev/vol_id -u /dev/hda1
48a6a316-9ca9-4214-b5c6-e7b33a77e860
```

/sbin/tune2fs

Putem de asemeni folosi **tune2fs** pentru a afla **uuid** a unui sistem fișier.

```
[root@RHEL5 ~]# tune2fs -l /dev/sda1 | grep UUID
Filesystem UUID:          11cfc8bc-07c0-4c3f-9f64-78422ef1dd5c
[root@RHEL5 ~]# /lib/udev/vol_id -u /dev/sda1
11cfc8bc-07c0-4c3f-9f64-78422ef1dd5c
```

8.2. uuid în /etc/fstab

Puteti folosi **uuid** să fiți siguri că un volum este identificat unic universal în **/etc/fstab**. Numele dispozitiv poate să se schimbe depinzînd de dispozitivele disk care sînt prezente la timpul butării, dar un **uuid** nu se schimbă niciodată.

Mai întîi folosim **tune2fs** pentru a găsi **uuid**.

```
[root@RHEL5 ~]# tune2fs -l /dev/sdc1 | grep UUID
Filesystem UUID:          7626d73a-2bb6-4937-90ca-e451025d64e8
```

Apoi verificăm dacă este adăugat cum trebuie în **/etc/fstab**, **uuid-ul** înlocuiește variabila nume dispozitiv **/dev/sdc1**.

```
[root@RHEL5 ~]# grep UUID /etc/fstab
UUID=7626d73a-2bb6-4937-90ca-e451025d64e8 /home/pro42 ext3 defaults 0 0
```

Acum putem monta volumul utilizînd punctul montare definit în **/etc/fstab**.

```
[root@RHEL5 ~]# mount /home/pro42
[root@RHEL5 ~]# df -h | grep 42
/dev/sdc1          397M    11M 366M 3% /home/pro42
```

Testul real acum, este să extragem **/dev/sdb** din sistem, să rebutăm mașina și să vedem ce se întîmplă. După reboot, disk-ul cunoscut mai înainte ca **/dev/sdc** este acum **/dev/sdb**.

```
[root@RHEL5 ~]# tune2fs -l /dev/sdb1 | grep UUID
Filesystem UUID:          7626d73a-2bb6-4937-90ca-e451025d64e8
```

Și datorită **uuid-ului** din **/etc/fstab**, punctul montare este montat pe același disk ca mai înainte.

```
[root@RHEL5 ~]# df -h | grep sdb  
/dev/sdb1          397M   11M   366M   3%  /home/pro42
```

8.3. uuid în menu.lst

Încarnări recente a distribuției Ubuntu vor folosi un **uuid** pentru a identifica sistemul fișier root. Acest exemplu arată cum un **root=/dev/sda1** este înlocuit cu un **uuid**.

```
title Ubuntu 9.10, kernel 2.6.31-19-generic  
uuid f001ba5d-9077-422a-9634-8d23d57e782a  
kernel /boot/vmlinuz-2.6.31-19-generic \  
root=UUID=f001ba5d-9077-422a-9634-8d23d57e782a ro quiet splash  
initrd /boot/initrd.img-2.6.31-19-generic
```

Captura de ecran de mai sus conține doar patru linii. Linia începând cu **root=** este continuarea liniei **kernel**.

8.4. practică: uuid și sisteme fișier

1. Găsiți **uuid** a uneia dintre partițiile **ext3** cu **tune2fs** și **vol_id**.
2. Folosiți acest **uuid** în **/etc/fstab** și testați dacă funcționează cu un simplu **mount**.
3. (opțional) Testați-l de asemeni prin extragerea disk-ului (astfel ca numele dispozitiv să fie schimbat). Puteți edita setările în VMware/VirtualBox pentru a extrage un hard disk.
4. Afișați **root=** directivă în **/boot/grub/menu.lst**. (Vom vedea mai târziu în curs cum să menținem acest fișier.)
5. (opțional) Înlocuiți **/dev/xxx** în **/boot/grub/menu.lst** cu un **uuid** (folosiți o extra stanza pentru asta). Testați dacă funcționează.

8.5. soluție: uuid și sisteme fișier

1. Găsiți **uuid** a uneia dintre partițiile **ext3** cu **tune2fs** și **vol_id**.

```
root@rhel55:~# /lib/udev/vol_id -u /dev/hda1
60926898-2c78-49b4-a71d-c1d6310c87cc
```

```
root@ubu1004:~# tune2fs -l /dev/sda2 | grep UUID
Filesystem UUID:          3007b743-1dce-2d62-9a59-cf25f85191b7
```

2. Folosiți acest **uuid** în **/etc/fstab** și testați dacă funcționează cu un simplu **mount**.

```
tail -1 /etc/fstab
UUID=60926898-2c78-49b4-a71d-c1d6310c87cc /home/pro42 ext3 defaults 0 0
```

3. (opțional) Testați-l de asemeni prin extragerea disk-ului (astfel ca numele dispozitiv să fie schimbat). Puteți edita setările în VMware/VirtualBox pentru a extrage un hard disk.

4. Afișați **root=** directivă în **/boot/grub/menu.lst**. (Vom vedea mai târziu în curs cum să menținem acest fișier.)

```
paul@deb503:~$ grep ^[^\#] /boot/grub/menu.lst | grep root=
kernel /boot/vmlinuz-2.6.26-2-686 root=/dev/hda1 ro selinux=1 quiet
kernel /boot/vmlinuz-2.6.26-2-686 root=/dev/hda1 ro selinux=1 single
```

5. (opțional) Înlocuiți **/dev/xxx** în **/boot/grub/menu.lst** cu un **uuid** (folosiți o extra stanză pentru asta). Testați dacă funcționează.

Capitolul 9. introducere la raid

Conținut

9.1. hardware sau software.	72
9.2. nivele raid.	72
9.3. construirea unui aranjament software raid 5.	73
9.4. practică: raid.	77
9.5. soluție: raid.	78

9.1. hardware sau software

RAID sau **Redundant Array of Independent** (original Inexpensive) **Disks** [**Disk-uri Redundante Aranjate Independent** (original Ieftine)] poate fi setat folosind hardware sau software. Hardware RAID este mai scump, dar oferă o performanță mai bună. Software RAID este mai ieftin și mai ușor de folosit, dar utilizează mai mult CPU și memorie.

În urmă cu zece ani când nimeni nu se certa despre cea mai bună alegere hardware fiind RAID, asta s-a schimbat de când tehnologii ca mdadm, lvm și chiar zfs s-au focalizat mai mult pe ușurința în administrare. Încărcarea în cpu pentru software RAID a fost mare, dar procesoarele au devenit mult mai rapide.

9.2. nivele raid

raid 0

raid 0 folosește două sau mai multe disk-uri, și este deseori numit **stabilizare** (sau setat stabil sau volum demontat). Datele sînt împărțite în **bucăți**, aceste bucăți sînt în cele din urmă împrăștiate de-a lungul fiecărui disk în aranjament. Avantajul principal al **raid 0** este că puteți crea **disk-uri mai mari**. **raid 0** este singurul raid fără redundanță.

jbod

jbod folosește două sau mai multe disk-uri, și este deseori numit **concatenat** (deschis, setat, sau volum rază de acțiune). Datele sînt scrise pe primul disk, pînă cînd devine plin. Apoi date sînt scrise pe al doilea disk... Principalul avantaj a lui **jbod** (Just a Bunch of Disks) este că puteți crea **disk-uri mai mari**. JBOD nu oferă redundanță.

raid 1

raid 1 folosește exact două disk-uri, și este deseori numit **mirror (oglindă)** (sau setare oglindă sau volum oglindă). Toate datele scrise în aranjament sînt scrise pe fiecare disk. Principalul avantaj al raid 1 este **redundanța**. Principalul dezavantaj este că se pierde cel puțin jumătate din spațiul de disk disponibil (cu alte cuvinte, dublați cel puțin costul).

raid 2, 3 și 4?

raid 2 folosește stabilizare la nivel bit, **raid 3** nivel bit, și **raid 4** este la fel cu **raid 5**, dar cu un disk cu paritate dedicat. Asta e în realitate mai încet decît **raid 5**, pentru că fiecare scriere ar trebui să aibă paritatea la acest disk. Este rar dacă ați vedea vreodată aceste nivele **raid** în producție.

raid 5

raid 5 folosește **trei** sau mai multe disk-uri, fiecare divizat în bucăți. De fiecare dată cînd bucățile sînt scrise în aranjament, unul dintre disk-uri va primi o bucată (de software) **pară**. Spre deosebire de **raid 4**, bucata pară va exista în toate disk-urile. Principalul avantaj a acestui lucru este că **raid 5** va permite recuperarea datelor în mod integral în cazul în care **unul** dintre disk-uri se va defecta.

raid 6

raid 6 este foarte similar cu **raid 5**, dar folosește două bucăți de paritate. **raid 6** protejează împotriva defectării a două disk-uri. Solaris Oracle **zfs** numește asta **raidz2** (și are de asemenea **raidz3** cu paritate triplă).

raid 0+1

raid 0+1 este o oglindă (1) de stabilizări (0). Asta înseamnă că mai întâi creați două **setări stabile raid 0**, și apoi le setați ca volum oglindă. De exemplu, când aveți șase disk-uri de 100 GB, atunci stabilizările setate au fiecare 300 GB. Combinat într-o oglindă, asta face un total de 300 GB. **raid 0+1** va supraviețui unui singur disk stricat. Va supraviețui doar la eșuarea celui de-al doilea disk dacă acest disk este în aceeași stabilizare setată ca primul disk eșuat.

raid 1+0

raid 1+0 este o stabilizare (0) de oglinzi (1). De exemplu, când aveți șase disk-uri de 100 GB, atunci creați mai întâi trei *oglinzi* de 100 GB fiecare. Apoi le stabiliți împreună într-un disk de 300 GB. În acest exemplu, atîta vreme cît nu toate disk-urile din aceeași oglindă eșuează, el poate să supraviețuiască eșuării a pînă la trei disk-uri.

raid 50

raid 5+0 este o stabilizare (0) a aranjamentelor **raid 5**. Să presupunem că aveți nouă disk-uri de 100 GB, atunci puteți crea trei aranjamente **raid 5** de 200 GB fiecare. Puteți să le combinați atunci într-o singură mare stabilizare setat stabilă.

multe altele

Există multe alte combinații **raid**, ca **raid 30**, **51**, **60**, **100**, **150**, ...

9.3. construirea un aranjament software raid 5

avem trei disk-uri?

Mai întâi, trebuie să atașați cîteva disk-uri computerului. În acest scenariu, sînt adăugate trei noi disk-uri de opt gigabiți fiecare. Verificați cu **fdisk -l** dacă ele sînt conectate.

```
[root@rhel6c ~]# fdisk -l 2> /dev/null | grep MB
Disk /dev/sdb: 8589 MB, 8589934592 bytes
Disk /dev/sdc: 8589 MB, 8589934592 bytes
Disk /dev/sdd: 8589 MB, 8589934592 bytes
```

partiția tip fd

Următorul pas este să creăm o partiție de tip **fd** pe fiecare disk. Tipul **fd** există pentru a seta partiția ca **Linux RAID autodetect**. Vedeți această captură de ecran (trunchiată):

```
[root@rhel6c ~]# fdisk /dev/sdd
...
Command (m for help): n
Command action
    e           extended
    p           primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-1044, default 1):
Using default value 1
Last cylinder, +cylinders or +size{K,M,G} (1-1044, default 1044):
Using default value 1044
Command (m for help): t
Selected partition 1
Hex code (type L to list codes): fd
Changed system type of partition 1 to fd (Linux raid autodetect)

Command (m for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.
Syncing disks.
```

verificați toate cele trei partiții

Acum toate cele trei disk-uri sînt gata pentru **raid 5**, așa că va trebui să-i transmitem sistemului ce să facă cu aceste disk-uri.

```
[root@rhel6c ~]# fdisk -l 2> /dev/null | grep raid
/dev/sdb1      1      1044      8385898+  fd Linux raid autodetect
/dev/sdc1      1      1044      8385898+  fd Linux raid autodetect
/dev/sdd1      1      1044      8385898+  fd Linux raid autodetect
```

creați raid 5

În pasul următor trebuia să fie *creați tabelul raid în /etc/raidtab*. Astăzi, putem să tastăm comanda **mdadm** cu parametrii corecți.

Comanda de mai jos este împărțită în două linii pentru a încăpea în acest document, dar ar trebui să îl tastați pe o singură linie, fără backslash (\).

```
[root@rhel6c ~]# mdadm --create /dev/md0 --chunk=64 --level=5 --raid-\
devices=3 /dev/sdb1 /dev/sdc1 /dev/sdd1
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

Mai jos este o captură de ecran parțială – cum **fdisk -l** vede **raid 5**.

```
[root@rhel6c ~]# fdisk -l /dev/md0
```

```
Disk /dev/md0: 17.2 GB, 17172135936 bytes
2 heads, 4 sectors/track, 4192416 cylinders
Units = cylinders of 8 * 512 = 4096 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 65536 bytes / 131072 bytes
Disk identifier: 0x00000000
```

Disk /dev/md0 doesn't contain a valid partition table

Am putea folosi acest aranjament **raid 5** în următorul subiect: **lvm**.

/proc/mdstat

Stadiul în care se află dispozitivele raid poate fi văzut în **/proc/mdstat**. Acest exemplu arată un **raid 5** în proces de reconstruire.

```
[root@rhel6c ~]# cat /proc/mdstat
Personalities : [raid6] [raid5] [raid4]
md0 : active raid5 sdd1[3] sdc1[1] sdb1[0]
      16769664 blocks super 1.2 level 5, 64k chunk, algorithm 2 [3/2] [UU_]
      [=====>.....] recovery = 62.8% (5266176/8384832) finish=0\
      .3min speed=139200K/sec
```

Acest exemplu arată un software activ **raid 5**.

```
[root@rhel6c ~]# cat /proc/mdstat
Personalities : [raid6] [raid5] [raid4]
md0 : active raid5 sdd1[3] sdc1[1] sdb1[0]
      16769664 blocks super 1.2 level 5, 64k chunk, algorithm 2 [3/3] [UUU]
```

mdadm --detail

Folosiți **mdadm --detail** pentru a obține informații despre un dispozitiv raid.

```
[root@rhel6c ~]# mdadm --detail /dev/md0
/dev/md0:
```

Version : 1.2

Creation Time : Sun Jul 17 13:48:41 2011

Raid Level : raid5

Array Size : 16769664 (15.99 GiB 17.17 GB)

Used Dev Size : 8384832 (8.00 GiB 8.59 GB)

Raid Devices : 3

Total Devices : 3

Persistence : Superblock is persistent

Update Time : Sun Jul 17 13:49:43 2011

State : clean

Active Devices : 3

Working Devices : 3

Failed Devices : 0

Spare Devices : 0

Layout : left-symmetric

Chunk Size : 64K

Name : rhel6c:0 (local to host rhel6c)

UUID : c10fd9c3:08f9a25f:be913027:999c8e1f

Events : 18

Number	Major	Minor	RaidDevice	State	
0	8	17	0	active sync	/dev/sdb1
1	8	33	1	active sync	/dev/sdc1
3	8	49	2	active sync	/dev/sdd1

ștergerea unui software raid

Software-ul raid este vizibil în **/proc/mdstat** când este activ. Pentru a șterge raid în mod complet pentru a putea utiliza disk-urile pentru alte scopuri, îl opriți (deactivați) cu **mdadm**.

```
[root@rhel6c ~]# mdadm --stop /dev/md0
```

```
mdadm: stopped /dev/md0
```

Disk-urile pot fi acum repartiționate.

9.4. practică: raid

1. Adăugați trei disk-uri virtuale de 1 GB fiecare unei mașini virtuale.
2. Creați un software **raid 5** pe cele trei disk-uri. (Nu este necesar să puneți un sistem fișier pe el).
3. Verificați cu **fdisk** și în **/proc** dacă **raid 5** există.
4. (opțional) Opriți și ștergeți **raid 5**.
5. (opțional) Creați un **raid 1** pentru oglinda a două disk-uri.

9.5. soluție: raid

1. Adăugați trei disk-uri virtuale de 1 GB fiecare unei mașini virtuale.
2. Creați un software **raid 5** pe cele trei disk-uri. (Nu este necesar să puneți un sistem fișier pe el).
3. Verificați cu **fdisk** și în **/proc** dacă **raid 5** există.
4. (opțional) Opriți și ștergeți **raid 5**.
5. (opțional) Creați un **raid 1** pentru oglinda a două disk-uri.

```
[root@rhel6c ~]# mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb1
/dev/sdc1
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
[root@rhel6c ~]# cat /proc/mdstat
Personalities : [raid6] [raid5] [raid4] [raid1]
md0 : active raid1 sdc1[1] sdb1[0]
      8384862 blocks super 1.2 [2/2] [UU]
      [====>.....] resync = 20.8% (1745152/8384862) \
finish=0.5min speed=218144K/sec
```

Capitolul 10. management volume logice

Conținut

10.1.	introducere în lvm.	.80
10.2.	terminologie lvm.	.80
10.3.	exemplu: folosire lvm.	.81
10.4.	exemplu: extindere a unui volum logic.	.82
10.5.	exemplu: redimensionare a unui volum fizic.	.84
10.6.	exemplu: oglinda unui volum logic.	.86
10.7.	exemplu: fotografia unui volum logic.	.86
10.8.	verificare a volumelor fizice existente.	.87
10.9.	verificare a volumelor grup existente.	.88
10.10.	verificare a volumelor logice existente.	.89
10.11.	management al volumelor fizice.	.90
10.12.	management al volumelor grup.	.92
10.13.	management volume logice.	.93
10.14.	practică: lvm.	.96

10.1. introducere în lvm

probleme cu partițiile standard

Există unele probleme când lucrăm cu hard-disk-uri și partiții standard. Să luăm în considerare un sistem cu un dispozitiv hard-disk mic și unul mare, partiționat astfel. Primul disk (/dev/sda) este partiționat în două, al doilea disk (/dev/sdb) are trei partiții.

Tabelul 10.1. exemplu de partiționare disk

/dev/sda		/dev/sdb			
/dev/sda1	/dev/sda2	/dev/sdb1	/dev/sdb2	/dev/sdb3	unused
/boot	/	/var	/home	/project42	
ext2	ext3	ext2	reiserfs	ext3	

În exemplul de mai sus, luați în considerare că vreți să măriți spațiul disponibil pentru **/project42**. Ce puteți face? Soluția vă va forța mereu să demontați sistemul de fișiere, să faceți un backup a datelor, să scoateți și să recreați partițiile, și apoi să restaurați datele și să remontați sistemul fișier.

soluție cu lvm

Folosind **lvm** va crea un strat virtual între sistemele de fișier montate și dispozitivele hardware. Acest strat virtual va permite unui administrator să mărească sistemul de fișier montat în folosire. Când **lvm** este utilizat cum trebuie, atunci nu mai există nevoia de a demonta sistemul de fișier pentru a-l mări.

Tabelul 10.2. exemplu LVM

/dev/sda		/dev/sdb		
		Volum Grup		
/boot	/	/var	/home	/project42
ext2	ext3	ext2	reiserfs	ext3

despre lvm

Cele mai multe implementări **lvm** suportă **stocarea fizică grupată, ajustarea volumelor logice și migrarea datelor**.

Stocarea fizică grupată este un nume nostim pentru a grupa dispozitive multiple (hard-disk-uri) într-un dispozitiv logic de stocare în masă. Pentru a mări acest grup fizic, hard-disk-urile sau chiar partițiile singulare pot fi adăugate mai târziu. Mărimea **volumelor lvm** pe acest **grup fizic** este independentă de mărimea individuală a componentelor. Mărimea totală a grupului este limită.

Una dintre cele mai frumoase proprietăți a **lvm** este ajustarea volumului logic. Puteți extinde mărimea unui **volum lvm**, uneori chiar fără nici un downtime. În plus, puteți să migrați datele separat de un dispozitiv hard-disk eşuat.

10.2. terminologie lvm

volum fizic (pv)

Un **volum fizic** este un disk, o partiție sau un dispozitiv RAID (hardware sau software). Toate aceste dispozitive pot deveni un membru al unui **volum grup**.

volum grup (vg)

Un **volum grup** este un strat abstract între **dispozitive fizice** și **volum logice**.

volum logic (lv)

Un **volum logic** este creat într-un **volum grup**. Volumele logice care conțin un sistem fișier pot fi montate. Folosirea volumelor logice este similară cu utilizarea partițiilor (ambele sînt dispozitive bloc standard) și se realizează cu aceleași comenzi standard (mkfs, mount, fsck, df, ...).

10.3. exemplu: folosire lvm

Acest exemplu arată cum puteți folosi un dispozitiv (în acest caz /dev/sdc, dar ar putea fi /dev/sdb sau orice alt disk sau partiție) cu lvm, cum să creați un volum grup (vg) și cum să creați și să utilizați un volum logic (vg/lvol0).

Primul lucru care este de făcut, este să creați volume fizice care pot să se alăture volumului grup cu **pvcreate**. Această comandă face un disk sau partiție disponibile pentru utilizare în volume grupuri. Captura de ecran arată cum să afișăm dispozitivul disk SCSI în LVM.

```
root@RHEL4:~# pvcreate /dev/sdc
Physical volume "/dev/sdc" successfully created
```

Notă pentru utilizatorii casnici: lvm va lucra bine folosind disk-ul complet, dar alt sistem de operare pe același computer nu va recunoaște lvm și va marca disk-ul ca fiind gol! Puteți să evitați acest lucru creînd o partiție care cuprinde întregul disk, apoi să executați pvcreate pe partiție în locul disk-ului.

Apoi **vgcreate** crează un volum grup folosind un singur dispozitiv. De notat că mai multe dispozitive ar putea fi adăugate volumului grup.

```
root@RHEL4:~# vgcreate vg /dev/sdc
Volume group "vg" successfully created
```

În ultimul pas **lvcreate** crează un volum logic.

```
root@RHEL4:~# lvcreate --size 500m vg
Logical volume "lv010" created
```

Volumul logic /dev/vg/lv010 poate fi acum formatat cu ext2, și montat pentru utilizare normală.

```
root@RHELv4u2:~# mke2fs -m0 -j /dev/vg/lv010
mke2fs 1.35 (28-Feb-2004)
Filesystem label=
OS type: Linux
Block size=1024 (log=0)
Fragment size=1024 (log=0)
128016 inodes, 512000 blocks
0 blocks (0.00%) reserved for the super user
First data block=1
Maximum filesystem blocks=67633152
63 block groups
8192 blocks per group, 8192 fragments per group
2032 inodes per group
Superblock backups stored on blocks:
8193, 24577, 40961, 57345, 73729, 204801, 221185, 401409
```

```
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
This filesystem will be automatically checked every 37 mounts or 180 days,
whichever comes first. Use tune2fs -c or -i to override.
```

```
root@RHELv4u2:~# mkdir /home/project10
root@RHELv4u2:~# mount /dev/vg/lv010 /home/project10/
root@RHELv4u2:~# df -h | grep proj
/dev/mapper/vg-lv010 485M    11M    474M    3%    /home/project10
```

Un volum logic este foarte asemănător cu o partiție, el poate fi formatat cu un sistem fișier, și poate fi montat astfel ca utilizatorii să-l poată accesa.

10.4. exemplu: extindere a unui volum logic

Un volum logic poate fi extins fără a demonta sistemul de fișier. Dacă volumul poate fi extins sau nu depinde de sistemul de fișier pe care îl folosește. Volumele care sînt montate ca vfat sau etx2 nu pot fi extinse, astfel că în exemplul de aici folosim sistemul de fișier ext3.

Comanda fdisk ne arată disk-urile scsi nou atașate care vor fi folosite de către volumul nostru lvm. Acest volum va fi apoi extins. Mai întîi, priviți la aceste disk-uri.

```
[root@RHEL5 ~]# fdisk -l | grep sd[bc]
Disk /dev/sdb doesn't contain a valid partition table
Disk /dev/sdc doesn't contain a valid partition table
Disk /dev/sdb: 1181 MB, 1181115904 bytes
Disk /dev/sdc: 429 MB, 429496320 bytes
```

Știți deja cum să partiționați un disk, mai jos primul disk este partiționat (într-o singură mare partiție primară), al doilea disk este lăsat neatins.

```
[root@RHEL5 ~]# fdisk -l | grep sd[bc]
Disk /dev/sdc doesn't contain a valid partition table
Disk /dev/sdb: 1181 MB, 1181115904 bytes
/dev/sdb1          1          143    1148616    83    Linux
Disk /dev/sdc: 429 MB, 429496320 bytes
```

Știți de asemeni cum să pregătiți disk-urile pentru lvm cu **pvcreate**, și cum să creați un volum grup cu **vgcreate**. Acest exemplu adaugă și disk-ul partiționat și disk-ul neatins la volumul grup numit **vg2**.

```
[root@RHEL5 ~]# pvcreate /dev/sdb1
Physical volume "/dev/sdb1" successfully created
[root@RHEL5 ~]# pvcreate /dev/sdc
Physical volume "/dev/sdc" successfully created
[root@RHEL5 ~]# vgcreate vg2 /dev/sdb1 /dev/sdc
Volume group "vg2" successfully created
```

Puteți folosi **pvdisk** pentru a verifica dacă și disk-ul și partiția aparțin volumului grup.

```
[root@RHEL5 ~]# pvdisk | grep -B1 vg2
PV Name          /dev/sdb1
VG Name          vg2
--
PV Name          /dev/sdc
VG Name          vg2
```

Și sînteți familiar cu comanda **lvcreate** pentru a crea un volum logic mic și cu comanda **mke2fs** pentru a pune ext2 pe el.

```
[root@RHEL5 ~]# lvcreate --size 200m vg2
Logical volume "lvol0" created
[root@RHEL5 ~]# mke2fs -m20 -j /dev/vg2/lvol0
...
```

Precum vedeți, avem un volum logic montat care după ce ne spune comanda **df** are aproape 200 megabiți în mărime.

```
[root@RHEL5 ~]# mkdir /home/resizetest
[root@RHEL5 ~]# mount /dev/vg2/lvol0 /home/resizetest/
[root@RHEL5 ~]# df -h | grep resizetest
194M    5.6M   149M    4%    /home/resizetest
```

Extinderea volumului este ușoară cu **lvextend**.

```
[root@RHEL5 ~]# lvextend -L +100 /dev/vg2/lvol0
Extending logical volume lvol0 to 300.00 MB
Logical volume lvol0 successfully resized
```

Dar cum puteți vedea, există o mică problemă: se pare că **df** nu este capabil să afișeze volumul extins în întreaga lui mărime. Asta pentru că sistemul fișier este setat doar pentru extinderea volumului înainte ca extinderea să fie adăugată.

```
[root@RHEL5 ~]# df -h | grep resizetest
194M    5.6M   149M    4%    /home/resizetest
```

Cu **lvdisplay** oricum putem vedea dacă volumul este într-adevăr extins.

```
[root@RHEL5 ~]# lvsdisplay /dev/vg2/lvol0 | grep Size
LV Size          300.00 MB
```

Pentru a termina extinderea, aveți nevoie ca **resize2fs** să cuprindă sistemul fișier în întreaga mărime a volumului logic.

```
[root@RHEL5 ~]# resize2fs /dev/vg2/lvol0
resize2fs 1.39 (29-May-2006)
Filesystem at /dev/vg2/lvol0 is mounted on /home/resizetest; on-line re\
sizing required
Performing an on-line resize of /dev/vg2/lvol0 to 307200 (1k) blocks.
The filesystem on /dev/vg2/lvol0 is now 307200 blocks long.
```

Felicitări, ați extins cu succes un volum logic.

```
[root@RHEL5 ~]# df -h | grep resizetest
291M 6.1M 225M 3% /home/resizetest
[root@RHEL5 ~]#
```

10.5. exemplu: redimensionare a unui volum fizic

Aceasta este o umilă demonstrație care arată cum să redimensionați un volum fizic cu lvm (după ce îl redimensionați cu fdisk). Demonstrația începe cu o partiție de 100 MB numită /dev/sde1. Folosim fdisk pentru a-l crea, și pentru a verifica mărimea.

```
[root@RHEL5 ~]# fdisk -l 2>/dev/null | grep sde1
/dev/sde1          1          100      102384      83      Linux
[root@RHEL5 ~]#
```

Acum putem folosi pvcreate pentru a crea volumul fizic, urmat de pvs pentru a verifica crearea.

```
[root@RHEL5 ~]# pvcreate /dev/sde1
Physical volume "/dev/sde1" successfully created
[root@RHEL5 ~]# pvs | grep sde1
/dev/sde1          lvm2 --          99.98M 99.98M
[root@RHEL5 ~]#
```

Următorul pas este să folosim fdisk pentru a lărgi partiția (în realitate o ștergem și apoi recreăm /dev/sde1 cu mai mulți cilindri).

```
[root@RHEL5 ~]# fdisk /dev/sde
```

```
Command (m for help): p
```

```
Disk /dev/sde: 858 MB, 858993152 bytes
64 heads, 32 sectors/track, 819 cylinders
Units = cylinders of 2048 * 512 = 1048576 bytes
```

Device	Boot	Start	End	Blocks	Id	System
/dev/sde1		1	100	102384	83	Linux

```
Command (m for help): d
Selected partition 1
```

```
Command (m for help): n
Command action
e      extended
p      primary partition (1-4)
```

```
p
Partition number (1-4):
Value out of range.
Partition number (1-4): 1
First cylinder (1-819, default 1):
Using default value 1
Last cylinder or +size or +sizeM or +sizeK (1-819, default 819): 200
```

```
Command (m for help): w
The partition table has been altered!
```

```
Calling ioctl() to re-read partition table.
Syncing disks.
[root@RHEL5 ~]#
```

Cînd folosim acum fdisk și pvs pentru a verifica mărimea partiției și Volumul Fizic, atunci există o diferență în mărime. LVM încă folosește vechea mărime.

```
[root@RHEL5 ~]# fdisk -l 2>/dev/null | grep sde1
/dev/sde1          1          200      204784          83      Linux
[root@RHEL5 ~]# pvs | grep sde1
/dev/sde1          lvm --   99.98M 99.98M
[root@RHEL5 ~]#
```

Executînd pvresize pe Volumul Fizic va face ca lvm să își dea seama de schimbarea mărimii partiției. Mărimea corectă poate fi afișată cu pvs.

```
[root@RHEL5 ~]# pvresize /dev/sde1
Physical volume "/dev/sde1" changed
 1 physical volume(s) resized / 0 physical volume(s) not resized
[root@RHEL5 ~]# pvs | grep sde1
/dev/sde1          lvm2 --   199.98M 199.98M
[root@RHEL5 ~]#
```

10.6. exemplu: oglinda unui volum logic

Începem prin crearea a trei volume fizice pentru lvm. Apoi verificăm crearea și mărimea cu pvs. Trei disk-uri fizice pentru că lvm folosește două disk-uri pentru oglindă și un al treilea disk pentru fișierul-jurnal al oglinzii!

```
[root@RHEL5 ~]# pvcreate /dev/sdb /dev/sdc /dev/sdd
Physical volume "/dev/sdb" successfully created
Physical volume "/dev/sdc" successfully created
Physical volume "/dev/sdd" successfully created
[root@RHEL5 ~]# pvs
```

PV	VG	Fmt	Attr	PSize	PFree
/dev/sdb		lvm2	--	409.60M	409.60M
/dev/sdc		lvm2	--	409.60M	409.60M
/dev/sdd		lvm2	--	409.60M	409.60M

Apoi creăm volumul grup și verificăm din nou cu pvs. Observați cum cele trei volume fizice aparțin acum la vg33, și cum mărimea este rotunjită (în pași ai mărimii extensie, aici 4 MB).

```
[root@RHEL5 ~]# vgcreate vg33 /dev/sdb /dev/sdc /dev/sdd
Volume group "vg33" successfully created
[root@RHEL5 ~]# pvs
```

PV	VG	Fmt	Attr	PSize	PFree
/dev/sda2	VolGroup00	lvm2	a-	15.88G	0
/dev/sdb	vg33	lvm2	a-	408.00M	408.00M
/dev/sdc	vg33	lvm2	a-	408.00M	408.00M
/dev/sdd	vg33	lvm2	a-	408.00M	408.00M

```
[root@RHEL5 ~]#
```

Ultimul pas este să creăm volumul logic cu **lvcreate**. Observați întrerupătorul **-m 1** pentru a crea o oglindă. Observați de asemeni schimbarea în spațiu liber în toate cele trei volume fizice!

```
[root@RHEL5 ~]# lvcreate --size 300m -n lvmir -m 1 vg33
Logical volume "lvmir" created
[root@RHEL5 ~]# pvs
```

PV	VG	Fmt	Attr	PSize	PFree
/dev/sda2	VolGroup00	lvm2	a-	15.88G	0
/dev/sdb	vg33	lvm2	a-	408.00M	108.00M
/dev/sdc	vg33	lvm2	a-	408.00M	108.00M
/dev/sdd	vg33	lvm2	a-	408.00M	404.00M

Puteți vedea statusul copierii oglinzii cu lvs. Acum arată o copiere 100%.

```
[root@RHEL5 ~]# lvs vg33/lvmir
```

LV	VG	Attr	LSize	Origin	Snap%	Move	Log	Copy%
lvmir	vg33	mwi-ao	300.00M				lvmir_mlog	100.00

10.7. exemplu: fotografia unui volum logic

O fotografie (**snapshot**) este o copie virtuală a tuturor datelor la un anumit punct în timp pe un volum. O fotografie a unui volum logic va reține o copie a tuturor fișierelor schimbate în volumul logic fotografiat.

Exemplul de mai jos crează o fotografie a volumului logic bigLV.

```
[root@RHEL5 ~]# lvcreate -L100M -s -n snapLV vg42/bigLV
Logical volume "snapLV" created
[root@RHEL5 ~]#
```

Puteți vedea cu `lvs` că fotografia `snapLV` este într-adevăr o fotografie a `bigLV`. Cu câteva momente după ce facem fotografia, există puține schimbări în `bigLV` (0.02 %).

```
[root@RHEL5 ~]# lvs
LV          VG      Attr      Lsize   Origin    Snap%   Move   Log    Copy%
bigLV       vg42     owi-a-    200.00M
snapLV      vg42     swi-a-    100.00M  bigLV      0.02
```

Dar după ce folosim `bigLV` pentru puțin timp, mai multe schimbări au loc. Asta înseamnă că volumul fotografiat trebuie să păstreze mai mult din datele originale (10.22 la sută).

```
[root@RHEL5 ~]# lvs | grep vg42
bigLV       vg42     owi-ao    200.00M
snapLV      vg42     swi-a-    100.00M  bigLV      10.22
[root@RHEL5 ~]#
```

Acum puteți folosi utilitare de backup normale (`dump`, `tar`, `cpio`, ...) pentru a face un backup a fotografiei volumului logic. Acest backup va conține toate datele așa cum existau pe `bigLV` în timpul în care fotografia a fost făcută. Când backup-ul s-a terminat, puteți șterge fotografia.

```
[root@RHEL5 ~]# lvremove vg42/snapLV
Do you really want to remove active logical volume "snapLV"? [y/n]: y
Logical volume "snapLV" successfully removed
[root@RHEL5 ~]#
```

10.8. verificare a volumelor fizice existente

lvmdiskscan

Pentru a obține o listă a dispozitivelor bloc care pot fi utilizate cu LVM, folosiți **lvmdiskscan**. Exemplul de mai jos folosește `grep` pentru a limita rezultatul la dispozitive SCSI.

```
[root@RHEL5 ~]# lvmdiskscan | grep sd
/dev/sda1    [      101.94 MB]
/dev/sda2    [      15.90 GB] LVM physical volume
/dev/sdb     [      409.60 MB]
/dev/sdc     [      409.60 MB]
/dev/sdd     [      409.60 MB] LVM physical volume
/dev/sde1    [       95.98 MB]
/dev/sde5    [     191.98 MB]
/dev/sdf     [     819.20 MB] LVM physical volume
/dev/sdg1    [     818.98 MB]
[root@RHEL5 ~]#
```

pvs

Cea mai ușoară cale pentru a verifica dacă dispozitivele sînt recunoscute de LVM este cu comanda **pvs**. Captura de ecran de mai jos arată că doar `/dev/sda2` este recunoscută în mod curent pentru a fi utilizată cu LVM. Ea arată că `/dev/sda2` face parte din `Volgroup00` și are aproape 16 GB ca mărime. Ea de asemeni arată că

/dev/sdc și /dev/sdd sînt parte a vg33. Dispozitivul /dev/sdb este recunoscut de lvm, dar nu este legat de nici un volum grup.

```
[root@RHEL5 ~]# pvs
PV          VG          Fmt    Attr    PSize    PFree
/dev/sda2   VolGroup00  lvm2    a-      15.88G    0
/dev/sdb                lvm2    --      409.60M   409.60M
/dev/sdc     vg33             lvm2    a-      408.00M   408.00M
/dev/sdd     vg33             lvm2    a-      408.00M   408.00M
[root@RHEL5 ~]#
```

pvscan

Comanda **pvscan** va scana toate disk-urile pentru a găsi volume fizice existente. Informația este similară cu pvs, în plus obțineți o linie cu mărimea totală.

```
[root@RHEL5 ~]# pvscan
PV /dev/sdc     VG vg33             lvm2    [408.00 MB / 408.00 MB free]
PV /dev/sdd     VG vg33             lvm2    [408.00 MB / 408.00 MB free]
PV /dev/sda2    VG VolGroup00      lvm2    [15.88 GB / 0 free]
PV /dev/sdb                lvm2    [409.60 MB]
Total: 4 [17.07 GB] / in use: 3 [16.67 GB] / in no VG: 1 [409.60 MB]
[root@RHEL5 ~]#
```

pvdisk

Folosiți **pvdisk** pentru a obține mai multă informație despre volumele fizice. Puteți de asemenea folosi **pvdisk** fără un argument pentru a afișa informație despre toate volumele (lvm) fizice.

```
[root@RHEL5 ~]# pvdisk /dev/sda2
--- Physical volume ---
PV Name          /dev/sda2
VG Name          VolGroup00
PV Size          15.90 GB / not usable 20.79 MB
Allocatable      yes (but full)
PE Size (Kbyte)  32768
Total PE         508
Free PE          0
Allocated PE     508
PV UUID          TobYfp-Ggg0-Rf8r-xtLd-5XgN-RSPc-8vkTHD
```

```
[root@RHEL5 ~]#
```

10.09. verificare a volumelor grup existente

vgs

Folosirea **vgs** este similară cu **pvs** pentru a afișa o scurtă imagine de ansamblu a tuturor volumelor grup. Există doar un singur volum grup în captura de ecran de mai jos, este numit VolGroup00 și are ca mărime aproape 16 GB.

```
[root@RHEL5 ~]# vgs
VG          #PV  #LV  #SN  Attr    VSize    VFree
VolGroup00   1    2    0  wz-- n-  15.88G    0
[root@RHEL5 ~]#
```


vgscan

Comanda **vgscan** va scana toate disk-urile pentru a găsi volume grup existente. Va aduce la zi de asemeni fișierul **/etc/lvm/.cache**. Acest fișier conține o listă a tuturor dispozitivelor curente lvm.

```
[root@RHEL5 ~]# vgscan
Reading all physical volumes. This may take a while...
Found volume group "VolGroup00" using metadata type lvm2
[root@RHEL5 ~]#
```

LVM va rula vgscan automatic la boot-up, așa că dacă adăugați dispozitive hot swap, atunci va trebui să executați vgscan pentru a aduce la zi /etc/lvm/.cache cu noile dispozitive.

vgdisplay

Comanda **vgdisplay** vă va da mai multe informații detaliate despre un volum grup (sau despre toate volumurile grup dacă omiteți argumente).

```
[root@RHEL5 ~]# vgdisplay VolGroup00
--- Volume group ---
VG Name                VolGroup00
System ID
Format                 lvm2
Metadata Areas         1
Metadata Sequence No   3
VG Access               read/write
VG Status               resizable
MAX LV                 0
Cur LV                 2
Open LV                 2
Max Pv                 0
Cur Pv                 1
Act Pv                 1
VG Size                 15.88 GB
PE Size                 32.00 MB
Total PE                508
Alloc PE / Size         508 / 15.88 GB
Free PE / Size          0 / 0
VG UUID                 qsXvJb-71qV-9l7U-ishX-FobM-qptE-VXmKIg
```

```
[root@RHEL5 ~]#
```

10.10. verificare a volumelor logice existente

lvs

Folosiți **lvs** pentru o scurtă privire asupra tuturor volumelor logice existente. Mai jos puteți vedea două volume logice numite LogVol00 și LogVol01.

```
[root@RHEL5 ~]# lvs
  LV          VG           Attr      Lsize   Origin Snap%  Move Log Copy%
  LogVol00    VolGroup00 -wi-ao    14.88G
  LogVol01    VolGroup00 -wi-ao     1.00G
[root@RHEL5 ~]#
```

lvscan

Comanda **lvscan** va scana toate disk-urile pentru a afla existența volumelor logice.

```
[root@RHEL5 ~]# lvscan
ACTIVE          '/dev/VolGroup00/LogVol00' [14.88 GB] inherit
ACTIVE          '/dev/VolGroup00/LogVol01' [1.00 GB] inherit
[root@RHEL5 ~]#
```

lvdisplay

Mai multe informații detaliate despre volumele logice sînt disponibile prin intermediul comenzii **lvdisplay(1)**.

```
[root@RHEL5 ~]# lvdisplay VolGroup00/LogVol01
--- Logical volume ---
LV Name                /dev/VolGroup00/LogVol01
VG Name                VolGroup00
LV UUID                RnTGK6-xWsi-t530-ksJx-7cax-co5c-A1KlDp
LV Write Access        read/write
LV Status              available
# open                 1
LV Size                1.00 GB
Current LE             32
Segments              1
Allocation             inherit
Read ahead sectors     0
Block device           253:1
```

```
[root@RHEL5 ~]#
```

10.11 management al volumelor fizice

pvcreate

Folosiți comanda **pvcreate** pentru a adăuga dispozitive la lvm. Acest exemplu arată cum să adăugăm un disk (sau un dispozitiv hardware RAID) la lvm.

```
[root@RHEL5 ~]# pvcreate /dev/sdb
Physical volume "/dev/sdb" successfully created
[root@RHEL5 ~]#
```

Acest exemplu arată cum să adăugăm o partiție la lvm.

```
[root@RHEL5 ~]# pvcreate /dev/sdc1
Physical volume "/dev/sdc1" successfully created
[root@RHEL5 ~]#
```

Puteți de asemeni să adăugați disk-uri multiple sau partiții ca țintă la pvcreate. Acest exemplu adaugă trei diskuri la lvm.

```
[root@RHEL5 ~]# pvcreate /dev/sde /dev/sdf /dev/sdg
Physical volume "/dev/sde" successfully created
Physical volume "/dev/sdf" successfully created
Physical volume "/dev/sdg" successfully created
[root@RHEL5 ~]#
```

pvremove

Folosiți comanda **pvremove** pentru a șterge volume fizice din lvm. Aceste dispozitive ar putea să nu fie în utilizare.

```
[root@RHEL5~]# pvremove /dev/sde /dev/sdf /dev/sdg
Labels on physical volume "/dev/sde" successfully wiped
Labels on physical volume "/dev/sdf" successfully wiped
Labels on physical volume "/dev/sdg" successfully wiped
[root@RHEL5~]#
```

pvresize

Cînd ați utilizat fdisk pentru a redimensiona o partiție pe un disk, atunci trebuie să folosiți **pvresize** pentru a face ca lvm să recunoască noua mărime a volumului fizic care reprezintă această partiție.

```
[root@RHEL5 ~]# pvresize /dev/sde1
Physical volume "/dev/sde1" changed
1 physical volume(s) resized / 0 physical volume(s) not resized
```

pvchange

Cu **pvchange** puteți preveni alocarea unui volum fizic într-un nou volum grup sau volum logic. Asta poate fi folositor dacă planificați să ștergeți un volum fizic.

```
[root@RHEL5 ~]# pvchange -xn /dev/sdd
Physical volume "/dev/sdd" changed
1 physical volume changed / 0 physical volumes not changed
[root@RHEL5 ~]#
```

Pentru a face reversul deciziei anterioare, acest exemplu arată cum să refaceți volumul fizic pentru a permite alocare.

```
[root@RHEL5 ~]# pvchange -xy /dev/sdd
Physical volume "/dev/sdd" changed
1 physical volume changed / 0 physical volumes not changed
[root@RHEL5 ~]#
```

pvmove

Cu **pvmove** puteți muta volumele logice din interiorul unui volum grup în alt volum fizic. Asta trebuie făcut înainte de a șterge un volum fizic.

```
[root@RHEL5 ~]# pvs | grep vg1
/dev/sdf      vg1    lvm2    a-    816.00M      0
/dev/sdg      vg1    lvm2    a-    816.00M    816.00M
[root@RHEL5 ~]# pvmove /dev/sdf
/dev/sdf:    Moved: 70.1%
/dev/sdf:    Moved: 100.0%
[root@RHEL5 ~]# pvs | grep vg1
/dev/sdf      vg1    lvm2    a-    816.00M    816.00M
/dev/sdg      vg1    lvm2    a-    816.00M      0M
```

10.12. management al volumelor grup

vgcreate

Folosiți comanda **vgcreate** pentru a crea un volum grup. Puteți imediat să numiți toate volumele fizice care cuprind volumul grup.

```
[root@RHEL5 ~]# vgcreate vg42 /dev/sde /dev/sdf
Volume group "vg42" successfully created
[root@RHEL5 ~]#
```

vgextend

Folosiți comanda **vgextend** pentru a extinde un volum grup existent cu un volum fizic.

```
[root@RHEL5 ~]# vgextend vg42 /dev/sdg
Volume group "vg42" successfully extended
[root@RHEL5 ~]#
```

vgremove

Folosiți comanda **vgremove** pentru a șterge volumele grupuri din lvm. Grupurile volum ar putea să nu fie în utilizare.

```
[root@RHEL5 ~]# vgremove vg42
Volume group "vg42" successfully removed
[root@RHEL5 ~]#
```

vgreduce

Folosiți comanda **vgreduce** pentru a șterge un volum fizic din volumul grup.

Următorul exemplu adaugă volumul fizic /dev/sdg la volumul grup vg1 folosind vgextend. Și apoi îl șterge din nou folosind vgreduce.

```
[root@RHEL5 ~]# pvs | grep sdg
/dev/sdg          lvm2 --      819.20M 819.20M
[root@RHEL5 ~]# vgextend vg1 /dev/sdg
Volume group "vg1" successfully extended
[root@RHEL5 ~]# pvs | grep sdg
/dev/sdg  vg1      lvm2 a-      816.00M 816.00M
[root@RHEL5 ~]# vgreduce vg1 /dev/sdg
Removed "/dev/sdg" from volume group "vg1"
[root@RHEL5 ~]# pvs | grep sdg
/dev/sdg          lvm2 --      819.20M 819.20M
```

vgchange

Folosiți comanda **vgchange** pentru a schimba parametrii unui volum grup.

Acest exemplu arată cum să prevenim volumele fizice să fie adăugate sau șterse din volumul grup vg1.

```
[root@RHEL5 ~]# vgchange -xn vg1
Volume group "vg1" successfully changed
[root@RHEL5 ~]# vgextend vg1 /dev/sdg
Volume group vg1 is not resizable.
```

Puteți de asemeni folosi **vgchange** pentru a schimba cele mai multe alte proprietăți a unui volum grup. Acest exemplu schimbă numărul maxim a volumelor logice și numărul maxim al volumelor fizice pe care **vg1** le poate deservi.

```
[root@RHEL5 ~]# vgdisplay vg1 | grep -i max
MAX LV          0
MAX PV          0
[root@RHEL5 ~]# vgchange -l16 vg1
Volume group "vg1" successfully changed
[root@RHEL5 ~]# vgchange -p8 vg1
Volume group "vg1" successfully changed
[root@RHEL5 ~]# vgdisplay vg1 | grep -i max
MAX LV          16
Max PV          8
```

vgmerge

Fuzionarea a două volume grupuri într-unul singur se face cu **vgmerge**. Exemplul următor fuzionează **vg2** în **vg1**, păstrând toate proprietățile lui **vg1**.

```
[root@RHEL5 ~]# vgmerge vg1 vg2
Volume group "vg2" successfully merged into "vg1"
[root@RHEL5 ~]#
```

10.13. management volume logice

lvcreate

Folosiți comanda **lvcreate** pentru a crea volume logice într-un volum grup. Acest exemplu crează un volum logic de 8 GB într-un volum grup **vg42**.

```
[root@RHEL5 ~]# lvcreate -L5G vg42
Logical volume "lv01" created
[root@RHEL5 ~]#
```

Precum puteți vedea, **lvm** în mod automat numește volumul logic **lv01**. Următorul exemplu crează un volum logic de 200 MB numit **MyLV** în volumul grup **vg42**.

```
[root@RHEL5 ~]# lvcreate -L200M -nMyLV vg42
Logical volume "MyLV" created
[root@RHEL5 ~]#
```

Următorul exemplu face același lucru, dar cu o sintaxă diferită.

```
[root@RHEL5 ~]# lvcreate --size 200M -n MyLV vg42
Logical volume "MyLV" created
[root@RHEL5 ~]#
```

Acest exemplu crează un volum logic care ocupă 10 la sută din volumul grup.

```
[root@RHEL5 ~]# lvcreate -l 10%VG -n MyLV2 vg42
Logical volume "MyLV2" created
[root@RHEL5 ~]#
```

Acest exemplu crează un volum logic care ocupă 30 la sută din spațiul liber rămas în volumul grup.

```
[root@RHEL5 ~]# lvcreate -l 30%FREE -n MyLV3 vg42
Logical volume "MyLV3" created
[root@RHEL5 ~]#
```

lvremove

Folosiți comanda **lvremove** pentru a șterge volume logice dintr-un volum grup. Ștergerea unui volum logic cere numele volumului grup.

```
[root@RHEL5 ~]# lvremove vg42/MyLV
Do you really want to remove active logical volume "MyLV"? [y/n]: y
Logical volume "MyLV" successfully removed
[root@RHEL5 ~]#
```

Ștergerea volumelor logice multiple va solicita confirmare pentru fiecare volum individual.

```
[root@RHEL5 ~]# lvremove vg42/MyLV vg42/MyLV2 vg42/MyLV3
Do you really want to remove active logical volume "MyLV"? [y/n]: y
Logical volume "MyLV" successfully removed
Do you really want to remove active logical volume "MyLV2"? [y/n]: y
Logical volume "MyLV2" successfully removed
Do you really want to remove active logical volume "MyLV3"? [y/n]: y
Logical volume "MyLV3" successfully removed
[root@RHEL5 ~]#
```

lvextend

Extinderea volumului este ușoară cu **lvextend**. Acest exemplu extinde un volum logic de 200 MB cu 100 MB.

```
[root@RHEL5 ~]# lvsdisplay /dev/vg2/lvol0 | grep Size
LV Size                200.00 MB
[root@RHEL5 ~]# lvextend -L +100 /dev/vg2/lvol0
Extending logical volume lvol0 to 300.00 MB
Logical volume lvol0 successfully resized
[root@RHEL5 ~]# lvsdisplay /dev/vg2/lvol0 | grep Size
LV Size                300.00 MB
```

Următorul exemplu crează un volum logic de 100 MB, și apoi îl extinde la 500 MB.

```
[root@RHEL5 ~]# lvcreate --size 100M -n extLV vg42
Logical volume "extLV" created
[root@RHEL5 ~]# lvextend -L 500M vg42/extLV
Extending logical volume extLV to 500.00 MB
Logical volume extLV successfully resized
[root@RHEL5 ~]#
```

Acest exemplu dublează mărimea unui volum logic.

```
[root@RHEL5 ~]# lvextend -l+100%LV vg42/extLV
Extending logical volume extLV to 1000.00 MB
Logical volume extLV successfully resized
[root@RHEL5 ~]#
```

lvrename

Redenumirea unui volum logic se face cu **lvrename**. Acest exemplu redenumeste extLV în bigLV în volumul grup vg42.

```
[root@RHEL5 ~]# lvrename vg42/extLV vg42/bigLV
Renamed "extLV" to "bigLV" in volume group "vg42"
[root@RHEL5 ~]#
```

10.14. practică: lvm

1. Creați un volum grup care conține un disk complet și o partiție pe un alt disk.
2. Creați două volume logice (unul mic și unul mai mare) în acest volum grup. Formatați-le cu ext3, montați-le și apoi copiați niște fișiere în ele.
3. Verificați folosirea cu fdisk, mount, pvs, vgs, lvs, pvdisplay, vgdisplay, lvdisplay și df. fdisk vă dă informații despre lvm?
4. Măriți volumul logic mic cu 50%, și verificați!
5. Priviți la alte comenzi care încep cu vg*, pv* sau lv*.
6. Creați o oglindă și un volum logic demontat.
7. Preschimbați un volum logic linear într-o oglindă.
8. Preschimbați o oglindă volum logic în unul linear.
9. Creați o fotografie a unui volum logic, faceți un backup al fotografiei. Apoi ștergeți unele fișiere de pe volumul logic, apoi restaurați backup-ul.
10. Mutați volumul grup pe un alt disk (păstrați volumele logice montate).
11. Dacă timpul permite, împărțiți un volum grup cu vgsplit, apoi fuzionați-l din nou cu vgmerge.

Capitolul 11. dispozitive iSCSI

Conținut

11.1. terminologie iSCSI.	98
11.2. instalare țintă iSCSI.	98
11.3. setare țintă iSCSI.	98
11.4. setare inițiator client iSCSI.	100
11.5. folosire dispozitive iSCSI.	101
11.6. practică: dispozitive iSCSI.	103
11.7. soluție: dispozitive iSCSI.	104

Acest capitol vă învață cum să setați un **server țintă iSCSI** și un **inițiator client iSCSI**.

11.1. terminologie iSCSI

iSCSI este un protocol care deschide SCSI deasupra IP. Asta înseamnă că puteți avea dispozitive locale SCSI (ca /dev/sdb) fără să aveți hardware de stocare în computerul local.

Computerul care găzduiește hardware fizic de stocare a datelor se numește **țintă iSCSI**. Fiecare dispozitiv individual adresabil iSCSI pe ținta server va obține un **număr LUN**.

Clientul computer iSCSI care se conectează la ținta server este numit un **Inițiator**. Un inițiator va trimite comenzi SCSI deasupra IP în loc de a face acest lucru direct pe hardware. Inițiatorul se va conecta la țintă.

11.2. instalare țintă iSCSI

Instalarea de software pentru serverul țintă cere **iscsitarget** pe Ubuntu și Debian, și un extra modul kernel **iscsitarget-dkms** doar pe Debian.

```
root@debby6:~# aptitude install iscsitarget
The following NEW packages will be installed:
  iscsitarget
0 packages upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 69.4 kB of archives. After unpacking 262 kB will be used.
Get:1 http://ftp.belnet.be/debian/ squeeze/main iscsitarget i386 1.4.20.2-1
[69.4 kB]
Fetched 69.4 kB in 0s (415 kB/s)
Selecting previously deselected package iscsitarget.
(Reading database ... 36441 files and directories currently installed.)
Unpacking iscsitarget (from .../iscsitarget_1.4.20.2-1_i386.deb) ...
Processing triggers for man-db ...
Setting up iscsitarget (1.4.20.2-1) ...
iscsitarget not enabled in "/etc/default/iscsitarget", not starting...(warning).
```

Pe Debian 6 vom avea nevoie de **aptitude install iscsitarget-dkms** pentru modulele de kernel, pe Debian 5 asta e **aptitude install iscsitarget-modules-`uname -a`**. Ubuntu include modulele kernel în pachetul principal.

Ținta server iSCSI este dezactivată prin default, așa că o activăm.

```
root@debby6:~# cat /etc/default/iscsitarget
ISCSITARGET_ENABLE=false
root@debby6:~# vi /etc/default/iscsitarget
root@debby6:~# cat /etc/default/iscsitarget
ISCSITARGET_ENABLE=true
```

11.3. setare țintă iSCSI

Puteți folosi volumele LVM (/dev/md0/lvol0), partiții fizice (/dev/sda), dispozitive raid (/dev/md0) sau doar fișiere depline pentru stocare. În această demonstrație, utilizăm fișiere create cu **dd**.

Acesastă captură de ecran arată cum să creăm trei fișiere mici (100 MB, 200 MB și 300 MB).

```

root@debby6:~# mkdir /iscsi
root@debby6:~# dd if=/dev/zero of=/iscsi/lun1.img bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB) copied, 0.315825 s, 332 MB/s
root@debby6:~# dd if=/dev/zero of=/iscsi/lun2.img bs=1M count=200
200+0 records in
200+0 records out
209715200 bytes (210 MB) copied, 1.08342 s, 194 MB/s
root@debby6:~# dd if=/dev/zero of=/iscsi/lun3.img bs=1M count=300
300+0 records in
300+0 records out
314572800 bytes (315 MB) copied, 1.36209 s, 231 MB/s

```

Trebuie să declarăm aceste trei fișiere ca ținte iSCSI în **/etc/iet/ietd.conf** (fișierul vechi era **/etc/ietd.conf**).

```

root@debby6:/etc/iet# cp ietd.conf ietd.conf.original
root@debby6:/etc/iet# > ietd.conf
root@debby6:/etc/iet# vi ietd.conf
root@debby6:/etc/iet# cat ietd.conf
Target iqn.2010-02.be.linux-training:storage.lun1
    IncomingUser isuser hunter2
    OutgoingUser
    Lun 0 Path=/iscsi/lun1.img,Type=fileio
    Alias LUN1

```

```

Target iqn.2010-02.be.linux-training:storage.lun2
    IncomingUser isuser hunter2
    OutgoingUser
    Lun 0 Path=/iscsi/lun2.img,Type=fileio
    Alias LUN2

```

```

Target iqn.2010-02.be.linux-training:storage.lun3
    IncomingUser isuser hunter2
    OutgoingUser
    Lun 0 Path=/iscsi/lun3.img,Type=fileio
    Alias LUN3

```

Trebuie de asemenea să adăugăm dispozitivele noastre în fișierul **/etc/initiators.allow**.

```

root@debby6:/etc/iet# cp initiators.allow initiators.allow.original
root@debby6:/etc/iet# > initiators.allow
root@debby6:/etc/iet# vi initiators.allow
root@debby6:/etc/iet# cat initiators.allow
iqn.2010-02.be.linux-training:storage.lun1
iqn.2010-02.be.linux-training:storage.lun2
iqn.2010-02.be.linux-training:storage.lun3

```

Acum e timpul să pornim serverul:

```

root@debby6:/etc/iet# /etc/init.d/iscsitarget start
Starting iSCSI enterprise target service:.
.
root@debby6:/etc/iet#

```

Verificăm activarea dispozitivelor de stocare în **/proc/net/iet**:

```
root@debby6:/etc/iet# cat /proc/net/iet/volume
tid:3 name:iqn.2010-02.be.linux-training:storage.lun3
lun:0 state:0 iotype:fileio iomode:wt blocks:614400 blocksize:\
512 path:/iscsi/lun3.img
tid:2 name:iqn.2010-02.be.linux-training:storage.lun2
lun:0 state:0 iotype:fileio iomode:wt blocks:409600 blocksize:\
512 path:/iscsi/lun2.img
tid:1 name:iqn.2010-02.be.linux-training:storage.lun1
lun:0 state:0 iotype:fileio iomode:wt blocks:204800 blocksize:\
512 path:/iscsi/lun1.img
root@debby6:/etc/iet# cat /proc/net/iet/session
tid:3 name:iqn.2010-02.be.linux-training:storage.lun3
tid:2 name:iqn.2010-02.be.linux-training:storage.lun2
tid:1 name:iqn.2010-02.be.linux-training:storage.lun1
```

11.4. setare inițiator client iSCSI

Mai întâi instalăm software-ul de client iSCSI (pe alt computer decât ținta).

```
root@ubull04:~# aptitude install open-iscsi
Reading package lists... Done
Building dependency tree
Reading state information... Done
Reading extended state information
Initializing package states... Done
The following NEW packages will be installed:
  open-iscsi open-iscsi-utils{a}
```

Apoi setăm clientul iSCSI să se pornească automat.

```
root@ubull04:/etc/iscsi# cp iscsid.conf iscsid.conf.original
root@ubull04:/etc/iscsi# vi iscsid.conf
root@ubull04:/etc/iscsi# grep ^node.startup iscsid.conf
node.startup = automatic
```

Sau putem să-l pornim manual.

```
root@ubull04:/etc/iscsi/nodes# /etc/init.d/open-iscsi start
* Starting iSCSI initiator service iscsid
* Setting up iSCSI targets
root@ubull04:/etc/iscsi/nodes#
```

Acum ne putem conecta la serverul țintă și să utilizăm **iscsiadm** pentru a descoperi dispozitivele pe care le oferă:

```
root@ubull04:/etc/iscsi# iscsiadm -m discovery -t st -p 192.168.1.31
192.168.1.31:3260,1 iqn.2010-02.be.linux-training:storage.lun2
192.168.1.31:3260,1 iqn.2010-02.be.linux-training:storage.lun1
192.168.1.31:3260,1 iqn.2010-02.be.linux-training:storage.lun3
```

Putem să utilizăm același **iscsiadm** pentru a edita fișierele în **/etc/iscsi/nodes/**.

```

root@ubull104:/etc/iscsi# iscsiadm -m node --targetname "iqn.2010-02.be.linu\
x-training:storage.lun1" --portal "192.168.1.31:3260" --op=update --name no\
de.session.auth.authmethod --value=CHAP
root@ubull104:/etc/iscsi# iscsiadm -m node --targetname "iqn.2010-02.be.linu\
x-training:storage.lun1" --portal "192.168.1.31:3260" --op=update --name no\
de.session.auth.username --value=isuser
root@ubull104:/etc/iscsi# iscsiadm -m node --targetname "iqn.2010-02.be.linu\
x-training:storage.lun1" --portal "192.168.1.31:3260" --op=update --name no\
de.session.auth.password --value=hunter2

```

Repetati comanda de deasupra pentru celelalte doua dispozitive.

Reporniți serviciul inițiator pentru a intra în sistemul țintă.

```

root@ubull104:/etc/iscsi/nodes# /etc/init.d/open-iscsi restart
* Disconnecting iSCSI targets [OK]
* Stopping iSCSI initiator service [OK]
* Starting iSCSI initiator service iscsid [OK]
* Setting up iSCSI targets

```

Folosiți **fdisk -l** pentru a vă bucura de trei noi dispozitive iSCSI.

```

root@ubull104:/etc/iscsi/nodes# fdisk -l 2> /dev/null | grep Disk
Disk /dev/sda: 17.2 GB, 17179869184 bytes

```

```

Disk identifier: 0x0001983f
Disk /dev/sdb: 209 MB, 209715200 bytes
Disk identifier: 0x00000000
Disk /dev/sdd: 314 MB, 314572800 bytes
Disk identifier: 0x00000000
Disk /dev/sdc: 104 MB, 104857600 bytes
Disk identifier: 0x00000000

```

Ținta (serverul) arată acum sesiunile active.

```

root@debby6:/etc/iet# cat /proc/net/iet/session
tid:3 name:iqn.2010-02.be.linux-training:storage.lun3
sid:5348024611832320 initiator:iqn.1993-08.org.debian:01:8983ed2d770
cid:0 ip:192.168.1.35 state:active hd:none dd:none
tid:2 name:iqn.2010-02.be.linux-training:storage.lun2
sid:4785074624856576 initiator:iqn.1993-08.org.debian:01:8983ed2d770
cid:0 ip:192.168.1.35 state:active hd:none dd:none
tid:1 name:iqn.2010-02.be.linux-training:storage.lun1
sid:5066549618344448 initiator:iqn.1993-08.org.debian:01:8983ed2d770
cid:0 ip:192.168.1.35 state:active hd:none dd:none
root@debby6:/etc/iet#

```

11.5. folosire dispozitive iSCSI

Nu există nici o diferență între a folosi dispozitive SCSI sau iSCSI odată ce ele sînt conectate: partiție, make filesystem, mount.

```
root@ubul104:/etc/iscsi/nodes# history | tail -13
94 fdisk /dev/sdc
95 fdisk /dev/sdd
96 fdisk /dev/sdb
97 mke2fs /dev/sdb1
98 mke2fs -j /dev/sdc1
99 mkfs.ext4 /dev/sdd1
100 mkdir /mnt/is1
101 mkdir /mnt/is2
102 mkdir /mnt/is3
103 mount /dev/sdb1 /mnt/is1
104 mount /dev/sdc1 /mnt/is2
105 mount /dev/sdd1 /mnt/is3
106 history | tail -13
root@ubul104:/etc/iscsi/nodes# mount | grep is
/dev/sdb1 on /mnt/is1 type ext2 (rw)
/dev/sdc1 on /mnt/is2 type ext3 (rw)
/dev/sdd1 on /mnt/is3 type ext4 (rw)
```

11.6. practică: dispozitive iSCSI

1. Setați o țintă (folosind un dispozitiv LVM și unul SCSI) și un inițiator care să le conecteze pe amândouă.

11.7. soluție: dispozitive iSCSI

1. Setati o țintă (folosind un dispozitiv LVM și unul SCSI) și un inițiator care să le conecteze pe amândouă.

Decideți (cu un coleg) ca un computer să fie Ținta și alt computer să fie Inițiatorul.

Pe computerul Țintă:

Mai întâi instalați iscsitarget folosind utilitarele standard pentru a instala software din distribuție. Apoi folosiți cunoștințele din capitolul anterior pentru a seta un volum logic (/dev/vg/lvol0) și utilizați capitolul RAID pentru a seta /dev/md0. Apoi faceți următorul pas:

```
vi /etc/default/iscsitarget (setați enable la true)
```

Adăugați dispozitivele în /etc/iet/ietf.conf.

```
root@debby6:/etc/iet# cat ietd.conf
Target iqn.2010-02.be.linux-training:storage.lun1
    IncomingUser isuser hunter2
    OutgoingUser
    Lun 0 Path=/dev/vg/lvol0,Type=fileio
    Alias LUN1
Target iqn.2010-02.be.linux-training:storage.lun2
    IncomingUser isuser hunter2
    OutgoingUser
    Lun 0 Path=/dev/md0,Type=fileio
    Alias LUN2
```

Adăugați ambele dispozitive în /etc/iet/initiators.allow.

```
root@debby6:/etc/iet# cat initiators.allow
iqn.2010-02.be.linux-training:storage.lun1
iqn.2010-02.be.linux-training:storage.lun2
```

Acum porniți daemonul iscsitarget și mutați-vă la Inițiator.

Pe computerul Inițiator:

Instalați open-iscsi și porniți daemonul.

Apoi folosiți **iscsiadm -m discovery -t st -p 'target-ip'** pentru a vedea dispozitivele iscsi pe Țintă.

Editați fișierele /etc/iscsi/nodes așa cum este arătat în carte. Apoi reporniți daemonul iSCSI și executați **fdisk -l** pentru a vedea dispozitivele iSCSI.

Partea III. management butare

Capitolul 12. încărcător de boot

Conținut

12.1. terminologie boot.	107
12.2. grub.	109
12.3. lilo.	113
12.4. practică: încărcător de boot.	114
12.5. soluție: încărcător de boot.	115

12.1. terminologie boot

Ordinea exactă a lucrurilor care se întâmplă când pornim un sistem computer depinde de arhitectura hardware (**Intel x86** este diferit de **Sparc Sun** etc.), de încărcătorul de boot (**grub** e diferit de **lilo**) și de sistemul de operare (**Linux**, **Solaris**, **BSD** etc). Cea mai mare parte a acestui capitol se focalizează pe butarea **Linux** pe **Intel x86** cu **grub**.

post

Un computer începe să buteze în momentul în care deschideți sursa de alimentare (nu glumim). Acest prim proces este numit **post** sau **power on self test**. Dacă totul decurge bine acest lucru duce la **bios**. Dacă nu decurge atât de bine, atunci s-ar putea să nu auziți nimic, sau să auziți bipuri, sau să vedeți un mesaj de eroare, sau poate să vedeți fum ieșind din computer (hardware care arde miroase urât!).

bios

Toate computerele **Intel x86** vor avea un **sistem basic input/output** sau **bios** pentru a detecta, identifica și inițializa hardware. Apoi **bios** merge să caute un dispozitiv de pe care să buteze (**boot device**). Asta poate fi un floppy disk, un hard-disk, cdrom, placă de rețea sau dispozitiv usb.

În timp ce **bios** rulează puteți vedea un mesaj pe ecran transmițându-vă care tastă să apăsați (deseori **Del** sau **F2**) pentru a intra în **setarea bios**.

PhoenixBIOS Setup Utility							
Main	Advanced	Security	Power	Boot	Exit		
System Time: [18:06:55] System Date: [06/01/2009] Legacy Diskette A: [1.44/1.25 MB 3½"] Legacy Diskette B: [Disabled] ▶ Primary Master [None] ▶ Primary Slave [None] ▶ Secondary Master [UMware Virtual ID] ▶ Secondary Slave [None] ▶ Keyboard Features System Memory: 640 KB Extended Memory: 261120 KB Boot-time Diagnostic Screen: [Disabled]						Item Specific Help	
						<Tab>, <Shift-Tab>, or <Enter> selects field.	
F1	Help	↑↓	Select Item	-/+	Change Values	F9	Setup Defaults
Esc	Exit	↔	Select Menu	Enter	Select ▶ Sub-Menu	F10	Save and Exit

openboot

Sistemele Sun **Sparc** încep cu **openboot** pentru a testa hardware și a buta sistemul de operare. **Bill Callkins** explică **openboot** în cărțile lui Administrarea Sistemului Solaris. Detaliile **openboot** nu au focalizare în acest curs.

boot password

bios vă permite să setați o parolă. Nu uitați această parolă, sau va trebui să

deschideți hardware-ul și să o resetați. Puteți uneori să setați o parolă pentru a buta sistemul, și o altă parolă pentru a proteja **biosul** să nu fie modificat.

dispozitiv boot

bios-ul va căuta un **dispozitiv boot** în ordinea configurată în setarea bios. De obicei un sistem de operare pe un server de producție butează de pe un hard-disk.

PhoenixBIOS Setup Utility					
Main	Advanced	Security	Power	Boot	Exit
+Removable Devices +Hard Drive CD-ROM Drive Network boot from AMD Am79C970A				Item Specific Help Keys used to view or configure devices: <Enter> expands or collapses devices with a + or - <Ctrl+Enter> expands all <Shift + 1> enables or disables a device. <+> and <-> moves the device up or down. <n> May move removable device between Hard Disk or Removable Disk <d> Remove a device that is not installed.	
F1	Help	↑↓	Select Item	-/+	Change Values
Esc	Exit	↔	Select Menu	Enter	Select ► Sub-Menu
F9	Setup Defaults				F10
				Save and Exit	

master boot record

master boot record sau **mbr** este primul sector al unui hard-disk. Partiționarea unui disk în partiții **primare**, și partiția activă sînt definite în **mbr**.

mbr are o lungime de 512 biți și poate fi copiat cu **dd**.

```
dd if=/dev/sda of=bootsect.mbr count=1 bs=512
```

bootloader

mbr este executat de către **bios** și conține fie (un mic) încărcător de boot (**boot loader**) fie un cod pentru a încărca un **bootloader**.

Dacă ne uităm la **mbr** cu **od** putem afla informații despre **bootloader**.

```
paul@laika:~$ sudo dd if=/dev/sda count=1 bs=16 skip=24 2>/dev/null|od -c
00000000 376  G  R  U  B      \0  G  e  o  m  \0  H  a  r  d
00000020
```

Există disponibile o varietate de încărcători de boot, cel mai comun pe arhitectura **Intel** este **grub**, care înlocuiește **lilo** de cele mai multe ori. Când instalați **Linux** pe o arhitectură **Sparc**, puteți alege **silos**, sistemele **Itanium** pot folosi **elilo**, **IBM S/390** și **zSeries** folosesc **z/IPL**, **Alpha** folosește **milo** și arhitecturile **PowerPC** folosesc **yaboot** (yet another boot loader).

Cd-urile butabile și dvd-urile folosesc deseori **syslinux**.

kernel

Scopul tuturor acestora este de a încărca un sistem de operare, sau mai degrabă **kernel-ul** unui sistem de operare. Un încărcător de boot ca **grub** va copia un kernel de pe hard-disk pe memorie, și apoi va da controlul computerului spre kernel (va executa kernel-ul).

De îndată ce kernel-ul Linux este încărcat, încărcătorul de boot dă controlul asupra lui. Din acest punct încolo, kernel-ul controlează sistemul. După ce discutăm încărcătorii de boot, vom continua cu sistemul init care pornește toți daemonii.

12.2. grub

despre grub

Cel mai comun încărcător de boot pe sistemele Linux astăzi este **grub**. Pe aproape toate sistemele bazate pe Intel grub înlocuiește **lilo** (**linux loader**). Chiar și **Solaris** a trecut la **grub** pe arhitectura **x86**.

Unul dintre marile avantaje a lui **grub** față de **lilo** este capabilitatea de a schimba configurația în timpul butării (apăsând **e** pentru a edita linia de comandă boot).

/boot/grub/menu.lst

Fișierul de configurație a lui **grub** este numit **menu.lst** și este localizat în **/boot/grub**. Screenshot-ul de mai jos arată locația și mărimea **menu.lst** pe Debian.

```
root@barry:~# ls -l /boot/grub/menu.lst
-rw-r--r-- 1 root root 5155 2009-03-31 18:20 /boot/grub/menu.lst
```

/boot/grub/grub.conf

Unele distribuții ca Red Hat Enterprise Linux 5 folosesc **grub.conf** și dau un link simbolic în **menu.lst**. Acesta este același fișier, doar numele s-a schimbat de la **grub.conf** în **menu.lst**. Observați de asemeni în această captură de ecran că acest fișier este mult mai mic pe Red Hat.

```
[root@RHEL52 grub]# ls -l grub.conf menu.lst
-rw----- 1 root root 1346 Jan 21 04:20 grub.conf
lrwxrwxrwx 1 root root 11 Oct 11 2008 menu.lst -> ./grub.conf
```

comenzi meniu

Comenzile meniu trebuie să fie întotdeauna cel mai sus în fișierul de configurație **grub**.

default

Comanda **default** setează o **intrare** default pentru a începe. Prima intrare are numărul 0.

```
default 0
```

fallback

În cazul în care **default** nu butează, folosiți intrarea **fallback** în schimb.

fallback 1

timeout

timeout va aștepta un număr de secunde înainte de a buta intrarea **default**.

timeout 5

hiddenmenu

hiddenmenu va ascunde meniul **grub** doar dacă utilizatorul apasă **Esc** înainte ca **timeout** să expire.

hiddenmenu

title

Cu **title** putem începe o nouă **intrare** sau **stanza**.

title Debian Lenny

parolă

Puteți adăuga o **parolă** pentru a preveni selecția interactivă a mediului de boot în timp ce **grub** rulează.

password --md5 \$1\$Ec.id/\$T2C2ahI/EG3WRRsmmu/HN/

Folosiți shell-ul interactiv **grub** pentru a crea parola hash.

grub> md5crypt

Password: *****

Encrypted: \$1\$Ec.id/\$T2C2ahI/EG3WRRsmmu/HN/

comenzi stanza

Fiecare **sistem de operare** sau **kernel** pe care vreți să-l butați cu **grub** va avea o **stanza** cunoscută și ca **intrare** a câtorva linii. Aici sînt listate cîteva dintre cele mai comune comenzi **stanza**.

boot

Din punct de vedere tehnic comanda **boot** este obligatorie numai cînd rulăm **linia de comandă grub**. Această comandă nu are nici un parametru și poate fi setată doar ca ultima comandă a unei stanzas.

boot

kernel

Comanda **kernel** țintește spre locația kernel-ului. Pentru a buta Linux asta înseamnă a buta o imagine **gzip** compresată **zImage** sau o imagine **bzip2** compresată **bzImage**.

Această captură de ecran arată o comandă **kernel** tipică folosită pentru a încărca un kernel Debian.

kernel /boot/vmlinuz-2.6.17-2-686 root=/dev/hda1 ro

Și iată cum Red Hat folosește comanda **kernel**.

```
kernel /vmlinuz-2.6.18-128.el5 ro root=/dev/VolGroup00/LogVol00 rhgb quiet  
initrd
```

Multe instalări **Linux** vor avea nevoie de un **ramdisk inițial** la butare. Asta poate fi setat în **grub** cu comanda **initrd**.

Aici este un screenshot a Debian 4.0.

```
initrd /boot/initrd.img-2.6.17-2-686
```

Și la fel pentru Red Hat Enterprise Linux 5.3.

```
initrd /initrd-2.6.18-128.el5.img
```

root

Comanda **root** acceptă dispozitivul root ca un parametru.

Comanda **root** va ținti spre hard-disk și partiția de utilizat, cu **hd0** ca primul dispozitiv hard-disk și **hd1** ca al doilea dispozitiv hard-disk. Aceeași numerotare este folosită pentru partiții, astfel **hd0,0** este prima partiție de pe primul disk și **hd0,1** este a doua partiție pe acel disk.

```
root (hd0,0)
```

savedefault

Comanda **savedefault** poate fi folosită împreună cu **default saved** ca o comandă meniu. Această combinație va seta stanza acum butată ca următoarea stanza default de butat.

```
default saved  
timeout 10
```

```
title Linux  
root (hd0,0)  
kernel /boot/vmlinuz  
savedefault
```

```
title DOS  
root (hd0,1)  
makeactive  
chainloader +1  
savedefault
```

încărcarea în lanț

Cu butarea **grub**, există două alegeri: încărcarea unui sistem de operare sau **încărcarea în lanț** a unui alt încărcător de boot. **Încărcarea în lanț** a lui grub încarcă sectorul boot a unei partiții (care conține un sistem de operare).

Unele sisteme de operare mai vechi cer o **partiție primară** care să fie setată ca **activă**. Numai o singură partiție poate fi setată ca **activă** astfel ca **grub** să facă asta imediat înainte de **încărcarea în lanț**.

Această captură de ecran arată cum să setăm prima partiție primară **activă**

cu **grub**.

```
root (hd0,0)
makeactive
```

Încărcarea în lanț face referire la grub care încarcă un alt încărcător de boot al unui alt sistem de operare. Întrerupătorul **încărcătorului de boot în lanț** primește o singură opțiune: numărul sectoarelor pentru a citi și buta. Pentru **DOS** și **OS/2** un singur sector este îndeajuns. Țineți minte că **DOS** cere partiția boot/root să fie activă!

Aici este un exemplu concret pentru a **încărca în lanț** un sistem de operare vechi.

```
title MS-DOS 6.22
root (hd0,1)
makeactive
chainloader +1
```

exemple stanza

Acesta este un screenshot a unei stanza tipică **Debian 4.0**.

```
title Debian GNU/Linux, kernel 2.6.17-2-686
root (hd0,0)
kernel /boot/vmlinuz-2.6.17-2-686 root=/dev/hda1 ro
initrd /boot/initrd.img-2.6.17-2-686
```

Aici este un screenshot a unei stanza tipică **Red Hat Enterprise Linux**.

```
title Red Hat Enterprise Linux Server (2.6.18-128.el5)
root (hd0,0)
kernel /vmlinuz-2.6.18-98.el5 ro root=/dev/VolGroup00/LogVol00 rhgb quiet
initrd /initrd-2.6.18-98.el5.img
```

editare grub în timpul butării

În timpul butării, când meniul **grub** este afișat, puteți tasta **e** pentru a edita stanza curentă. Asta vă dă posibilitatea să adăugați parametri kernel-ului.

Unul dintre astfel de parametri, folositor când ați pierdut parola de root, este **single**. Acesta va buta kernel-ul în mod utilizator single (deși unele distribuții încă vor solicita să tastați parola root).

```
kernel /boot/vmlinuz-2.6.17-2-686 root=/dev/hda1 ro single
```

O altă opțiune de a resta o parolă de root este să utilizăm un parametru **init=/bin/bash**

```
kernel /boot/vmlinuz-2.6.17-2-686 root=/dev/hda1 ro init=/bin/bash
```

instalare grub

Executați comanda **grub-install** pentru a instala **grub**. Comanda solicită o destinație pentru a suprascrie **sectorul de boot** pe **mbr**.

```
# grub-install /dev/hda
```


12.3. lilo

Linux loader

lilo era cel mai utilizat încărcător de boot Linux, dar este constant înlocuit cu **grub** în x86.

lilo.conf

Aici este un exemplu tipic al fișierului **lilo.conf**. **Întrerupătorul de timp** primește un număr în zecimi de secundă. Astfel că diferența de timp este 3 secunde, nu 30!

```
boot = /dev/hda
delay = 30
```

```
image = /boot/vmlinuz
root = /dev/hda1
label = Red Hat 5.2
```

```
image = /boot/vmlinuz
root = /dev/hda2
label = S.U.S.E. 8.0
```

```
other = /dev/hda4
table = /dev/hda
label = MS-DOS 6.22
```

Fișierul de configurare arată trei exemple stanza. Primul butează Red Hat de pe prima partiție pe primul hard-disk (hda1). A doua stanza butează Suse 8.0 de pe următoarea partiție. Ultima încarcă MS-DOS.

12.4. practică: încărcător de boot

1. Faceți o copie a fișierelor kernel-ului, `initrd` și `System.map` în `/boot`. Puneți copiile de asemeni în `/boot` dar înlocuiți 2.6.x cu 3.0 (imaginați-vă că Linux 3.0 a apărut).
2. Adăugați o stanza în `grub` pentru fișierele 3.0. Asigurați-vă că titlul este diferit.
3. Setați pauza meniului boot la 30 de secunde.
4. Rebutați și testați noua stanza.

12.5. soluție: încărcător de boot

1. Faceți o copie a fișierelor kernel-ului, initrd și System.map în /boot. Puneți copiile de asemeni în /boot dar înlocuiți 2.6.x cu 3.0 (imaginați-vă că Linux 3.0 a apărut).

```
cd /boot
cp vmlinuz-2.6.18-8.e15 vmlinuz-3.0
cp initrd-2.6.18-8.e15.img initrd-3.0.img
cp System.map-2.6.18-8.e15 System.map-3.0
```

Nu uitați că fișierul initrd se termină în .img.

2. Adăugați o stanză în grub pentru fișierele 3.0. Asigurați-vă că titlul este diferit.

```
[root@RHEL5 ~]# grep 3.0 /boot/grub/menu.lst
title Red Hat Enterprise Linux Server (3.0)
kernel /vmlinuz-3.0 ro root=/dev/VolGroup00/LogVol00 rhgb quiet
initrd /initrd-3.0.img
```

3. Setări pauza meniului boot la 30 de secunde.

```
[root@RHEL5 ~]# grep time /boot/grub/menu.lst
timeout=30
```

4. Rebutați și testați noua stanză.

Capitolul 13. init și runlevel

Conținut

13.1.	despre sysv init.	117
13.2.	init(țializare) sistem.	117
13.3.	daemon sau demon ?.	121
13.4.	start și stop daemoni.	121
13.5.	chkconfig.	121
13.6.	update-rc.d.	123
13.7.	bum.	124
13.8.	runlevel.	125
13.9.	practică: init.	127
13.10.	soluție: init.	128

13.1. despre sysv init

Multe distribuții Linux folosesc scripruri **init** pentru a deschide daemoni în același fel în care făcea **Unix System V**. Acest capitol va explica în detaliu cum funcționează asta.

Init pornește daemoni folosind script-uri, unde fiecare script pornește un daemon, și unde fiecare script așteaptă ca script-ul anterior să se finalizeze. Acest proces serial al pornirii daemonilor este încet, și cu toate că butarea cu timp lung nu este o problemă pe servere unde uptime este măsurat în ani, câștigarea terenului Linux pentru utilizatorii desktop a rezultat în plângeri din partea utilizatorilor.

Pentru a îmbunătăți viteza de start, **Canonical** a dezvoltat **upstart**, care a fost prima dată folosit în Ubuntu. Solaris folosea de asemeni init până la Solaris 9, pentru Solaris 10 **Sun** a dezvoltat **Service Management Facility**. Ambele sisteme pornesc daemoni în paralel și pot înlocui scripturile init SysV. Există de asemeni un efort continuu pentru a crea **initng** (init next generation).

13.2. init(țializare) sistem

proces id 1

Kernel-ul primește controlul sistemului de la încărcătorul de boot. După un scurt timp kernel-ul pornește **daemonul init**. Daemonul **init (/sbin/init)** este primul daemon care este pornit și primește **procesul id 1** (PID 1). **Init** nu moare niciodată.

configurare în /etc/inittab

Când **/sbin/init** este pornit, va citi mai întâi fișierul lui de configurație în **/etc/inittab**. În acel fișier, va căuta valoarea lui **initdefault** (3 în screenshot-ul de mai jos).

```
[paul@rhel4 ~]$ grep ^id /etc/inittab
id:3:initdefault:
```

initdefault

Valoarea găsită în **initdefault** indică **runlevel-ul** default. Unele distribuții Linux au o scurtă descriere a runlevel-urilor în **/etc/inittab**, ca cea de aici pe Red Hat Enterprise Linux 4.

```
# Default runlevel. The runlevels used by RHS are:
# 0 - halt (Do NOT set initdefault to this)
# 1 - Single user mode
# 2 - Multiuser, without NFS (The same as 3, if you don't have network)
# 3 - Full multiuser mode
# 4 - Unused
# 5 - X11
# 6 - reboot (Do NOT set initdefault to this)
```

Runlevel 0 înseamnă că sistemul se închide. **Runlevel 1** este folosit pentru depanare, doar utilizatorul root poate intra în sistem, și numai la consolă.

Runlevel 3 este tipic pentru servere, și de aici **runlevel 5** este tipic pentru desktop-uri (logon grafic). În afară de runlevel 0, 1 și 6, utilizarea poate varia depinzând de distribuție. Debian și sistemele Linux derivate au rețea deplină și logon GUI pe runlevel-urile 2 până la 5. Așa că verificați mereu înțelesul deplin a runlevel-urilor pe sistem.

script sysinit

/etc/rc.d/rc.sysinit

Următoarea linie în **/etc/inittab** în Red Hat și derivatele lui este următoarea.

```
si::sysinit:/etc/rc.d/rc.sysinit
```

Asta înseamnă că independent de runlevel-ul selectat, **init** va rula scriptul **/etc/rc.d/rc.sysinit**. Acest script inițializează hardware, setează unele medii de bază, populează **/etc/mtab** în timp ce montează sisteme de fișier, pornește swap și alte lucruri.

```
[paul@rhel ~]$ egrep -e"^# Ini" -e"^# Sta" -e"^# Che" /etc/rc.d/rc.sysinit
# Check SELinux status
# Initialize hardware
# Start the graphical boot, if necessary; /usr may not be mounted yet...
# Initialize ACPI bits
# Check filesystems
# Start the graphical boot, if necessary and not done yet.
# Check to see if SELinux requires a relabel
# Initialize pseudo-random number generator
# Start up swapping.
# Initialize the serial ports.
```

Comanda **egrep** de mai sus ar putea de asemeni să fie scrisă cu **grep** astfel:

```
grep "^# \(Ini\|Sta\|Che\)".
```

/etc/init.d/rcS

Debian are următoarea linie după **initdefault**.

```
si::sysinit:/etc/init.d/rcS
```

Scriptul **/etc/init.d/rcS** se va executa mereu pe Debian (independent de runlevel-ul selectat). Acest script execută în realitate toate scripturile în directorul **/etc/rcS.d/** în ordine alfabetică.

```
root@barry:~# cat /etc/init.d/rcS
#!/bin/sh
#
# rcS
#
# Call all S??* scripts in /etc/rcS.d/ in numerical/alphabetical order
#
exec /etc/init.d/rc S
```

script-uri rc

Init va continua să citească **/etc/inittab** și întâlnește această secțiune pe Debian Linux.

```
l0:0:wait:/etc/init.d/rc 0
l1:1:wait:/etc/init.d/rc 1
l2:2:wait:/etc/init.d/rc 2
l3:3:wait:/etc/init.d/rc 3
l4:4:wait:/etc/init.d/rc 4
l5:5:wait:/etc/init.d/rc 5
l6:6:wait:/etc/init.d/rc 6
```

Pe Red Hat Enterprise Linux este identic cu excepția că **init.d** este **rc.d**.

```
l0:0:wait:/etc/rc.d/rc 0
l1:1:wait:/etc/rc.d/rc 1
l2:2:wait:/etc/rc.d/rc 2
l3:3:wait:/etc/rc.d/rc 3
l4:4:wait:/etc/rc.d/rc 4
l5:5:wait:/etc/rc.d/rc 5
l6:6:wait:/etc/rc.d/rc 6
```

În ambele cazuri, asta înseamnă că **init** va începe script-ul rc cu runlevel ca singurul parametru. În realitate **/etc/inittab** are câmpuri separate de două puncte. Al doilea câmp determină runlevel-ul în care linia ar trebui executată. Astfel în ambele cazuri, doar o singură linie din șapte va fi executată, depinzând de runlevel-ul setat de **initdefault**.

directoare rc

Când priviți la oricare dintre directoarele **/etc/rcX.d/**, atunci veți vedea multe dintre (legături la) script-uri a căror nume încep fie cu K literă mare fie cu S cu literă mare.

```
[root@RHEL52 rc3.d]# ls -l | tail -4
lrwxrwxrwx 1 root root 19 Oct 11 2008 S98haldaemon -> ../init.d/haldaemon
lrwxrwxrwx 1 root root 19 Oct 11 2008 S99firstboot -> ../init.d/firstboot
lrwxrwxrwx 1 root root 11 Jan 21 04:16 S99local -> ../rc.local
lrwxrwxrwx 1 root root 16 Jan 21 04:17 S99smartd -> ../init.d/smartd
```

Directoarele **/etc/rcX.d/** conțin doar legături către script-uri în **/etc/init.d/**. Link-urile (legăturile) permit scriptului să aibă un nume diferit. Când intrăm într-un runlevel, toate scripturile care încep cu litera mare K sau litera mare S vor fi începute în ordine alfabetică. Acelea care încep cu K vor fi pornite primele, cu **stop** ca singurul parametru. Script-urile care rămân cu S vor fi pornite cu **start** ca parametru singular.

Toate acestea se fac de către scriptul **/etc/rc.d/rc** pe Red Hat și de către scriptul **/etc/init.d/rc** pe Debian.

mingetty

mingetty în /etc/inittab

Aproape de sfârșitul din **/etc/inittab** există o secțiune pentru a porni și a reproduce câțiva daemoni **mingetty**.

```
[root@RHEL4b ~]# grep getty /etc/inittab
# Run gettys in standard runlevels
1:2345:respawn:/sbin/mingetty tty1
2:2345:respawn:/sbin/mingetty tty2
3:2345:respawn:/sbin/mingetty tty3
4:2345:respawn:/sbin/mingetty tty4
5:2345:respawn:/sbin/mingetty tty5
6:2345:respawn:/sbin/mingetty tty6
```

mingetty și /bin/login

Acest **/sbin/mingetty** va afișa un mesaj pe o consolă virtuală și vă va permite să tastați un **userid**. Apoi execută comanda **/bin/login** cu acel **userid**. Programul **/bin/login** va verifica dacă utilizatorul există în **/etc/passwd** și va solicita (și verifica) o parolă. Dacă parola e corectă, **/bin/login** trece controlul către shell-ul listat în **/etc/passwd**.

relansare mingetty

Daemonii **mingetty** sînt porniți de **init** și observați pînă cînd mor (utilizatorul iese din shell și este delogat). Cînd asta se întîmplă, daemonul **init** va **relansa** un nou **mingetty**. Astfel că chiar dacă **ucideți** un daemon **mingetty**, el va fi restartat în mod automat.

Acest exemplu arată că **init** relansează daemoni **mingetty**. Priviți la PID-urile ultimelor două procese **mingetty**.

```
[root@RHEL52 ~]# ps -C mingetty
  PID   TTY      TIME    CMD
 2407   tty1      00:00:00 mingetty
 2408   tty2      00:00:00 mingetty
 2409   tty3      00:00:00 mingetty
 2410   tty4      00:00:00 mingetty
 2411   tty5      00:00:00 mingetty
 2412   tty6      00:00:00 mingetty
```

Cînd **închidem** ultimele două **mingetty**, atunci **init** va observa asta și le va porni din nou (cu un PID diferit).

```
[root@RHEL52 ~]# kill 2411 2412
[root@RHEL52 ~]# ps -C mingetty
  PID   TTY      TIME    CMD
 2407   tty1      00:00:00 mingetty
 2408   tty2      00:00:00 mingetty
 2409   tty3      00:00:00 mingetty
 2410   tty4      00:00:00 mingetty
 2821   tty5      00:00:00 mingetty
 2824   tty6      00:00:00 mingetty
```

deautorizarea unui mingetty

Puteți deautoriza un **mingetty** pentru un anumit **tty** ștergînd runlevel-ul din al doilea cîmp în linia lui în **/etc/inittab**. Nu uitați să informați **init** despre schimbarea configurației fișierului cu **kill -1 1**.

Acest exemplu de mai jos arată cum să deautorizăm **mingetty** pe **tty3** pînă la **tty6** în runlevel-urile 4 și 5.


```
[root@RHEL52 ~]# grep getty /etc/inittab
# Run gettys in standard runlevels
1:2345:respawn:/sbin/mingetty tty1
2:2345:respawn:/sbin/mingetty tty2
3:23:respawn:/sbin/mingetty tty3
4:23:respawn:/sbin/mingetty tty4
5:23:respawn:/sbin/mingetty tty5
6:23:respawn:/sbin/mingetty tty6
```

13.3. daemon sau demon?

Un **daemon** este un proces care se execută în background, fără o legătură cu un GUI sau terminal. Daemonii sînt de obicei porniți la butarea sistemului, și stau vii pînă ce sistemul se închide. În literatura tehnică mai recentă, la daemoni se face deseori referire ca **servicii**.

Daemonii Unix nu trebuie confundați cu demonii. **Evi Nemeth**, co-autor al UNIX System Administration Handbook are de spus următoarele despre daemoni:

Multă lume egalizează cuvîntul "daemon" cu cuvîntul "demon", implicînd un fel de conexiune satanică între UNIX și infern. Aceasta este o înțelegere strident greșită. "Daemon" este în realitate o formă mult mai veche a cuvîntului "demon"; daemonii nu au nici o înclinare particulară spre bine sau rău, ci însă ajută să definească caracterul unei persoane sau personalității. Conceptul anticilor greci a unui "daemon personal" era similar cu conceptul modern al unui "înger păzitor"

13.4. start și stop daemoni

Scripturile K și S sînt legături la adevăratele scripturi în **/etc/init.d/**. Acestea pot fi de asemeni folosite cînd sistemul rulează pentru a porni și închide daemoni (sau servicii). Mulți dintre ei acceptă următorii parametri: start, stop, restart, status.

De exemplu în această captură de ecran restartăm daemonul samba.

```
root@laika:~# /etc/init.d/samba restart
* Stopping Samba daemons. . . [ OK ]
* Starting Samba daemons. . . [ OK ]
```

Puteți obține același rezultat pe RHEL/Fedora cu comanda **service**.

```
[root@RHEL4b ~]# service smb restart
Shutting down SMB services: [ OK ]
Shutting down NMB services: [ OK ]
Starting SMB services: [ OK ]
Starting NMB services: [ OK ]
```

Ați putea de asemeni să vă uitați la **chkconfig**, **update-rc.d**.

13.5. chkconfig

Scopul lui **chkconfig** este să le facă ușor administratorilor de sistem să facă managementul manual al tuturor legăturilor și scripturilor în **/etc/init.d** și **/etc/rcX.d/**.

chkconfig --list

Aici folosim **chkconfig** pentru a lista statusul unui serviciu în diferite runlevel-uri. Puteți vedea că daemonul (sau serviciul) **crond** este activat doar în runlevel-urile 2 pînă la 5.

```
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off 1:off 2:on 3:on 4:on 5:on 6:off
```

Cînd comparați screenshot-ul de deasupra cu cel de mai jos, puteți vedea că **off** echivalează cu o legătură K scriptului, și **on** echivalează cu o legătură S.

```
[root@RHEL52 etc]# find ./rc?.d/ -name \*crond -exec ls -l {} \;|cut -b40-
./rc0.d/K60crond -> ../init.d/crond
./rc1.d/K60crond -> ../init.d/crond
./rc2.d/S90crond -> ../init.d/crond
./rc3.d/S90crond -> ../init.d/crond
./rc4.d/S90crond -> ../init.d/crond
./rc5.d/S90crond -> ../init.d/crond
./rc6.d/K60crond -> ../init.d/crond
```

configurare runlevel

Aici vedeți cum să utilizați **chkconfig** pentru a deautoriza (sau autoriza) un serviciu într-un anumit runlevel.

Acest screenshot arată cum să deautorizăm **crond** în runlevel 3.

```
[root@RHEL52 ~]# chkconfig --level 3 crond off
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off 1:off 2:on 3:off 4:on 5:on 6:off
```

Acest screenshot arată cum să autorizăm **crond** în runlevel-urile 3 și 4.

```
[root@RHEL52 ~]# chkconfig --level 34 crond on
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off 1:off 2:on 3:on 4:on 5:on 6:off
```

configurare chkconfig

Fiecare script în **/etc/init.d/** poate avea linii (comentate) pentru a transmite **chkconfig** ce să facă cu serviciul. Linia cu **# chkconfig:** conține runlevel-urile în care serviciul ar trebui pornit (2345), urmat de prioritatea pentru start (90) și stop (60).

```
[root@RHEL52 ~]# head -9 /etc/init.d/crond | tail -5
# chkconfig: 2345 90 60
# description: cron is a standard UNIX program that runs user-specified
#               programs at periodic scheduled times. vixie cron adds a
#               number of features to the basic UNIX cron, including better
#               security and more powerful configuration options.
```

autorizați și deautorizați servicii

Serviciile pot fi autorizate sau deautorizate în toate runlevel-urile cu o singură comandă. Runlevel-urile 0, 1 și 6 sînt întotdeauna servicii de oprire (sau aducere a scripturilor cu **stop**) chiar dacă numele lor încep cu S literă mare.

```
[root@RHEL52 ~]# chkconfig crond off
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off  1:off  2:off  3:off  4:off  5:off  6:off
[root@RHEL52 ~]# chkconfig crond on
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off  1:off  2:on   3:on   4:on   5:on   6:off
```

13.6. update-rc.d

despre update-rc.d

Echivalentul Debian al lui **chkconfig** este numit **update-rc.d**. Acest utilitar este făcut să fie utilizat în scripturi, dacă preferați un utilitar grafic atunci priviți la **bum**.

Cînd sînt legături existente în **/etc/rcX.d/** atunci **update-rc.d** nu face nimic. Asta pentru a evita acele scripturi post instalare folosind **update-rc.d** care suprascriu schimbările făcute de un administrator de sistem.

```
root@barry:~# update-rc.d cron remove
update-rc.d: /etc/init.d/cron exists during rc.d purge (use -f to force)
```

După cum vedeți în următoarea captură de ecran, nimic nu s-a schimbat pentru daemonul cron.

```
root@barry:~# find /etc/rc?.d/ -name '*cron' -exec ls -l {} \;|cut -b44-
/etc/rc0.d/K11cron -> ../init.d/cron
/etc/rc1.d/K11cron -> ../init.d/cron
/etc/rc2.d/S89cron -> ../init.d/cron
/etc/rc3.d/S89cron -> ../init.d/cron
/etc/rc4.d/S89cron -> ../init.d/cron
/etc/rc5.d/S89cron -> ../init.d/cron
/etc/rc6.d/K11cron -> ../init.d/cron
```

ștergerea unui serviciu

Acum putem șterge **cron** din toate runlevel-urile. Țineți minte că modalitatea corectă de a deautoriza un serviciu este de a pune scripturile K pe toate runlevel-urile!

```
root@barry:~# update-rc.d -f cron remove
Removing any system startup links for /etc/init.d/cron ...
/etc/rc0.d/K11cron
/etc/rc1.d/K11cron
/etc/rc2.d/S89cron
/etc/rc3.d/S89cron
/etc/rc4.d/S89cron
/etc/rc5.d/S89cron
/etc/rc6.d/K11cron
root@barry:~# find /etc/rc?.d/ -name '*cron' -exec ls -l {} \;|cut -b44-
root@barry:~#
```

autorizarea unui serviciu

Această captură de ecran arată cum să utilizăm **update-rc.d** pentru a autoriza un serviciu în runlevel-urile 2, 3, 4 și 5 și să deautorizăm serviciul în runlevel-urile 0, 1 și 6.

```

root@barry:~# update-rc.d cron defaults
Adding system startup for /etc/init.d/cron ...
/etc/rc0.d/K20cron -> ../init.d/cron
/etc/rc1.d/K20cron -> ../init.d/cron
/etc/rc6.d/K20cron -> ../init.d/cron
/etc/rc2.d/S20cron -> ../init.d/cron
/etc/rc3.d/S20cron -> ../init.d/cron
/etc/rc4.d/S20cron -> ../init.d/cron
/etc/rc5.d/S20cron -> ../init.d/cron

```

faceți un serviciu la comandă

Și aici este un exemplu despre cum să setați configurația făcută la comandă pentru daemonul cron.

```

root@barry:~# update-rc.d -n cron start 11 2 3 4 5 . stop 89 0 1 6 .
Adding system startup for /etc/init.d/cron ...
/etc/rc0.d/K89cron -> ../init.d/cron
/etc/rc1.d/K89cron -> ../init.d/cron
/etc/rc6.d/K89cron -> ../init.d/cron
/etc/rc2.d/S11cron -> ../init.d/cron
/etc/rc3.d/S11cron -> ../init.d/cron
/etc/rc4.d/S11cron -> ../init.d/cron
/etc/rc5.d/S11cron -> ../init.d/cron

```

13.7. bum

Această captură de ecran arată **bum** în modul avansat.

The screenshot shows the 'bum' service configuration window. The 'Advanced' mode is selected. The table lists the following services:

Activate	Service name	Single user	Run level 2*	Run level 3	Run level 4	Run level 5	Reboot	Halt	Running
☒	hns-common	K79	S21	S21	S21	S21	K79	K79	?
☒	fam	K21	S21	S21	S21	S21	K21	K21	?
☒	ntp	K23	S23	S23	S23	S23	K23	K23	?
☒	cron	K11	S89	S89	S89	S89	K11	K11	?
☒	atd	K11	S89	S89	S89	S89	K11	K11	?
☒	apache2	K09	S91	S91	S91	S91	K09	K09	?
☒	apache-perl	K91	S91	S91	S91	S91	K91	K91	?
☐	bootclean	---	---	---	---	---	---	---	?

cron: Runs system housekeeping chores on specified dates/times

cron is a background process ('daemon') that runs programs at regular intervals (for example, every minute, day, week or month); which processes are run and at what times are specified in the 'crontab'.

Users may also install crontabs so that processes are run on their behalf, though this feature can be disabled or restricted to particular users.

At the bottom, there are buttons for 'Apply' and 'Quit', and a checkbox for 'Advanced' which is currently checked.

13.8. runlevel

afișarea runlevel

Puteți vedea runlevel-ul curent cu comenzile **runlevel** sau **who -r**.

Comanda **runlevel** este tipică Linux-ului și va afișa ieșirea runlevel-ului anterior și a celui curent. Dacă nu a existat nici un runlevel anterior, atunci îl va marca cu litera N.

```
[root@RHEL4b ~]# runlevel
N 3
```

Istoria lui **who -r** datează din anii 1970 în Unix, și încă mai funcționează pe Linux.

```
[root@RHEL4b ~]# who -r
run-level 3 Jul 28 09:15 last=S
```

schimbând runlevel-ul

Puteți schimba un alt runlevel cu comanda **telinit**. Pe Linux **/sbin/telinit** are de obicei o legătură (hard) la **/sbin/init**.

Această captură de ecran arată cum să schimbăm de la runlevel 2 la runlevel 3 fără reboot.

```
root@barry:~# runlevel
N 2
root@barry:~# init 3
root@barry:~# runlevel
2 3
```

/sbin/shutdown

Comanda **shutdown** este folosită pentru a închide cum trebuie un sistem.

Întrerupători comuni folosiți cu **shutdown** sînt **-a**, **-t**, **-h** și **-r**.

Întrerupătorul **-a** forțează **/sbin/shutdown** să folosească **/etc/shutdown.allow**.

Întrerupătorul **-t** este folosit să definească numărul de secunde dintre trimiterea semnalului **TERM** și a semnalului **KILL**. Întrerupătorul **-h** oprește sistemul în loc să se schimbe în runlevel 1. Întrerupătorul **-r** transmite **/sbin/shutdown** să rebuteze după ce se închide.

Această captură de ecran arată cum să utilizăm **shutdown** cu cinci secunde între semnalele **TERM** și **KILL**.

```
root@barry:~# shutdown -t5 -h now
```

now este argumentul timp. Acesta poate fi **+m** pentru numărul de minute de așteptare înainte de oprire (cu **now** ca alias pentru **+0**). Comanda de asemeni va accepta hh:mm în loc de **+m**.

halt, reboot și poweroff

Executabilul **/sbin/reboot** este același cu **/sbin/halt** și **/sbin/poweroff**. Depinzînd de numele pe care îl folosim pentru a invoca comanda, el se poate comporta diferit.

Cînd sîntem în runlevel 0 sau 6, **halt**, **reboot** și **poweroff** va transmite kernel-ului să **oprească**, să **rebuteze** sau să **încidă** sistemul.

Cînd nu sîntem în runlevel 0 sau 6, tastînd **reboot** ca root în realitate invocă comanda **shutdown** cu întrerupătorul **-r** și tastînd **poweroff** va întrerupe electricitatea cînd închidem sistemul.

/var/log/wtmp

halt, **reboot** și **poweroff** toate se scriu în **/var/log/wtmp**. Pentru a privi la **/var/log/wtmp**, trebuie să utilizăm **last**.

```
[root@RHEL52 ~]# last | grep reboot
reboot    system boot  2.6.18-128.el5  Fri May 29 11:44    (192+05:01)
reboot    system boot  2.6.18-128.el5  Wed May 27 12:10    (06:49)
reboot    system boot  2.6.18-128.el5  Mon May 25 19:34    (1+15:59)
reboot    system boot  2.6.18-128.el5  Mon Feb  9 13:20    (106+21:13)
```

Ctrl-Alt-Del

Cînd **rc** a terminat de pornit toate acele script-uri, **init** va continua să citească **/etc/inittab**. Următoarea linie este despre ce să facă cînd utilizatorul apasă **Ctrl-Alt-Delete** pe tastatură. Aici este ceea ce face Debian 4.0.

```
root@barry:~# grep -i ctrl /etc/inittab
# What to do when CTRL-ALT-DEL is pressed.
ca:12345:ctrlaltdel:/sbin/shutdown -t1 -a -r now
```

Ceea ce este foarte similar cu acțiunea default a Red Hat Enterprise Linux 5.2.

```
[root@RHEL52 ~]# grep -i ctrl /etc/inittab
# Trap CTRL-ALT-DELETE
ca::ctrlaltdel:/sbin/shutdown -t3 -r now
```

O diferență notabilă este că Debian forțează **shutdown** să folosească **/etc/shutdown.allow**, pe cînd Red Hat permite oricui să invoce **shutdown** apăsînd **Ctrl-Alt-Delete**.

UPS și pierderea electricității

```
[root@RHEL52 ~]# grep ^p /etc/inittab
pf::powerfail:/sbin/shutdown -f -h +2 "Power Failure; System Shutting Down"
pr:12345:powerokwait:/sbin/shutdown -c "Power Restored; Shutdown Cancelled"
```

Va citi comenzi despre ceea ce să execute în cazul **powerfailure**, **powerok** și **Ctrl-Alt-Delete**. Procesul **init** nu se oprește niciodată să acorde atenție asupra pierderii electricității și a acelor taste triple combinate.

```
root@barry:~# grep ^p /etc/inittab
pf::powerwait:/etc/init.d/powerfail start
pn::powerfailnow:/etc/init.d/powerfail now
po::powerokwait:/etc/init.d/powerfail stop
```

13.9. practică: init

1. Schimbați **/etc/inittab** astfel ca doar două **mingetty** să fie relansate. Închideți celelalte **mingetty** și verificați dacă ele nu se întorc.
2. Folosiți Red Hat Enterprise Linux ca mașină virtuală. Intrați în runlevel 5, afișați runlevel-ul curent și cel anterior, apoi mergeți înapoi în runlevel 3.
3. Este script-ul **sysinit** pe setarea computerului sau schimbă mediul variabilă **PATH**?
4. Listați toate script-urile **init.d** care sînt pornite în runlevel 2.
5. Scrieți un script care acționează ca un script daemon în **/etc/init.d/**. Ar trebui să aibă o declarație pentru a acționa asupra start/stop/restart și status. Testați scriptul!
6. Folosiți **chkconfig** pentru a seta script-ul să înceapă în runlevel-urile 3, 4 și 5 și să se oprească în oricare alt runlevel.

13.10. soluție: init

1. Schimbați **/etc/inittab** astfel ca doar două mingetty să fie relansate. Închideți celelalte **mingetty** și verificați dacă ele nu se întorc.

Închizînd mingetty va rezulta că init le va relansa. Puteți edita **/etc/inittab** astfel încît să arate ca în screenshotul de mai jos. Nu uitați de asemeni să executați **kill -1 1**.

```
[root@RHEL5 ~]# grep tty /etc/inittab
# Run gettys in standard runlevels
1:2345:respawn:/sbin/mingetty tty1
2:2345:respawn:/sbin/mingetty tty2
3:2:respawn:/sbin/mingetty tty3
4:2:respawn:/sbin/mingetty tty4
5:2:respawn:/sbin/mingetty tty5
6:2:respawn:/sbin/mingetty tty6
[root@RHEL5 ~]#
```

2. Folosiți Red Hat Enterprise Linux ca mașină virtuală. Intrați în runlevel 5, afișați runlevel-ul curent și cel anterior, apoi mergeți înapoi în runlevel 3.

```
init 5 (uitați-vă la consolă pentru schimbarea care are loc)
runlevel
init 3 (din nou puteți urmări asta pe consolă)
```

3. Este script-ul **sysinit** pe setarea computerului sau schimbă mediul variabilă **PATH**?

Pe Red Hat, grep pentru **PATH** în **/etc/rc.sysinit**, pe Debian/Ubuntu verificați **/etc/rc.local** și **/etc/init.d/rc.local**. Răspunsul este probabil nu, dar pe RHEL5 scriptul **rc.sysinit** setează variabila **HOSTNAME**.

```
[root@RHEL5 etc]# grep HOSTNAME rc.sysinit
```

4. Listați toate scripturile **init.d** care sînt pornite în runlevel 2.

```
root@RHEL5 ~# chkconfig --list | grep '2:on'
```

5. Scrieți un script care acționează ca un script daemon în **/etc/init.d/**. Ar trebui să aibă o declarație pentru a acționa asupra start/stop/restart și status. Testați scriptul!

Script-ul ar putea arăta astfel:


```
#!/bin/bash
#
# chkconfig: 345 99 01
# description: pold demo script
#
# /etc/init.d/pold

#
case "$1" in
start)
    echo -n "Starting pold..."
    sleep 1;
    touch /var/lock/subsys/pold
    echo "done."
    echo pold started >> /var/log/messages
    ;;
stop)
    echo -n "Stopping pold..."
    sleep 1;
    rm -rf /var/lock/subsys/pold
    echo "done."
    echo pold stopped >> /var/log/messages
    ;;
*)
    echo "Usage: /etc/init.d/pold {start|stop}"
    exit 1
;;
esac
exit 0

touch /var/lock/subsys/pold este obligatoriu și trebuie să fie același nume de
fișier ca numele scriptului, dacă doriți ca secvența stop (legătura K01pold) să
fie executată.
```

6. Folosiți **chkconfig** pentru a seta scriptul să înceapă în runlevelurile 3, 4 și 5 și să se oprească în oricare alt runlevel.

```
chkconfig --add pold
```

Comanda de deasupra va funcționa doar dacă în liniile din script-ul pold există #chkconfig: și #description:.

Partea IV. management sistem

Capitolul 14. planificare

Conținut

14.1. job-uri pe rînd cu at	132
14.2. cron	133
14.3. practică: planificare.	135
14.4. soluție: planificare.	136

Administratorii Linux folosesc comanda **at** pentru a planifica job-uri pe rînd. Job-urile recurente sînt cel mai bine planificate cu **cron**. Cele două secțiuni vor discuta ambele utilitare.

14.1. job-uri pe rînd cu at

at

Planificările simple pot fi făcute cu comanda **at**. Această captură de ecran arată planificarea comenzii **date** la 22:01 și a comenzii **sleep** la 22:03.

```
root@laika:~# at 22:01
at> date
at> <EOT>
job 1 at Wed Aug 1 22:01:00 2007
root@laika:~# at 22:03
at> sleep 10
at> <EOT>
job 2 at Wed Aug 1 22:03:00 2007
root@laika:~#
```

În viața reală sperăm că veți face planificare a unor comenzi mai utile ;-)

atq

Cu comenzile **atq** sau **at -l** este ușor de verificat cînd sînt planificate job-uri.

```
root@laika:~# atq
1      Wed Aug 1 22:01:00 2007 a root
2      Wed Aug 1 22:03:00 2007 a root
root@laika:~# at -l
1      Wed Aug 1 22:01:00 2007 a root
2      Wed Aug 1 22:03:00 2007 a root
root@laika:~#
```

Comanda **at** înțelege cuvinte englezești ca *mîine* și *teatime* pentru a planifica comenzi în ziua următoare și la patru după-amiază.

```
root@laika:~# at 10:05 tomorrow
at> sleep 100
at> <EOT>
job 5 at Thu Aug 2 10:05:00 2007
root@laika:~# at teatime tomorrow
at> tea
at> <EOT>
job 6 at Thu Aug 2 16:00:00 2007
root@laika:~# atq
6      Thu Aug 2 16:00:00 2007 a root
5      Thu Aug 2 10:05:00 2007 a root
root@laika:~#
```

atrm

Job-urile din coadă **at** pot fi șterse cu **atrm**.

```

root@laika:~# atq
6      Thu Aug 2  16:00:00 2007 a root
5      Thu Aug 2  10:05:00 2007 a root
root@laika:~# atrm 5
root@laika:~# atq
6      Thu Aug 2  16:00:00 2007 a root
root@laika:~#

```

at.allow și at.deny

Puteți de asemeni folosi fișierele **/etc/at.allow** și **/etc/at.deny** pentru a aranja cine poate planifica job-uri cu **at**.

Fișierul **/etc/at.allow** poate conține o listă de utilizatori cărora le este permis să planifice sarcini **at**. Când fișierul **/etc/at.allow** nu există, atunci oricine poate utiliza **at** doar dacă numele lor de utilizator nu este listat în **/etc/at.deny**.

Dacă nici unul dintre aceste fișiere nu există, atunci oricine poate utiliza **at**.

14.2. cron

fișierul crontab

Comanda **crontab(1)** poate fi folosită pentru a menține fișierul **crontab(5)**. Fiecare utilizator poate avea propriul lui fișier **crontab** pentru a planifica job-uri la un anumit timp. Acest timp poate fi specificat cu cinci câmpuri în această ordine: minut, oră, ziua lunii, luna și ziua săptămânii. Dacă câmpul conține un asterisc (*), atunci acesta înseamnă toate valorile acelui câmp.

Următorul exemplu înseamnă: executarea script42 la ora 14:08, în fiecare zi a lunii, în fiecare lună și în fiecare zi a săptămânii.

```
8 14 * * * script42
```

Execută scriptul script8472 în prima zi a lunii la ora 01:00.

```
25 0 1 * * script8472
```

Execută acest script33 la fiecare două minute duminica (ambele cifre 0 și 7 fac referire la ziua de duminică).

```
*/2 * * * 0
```

În loc de aceste cinci câmpuri, puteți de asemeni să tastați aceste comenzi: **@reboot**, **@yearly** sau **@annually**, **@monthly**, **@weekly**, **@daily** sau **@midnight**, și **@hourly**.

comanda crontab

Utilizatorii nu ar trebui să editeze fișierul **crontab** în mod direct, în schimb ar trebui să tasteze **crontab -e** care va utiliza editorul definit în variabila mediu **EDITOR** sau **VISUAL**. Utilizatorii pot afișa tabelul lor cron cu **crontab -l**.

cron.allow și cron.deny

daemonul **cron crond** citește tabelurile cron, luând în considerare fișierele **/etc/cron.allow** și **/etc/cron.deny**.

Aceste fișiere lucrează în aceeași modalitate ca **at.allow** și **at.deny**. Când

fișierul **cron.allow** există, atunci username-ul trebuie să fie în el, altfel nu puteți utiliza **cron**. Când fișierul **cron.allow** nu există, atunci username-ul nu poate exista în fișierul **cron.deny** dacă vreți să utilizați **cron**.

/etc/crontab

Fișierul **/etc/crontab** conține intrări asupra timpului când să se execute job-uri pe oră/zile/săptămîni/luni. Va arăta la fel cu această ieșire.

```
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
20 3 * * *          root    run-parts --report /etc/cron.daily
40 3 * * 7          root    run-parts --report /etc/cron.weekly
55 3 1 * *          root    run-parts --report /etc/cron.monthly
```

/etc/cron.*

Directoarele arătate în următoarea captură de ecran conțin job-urile care sînt rulate la datele planificate în **/etc/crontab**. Directorul **/etc/cron.d** există pentru cauze speciale, pentru a planifica job-uri care cer un control mai fin decît pe ore/zile/săptămîni/luni.

```
paul@laika:~$ ls -ld /etc/cron.*
drwxr-xr-x 2 root root 4096 2008-04-11 09:14 /etc/cron.d
drwxr-xr-x 2 root root 4096 2008-04-19 15:04 /etc/cron.daily
drwxr-xr-x 2 root root 4096 2008-04-11 09:14 /etc/cron.hourly
drwxr-xr-x 2 root root 4096 2008-04-11 09:14 /etc/cron.monthly
drwxr-xr-x 2 root root 4096 2008-04-11 09:14 /etc/cron.weekly
```

14.3. practică: planificare

1. Planificați două job-uri cu **at**, afișați **at queue** și ștergeți un job.
2. Ca utilizator normal, folosiți **crontab -e** pentru a planifica un script să se execute la fiecare patru minute.
3. Ca root, afișați fișierul **crontab** a utilizatorului normal.
4. Ca utilizator normal din nou, ștergeți fișierul **crontab**.
5. Priviți la fișierele **cron** și directoarele din **/etc** și înțelegeți-le. Ce face comanda **run-parts**?

14.4. soluție: planificare

1. Planificați două job-uri cu **at**, afișați **at queue** și ștergeți un job.

```
root@rhel55 ~# at 9pm today
at> echo go to bed >> /root/todo.txt
at> <EOT>
job 1 at 2010-11-14 21:00
root@rhel55 ~# at 17h31 today
at> echo go to lunch >> /root/todo.txt
at> <EOT>
job 2 at 2010-11-14 17:31
root@rhel55 ~# atq
2 2010-11-14 17:31 a root
1 2010-11-14 21:00 a root
root@rhel55 ~# atrm 1
root@rhel55 ~# atq
2 2010-11-14 17:31 a root
root@rhel55 ~# date
Sun Nov 14 17:31:01 CET 2010
root@rhel55 ~# cat /root/todo.txt
go to lunch
```

2. Ca utilizator normal, folosiți **crontab -e** pentru a planifica un script să se execute la fiecare patru minute.

```
paul@rhel55 ~$ crontab -e
no crontab for paul - using an empty one
crontab: installing new crontab
```

3. Ca root, afișați fișierul **crontab** a utilizatorului normal.

```
root@rhel55 ~# crontab -l -u paul
*/4 * * * * echo `date` >> /home/paul/crontest.txt
```

4. Ca utilizator normal din nou, ștergeți fișierul **crontab**.

```
paul@rhel55 ~$ crontab -r
paul@rhel55 ~$ crontab -l
no crontab for paul
```

5. Priviți la fișierele cron și directoarele din **/etc** și înțelegeți-le. Ce face comanda **run-parts**?

run-parts execută un script într-un director.

Capitolul 15. jurnalizare

Conținut

15.1. jurnalizarea accesărilor sistemului.	138
15.2. syslogd.	140
15.3. logger.	142
15.4. observarea fișierelor-jurnal.	142
15.5. fișiere-jurnal rotație	143
15.6. practică: jurnalizare.	144
15.7. soluție: jurnalizare.	145

Acest capitol are trei subiecte distincte.

Mai întâi ne uităm la jurnalizarea accesărilor sistemului; cum putem ști cine intră în sistem, când și de unde. Și cine nu este intrat în sistem, cine eșuează la **su** sau **ssh**.

Apoi discutăm cum să configurăm daemonul syslog, și cum să îl testăm cu **logger**.

Ultima parte este în principal despre **fișiere-jurnal rotație** și menționează comenzile **tail -f** și **watch** pentru **fișiere-jurnal observație**.

15.1. jurnalizarea accesărilor sistemului

Pentru a ține evidența asupra a cine este intrat în sistem, Linux poate menține fișierele `/var/log/wtmp`, `/var/log/btmp`, `/var/run/utmp` și `/var/log/lastlog`.

`/var/run/utmp` (who)

Folosiți comanda **who** pentru a vedea fișierul `/var/run/utmp`. Această comandă vă arată toți utilizatorii intrați în sistem **în mod curent**. Observați că fișierul `utmp` este în `/var/run` și nu în `/var/log`.

```
[root@rhel4 ~]# who
paul      pts/1      Feb  14  18:21 (192.168.1.45)
sandra    pts/2      Feb  14  18:11 (192.168.1.42)
inge      pts/3      Feb  14  12:01 (192.168.1.33)
els       pts/4      Feb  14  14:33 (192.168.1.19)
```

`/var/log/wtmp` (last)

Fișierul `/var/log/wtmp` este adus la zi de **programul login**. Folosiți **last** pentru a vedea fișierul `/var/run/wtmp`.

```
[root@rhel4a ~]# last | head
paul      pts/1      192.168.1.45      Wed Feb 14 18:39   still logged in
reboot    system boot  2.6.9-42.0.8.Els  Wed Feb 14 18:21   (01:15)
nicolas   pts/5      pc-dss.telematic  Wed Feb 14 12:32 - 13:06 (00:33)
stefaan   pts/3      pc-sde.telematic  Wed Feb 14 12:28 - 12:40 (00:12)
nicolas   pts/3      pc-nae.telematic  Wed Feb 14 11:36 - 12:21 (00:45)
nicolas   pts/3      pc-nae.telematic  Wed Feb 14 11:34 - 11:36 (00:01)
dirk      pts/5      pc-dss.telematic  Wed Feb 14 10:03 - 12:31 (02:28)
nicolas   pts/3      pc-nae.telematic  Wed Feb 14 09:45 - 11:34 (01:48)
dimitri   pts/5      rhel4             Wed Feb 14 07:57 - 08:38 (00:40)
stefaan   pts/4      pc-sde.telematic  Wed Feb 14 07:16 - down  (05:50)
[root@rhel4a ~]#
```

Comanda `last` de asemeni poate fi utilizată pentru a obține o listă a ultimelor rebutări.

```
[paul@rekkie ~]$ last reboot
reboot    system boot 2.6.16-rekkie    Mon Jul 30 05:13  (370+08:42)
```

```
wtmp begins Tue May 30 23:11:45 2006
[paul@rekkie ~]$
```

`/var/log/lastlog` (lastlog)

Folosiți **lastlog** pentru a vedea fișierul `/var/log/lastlog`.

```
[root@rhel4a ~]# lastlog | tail
tim          pts/5 10.170.1.122      Tue Feb 13 09:36:54 +0100 2007
rm           pts/6 rhel4              Tue Feb 13 10:06:56 +0100 2007
henk         **Never logged in**
stefaan      pts/3 pc-sde.telematic Wed Feb 14 12:28:38 +0100 2007
dirk         pts/5 pc-dss.telematic Wed Feb 14 10:03:11 +0100 2007
arsene       **Never logged in**
nicolas      pts/5 pc-dss.telematic Wed Feb 14 12:32:18 +0100 2007
dimitri      pts/5 rhel4          Wed Feb 14 07:57:19 +0100 2007
bashuserm    pts/7 rhel4              Tue Feb 13 10:35:40 +0100 2007
kornuserm    pts/5 rhel4              Tue Feb 13 10:06:17 +0100 2007
[root@rhel4a ~]#
```

/var/log/btmp (lastb)

Există de asemeni comanda **lastb** pentru a afișa fișierul **var/log/btmp**. Acest fișier este adus la zi de către programul login când se tastează parola greșită, astfel el conține încercări de logare eșuate. Multe computere nu vor avea acest fișier, rezultând nejournalizarea încercărilor de logare eșuate.

```
[root@RHEL4b ~]# lastb
lastb: /var/log/btmp: No such file or directory
Perhaps this file was removed by the operator to prevent logging lastb\
info.
[root@RHEL4b ~]#
```

Motivul dat pentru asta este că utilizatorii uneori tastează parola lor din greșeală în loc să intre în sistem, astfel acest fișier cu drepturi de citire pentru toți pune un risc de securitate. Puteți autoriza intrările în sistem login eșuate creînd în mod simplu fișierul. Executînd un **chmod o-r /var/log/btmp** îmbunătățește securitatea.

```
[root@RHEL4b ~]# touch /var/log/btmp
[root@RHEL4b ~]# ll /var/log/btmp
-rw-r--r-- 1 root root 0 Jul 30 06:12 /var/log/btmp
[root@RHEL4b ~]# chmod o-r /var/log/btmp
[root@RHEL4b ~]# lastb
```

```
btmp begins Mon Jul 30 06:12:19 2007
[root@RHEL4b ~]#
```

Accesările eșuate via ssh, rlogin sau su nu sînt înregistrate în /var/log/btmp. Accesările eșuate prin tty sînt înregistrate.

```
[root@RHEL4b ~]# lastb
HalvarFl tty3      Mon Jul 30 07:10 - 07:10 (00:00)
Maria tty1         Mon Jul 30 07:09 - 07:09 (00:00)
Roberto tty1       Mon Jul 30 07:09 - 07:09 (00:00)
```

```
btmp begins Mon Jul 30 07:09:32 2007
[root@RHEL4b ~]#
```

intrare în sistem su și ssh

Depinzînd de distribuție, puteți avea de asemeni fișierul **/var/log/secure** umplut cu mesaje de la facilitățile auth și/sau authpriv syslog. Acest fișier-jurnal va include încercările eșuate su și/sau ssh de a accesa sistemul. Unele distribuții pun acestea în **/var/log/auth.log**, verificați configurația syslog.

```
[root@RHEL4b ~]# cat /var/log/secure
Jul 30 07:09:03 sshd[4387]: Accepted publickey for paul from ::ffff:19\
2.168.1.52 port 33188 ssh2
Jul 30 05:09:03 sshd[4388]: Accepted publickey for paul from ::ffff:19\
2.168.1.52 port 33188 ssh2
Jul 30 07:22:27 sshd[4655]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 05:22:27 sshd[4656]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 07:22:30 sshd[4655]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 05:22:30 sshd[4656]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 07:22:33 sshd[4655]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 05:22:33 sshd[4656]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 08:27:33 sshd[5018]: Invalid user roberto from ::ffff:192.168.1\
.52
Jul 30 06:27:33 sshd[5019]: input_userauth_request: invalid user rober\
to
Jul 30 06:27:33 sshd[5019]: Failed none for invalid user roberto from \
::ffff:192.168.1.52 port 41064 ssh2
Jul 30 06:27:33 sshd[5019]: Failed publickey for invalid user roberto \
from ::ffff:192.168.1.52 port 41064 ssh2
Jul 30 08:27:36 sshd[5018]: Failed password for invalid user roberto f\
rom ::ffff:192.168.1.52 port 41064 ssh2
Jul 30 06:27:36 sshd[5019]: Failed password for invalid user roberto f\
rom ::ffff:192.168.1.52 port 41064 ssh2
[root@RHEL4b ~]#
```

Puteți autoriza asta, cu un fișier special de jurnalizare adăugînd următoarea linie în syslog.conf.

```
auth.*,authpriv.*          /var/log/customsec.log
```

15.2. syslogd

despre syslog

Metoda standard de jurnalizare pe Linux este prin intermediul daemonului **syslogd**. Syslog a fost dezvoltat de **Eric Allman** pentru sendmail, dar a devenit repede un standard printre multe aplicații Unix și a fost mult mai târziu scris ca rfc 3164. Daemonul syslog poate primi mesaje pe **portul** udp **514** pentru multe aplicații (și dispozitive), și poate anexa fișiere-jurnal, imprimantă, să afișeze mesaje pe terminale și să înainteze fișiere-jurnal către alți daemoni syslogd pe alte mașini. Daemonul syslogd este configurat în **/etc/syslog.conf**.

Fiecare linie în fișierul configurației folosește o **facilitate** pentru a determina de unde vine mesajul. De asemeni conține un **nivel** pentru severitatea mesajului, și o **acțiune** pentru a decide ce să facă cu mesajul.

facilități

man syslog.conf va explica diferitele facilități default pentru anumiți daemoni, cum ar fi mail, lpr, news și mesaje kern(el). Facilitatea local0 la local7 poate fi utilizată pentru dispozitive (sau orice dispozitiv network care suportă

syslog). Aici este o listă a tuturor facilităților pentru syslog.conf versiunea 1.3. Cuvîntul-cheie security este învechit.

auth (security)
authpriv
cron
daemon
ftp
kern
lpr mail
mark (internal use only)
news
syslog
user
uucp
local0-7

nivele

Cel mai rău mesaj de securitate pe care poate să-l aibă un mesaj este **emerg** urmat de **alert** și **crit**. Cea mai joasă prioritate ar trebui să le aibă mesajele **info** și **debug**. Specificarea unei severități va jurnaliza de asemeni toate mesajele cu o severitate mai mare. Puteți pune prefix severității cu = pentru a obține doar mesajele care se potrivesc cu acea severitate. Puteți de asemeni specifica **.none** pentru a preveni o acțiune specifică de la orice mesaj pentru o anume facilitate.

Aici este o listă a tuturor nivelelor, în ordine ascendentă. Cuvintele-cheie warn, error și panic sînt învechite.

debug
info
notice
warning (warn)
err (error)
crit
alert
emerg (panic)

acțiuni

Acțiunea default este să trimită un mesaj utilizatorului listat ca acțiune. Când acțiunea are ca prefix un / atunci syslog va trimite mesajul într-un fișier (care poate fi un fișier regular, dar de asemeni o imprimantă sau terminal). Prefixul cu semnul @ va trimite mesajul la un alt server syslog. Aici este o listă a tuturor acțiunilor posibile.

root,user1	listă a utilizatorilor, separați de virgule
*	mesaj către toți utilizatorii intrați în sistem
/	fișier (poate fi o imprimantă, o consolă, un tty, ...)
-/	fișier, dar nu sincronizează după fiecare scriere
	conductă
@	alt hostname syslog

În plus, puteți pune prefix acțiunilor cu a - pentru a omite sincronizarea fișierului după fiecare jurnalizare.

configurație

Mai jos este un exemplu de configurație a mesajelor speciale local4 în **/etc/syslog.conf**.

```
local4.crit      /var/log/critandabove
local4.=crit     /var/log/onlycrit
local4.*         /var/log/alllocal4
```

Nu uitați să reporniți serverul.

```
[root@rhel4a ~]# /etc/init.d/syslog restart
Shutting down kernel logger:      [OK]
Shutting down system logger:      [OK]
Starting system logger:           [OK]
Starting kernel logger:           [OK]
[root@rhel4a ~]#
```

15.3. logger

Comanda **logger** poate fi utilizată pentru a genera mesaje test syslog. Puteți de asemenea să o utilizați în scripturi. Un exemplu al testării syslogd cu utilitarul **logger**.

```
[root@rhel4a~]# logger -p local4.debug "l4 debug"
[root@rhel4a~]# logger -p local4.crit "l4 crit"
[root@rhel4a~]# logger -p local4.emerg "l4 emerg"
[root@rhel4a~]#
```

Rezultatul testelor cu **logger**.

```
[root@rhel4a ~]# cat /var/log/critandabove
Feb 14 19:55:19 rhel4a paul: l4 crit
Feb 14 19:55:28 rhel4a paul: l4 emerg
[root@rhel4a ~]# cat /var/log/onlycrit
Feb 14 19:55:19 rhel4a paul: l4 crit
[root@rhel4a ~]# cat /var/log/alllocal4
Feb 14 19:55:11 rhel4a paul: l4 debug
Feb 14 19:55:19 rhel4a paul: l4 crit
Feb 14 19:55:28 rhel4a paul: l4 emerg
[root@rhel4a ~]#
```

15.4. observarea fișierelor-jurnal

Ați putea dori să folosiți comanda **tail -f** pentru a privi la ultimele linii a unui fișier-jurnal. Opțiunea **-f** va afișa în mod dinamic liniile care sînt adăugate jurnalului.

```
paul@ubu1010:~$ tail -f /var/log/udev
SEQNUM=1741
SOUND_INITIALIZED=1
ID_VENDOR_FROM_DATABASE=nVidia Corporation
ID_MODEL_FROM_DATABASE=MCP79 High Definition Audio
ID_BUS=pci
ID_VENDOR_ID=0x10de
ID_MODEL_ID=0x0ac0
ID_PATH=pci-0000:00:08.0
SOUND_FORM_FACTOR=internal
```

Puteți să repetați în mod automat comenzi punându-le înainte comanda **watch**. Când executăm următoarea comandă:

```
[root@rhel6 ~]# watch who
```

va apărea pe ecran ceva asemănător, repetînd ieșirea comenzii **who** la fiecare două secunde:

```
Every 2.0s: who      Sun Jul 17 15:31:03 2011

root      tty1      2011-07-17   13:28
paul      pts/0      2011-07-17   13:31  (192.168.1.30)
paul      pts/1      2011-07-17   15:19  (192.168.1.30)
```

15.5. fișiere-jurnal rotație

Multe din fișierele-jurnal cresc mereu în mărime. Pentru a le păstra în limite, ați putea dori să folosiți **logrotate** pentru a rota, comprima, șterge și trimite prin e-mail fișiere jurnal. Mai multe informații despre comanda **logrotate** sînt în **/etc/logrotate.conf**. Configurări individuale pot fi găsite în directorul **/etc/logrotate.d/**.

În această captură de ecran e fișierul de configurație pentru fișierele-jurnal din **aptitude** pentru a configura rotările lunare, păstrînd ultimele șase și comprimînd vechile fișiere-jurnal.

```
paul@ubu1010:/var/log$ cat /etc/logrotate.d/aptitude
/var/log/aptitude {
rotate 6
monthly
compress
missingok
notifempty
}
```

Și această captură de ecran este rezultatul configurării de deasupra, pentru fișierul-jurnal din **aptitude**.

```
paul@ubu1010:/var/log$ ls -l /var/log/aptitude*
-rw-r--r-- 1 root root 18298 2011-07-17 13:32 /var/log/aptitude
-rw-r--r-- 1 root root 8163   2011-07-01 01:43 /var/log/aptitude.1.gz
-rw-r--r-- 1 root root 8163   2011-06-01 01:43 /var/log/aptitude.2.gz
-rw-r--r-- 1 root root 8163   2011-05-01 01:43 /var/log/aptitude.3.gz
```

15.6. practică: jurnalizare

1. Afișați fișierul /var/run/utmp cu comanda corectă (nu cu cat sau vi).
2. Afișați fișierul /var/log/wtmp.
3. Folosiți comenzile lastlog și lastb, înțelegeți diferența.
4. Examinați syslog pentru a găsi locația fișierului-jurnal conținând accesările ssh eșuate.
5. Configurați syslog pentru a pune local4.error și mesajele de deasupra în /var/log/l4e.log și local4.info doar în /var/log/l4i.log. Testați dacă funcționează cu utilitarul logger!
6. Configurați /var/log/Mysu.log, toate mesajele su la root ar trebui să se ducă în acel fișier-jurnal. Testați dacă funcționează!
7. Trimiteți mesaje local5 serverului syslog vecinului. Testați dacă funcționează.
8. Scrieți un script care execută logger către local4 în fiecare 15 secunde (mesaj diferit). Folosiți tail -f și priviți fișierele-jurnal pe local4.

15.7. soluție: jurnalizare

1. Afișați fișierul /var/run/utmp cu comanda corectă (nu cu cat sau vi).

```
who
```

2. Afișați fișierul /var/log/wtmp.

```
last
```

3. Folosiți comenzile lastlog și lastb, înțelegeți diferența.

lastlog: când utilizatorii au intrat ultima dată în sistem

lastb: încercări eșuate (bad) login

4. Examinați syslog pentru a găsi locația fișierului-jurnal conținând accesările sistemului ssh eșuate.

```
root@rhel53 ~# grep authpriv /etc/syslog.conf
authpriv.*      /var/log/secure
```

Debian/Ubuntu: /var/log/auth.log

Ubuntu 9.10 și Debian Lenny au trecut la utilizarea rsyslog.

```
root@ubuntu910:~# grep authpriv /etc/rsyslog.d/50-default.conf
auth,authpriv.*  /var/log/auth.log
```

```
root@deb503:~# grep authpriv /etc/rsyslog.conf
auth,authpriv.*  /var/log/auth.log
```

5. Configurați syslog pentru a pune local4.error și mesajele de deasupra în /var/log/l4e.log și local4.info doar în /var/log/l4i.log. Testați dacă funcționează cu utilitarul logger!

```
echo local4.error /var/log/l4e.log >> /etc/syslog.conf
```

```
echo local4.=info /var/log/l4i.log >> /etc/syslog.conf
```

```
/etc/init.d/syslog restart
```

```
logger -p local4.error "l4 error test"
```

```
logger -p local4.alert "l4 alert test"
```

```
logger -p local4.info "l4 info test"
```

```
cat /var/log/l4e.log
```

```
cat /var/log/l4i.log
```

6. Configurați /var/log/Mysu.log, toate mesajele su la root ar trebui să se ducă în acel fișier-jurnal. Testați dacă funcționează!

```
echo authpriv.*  /var/log/Mysu.log >> /etc/syslog.conf
```

Această comandă va jurnaliza mai mult decât folosirea su.

7. Trimiteți mesaje local5 serverului syslog vecinului. Testați dacă funcționează.

Pe RHEL5, editați /etc/sysconfig/syslog pentru a autoriza ascultarea remote pe server.

Pe Debian/Ubuntu editați /etc/default/syslog sau /etc/default/rsyslog.

pe client: logger -p local5.info "test local5 to neighbour"

8. Scrieți un script care execută logger către local4 în fiecare 15 secunde (mesaj diferit). Folosiți tail -f și priviți fișierele-jurnal pe local4.

```
root@rhel53 scripts# cat logloop
#!/bin/bash

for i in `seq 1 10`
do
logger -p local4.info "local4.info test number $i"
sleep 15
done

root@rhel53 scripts# chmod +x logloop
root@rhel53 scripts# ./logloop &
[1] 8264
root@rhel53 scripts# tail -f /var/log/local4.all.log
Mar 28 13:13:36 rhel53 root: local4.info test number 1
Mar 28 13:13:51 rhel53 root: local4.info test number 2
. . .
```

Capitolul 16. management memorie

Conținut

16.1. afișare memorie și cache.	.148
16.2. management spațiu swap.	.148
16.3. monitorizare memorie cu vmstat.	.150
16.4. practică: memorie.	.151
16.5. soluție: memorie.	.152

Acest capitol vă va informa cum să faceți managementul memoriei RAM și cache (memoriei tampon).

Vom începe cu unele utilitare simple pentru a afișa informație despre memorie: **free -om**, **top** și **cat /proc/meminfo**.

Vom continua cu managementul spațiului swap, folosind termeni ca **swapping**, **paging** și **memorie virtuală**.

Ultima parte este despre folosirea **vmstat** pentru a monitoriza utilizarea swap.

16.1. afișare memorie și cache

/proc/meminfo

Afișarea **/proc/meminfo** vă va informa despre memorie pe computerul Linux.

```
paul@ubu1010:~$ cat /proc/meminfo
MemTotal:      3830176 kB
MemFree:       244060 kB
Buffers:       41020 kB
Cached:        2035292 kB
SwapCached:    9892 kB
...
```

Prima linie conține suma totală a memoriei fizice RAM, a doua linie este memoria RAM neutilizată. Memoria intermediară (**buffers**) este RAM folosit pentru memorarea intermediară a fișierelor (buffering), **cached** (tampon) este suma RAM folosită pentru stocare și **SwapCached** (swap tampon) este suma swap utilizată ca memorie tampon. Fișierul ne dă mai multă informație înafara scopului acestui curs.

free

Utilitarul **free** poate afișa informația dată de **/proc/meminfo** într-un format mai ușor de citit. Exemplul de mai jos afișează pe scurt informație memorie în megabiți.

```
paul@ubu1010:~$ free -om
              total        used        free      shared    buffers      cached
Mem:          3740         3519          221           0          42        1994
Swap:         6234           82        6152
```

top

Utilitarul **top** este deseori folosit pentru a privi la procesele care consumă cel mai mult cpu, dar afișează de asemeni informație memorie pe linia patru și cinci (care poate fi declanșată apăsînd m).

Mai jos este o captură de ecran top de pe același ubu1010 de mai sus.

```
top - 10:44:34 up 16 days, 9:56, 6 users, load average: 0.13, 0.09, 0.12
Tasks: 166 total, 1 running, 165 sleeping, 0 stopped, 0 zombie
Cpu(s): 5.1%us, 4.6%sy, 0.6%ni, 88.7%id, 0.8%wa, 0.0%hi, 0.3%si, 0.0%st
Mem:   3830176k total, 3613720k used, 216456k free, 45452k buffers
Swap: 6384636k total, 84988k used, 6299648k free, 2050948k cached
```

16.2. management spațiu swap

despre spațiu swap

Cînd sistemul de operare are nevoie de mai multă memorie decît cea prezentă în mod fizic în RAM, el poate să utilizeze **spațiu swap**. Spațiu swap este localizat pe memorie lentă și ieftină. Notați că, deși hard-disk-urile sînt în mod obișnuit folosite pentru spațiu swap, timpul lor de acces este de o sută de mii de ori mai lent.

Spațiul swap poate fi un fișier, o partiție, sau o combinație de fișiere și partiții. Puteți vedea spațiul swap cu comanda **free**, sau cu **cat/proc/swaps**.

```
paul@ubu1010:~$ free -o | grep -v Mem
              total        used        free      shared    buffers     cached
Swap:      6384636      84988      6299648
paul@ubu1010:~$ cat /proc/swaps
Filename                Type              Size      Used      Priority
/dev/sda3                partition        6384636   84988     -1
```

Suma de spațiu swap de care aveți nevoie depinde în majoritate de serviciile pe care computerul le dă.

crearea unei partiții swap

Puteți activa sau deactiva spațiul swap cu comenzile **swapon** și **swapoff**. Spațiu swap nou poate fi creat cu comanda **mkswap**. Captura de ecran de mai jos arată crearea și activarea unei partiții swap.

```
root@RHELv4u4:~# fdisk -l 2> /dev/null | grep hda
Disk /dev/hda: 536 MB, 536870912 bytes
/dev/hda1                1            1040      524128+   83    Linux
root@RHELv4u4:~# mkswap /dev/hda1
Setting up swapspace version 1, size = 536702 kB
root@RHELv4u4:~# swapon /dev/hda1
```

Acum puteți vedea că **/proc/swaps** afișează toate spațiile swap separat, pe când comanda **free -om** face un sumar lizibil.

```
root@RHELv4u4:~# cat /proc/swaps
Filename                Type              Size      Used      Priority
/dev/mapper/VolGroup00-LogVol01  partition        1048568    0         -1
/dev/hda1                partition        524120     0         -2
root@RHELv4u4:~# free -om
              total        used        free      shared    buffers     cached
Mem:          249          245           4           0          125          54
Swap:         1535           0         1535
```

crearea unui fișier swap

Aici este încă un exemplu care vă arată cum să creați un **fișier swap**. Pe Solaris puteți folosi **mkfile** în loc de **dd**.

```
root@RHELv4u4:~# dd if=/dev/zero of=/smallswapfile bs=1024 count=4096
4096+0 records in
4096+0 records out
root@RHELv4u4:~# mkswap /smallswapfile
Setting up swapspace version 1, size = 4190 kB
root@RHELv4u4:~# swapon /smallswapfile
root@RHELv4u4:~# cat /proc/swaps
Filename                Type              Size      Used      Priority
/dev/mapper/VolGroup00-LogVol01  partition        1048568    0         -1
/dev/hda1                partition        524120     0         -2
/smallswapfile           file             4088       0         -3
```

spațiu swap în /etc/fstab

Dacă doriți ca aceste swapuri să fie permanente, atunci nu uitați să le adăugați în **/etc/fstab**. Liniile în **/etc/fstab** vor fi similare exemplului care urmează.

```

/dev/hda1      swap      swap      defaults      0 0
/swapfile      swap      swap      defaults      0 0

```

16.3 monitorizare memorie cu vmstat

Puteți găsi informație despre **utilizarea swap** folosind **vmstat**.
Mai jos este un exemplu simplu **vmstat** afișând informație în megabiți.

```

paul@ubu1010:~$ vmstat -S m
procs -----memory----- ---swap-- ----io---- -system- ----cpu----
r b swpd      free buff  cache si  so  bi   bo    in cs  us sy id wa
0 0   87       225  46   2097  0  0   2    5    14  8   6  5  89  1

```

Mai jos este un exemplu **vmstat** când (într-un alt terminal) root lansează un **find** /. Această comandă de asemeni generează mult i/o din disk (bi și bo sînt disk-uri bloc intrare și ieșire). Nu este nevoie de nici un swapping aici.

```

paul@ubu1010:~$ vmstat 2 100
procs -----memory----- ---swap-- ----io---- -system-- ----cpu-----
r b      swpd      free buff  cache si  so   bi   bo    in cs  us sy id wa
0 0      84984 1999436 53416 269536 0  0    2    5     2 10   6  5  89  1
0 0      84984 1999428 53416 269564 0  0    0    0   1713 2748  4  4  92  0
0 0      84984 1999552 53416 269564 0  0    0    0   1672 1838  4  6  90  0
0 0      84984 1999552 53424 269560 0  0    0   14   1587 2526  5  7  87  2
0 0      84984 1999180 53424 269580 0  0    0  100   1748 2193  4  6  91  0
1 0      84984 1997800 54508 269760 0  0   610    0   1836 3890 17 10  68  4
1 0      84984 1994620 55040 269748 0  0   250  168   1724 4365 19 17  56  9
0 1      84984 1978508 55292 269704 0  0   126    0   1957 2897 19 18  58  4
0 0      84984 1974608 58964 269784 0  0  1826  478   2605 4355  7  7  44 41
0 2      84984 1971260 62268 269728 0  0  1634  756   2257 3865  7  7  47 39

```

Mai jos este un exemplu **vmstat** când executăm (pe RHEL6) un program simplu de scurgere de memorie. Acum vedeți multă memorie care este swap (este în troc).

```
[paul@rhel6c ~]$ vmstat 2 100
```

```

procs -----memory----- ---swap-- ----io---- --system-- ----cpu-----
r b      swpd      free buff  cache si  so   bi   bo    in cs  us sy id wa st
0 3     245208  5280   232  1916  261    0    0    42   27  21   0  1  98  1  0
0 2     263372  4800    72   908 143840 128    0  1138  462  191  2 10  0  88  0
1 3     350672  4792    56   992 169280 256    0  1092  360  142  1 13  0  86  0
1 4     449584  4788    56  1024 95880  64    0   606  471  191  2 13  0  85  0
0 4     471968  4828    56  1140 44832  80    0   390  235   90  2 12  0  87  0
3 5     505960  4764    56  1136 68008  16    0   538  286  109  1 12  0  87  0

```

Codul de mai jos a fost folosit pentru a simula o scurgere de memorie (și a forța swapping-ul). Acest cod a fost găsit pe wikipedia fără autor.

```

paul@mac:~$ cat memleak.c
#include <stdlib.h>
int main(void)
{
while (malloc(50));
return 0;
}

```

16.4. practică: memorie

1. Folosiți **dmesg** pentru a găsi suma totală de memorie din computer.
2. Folosiți **free** pentru a afișa utilizarea memoriei în kilobiți (apoi în megabiți).
3. Pe o mașină virtuală, creați o partiție swap (s-ar putea să aveți nevoie de încă un disk virtual pentru asta).
4. Adăugați un fișier swap de 20 de megabiți sistemului.
5. Puneți toate spațiile swap în **/etc/fstab** și activați-le. Testați cu un reebot ca ele să fie montate.
6. Folosiți **free** pentru a verifica utilizarea swapului curent.
7. (opțional) Afișați utilizarea swap cu **vmstat** și **free -s** în timpul unei scurgeri de memorie.

16.5. soluție: memorie

1. Folosiți **dmesg** pentru a găsi suma totală de memorie din computer.

```
dmesg | grep Memory
```

2. Folosiți **free** pentru a afișa utilizarea memoriei în kilobiți (apoi în megabiți).

```
free ; free -m
```

3. Pe o mașină virtuală, creați o partiție swap (s-ar putea să aveți nevoie de încă un disk virtual pentru asta).

```
mkswap /dev/sdd1 ; swapon /dev/sdd1
```

4. Adăugați un fișier swap de 20 de megabiți sistemului.

```
dd if=/dev/zero of=/swapfile20mb bs=1024 count=20000  
mkswap /swapfile20mb  
swapon /swapfile20mb
```

5. Puneți toate spațiile swap în **/etc/fstab** și activați-le. Testați cu un reebot ca ele să fie montate.

```
root@computer# tail -2 /etc/fstab  
/dev/sdd1          swap swap defaults 0 0  
/swapfile20mb     swap swap defaults 0 0
```

6. Folosiți **free** pentru a verifica utilizarea swapului curent.

```
free -om
```

7. (opțional) Afișați utilizarea swap cu **vmstat** și **free -s** în timpul unei scurgeri de memorie.

Capitolul 17. management pachet

Conținut

17.1.	terminologie pachet.	.154
17.2.	management pachet .deb.	156
17.3.	apt-get.	.157
17.4.	aptitude.	160
17.5.	apt.	.161
17.6.	rpm.	.162
17.7.	yum.	.163
17.8.	alien.	.168
17.9.	download software din afara repozitoriului.	169
17.10.	compilare software.	169
17.11.	practică: management pachet.	.170
17.12.	soluție: management pachet.	171

Cele mai multe distribuții Linux au un sistem de **management al pachetului** cu **repozitoare** online care conțin mii de pachete. Asta face foarte ușor să se instaleze și să se șteargă aplicații, componente ale sistemului de operare, documentație și mult mai mult.

Discutăm despre cele două dintre cele mai folosite formate de pachete **.rpm** și **.deb** și utilitățile lor respective. Discutăm de asemenea pe scurt opțiunea obținerii software-ului din afara **repozitoriului**.

17.1. terminologie pachet

repozitoriu

Mult software și documentație pentru distribuția Linux sînt disponibile ca **pachete** în unul sau mai multe **repozitoare** distribuite central. Aceste **pachete** într-un astfel de **repozitoriu** sînt testate și foarte ușor de instalat (sau de șters) cu un installer grafic sau pe linia de comandă.

pachete .deb

Debian, Ubuntu, Mint și toate sistemele de operare derivate din Debian și Ubuntu folosesc pachete **.deb**. Pentru a aranja software pe aceste sisteme, puteți folosi **aptitude** sau **apt-get**, amîndouă dintre aceste utilitare sînt un front-end pentru **dpkg**.

pachete .rpm

Red Hat, Fedora, CentOS, OpenSUSE, Mandriva, Red Flag și alte sisteme folosesc pachete **.rpm**. Utilitarele pentru a face managementul pachetelor software pe aceste sisteme sînt **yum** și **rpm**.

dependențe

Unele pachete au nevoie de alte pachete pentru a funcționa. Utilitare ca **apt-get**, **aptitude** și **yum** vor instala toate **dependențele** de care aveți nevoie. Cînd folosiți **dpkg** sau **rpm**, sau cînd construim din **sursă**, trebuie să instalați dependențele singuri.

sursă deschisă

Aceste repozitoare conțin mult **software sursă deschisă** independentă. Deseori codul sursă este făcut special pentru a se integra mai bine cu distribuția. Cele mai multe distribuții de asemeni oferă acest cod sursă modificat ca un **pachet** în una sau mai multe **repozitoare sursă**.

Sînteți liber să mergeți la website-ul proiectului însuși (samba.org, apache.org, github.com ...) și să downloadați codul sursă **vanilla** (= fără schimbările făcute special pentru distribuție).

GUI software management

Utilizatorii finali au cîteva aplicații grafice disponibile via desktop (căutați 'add/remove software' sau similar).

Mai jos este un screenshot a Ubuntu Software Center rulînd pe Ubuntu 12.04. Utilitarele grafice nu sînt discutate în această carte.



17.2. management pachet .deb

despre deb

Cei mai mulți oameni folosesc **aptitude** sau **apt-get** pentru a face managementul familiei de distribuții Linux Debian/Ubuntu. Ambele sînt un front-end pentru **dpkg** și sînt ele însele un back-end pentru **synaptic** și alte utilitare grafice.

dpkg -l

Utilitarul low-level pentru a lucra cu pachete **.deb** este **dpkg**. Aici vedeți cum să obțineți o listă a tuturor pachetelor instalate pe un server Debian.

```
root@debian6:~# dpkg -l | wc -l
265
```

Comparați asta cu aceeași listă pe un computer desktop Ubuntu.

```
root@ubu1204~# dpkg -l | wc -l
2527
```

dpkg -l \$package

Aici este un exemplu despre cum să obținem informație despre un pachet individual. **ii** la început înseamnă că pachetul este instalat.

```
root@debian6:~# dpkg -l rsync | tail -1 | tr -s ' '
ii rsync 3.0.7-2 fast remote file copy program (like rcp)
```

dpkg -S

Puteți afla pachetul care a instalat un anumit fișier pe computer cu **dpkg -S**. Acest exemplu arată cum să găsim pachetul pentru trei fișiere pe un server Debian tipic.

```
root@debian6:~# dpkg -S /usr/share/doc/tmux/ /etc/ssh/ssh_config /sbin/ifconfig
tmux: /usr/share/doc/tmux/
openssh-client: /etc/ssh/ssh_config
net-tools: /sbin/ifconfig
```

dpkg -L

Puteți de asemeni găsi o listă a tuturor fișierelor care sînt instalate de către un anumit program. Mai jos este o listă pentru pachetul **tmux**.

```

root@debian6:~# dpkg -L tmux
/.
/etc
/etc/init.d
/etc/init.d/tmux-cleanup
/usr
/usr/share
/usr/share/lintian
/usr/share/lintian/overrides
/usr/share/lintian/overrides/tmux
/usr/share/doc
/usr/share/doc/tmux
/usr/share/doc/tmux/TODO.gz
/usr/share/doc/tmux/FAQ.gz
/usr/share/doc/tmux/changelog.Debian.gz
/usr/share/doc/tmux/NEWS.Debian.gz
/usr/share/doc/tmux/changelog.gz
/usr/share/doc/tmux/copyright
/usr/share/doc/tmux/examples
/usr/share/doc/tmux/examples/tmux.vim.gz
/usr/share/doc/tmux/examples/h-boetes.conf
/usr/share/doc/tmux/examples/n-marriott.conf
/usr/share/doc/tmux/examples/screen-keys.conf
/usr/share/doc/tmux/examples/t-williams.conf
/usr/share/doc/tmux/examples/vim-keys.conf
/usr/share/doc/tmux/NOTES
/usr/share/man
/usr/share/man/man1
/usr/share/man/man1/tmux.1.gz
/usr/bin
/usr/bin/tmux

```

dpkg

Ați putea folosi **dpkg -i** pentru a instala un pachet și **dpkg -r** pentru a șterge un pachet, dar ar trebui să țineți seama de dependențe manual. Folosind **apt-get** sau **aptitude** este mult mai ușor.

17.3. apt-get

Debian a utilizat **apt-get** pentru a face managementul pachetelor din 1998. Astăzi **Debian** și multe distribuții bazate pe **Debian** susțin încă activ **apt-get**, deși unii experți reclamă că **aptitude** este mai bun în operarea dependențelor decât **apt-get**.

Ambele comenzi folosesc aceleași fișiere de configurație și pot fi folosite în mod alternativ; oricând vedeți **apt-get** în documentație, simțiți-vă liber să tastați **aptitude**.

Vom începe cu **apt-get** și vom discuta **aptitude** în secțiunea următoare.

apt-get update

Când tastați **apt-get update** downloadați numele, versiunile și scurte descrieri a tuturor pachetelor disponibile pe toate repozițiile configurate pentru sistem.

În exemplul de mai jos puteți vedea unele repozitoare la url-ul **be.archive.ubuntu.com** pentru că acest computer a fost instalat în Belgia. Acest url poate fi diferit.

```
root@ubu1204~# apt-get update
Ign http://be.archive.ubuntu.com precise InRelease
Ign http://extras.ubuntu.com precise InRelease
Ign http://security.ubuntu.com precise-security InRelease
Ign http://archive.canonical.com precise InRelease
Ign http://be.archive.ubuntu.com precise-updates InRelease
...
Hit http://be.archive.ubuntu.com precise-backports/main Translation-en
Hit http://be.archive.ubuntu.com precise-backports/multiverse Translation-en
Hit http://be.archive.ubuntu.com precise-backports/restricted Translation-en
Hit http://be.archive.ubuntu.com precise-backports/universe Translation-en
Fetched 13.7 MB in 8s (1682 kB/s)
Reading package lists... Done
root@mac~#
```

Executați **apt-get update** de fiecare dată înainte de a face alte operații pachet.

apt-get upgrade

Una dintre cele mai frumoase posibilități a **apt-get** este că permite un update securizat a **întregului software instalat în mod curent** pe computer cu numai o **singură** comandă.

```
root@debian6:~# apt-get upgrade
Reading package lists... Done
Building dependency tree
Reading state information... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@debian6:~#
```

Screenshot-ul de deasupra arată că întregul software este actualizat la ultima versiune disponibilă pentru distribuția mea.

apt-get clean

apt-get păstrează o copie a pachetelor downloadate în **/var/cache/apt/archives**, după cum poate fi văzut în această captură de ecran.

```
root@ubu1204~# ls /var/cache/apt/archives/ | head
accountsservice_0.6.15-2ubuntu9.4_i386.deb
apport_2.0.1-0ubuntu14_all.deb
apport-gtk_2.0.1-0ubuntu14_all.deb
apt_0.8.16~exp12ubuntu10.3_i386.deb
apt-transport-https_0.8.16~exp12ubuntu10.3_i386.deb
apt-utils_0.8.16~exp12ubuntu10.3_i386.deb
bind9-host_1%3a9.8.1.dfsg.P1-4ubuntu0.4_i386.deb
chromium-browser_20.0.1132.47~r144678-0ubuntu0.12.04.1_i386.deb
chromium-browser-l10n_20.0.1132.47~r144678-0ubuntu0.12.04.1_all.deb
chromium-codecs-ffmpeg_20.0.1132.47~r144678-0ubuntu0.12.04.1_i386.deb
```

Tastînd **apt-get-clean** șterge toate fișierele .deb din acel director.

```
root@ubu1204~# apt-get clean
root@ubu1204~# ls /var/cache/apt/archives/*.deb
ls: cannot access /var/cache/apt/archives/*.deb: No such file or directory
```

apt-cache search

Folosiți **apt-cache search** pentru a căuta disponibilitatea unui pachet. Aici căutăm **rsync**.

```
root@ubu1204~# apt-cache search rsync | grep ^rsync
rsync - fast, versatile, remote (and local) file-copying tool
rsyncrypto - rsync friendly encryption
```

apt-get install

Puteți instala una sau mai multe aplicații adăugînd numele lor după **apt-get install**. Captura de ecran arată cum să instalăm pachetul **rsync**.

```
root@ubu1204~# apt-get install rsync
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  rsync
0 upgraded, 1 newly installed, 0 to remove and 8 not upgraded.
Need to get 299 kB of archives.
After this operation, 634 kB of additional disk space will be used.
Get:1 http://be.archive.ubuntu.com/ubuntu/ precise/main rsync i386 3.0.9-1ubuntu1 [299 k
Fetched 299 kB in 0s (740 kB/s)
Selecting previously unselected package rsync.
(Reading database ... 323649 files and directories currently installed.)
Unpacking rsync (from .../rsync_3.0.9-1ubuntu1_i386.deb) ...
Processing triggers for man-db ...
Processing triggers for ureadahead ...
Setting up rsync (3.0.9-1ubuntu1) ...
  Removing any system startup links for /etc/init.d/rsync ...
root@ubu1204~#
```

apt-get remove

Puteți șterge una sau mai multe aplicații adăugînd numele lor după **apt-get remove**. Captura de ecran arată cum să ștergem pachetul **rsync**.

```

root@ubul204~# apt-get remove rsync
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
  rsync ubuntu-standard
0 upgraded, 0 newly installed, 2 to remove and 8 not upgraded.
After this operation, 692 kB disk space will be freed.
Do you want to continue [Y/n]?
(Reading database ... 323681 files and directories currently installed.)
Removing ubuntu-standard ...
Removing rsync ...
* Stopping rsync daemon rsync
Processing triggers for ureadahead ...
Processing triggers for man-db ...
root@ubul204~#

```

Observați totuși că unele informații de configurație nu sînt șterse.

```

root@ubul204~# dpkg -l rsync | tail -1 | tr -s ' '
rc rsync 3.0.9-1ubuntu1 fast, versatile, remote (and local) file-copying tool

```

apt-get purge

Puteți curăța una sau mai multe aplicații adăugînd numele lor după **apt-get purge**. Curățarea de asemeni va șterge toate fișierele de configurație existente puse în legătură cu acea aplicație. Captura de ecran arată cum să curățăm pachetul **rsync**.

```

root@ubul204~# apt-get purge rsync
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
  rsync*
0 upgraded, 0 newly installed, 1 to remove and 8 not upgraded.
After this operation, 0 B of additional disk space will be used.
Do you want to continue [Y/n]?
(Reading database ... 323651 files and directories currently installed.)
Removing rsync ...
Purging configuration files for rsync ...
Processing triggers for ureadahead ...
root@ubul204~#

```

Observați că **dpkg** nu are nici o informație despre un pachet curățat, cu excepția că el este dezinstalat și nici o configurație nu este lăsată pe sistem.

```

root@ubul204~# dpkg -l rsync | tail -1 | tr -s ' '
rsync <none> (no description available)

```

17.4. aptitude

Cei mai mulți oameni folosesc **aptitude** pentru managementul pachetelor pe sistemele Debian, Mint și Ubuntu.

Pentru a sincroniza repozitoarele.

`aptitude update`

Pentru a face patch și a aduce la zi întregul software la cea mai nouă versiune pe Debian.

`aptitude upgrade`

Pentru a face patch și a îmbunătăți întregul software la cea mai nouă versiune pe Ubuntu și Mint.

`aptitude safe-upgrade`

Pentru a instala o aplicație cu toate dependențele.

`aptitude install $package`

Pentru a căuta repozitoarele pentru aplicații care conțin un anumit șir în numele sau descrierea lor.

`aptitude search $string`

Pentru a șterge o aplicație.

`aptitude remove $package`

Pentru a șterge o aplicație și toate fișierele de configurare.

`aptitude purge $package`

17.5. apt

Și **apt-get** și **aptitude** folosesc aceeași informație de configurare în **/etc/apt/**. Astfel că adăugînd un repozitoriu pentru una dintre ele, în mod automat le va adăuga pentru ambele.

/etc/apt/sources.list

Lista de resurse folosită de **apt-get** și **aptitude** este localizată în **/etc/apt/sources.list**. Acest fișier conține o listă de surse http sau ftp de unde pachetele pentru distribuție pot fi downloadate.

Astfel arată această listă pe serverul meu Debian.

```
root@debian6:~# cat /etc/apt/sources.list
deb http://ftp.be.debian.org/debian/ squeeze main
deb-src http://ftp.be.debian.org/debian/ squeeze main

deb http://security.debian.org/ squeeze/updates main
deb-src http://security.debian.org/ squeeze/updates main

# squeeze-updates, previously known as 'volatile'
deb http://ftp.be.debian.org/debian/ squeeze-updates main
deb-src http://ftp.be.debian.org/debian/ squeeze-updates main
```

Pe Ubuntu meu există de patru ori mai multe repozitoare online în folosire.

```
root@ubu1204~# wc -l /etc/apt/sources.list
63 /etc/apt/sources.list
```

Există mult mai mult de învățat despre **apt**, de explorat comenzi ca **add-apt-repository**, **apt-key** și **apropos apt**.

17.6. rpm

despre rpm

Red Hat package manager poate fi folosit pe linia de comandă cu **rpm** sau pe calea grafică mergînd la Applications- -System Settings- -Add/Remove Applications. Tastați **rpm --help** pentru a vedea unele dintre opțiuni.

Software distribuit în formatul **rpm** va fi numit **foo-version.platform.rpm**.

rpm -qa

Pentru a obține o listă a întregului software instalat folosiți comanda **rpm -qa**.

```
[root@RHEL52 ~]# rpm -qa | grep samba
system-config-samba-1.2.39-1.el5
samba-3.0.28-1.el5_2.1
samba-client-3.0.28-1.el5_2.1
samba-common-3.0.28-1.el5_2.1
```

rpm -q

Pentru a verifica dacă un pachet este instalat, folosiți **rpm -q**.

```
root@RHELv4u4:~# rpm -q gcc
gcc-3.4.6-3
root@RHELv4u4:~# rpm -q laika
package laika is not installed
```

rpm -q --redhatprovides

Pentru a verifica dacă un pachet este dat de Red Hat, folosiți opțiunea **--redhatprovides**.

```
root@RHELv4u4:~# rpm -q --redhatprovides bash
bash-3.0-19.3
root@RHELv4u4:~# rpm -q --redhatprovides gcc
gcc-3.4.6-3
root@RHELv4u4:~# rpm -q --redhatprovides laika
no package provides laika
```

rpm -Uvh

Pentru a instala sau a aduce la zi un pachet, folosiți întrerupătorii **-Uvh**. Întrerupătorul **-U** este același ca **-i** pentru instalare, cu excepția că versiunile mai vechi sînt șterse. Întrerupătorii **-vh** sînt pentru o ieșire mai frumoasă.

```
root@RHELv4u4:~# rpm -Uvh gcc-3.4.6-3
```

rpm -e

Pentru a șterge un pachet, folosiți întrerupătorul **-e**.

```
root@RHELv4u4:~# rpm -e gcc-3.4.6-3
```

rpm -e verifică dependențele, și astfel vă va preveni să nu ștergeți accidental pachete care sînt necesare altor pachete.

```
[root@RHEL52 ~]# rpm -e gcc-4.1.2-42.el5
error: Failed dependencies:
gcc = 4.1.2-42.el5 is needed by (installed) gcc-c++-4.1.2-42.el5.i386
gcc = 4.1.2-42.el5 is needed by (installed) gcc-gfortran-4.1.2-42.el5.i386
gcc is needed by (installed) systemtap-0.6.2-1.el5_2.2.i386
```

/var/lib/rpm

Baza de date **rpm** este localizată în **/var/lib/rpm**. Această bază de date conține toate metainformațiile despre pachetele care sînt instalate (via rpm). Păstrează toate fișierele, ceea ce autorizează ștergerea de software în mod complet.

rpm2cpio

Putem folosi **rpm2cpio** pentru a converti un **rpm** într-o arhivă **cpio**.

```
[root@RHEL53 ~]# file kernel.src.rpm
kernel.src.rpm: RPM v3 src PowerPC kernel-2.6.18-92.1.13.el5
[root@RHEL53 ~]# rpm2cpio kernel.src.rpm > kernel.cpio
[root@RHEL53 ~]# file kernel.cpio
kernel.cpio: ASCII cpio archive (SVR4 with no CRC)
```

Dar de ce ați vrea să faceți asta?

Poate doar pentru a vedea o listă de fișiere în fișierul **rpm**.

```
[root@RHEL53 ~]# rpm2cpio kernel.src.rpm | cpio -t | head -5
COPYING.modules
Config.mk
Module.kabi_i686
Module.kabi_i686PAE
Module.kabi_i686xen
```

Sau pentru a extrage un fișier dintr-un pachet **rpm**.

```
[root@RHEL53 ~]# rpm2cpio kernel.src.rpm | cpio -iv Config.mk
Config.mk
246098 blocks
```

17.7. yum

despre yum

Yellowdog Updater, Modified (yum) este o comandă mai ușoară pentru a lucra cu pachete **rpm**. Este instalată default pe Fedora și Red Hat Enterprise Linux încă din versiunea 5.2.

yum list

Tastați **yum list available** pentru a vedea o listă a pachetelor disponibile. Parametrul **available** este opțional.

```
[root@rhel55 ~]# yum list | wc -l
2471
```

Tastați **yum list \$package** pentru a obține toate versiunile (în repozitoare diferite) a unui pachet.

```
[root@rhel55 ~]# yum list samba
Loaded plugins: rhnplugin, security
Installed Packages
samba.i386                3.0.33-3.28.el5      installed
Available Packages
samba.i386                3.0.33-3.29.el5_5    rhel-i386-server-5
```

yum search

Pentru a căuta un pachet care conține un anumit șir în descriere sau nume folosiți **yum search \$string**.

```
[root@rhel55 ~]# yum search gcc44
Loaded plugins: rhnplugin, security
===== Matched: gcc44 =====
gcc44.i386 : Preview of GCC version 4.4
gcc44-c++.i386 : C++ support for GCC version 4.4
gcc44-gfortran.i386 : Fortran support for GCC 4.4 preview
```

yum provides

Pentru a căuta un pachet care conține un anumit fișier (ați putea avea nevoie de el pentru a compila obiecte) folosiți **yum provides \$filename**.

```
[root@rhel55 ~]# yum provides /usr/share/man/man1/gzip.1.gz
Loaded plugins: rhnplugin, security
Importing additional filelist information
gzip-1.3.5-9.el5.i386 : The GNU data compression program.
Repo                : rhel-i386-server-5
Matched from:
Filename           : /usr/share/man/man1/gzip.1.gz
...
```

yum install

Pentru a instala o aplicație, folosiți **yum install \$package**. Bineînțeles că **yum** va instala toate dependențele necesare.

```
[root@rhel55 ~]# yum install sudo
Loaded plugins: rhnplugin, security
Setting up Install Process
Resolving Dependencies
--> Running transaction check
---> Package sudo.i386 0:1.7.2p1-7.el5_5 set to be updated
--> Finished Dependency Resolution
```

Dependencies Resolved

```
=====
Package      Arch      Version      Repository      Size
Installing:
sudo         i386      1.7.2p1-7.el5_5  rhel-i386-server-5  230 k
```

Transaction Summary

```
=====
Install      1 Package(s)
Upgrade      0 Package(s)
```

Total download size: 230 k

Is this ok [y/N]: y

Downloading Packages:

```
sudo-1.7.2p1-7.el5_5.i386.rpm      | 230 kB      00:00
```

Running rpm_check_debug

Running Transaction Test

Finished Transaction Test

Transaction Test Succeeded

Running Transaction

```
Installing      : sudo      1/1
```

Installed:

```
sudo.i386 0:1.7.2p1-7.el5_5
```

Complete!

Puteți adăuga mai mult decât un singur parametru aici.

```
yum install $package1 $package2 $package3
```

yum update

Pentru a aduce toate aplicațiile la zi, downloadându-le sau instalându-le, tastați **yum update**. Tot software-ul care a fost instalat prin **yum** va fi adus la zi la cea mai nouă versiune care este disponibilă în repozitoriu.

```
yum update
```

Dacă vreți să faceți update doar unui singur pachet, folosiți **yum update \$package**.

```
[root@rhel55 ~]# yum update sudo
Loaded plugins: rhnplugin, security
Skipping security plugin, no data
Setting up Update Process
```

```
Resolving Dependencies
Skipping security plugin, no data
--> Running transaction check
---> Package sudo.i386 0:1.7.2p1-7.el5_5 set to be updated
--> Finished Dependency Resolution
```

Dependencies Resolved

```
=====
Package            Arch      Version      Repository      Size
=====
Updating:
sudo               i386      1.7.2p1-7.el5_5  rhel-i386-server-5  230 k
```

Transaction Summary

```
=====
Install           0 Package(s)
Upgrade           1 Package(s)
```

Total download size: 230 k

Is this ok [y/N]: y

Downloading Packages:

```
sudo-1.7.2p1-7.el5_5.i386.rpm | 230 kB 00:00
```

Running rpm_check_debug

Running Transaction Test

Finished Transaction Test

Transaction Test Succeeded

Running Transaction

```
  Updating           : sudo 1/2
```

```
  Cleanup            : sudo 2/2
```

Updated:

sudo.i386 0:1.7.2p1-7.el5_5

Complete!

grupuri software yum

Tastați **yum grouplist** pentru a vedea o listă a tuturor grupurilor software disponibile.

```
[root@rhel55 ~]# yum grouplist
Loaded plugins: rhnplugin, security
Setting up Group Process
Installed Groups:
```

- Administration Tools
- Authoring and Publishing
- DNS Name Server
- Development Libraries
- Development Tools
- Editors
- GNOME Desktop Environment
- GNOME Software Development
- Graphical Internet
- Graphics
- Legacy Network Server
- Legacy Software Development
- Legacy Software Support
- Mail Server
- Network Servers
- Office/Productivity
- Printing Support
- Server Configuration Tools
- System Tools
- Text-based Internet
- Web Server
- Windows File Server
- X Software Development
- X Window System

```
Available Groups:
```

- Engineering and Scientific
- FTP Server
- Games and Entertainment
- Java Development
- KDE (K Desktop Environment)
- KDE Software Development
- MySQL Database
- News Server
- OpenFabrics Enterprise Distribution
- PostgreSQL Database
- Sound and Video

```
Done
```

Pentru a instala un set de aplicații, luate împreună într-un grup, folosiți **yum groupinstall \$groupname**.

```
[root@rhel55 ~]# yum groupinstall 'Sound and video'
Loaded plugins: rhnplugin, security
Setting up Group Process
Package alsa-utils-1.0.17-1.el5.i386 already installed and latest version
Package sox-12.18.1-1.i386 already installed and latest version
Package 9:mkisofs-2.01-10.7.el5.i386 already installed and latest version
Package 9:cdrecord-2.01-10.7.el5.i386 already installed and latest version
Package cdrdao-1.2.1-2.i386 already installed and latest version
Resolving Dependencies
--> Running transaction check
---> Package cdda2wav.i386 9:2.01-10.7.el5 set to be updated
---> Package cdparanoia.i386 0:alpha9.8-27.2 set to be updated
---> Package sound-juicer.i386 0:2.16.0-3.el5 set to be updated
--> Processing Dependency: libmusicbrainz >= 2.1.0 for package: sound-juicer
--> Processing Dependency: libmusicbrainz.so.4 for package: sound-juicer
---> Package vorbis-tools.i386 1:1.1.1-3.el5 set to be updated
--> Processing Dependency: libao >= 0.8.4 for package: vorbis-tools
--> Processing Dependency: libao.so.2 for package: vorbis-tools
--> Running transaction check
---> Package libao.i386 0:0.8.6-7 set to be updated
---> Package libmusicbrainz.i386 0:2.1.1-4.1 set to be updated
--> Finished Dependency Resolution
...
```

Citiți pagina de manual a **yum** pentru mai multe informații despre managementul grupurilor în **yum**.

/etc/yum.conf și repozitoare

Configurarea repozitoarelor **yum** se face în **/etc/yum/yum.conf** și **/etc/yum/repos.d/**.

Configurarea lui **yum** însuși se face în **/etc/yum.conf**. Acest fișier va conține locația unui fișier-jurnal și un director cache pentru **yum** și poate conține de asemenea o listă a repozitoarelor.

Recent **yum** a început să accepte câteva fișiere **repo** cu fiecare fișier conținând o listă a **repozitoarelor**. Aceste fișiere **repo** sînt localizate în directorul **/etc/yum/repos.d/**.

O marcă importantă pentru **yum** este **enablerepo**. Folosiți această comandă dacă vreți să folosiți un repozitoriu care nu este autorizat prin default.

```
yum $command $foo --enablerepo=$repo
```

Un exemplu al conținutului fișierului repo: **MyRepo.repo**.

```
[$repo]
name=My Repository
baseurl=http://path/to/MyRepo
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-MyRep
```

17.8. alien

alien este software experimental care convertește pachete format **rpm** în **deb** (și alte formate).

Mai jos este un exemplu cum să folosim **alien** pentru a converti un pachet **rpm** într-un pachet **deb**.

```
paul@barry:~$ ls -l netcat*
-rw-r--r-- 1 paul paul 123912 2009-06-04 14:58 netcat-0.7.1-1.i386.rpm
paul@barry:~$ alien --to-deb netcat-0.7.1-1.i386.rpm
netcat_0.7.1-2_i386.deb generated
paul@barry:~$ ls -l netcat*
-rw-r--r-- 1 paul paul 123912 2009-06-04 14:58 netcat-0.7.1-1.i386.rpm
-rw-r--r-- 1 root root 125236 2009-06-04 14:59 netcat_0.7.1-2_i386.deb
```

*În viața reală, folosiți utilitarul **netcat** dat de distribuție, sau folosiți fișierul **.deb** de la website-ul lor.*

17.9. download software din afara repozitoriului

Primul și cel mai important lucru, oricând downloadați software, începeți cu citirea fișierului README!

În mod normal README va explica ce să faceți după download. Probabil veți obține un fișier **.tar.gz** sau unul **.tgz**. Citiți documentația, apoi puneți fișierul comprimat într-un director. Puteți folosi următoarele pentru a afla unde vrea să se instaleze pachetul.

```
tar tvzpf $downloadedFile.tgz
```

Le despachetați cu **tar xzf**, ea va crea un director numit **applicationName-1.2.3**

```
tar xzf $applicationName.tgz
```

Înlocuiți **z** cu un **j** când fișierul se termină în **.tar.bz2**. Comenzile **tar**, **gzip** și **bzip2** sînt explicate în detaliu în cursul Fundamente Linux.

Dacă downloadați un fișier **.deb**, atunci va trebui să folosiți **dpkg** pentru a-l instala, rpm-urile pot fi instalate cu comanda **rpm**.

17.10. compilare software

Primul și cel mai important lucru, oricând downloadați cod sursă pentru instalare, începeți cu citirea fișierului README!

De obicei pașii sînt întotdeauna aceiași trei: rulînd **./configure** urmat de **make** (care este compilarea în realitate) și apoi de **make install** pentru a copia fișierele în locația lor corespunzătoare.

```
./configure
make
make install
```

17.11. practică: management pachet

1. Găsiți aplicația grafică pe toate computerele pentru a adăuga și șterge aplicații.
2. Verificați pe ambele sisteme dacă gcc este instalat.
3. Folosiți aptitude sau yum pentru a căuta și instala aplicațiile 'dict', 'samba' și 'wesnoth'. Le-ați găsit pe toate?
4. Căutați pe internet 'webmin' și instalați-l.
5. Dacă timpul permite, dezinstalați samba de pe mașina Ubuntu, downloadați ultima versiune de la samba.org și instalați-o.

17.12. soluție: management pachet

1. Găsiți aplicația grafică pe toate computerele pentru a adăuga și șterge aplicații.

2. Verificați pe ambele sisteme dacă gcc este instalat.

```
dpkg -l | grep gcc
```

```
rpm -qa | grep gcc
```

3. Folosiți aptitude sau yum pentru a căuta și instala aplicațiile 'dict', 'samba' și 'wesnoth'. Le-ați găsit pe toate?

```
aptitude search wesnoth (Debian, Ubuntu și derivate din ele)
```

```
yum search wesnoth (Red Hat și derivatele din el)
```

4. Căutați pe internet 'webmin' și instalați-l.

Google ar trebui să vă indice webmin.com

Există mai multe formate disponibile acolo, alegeți .rpm, .deb sau .tgz.

5. Dacă timpul permite, dezinstalați samba de pe mașina Ubuntu, downloadați ultima versiune de la samba.org și instalați-o.

Partea V. management rețea

Capitolul 18. generalități networking

Conținut

18.1. straturi network.	.174
18.2. unicast, multicast, broadcast, anycast.	.178
18.3. lan-wan-man.	182
18.4. internet-intranet -extranet.	183
18.5. tcp/ip.	.184

Deși acest capitol nu este legat în mod direct de **Linux**, conține totuși concepte de networking generale care vă vor ajuta să depanați rețele pe **Linux**.

18.1. straturi network

șapte straturi OSI

Cînd vorbesc despre stratificarea protocolului, oamenii de obicei menționează șapte straturi ale protocolului **OSI** (Aplicație, Prezentare, Sesiune, Transport, Network, Data Link și Fizic). Vom discuta stratele 2 și 3 în adîncime, și ne vom focaliza mai puțin pe celelalte straturi. Motivul este că aceste straturi sînt importante pentru înțelegerea rețelelor. Veți auzi administratori folosind cuvinte ca „acesta este un dispozitiv strat 2” sau „acesta este un broadcast strat 3”, și ar trebui să fiți capabil să înțelegeți despre ce anume vorbesc ei.

patru straturi DoD

Modelul **DoD** (sau tcp/ip) are doar patru straturi, mapînd aproximativ **stratul de acces network** la straturile OSI 1 și 2 (Fizic și Datalink), stratul **internet** (IP) către **rețeaua stratificată** OSI, stratul **host-to-host** (gază-la-gază) (tcp, udp) la stratul 4 OSI (transport) și **stratul aplicație** la stratele OSI 5, 6 și 7.

Mai jos este o încercare de a pune straturile OSI și DoD lîngă unele protocoale și dispozitive.

OSI Model	DoD Model	protocols		devices/apps
layer 5, 6, 7	application	dns, dhcp, ntp, snmp, https, ftp, ssh, telnet, http, pop3... others		web server, mail server, browser, mail client...
layer 4	host-to-host	tcp	udp	gateway
layer 3	internet	ip, icmp, igmp		router, firewall layer 3 switch
layer 2	network access	arp (mac), rarp		bridge layer 2 switch
layer 1		ethernet, token ring		hub

scurtă introducere la stratul fizic

În stratul fizic, sau **stratul 1 (layer 1)**, este vorba despre voltaje, semnale electrice și conexiuni mecanice. Unele rețele încă pot folosi cabluri **coaxiale**, dar cele mai multe au migrat la **utp** (cat 5 sau mai bun) cu conectori **rj45**.

Dispozitive ca **relee** și **huburi** sînt parte a acestui strat. Nu puteți folosi software pentru a „vedea” un **releu** sau **hub** pe rețea. Singurul lucru pe care aceste dispozitive îl fac este să amplifice semnalul electric în cabluri. **Huburile pasive** sînt amplificatori multiport care amplifică un semnal electric care intră pe toate celelalte conexiuni. **Huburile active** fac asta citind și retransmițînd biți, fără a interpreta nici un înțeles în acei biți.

Tehnologii network ca **csmacd** și **semnal inelar (token ring)** sînt definite în acest strat.

Asta e tot ce avem de spus despre **stratul 1 (layer 1)** în această carte.

scurtă introducere la stratul data link

În stratul data link, sau **stratul 2 (layer 2)** este vorba despre construcții. O construcție are un **crc** (căutare **redundantă** **ciclică**). În cazul ethernet (802.3), fiecare placă de rețea este identificabilă de către o adresă unică **mac** (adresă de control de acces media) de 48 de biți.

Pe acest strat găsim dispozitive ca bridges (punți) și switches (comutatori). Un bridge este mai inteligent decît un hub deoarece un **bridge** poate face decizii

bazându-se pe adresa mac a computerelor. Un **switch** înțelege de asemeni adrese mac.

În această carte vom discuta comenzi ca **arp** și **ifconfig** pentru a explora acest strat.

scurtă introducere la stratul network

În **stratul 3 (layer 3)** este vorba despre pachete ip. Acest strat dă fiecărei gazde o adresă ip unică de 32 de biți. Dar **ip** nu este singurul protocol în acest strat, există de asemeni **icmp**, **igmp**, **ipv6** și altele. O listă completă poate fi găsită în fișierul **/etc/protocols**.

Pe acest strat găsim dispozitive ca **rutere** și switchuri layer 3, dispozitive care știu (și au) o adresă ip.

În tcp/ip la acest strat se obișnuiește să se facă referire **la stratul internet (internet layer)**.

scurtă introducere la stratul de transport

Vom discuta protocolurile **tcp** și **udp** în contextul stratului 4. Modelul DoD numește asta stratul host-to-host (gazdă-la-gazdă).

straturile 5, 6 și 7

Stratul de aplicație tcp/ip includ straturile 5, 6, și 7. Detaliile despre diferențe între aceste straturi sînt în afara scopului acestui curs.

straturile network în această carte

Straturile network în această carte sînt bazate pe explicația **Protocols in Frame [Protocoluri în Construcție]** în interceptorul **wireshark**. Cînd interceptăm un pachet dhcp, observăm următoarele în interceptor.

[Protocols in Frame: eth:ip:udp:bootp]

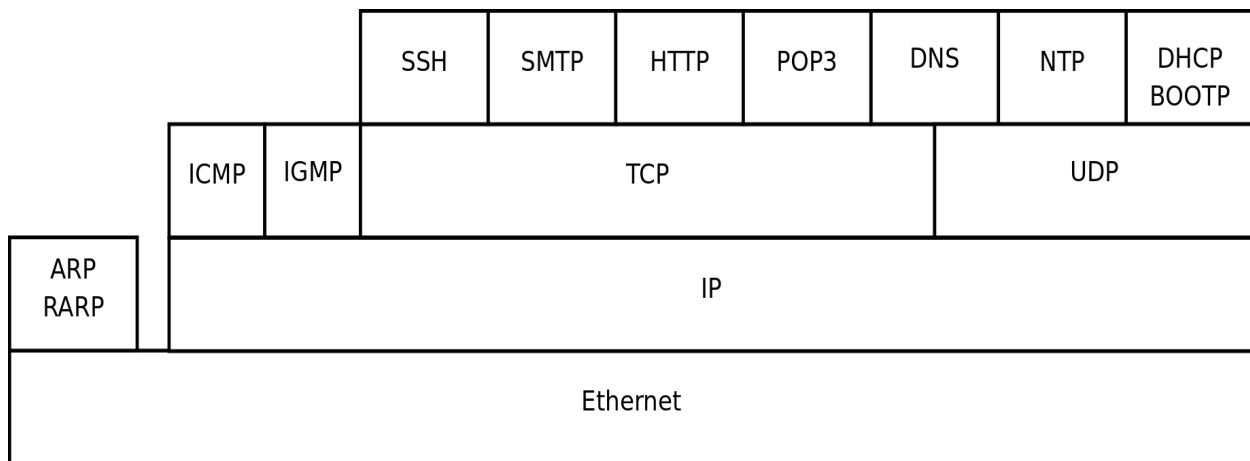
Interceptînd pachete ntp (Network Time Protocol) ne dă o linie, care ne face să concluzionăm să punem **ntp** alături de **bootp** în schema protocolului de mai jos.

[Protocols in Frame: eth:ip:udp:ntp]

Interceptînd un broadcast **arp** ne face să punem arp lîngă **ip**. Toate aceste protocoluri sînt explicate mai tîrziu în acest capitol.

[Protocols in Frame: eth:arp]

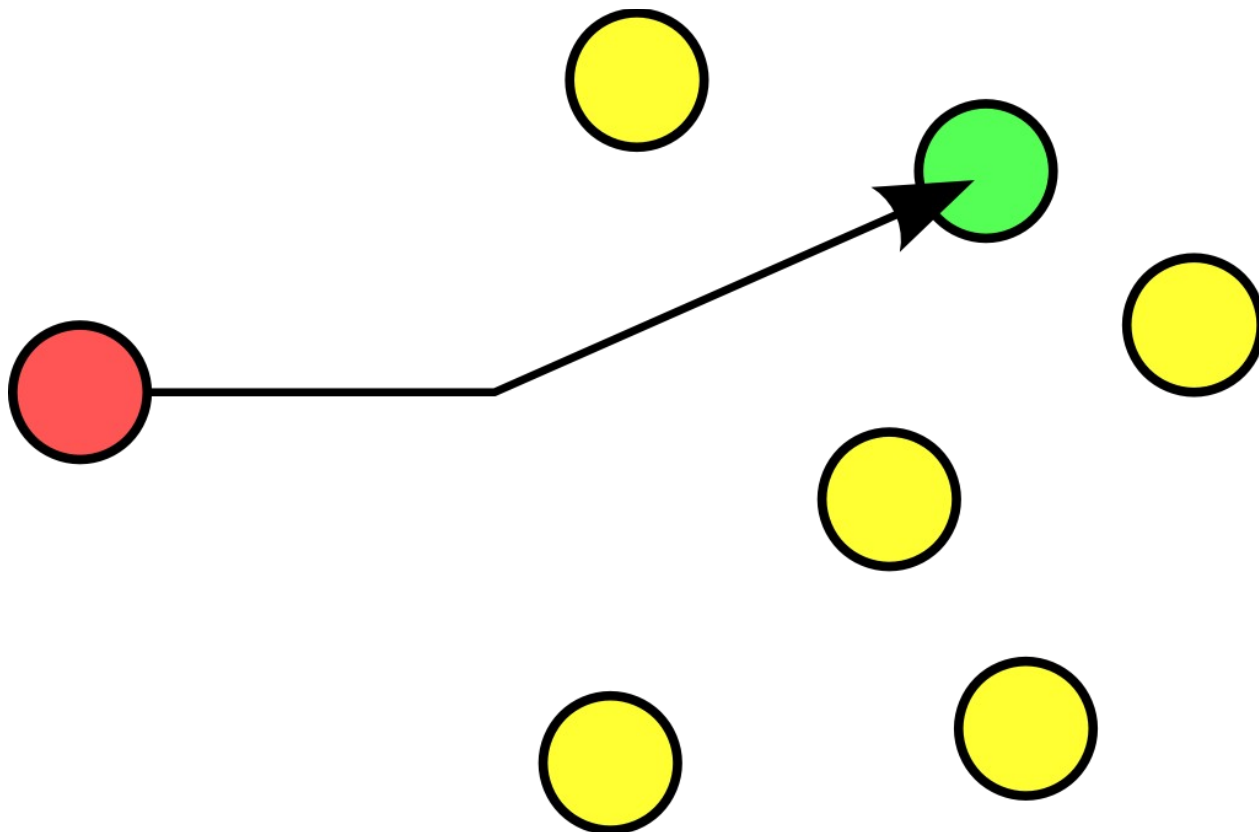
Mai jos este o hartă protocol bazată pe cunoașterea wireshark. Conține unele dintre cele mai comune protocoluri care sînt discutate în această carte. Harta nu conține toate protocolurile.



18.2. unicast, multicast, broadcast, anycast

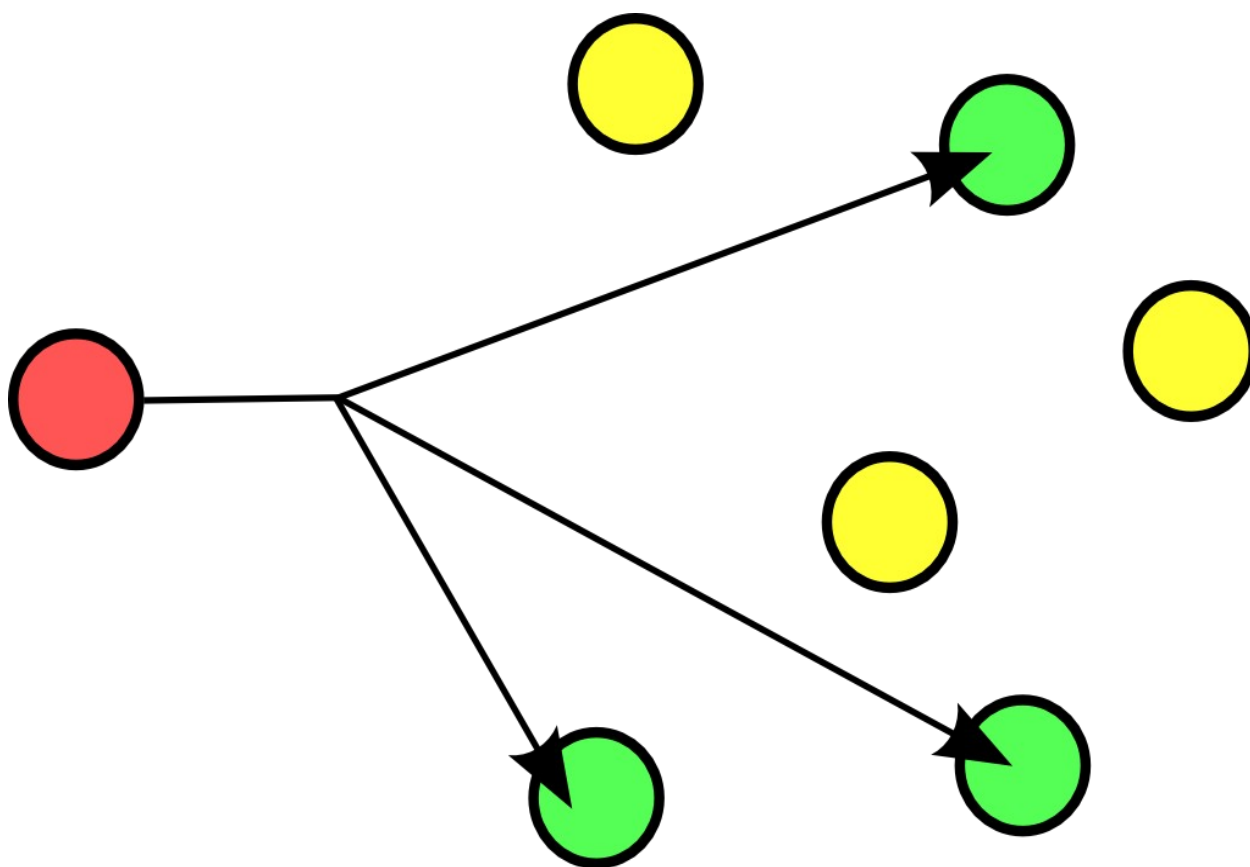
unicast

O comunicare **unicast** își are originea de la un computer și este destinată exact unui singur alt computer (sau gazdă). Este un lucru obișnuit pentru computere să aibă multe comunicări **unicast**.



multicast

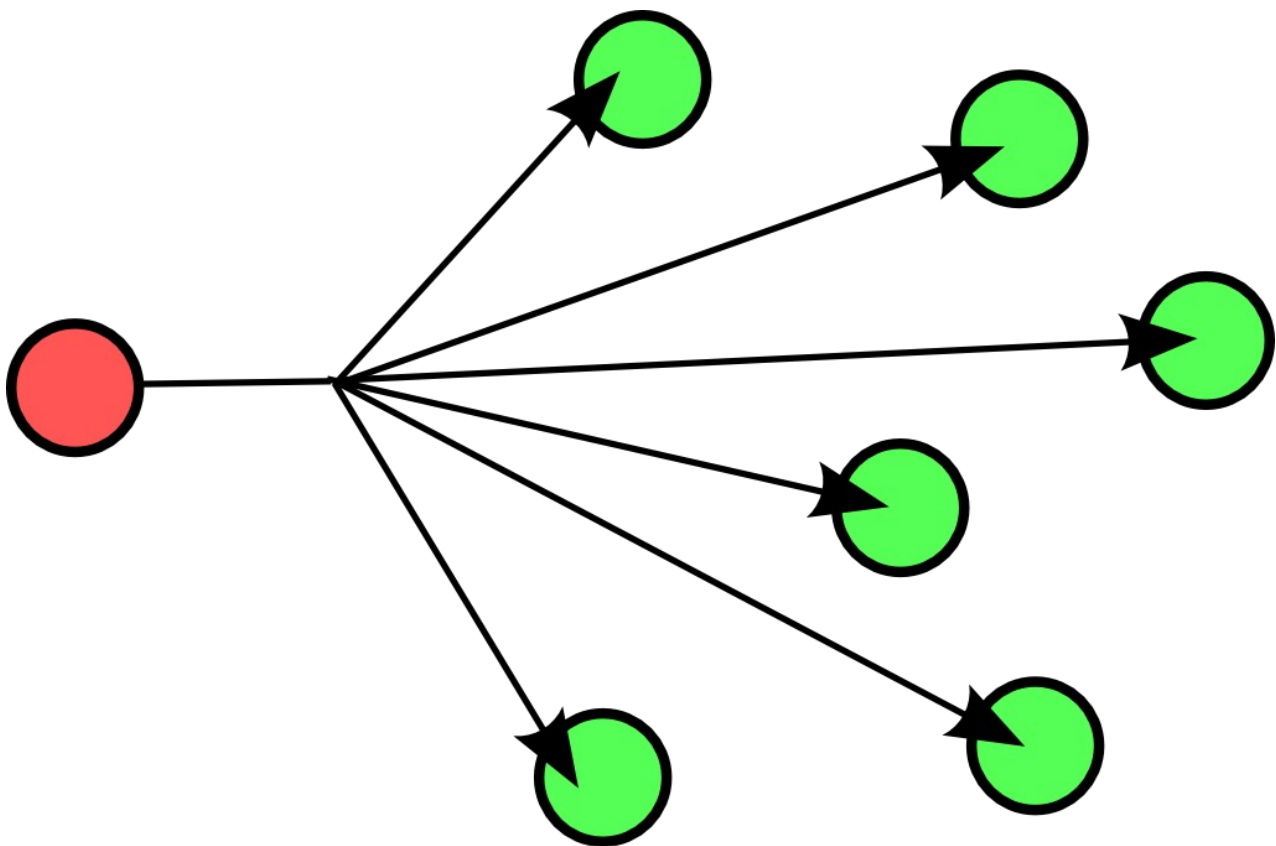
O comunicare **multicast** este destinată unui grup (de computere).



Cîteva exemple de **multicast** sînt Realplayer (fişiere .sdp) şi **ripv2** (un protocol de rutare).

broadcast

Un **broadcast** este făcut pentru toată lumea.

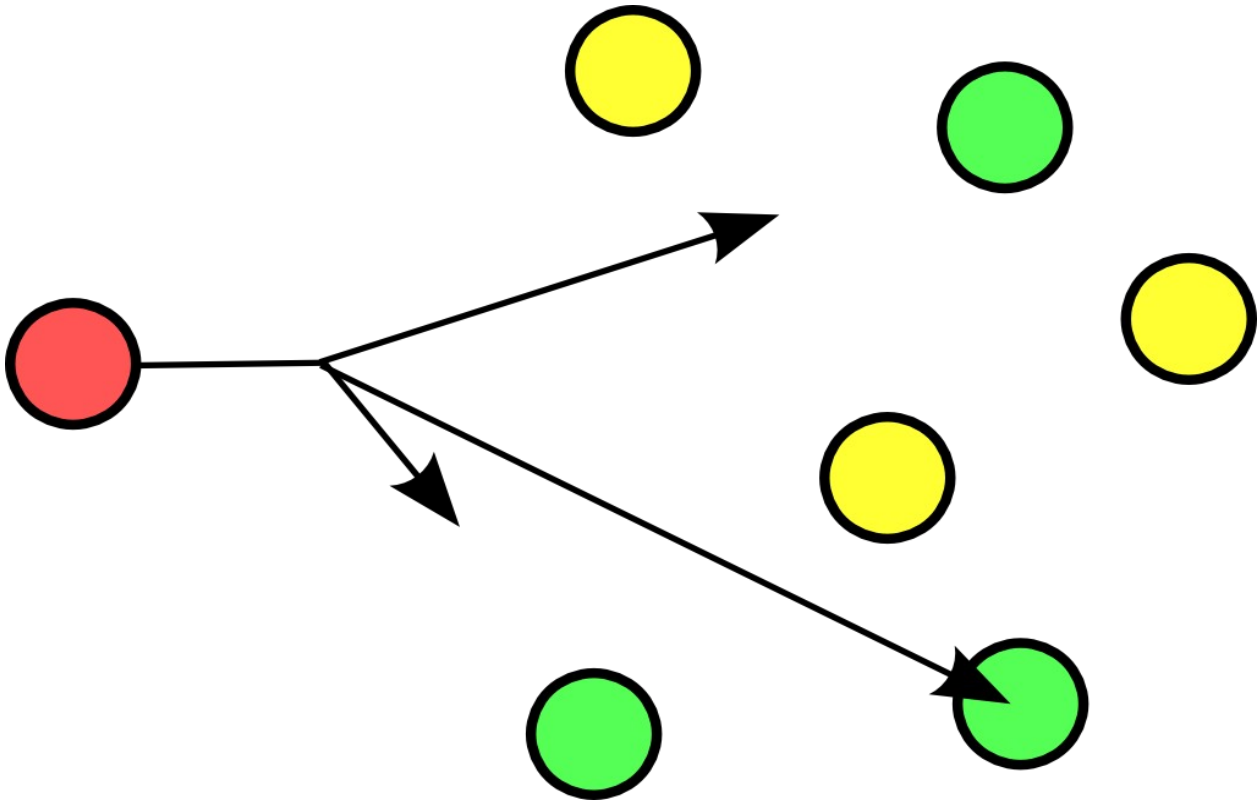


Un exemplu tipic aici este BBC (British Broadcasting Corporation) transmițând oricui. În comunicările de date un broadcast este în mod comun limitat la **lan**.

Fiți cu băgare de seamă, un **broadcast layer 2** este foarte diferit de un **broadcast layer 3**. Un broadcast layer 2 este recepționat de către toate plăcile network pe același segment (nu trece prin nici un ruter), dar un broadcast layer 3 este recepționat de toate gazdele în același subnet ip.

anycast

Serverele **root name** din internet folosesc **anycast**. Un semnal **anycast** trece prin cel mai apropiat grup bine definit (geografic).



Mulțumim colaboratorului anonim de pe wikipedia care a pus aceste desene în domeniul public.

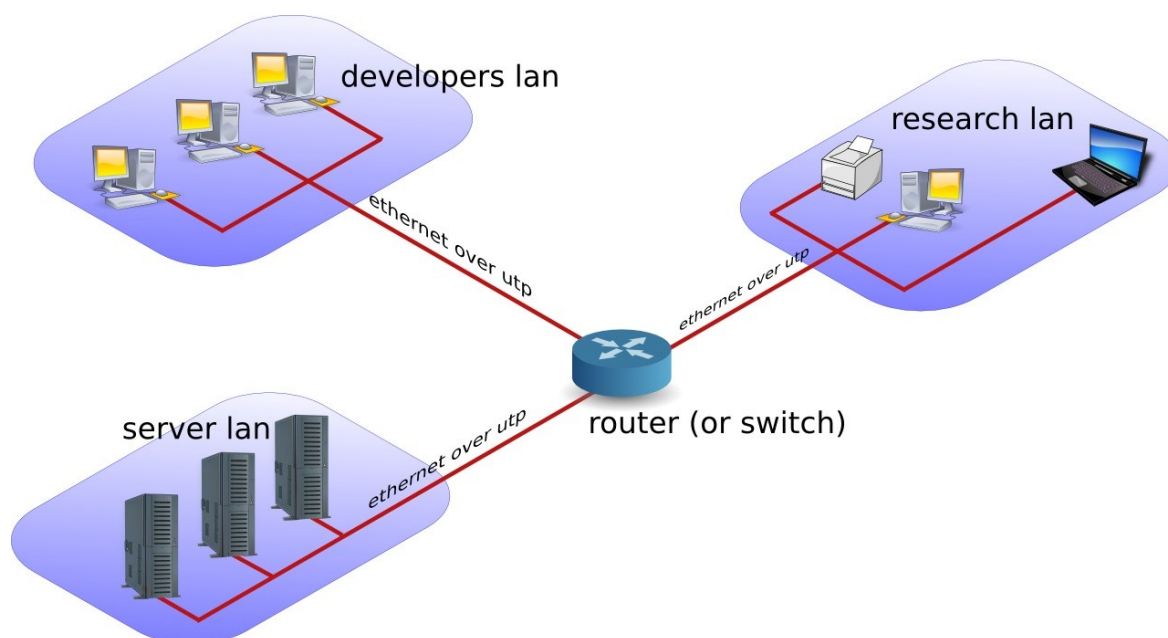
18.3. lan-wan-man

Termenul **lan** este folosit pentru local area networks (**zona rețelei locale**), în opoziție cu un **wan** – wide area networks (**zona rețelei largi**). Diferența dintre cei doi termeni este determinată de **distanța** dintre computere, și nu de numărul de computere dintr-o rețea. Unele protocoale ca **atm** sînt făcute pentru utilizare într-un **wan**, altele ca **ethernet** sînt făcute pentru a fi folosite într-un **lan**.

lan

Un **lan** (Local Area Network) este o rețea locală. Asta poate fi o singură cameră, sau un etaj, sau chiar o clădire mare. Numim **lan** atîta timp cît computerele sînt **aproape** unul de altul. Putem să definim **lan**, de asemeni, cînd toate computerele sînt conectate **ethernet**.

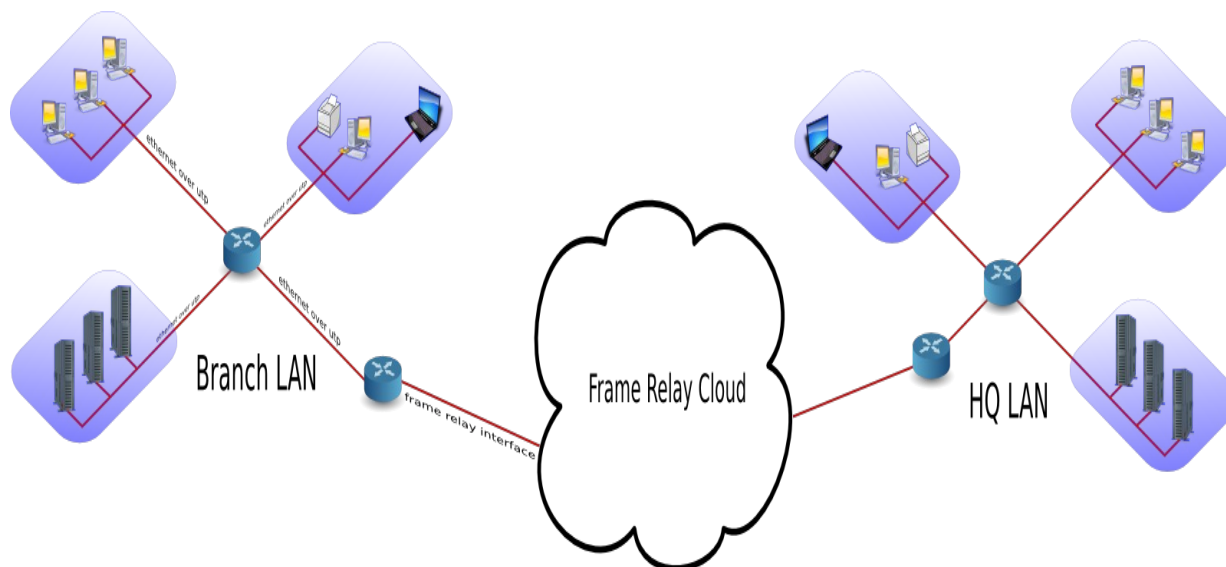
Un **lan** poate conține mai multe **lan-uri** mai mici. Desenul de mai jos arată trei **lan-uri** care împreună fac un singur lan.



wan

Un **wan** (Wide Area Network) (Zonă de Rețea Largă) este o rețea cu o mare distanță între computere (sau gazde). Aceste gazde sînt deseori conectate prin **linii închiriate**. Un **wan** nu utilizează **ethernet**, ci protocoale ca **fddi**, **frame relay**, **ATM** sau **X.25** pentru a conecta computere (și rețele).

Desenul de mai jos arată un birou branșă care este conectat prin intermediul **Frame Relay** cu administrație centrală.



Acronimul **wan** este de asemenea utilizat pentru rețele de suprafață mari ca **internetul**.

Cisco este cunoscut pentru tehnologia lor **wan**. Ei fabrică **rutere** care conectează multe rețele **lan** folosind protocoalele **wan**.

man

Un **man** (Metropolitan Area Network) (Zonă de Rețea Metropolitană) este ceva între un **lan** și un **wan**, deseori cuprinzînd mai multe clădiri în același campus sau în același oraș. Un **man** poate folosi **fddi** sau **ethernet** sau alte protocoale pentru conectivitate.

pan-wpan

Rețeaua dvs. de acasă este numită un **pan** (Personal Area Network) (Zonă de rețea Personală). Un **pan** wireless este un **wpan**.

18.4. internet-intranet-extranet

Internetul este o rețea globală. El conectează multe rețele folosind protocolul stivă **tcp/ip**.

Originea **internetului** este **arpanet**. **Arpanet** a fost creat în 1969, în acel an doar 4 computere erau conectate în rețea. În 1971 primul **e-mail** a fost trimis în **arpanet**. **E-mail-ul** a luat 75 la sută din tot traficul **arpanet** în 1973. 1973 a fost de asemenea anul în care **ftp** a fost introdus, și a văzut conectarea primelor țări europene (Norvegia și Anglia). În 2009 internetul a fost disponibil la 25 la sută din populația lumii. În 2011 este estimat că doar un sfert din paginile web internet sînt în limba engleză.

Un **intranet** este o rețea privată **tcp/ip**. Un **intranet** folosește aceleași protocoale ca **internetul**, dar este accesibil doar oamenilor din interiorul unei organizații.

Un **extranet** este similar cu un **intranet**, dar unele organizații trust (parteneri/clienți/furnizori/...) au acces la fel.

18.5. tcp/ip

istoria tcp/ip

În anii '60 dezvoltarea stivei protocol **tcp/ip** a fost început de către Departamentul Apărării al S.U.A. În anii '80 multe dintre companiile comerciale au dezvoltat propria lor stivă protocol: IBM a creat **sna**, Novell a avut **ipx/spx**, Microsoft a terminat **netbeui** și Apple a lucrat cu **appletalk**. Toate eforturile din anii '80 au eșuat să supraviețuiască anilor '90. La sfârșitul anilor '90, aproape toate computerele din lume erau capabile să vorbească în limbajul **tcp/ip**.

În opinia mea umilă, motivul principal pentru supraviețuirea **tcp/ip** deasupra tuturor altor protocoale este deschiderea lui. Oricine este liber să dezvolte și să folosească suita protocol **tcp/ip**.

rfc (request for comment)

Protocoalele care sînt utilizate pe internet sînt definite de **rfc**-uri. Un **rfc** sau **request for comment** descrie mersul intern al tuturor protocoalelor internet. **IETF** (**I**nternet **E**ngineering **T**ask **F**orce) este singurul editor al acestor protocoale din 1986.

Site-ul oficial al **rfc** este <http://www.rfc-editor.org>. Acest website conține toate **rfc**-urile în format text, de exemplu **rfc2132** (care definește **dhcp** și **bootp**) este accesibil la <http://www.rfc-editor.org/rfc/rfc2132.txt>.

multe protocoale

Pentru conexiuni stabile, folosiți **tcp**, în schimb **udp** este fără conexiuni dar mai rapid. Mesajele de eroare **icmp** sînt folosite de **ping**, grupurile multicast sînt aranjate de **igmp**.

Aceste protocoale sînt vizibile în câmpul protocol al ip header, și sînt listate în fișierul **/etc/protocols**.

```
paul@debian5:~$ grep tcp /etc/protocols
tcp      6      TCP      # transmission control protocol
```

multe servicii

Plăcile de rețea sînt identificate în mod unic de către **adresa** lor **mac**, numele de mașini de către **adresa ip** și aplicațiile după **numărul de port**.

Aplicațiile comune de nivel protocol ca **smtp**, **http**, **ssh**, **telnet** și **ftp** au **numere de port fixe**. Există o listă a **numerelor porturilor** în **/etc/services**.

```
paul@ubu1010:~$ grep ssh /etc/services
ssh      22/tcp      # SSH Remote Login Protocol
ssh      22/udp
```


Capitolul 19. configurare interfață de rețea

Conținut

19.1.	să folosim GUI sau să nu folosim GUI.	186
19.2.	configurare nic pe Debian/Ubuntu.	186
19.3.	configurare nic pe RedHat/Fedora.	188
19.4.	ifconfig.	189
19.5.	hostname.	192
19.6.	arp.	192
19.7.	route.	193
19.8.	ping.	193
19.9.	opțional: ethtool.	194
19.10.	practică: configurare interfață de rețea.	195
19.11.	soluție: configurare interfață de rețea.	196

Acest capitol explică cum să configurăm **plăcile interfață de rețea** să lucreze cu **tcp/ip**.

19.1. să folosim GUI sau să nu folosim GUI

Distribuții recente Linux deseori includ o aplicație grafică pentru a configura rețeaua. Unii oameni se plîng că aceste aplicații încurcă configurările network cînd sînt utilizate simultan cu configurările de la terminal. În special **Network Manager** (deseori înlocuit cu **wicd**) și **yast** sînt cunoscute pentru faptul că nu au grijă de schimbările configurației prin terminal.

Pentru că țelul acestui curs este administrarea **server**, vom asuma că serverele noastre Linux sînt întotdeauna administrate prin terminal.

Acest capitol se focalizează doar pe utilizarea terminalului pentru configurarea interfeței de rețea!

Din nefericire nu există o singură combinație de comenzi Linux și fișiere **/etc** care să funcționeze pe toate distribuțiile Linux. Vom vorbi despre rețea pe două (mari dar distincte) familii de distribuții Linux.

Începem cu **Debian/Ubuntu**, apoi continuăm cu **Fedora/RHEL**.

19.2. configurare nic pe Debian/Ubuntu

/etc/network/interfaces

Fișierul **/etc/network/interfaces** este un nucleu de configurare fișier a plăcii de rețea network pe Ubuntu și Debian.

client dhcp

Captura de ecran de mai jos arată că distribuția noastră curentă Ubuntu 11.04 este configurată pentru **dhcp** pe **eth0** (**prima placă interfață rețea sau nic**).

```
root@ubu1104srv:~# cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto eth0
iface eth0 inet dhcp
```

Configurarea plăcilor de rețea pentru **dhcp** este o practică bună pentru clienți, dar serverele de obicei cer o **adresă ip fixă**.

ip fix

Captura de ecran de mai jos arată **/etc/network/interfaces** configurat cu o **adresă ip fixă**.

```
root@ubul104srv:~# cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).
```

```
# The loopback network interface
auto lo
iface lo inet loopback
```

```
# The primary network interface
auto eth0
iface eth0 inet static
address 192.168.33.100
network 192.168.33.0
netmask 255.255.255.0
gateway 192.168.33.1
```

Captura de ecran de mai sus de asemeni arată că puteți face mai multe configurații decât doar adresa ip. Vedeți **interfaces(5)** pentru ajutor pentru setarea unui **gateway**, **netmask** (mască de rețea) sau orice alte opțiuni.

/sbin/ifdown

Este bine (dar nu obligatoriu) să deconectăm o interfață înainte de a-i schimba configurarea. Asta poate fi făcut cu comanda **ifdown**.

Comanda nu va da nici un semnal când dăm jos o interfață cu o adresă ip fixă. Oricum **ifconfig** nu va mai arăta deloc interfața.

```
root@ubul104srv:~# ifdown eth0
root@ubul104srv:~# ifconfig
lo    Link encap:Local Loopback
      inet addr:127.0.0.1 Mask:255.0.0.0
      inet6 addr: ::1/128 Scope:Host
      UP LOOPBACK RUNNING MTU:16436 Metric:1
      RX packets:106 errors:0 dropped:0 overruns:0 frame:0
      TX packets:106 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:0
      RX bytes:11162 (11.1 KB) TX bytes:11162 (11.1 KB)
```

O interfață care este deconectată nu poate fi folosită să ne conectăm la rețea.

/sbin/ifup

Mai jos este o captură de ecran a **ifup** care aduce interfața ethernet **eth0** la viață folosind dhcp. (Observați că asta e un screenshot **Ubuntu 10.10**, **Ubuntu 11.04** omite ieșirea **ifup** prin default).

```
root@ubu1010srv:/etc/network# ifup eth0
Internet Systems Consortium DHCP Client V3.1.3
Copyright 2004-2009 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/
```

```
Listening on LPF/eth0/08:00:27:cd:7f:fc
Sending on   LPF/eth0/08:00:27:cd:7f:fc
Sending on   Socket/fallback
DHCPREQUEST of 192.168.1.34 on eth0 to 255.255.255.255 port 67
DHCPNAK from 192.168.33.100
DHCPDISCOVER on eth0 to 255.255.255.255 port 67 interval 3
DHCPOFFER of 192.168.33.77 from 192.168.33.100
DHCPREQUEST of 192.168.33.77 on eth0 to 255.255.255.255 port 67
DHCPACK of 192.168.33.77 from 192.168.33.100
bound to 192.168.33.77 -- renewal in 95 seconds.
ssh stop/waiting
ssh start/running, process 1301
root@ubu1010srv:/etc/network#
```

Detalii despre **dhcp** sînt discutate pe larg într-un capitol separat în cursul **Servere Linux**.

19.3. configurare nic Red Hat/Fedora

/etc/sysconfig/network

Fișierul **/etc/sysconfig/network** este un fișier (de-a lungul tuturor plăcilor de rețea) de configurare global. El ne permite să definim dacă vrem rețea (**NETWORKING=yes | no**), care ar trebui să fie numele mașinii (**HOSTNAME=**) și care gateway să utilizăm (**GATEWAY=**).

```
[root@rhel6 ~]# cat /etc/sysconfig/network
NETWORKING=yes
HOSTNAME=rhel6
GATEWAY=192.168.1.1
```

Există o duzină de mai multe opțiuni setabile în acest fișier, detalii pot fi găsite în **/usr/share/doc/initscripts-*/sysconfig.txt**.

/etc/sysconfig/network-scripts/ifcfg-

Fiecare placă de rețea poate fi configurată individual folosind fișierele **/etc/sysconfig/network-scripts/ifcfg-***. Când dețineți doar o singură placă de rețea, atunci aceasta va fi probabil **/etc/sysconfig/network-scripts/ifconfig-eth0**.

client dhcp

Mai jos este un screenshot al **/etc/sysconfig/network-scripts/ifcfg-eth0** configurat pentru dhcp (**BOOTPROTO="dhcp"**). Notați de asemenea parametrul **NM_CONTROLLED** pentru a anula controlul acestui nic de către **Network Manager**.

Acest parametru nu este explicat (nici măcar menționat) în **/usr/share/doc/initscripts-*/sysconfig.txt**, dar multe altele sînt explicate.

```
[root@rhel6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE="eth0"
HWADDR="08:00:27:DD:0D:5C"
NM_CONTROLLED="no"
BOOTPROTO="dhcp"
ONBOOT="yes"
```

Variabila **BOOTPROTO** poate să fie setată la **dhcp** sau **bootp**, orice altceva va fi considerat **static** asta însemnând că nu ar trebui să fie nici un protocol folosit în timpul butării pentru a seta valoarea interfeței.
ip fix

Mai jos este un screenshot a unei configurații **ip fix** în **/etc/sysconfig/network-scripts/ifcfg-eth0**.

```
[root@rhel6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE="eth0"
HWADDR="08:00:27:DD:0D:5C"
NM_CONTROLLED="no"
BOOTPROTO="none"
IPADDR="192.168.1.99"
NETMASK="255.255.255.0"
GATEWAY="192.168.1.1"
ONBOOT="yes"
```

HWADDR poate fi folosit pentru a fi siguri că fiecare placă de rețea obține numele corect când mai multe plăci de rețea sînt prezente în computer. Nu poate fi utilizată pentru a desemna o **adresă mac** unei plăci de rețea. Pentru asta, trebuie să specificați variabila **MACADDR**. Nu utilizați **HWADDR** și **MACADDR** în același fișier **ifcfg-ethx**.

Parametrii **BROADCAST=** și **NETWORK=** din versiunile anterioare RHEL/Fedora sînt învechite.

/sbin/ifup și /sbin/ifdown

Comenzile **ifup** și **ifdown** vor seta o interfață deschisă sau închisă, utilizînd configurația discutată mai sus. Acestea sînt identice în comportarea lor pe Debian și Ubuntu.

```
[root@rhel6 ~]# ifdown eth0 && ifup eth0
[root@rhel6 ~]# ifconfig eth0
eth0 Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
      inet6 addr: fe80::a00:27ff:fedd:d5c/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:2452 errors:0 dropped:0 overruns:0 frame:0
      TX packets:1881 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:257036 (251.0 KiB) TX bytes:184767 (180.4 KiB)
```

19.4. ifconfig

Utilizarea comenzii **/sbin/ifconfig** fără nici un argument va afișa o listă a tuturor plăcilor de rețea active, incluzînd interfețele wireless și loopback. În captura de ecran de mai jos **eth0** nu are nici o adresă ip.

```

root@ubu1010:~# ifconfig
eth0 Link encap:Ethernet HWaddr 00:26:bb:5d:2e:52
    UP BROADCAST MULTICAST MTU:1500 Metric:1
    RX packets:0 errors:0 dropped:0 overruns:0 frame:0
    TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
    Interrupt:43 Base address:0xe000

eth1 Link encap:Ethernet HWaddr 00:26:bb:12:7a:5e
    inet addr:192.168.1.30 Bcast:192.168.1.255 Mask:255.255.255.0
    inet6 addr: fe80::226:bbff:fe12:7a5e/64 Scope:Link
    UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
    RX packets:11141791 errors:202 dropped:0 overruns:0 frame:11580126
    TX packets:6473056 errors:3860 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:3476531617 (3.4 GB) TX bytes:2114919475 (2.1 GB)
    Interrupt:23

lo    Link encap:Local Loopback
    inet addr:127.0.0.1 Mask:255.0.0.0
    inet6 addr: ::1/128 Scope:Host
    UP LOOPBACK RUNNING MTU:16436 Metric:1
    RX packets:2879 errors:0 dropped:0 overruns:0 frame:0
    TX packets:2879 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:0
    RX bytes:486510 (486.5 KB) TX bytes:486510 (486.5 KB)

```

Puteți de asemenea utiliza **ifconfig** pentru a obține informații despre doar o singură placă de rețea.

```

[root@rhel6 ~]# ifconfig eth0
eth0 Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
    inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
    inet6 addr: fe80::a00:27ff:fedd:d5c/64 Scope:Link
    UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
    RX packets:2969 errors:0 dropped:0 overruns:0 frame:0
    TX packets:1918 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:335942 (328.0 KiB) TX bytes:190157 (185.7 KiB)

```

Când **/sbin** nu este în **\$PATH** a unui utilizator obișnuit trebuie să tastați calea absolută, cum se vede aici pe Debian.

```

paul@debian5:~$ /sbin/ifconfig eth3
eth3 Link encap:Ethernet HWaddr 08:00:27:ab:67:30
    inet addr:192.168.1.29 Bcast:192.168.1.255 Mask:255.255.255.0
    inet6 addr: fe80::a00:27ff:feab:6730/64 Scope:Link
    UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
    RX packets:27155 errors:0 dropped:0 overruns:0 frame:0
    TX packets:30527 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:13095386 (12.4 MiB) TX bytes:25767221 (24.5 MiB)

```

up și down

Puteți de asemeni folosi **ifconfig** pentru a închide sau deschide o interfață. Diferența cu **ifup** este că **ifconfig eth0 up** va re-activa nic-ul păstrînd configurația (curentă) existentă, însă **ifup** va citi fișierul corect care conține o configurație (posibil nouă) și va utiliza acest fișier de configurare pentru a deschide interfața.

```
[root@rhel6 ~]# ifconfig eth0 down
[root@rhel6 ~]# ifconfig eth0 up
[root@rhel6 ~]# ifconfig eth0
eth0 Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
      inet6 addr: fe80::a00:27ff:fedd:d5c/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:2995 errors:0 dropped:0 overruns:0 frame:0
      TX packets:1927 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:339030 (331.0 KiB) TX bytes:191583 (187.0 KiB)
```

setarea unei adrese ip

Puteți seta **temporar** o adresă ip cu **ifconfig**. Această adresă ip este validă doar pînă la următorul ciclu **ifup/ifdown** sau pînă la următorul **reboot**.

```
[root@rhel6 ~]# ifconfig eth0 | grep 192
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
[root@rhel6 ~]# ifconfig eth0 192.168.33.42 netmask 255.255.0.0
[root@rhel6 ~]# ifconfig eth0 | grep 192
      inet addr:192.168.33.42 Bcast:192.168.255.255 Mask:255.255.0.0
[root@rhel6 ~]# ifdown eth0 && ifup eth0
[root@rhel6 ~]# ifconfig eth0 | grep 192
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
```

setarea adresei mac

Puteți de asemeni folosi **ifconfig** pentru a seta o altă **adresă mac** decît cea codată în placa de rețea. Acest screenshot arată cum.

```
[root@rhel6 ~]# ifconfig eth0 | grep HWaddr
eth0 Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
[root@rhel6 ~]# ifconfig eth0 hw ether 00:42:42:42:42:42
[root@rhel6 ~]# ifconfig eth0 | grep HWaddr
eth0 Link encap:Ethernet HWaddr 00:42:42:42:42:42
```

dhclient

Desktop-urile de acasă și client Linux deseori execută **/sbin/dhclient**. Acesta e un daemon care deschide o interfață network pentru a concesiona o configurare ip de la un **server dhcp**. Cînd adaptatorul de rețea este configurat pentru **dhcp** sau **bootp**, atunci **/sbin/ifup** va porni daemonul **dhclient**.

Cînd o concesiune este reînnoită, **dhclient** va suprascrie setarea **ifconfig** a adresei ip!

19.5. hostname

Fiecare gazdă primește un **hostname** (nume de mașină), deseori plasat într-un **nume spațiu DNS** formînd **fqdn** sau **Fully Qualified Domain Name** (Nume De Domeniu Calificat în întregime).

Acest screenshot arată comanda **hostname** și configurarea hostname-ului pe **Red Hat/Fedora**.

```
[root@rhel6 ~]# grep rhel /etc/sysconfig/network
HOSTNAME=rhel6
[root@rhel6 ~]# hostname
rhel6
```

Ubuntu/Debian folosește fișierul **/etc/hostname** pentru a configura **numele mașinii**.

```
paul@ubu1010:~$ cat /etc/hostname
ubu1010
paul@ubu1010:~$ hostname
ubu1010
```

Pe toate distribuțiile Linux puteți schimba **numele mașinii** folosind comanda **hostname \$newname**. Asta nu este o schimbare permanentă.

```
[root@rhel6 ~]# hostname server42
[root@rhel6 ~]# hostname
server42
```

Pe orice Linux puteți folosi **sysctl** pentru a afișa și seta hostname-ul.

```
[root@rhel6 ~]# sysctl kernel.hostname
kernel.hostname = server42
[root@rhel6 ~]# sysctl kernel.hostname=rhel6
kernel.hostname = rhel6
[root@rhel6 ~]# sysctl kernel.hostname
kernel.hostname = rhel6
[root@rhel6 ~]# hostname
rhel6
```

19.6. arp

Rezoluția **ip to mac** este controlată de protocolul **arp layer 2 broadcast**. Tabelul **arp** poate fi afișat cu **utilitarul arp**. Screenshot-ul de mai jos arată lista computerelor cu care acest computer a comunicat recent.

```
root@barry:~# arp -a
? (192.168.1.191) at 00:0C:29:3B:15:80 [ether] on eth1
agapi (192.168.1.73) at 00:03:BA:09:7F:D2 [ether] on eth1
anya (192.168.1.1) at 00:12:01:E2:87:FB [ether] on eth1
faith (192.168.1.41) at 00:0E:7F:41:0D:EB [ether] on eth1
kiss (192.168.1.49) at 00:D0:E0:91:79:95 [ether] on eth1
laika (192.168.1.40) at 00:90:F5:4E:AE:17 [ether] on eth1
pasha (192.168.1.71) at 00:03:BA:02:C3:82 [ether] on eth1
shaka (192.168.1.72) at 00:03:BA:09:7C:F9 [ether] on eth1
root@barry:~#
```


anya este un Firewall Cisco, faith este o imprimantă laser, kiss este un Kiss DP600, laika este un laptop și agapi, shaka și pasha sînt servere SPARC. Semnul întrebării este un Red Hat Enterprise Linux server rulînd într-o mașină virtuală.

Puteți folosi **arp -d** pentru a șterge o intrare în **tabelul arp**.

```
[root@rhel6 ~]# arp
Address      HWtype  HWaddress      Flags Mask    Iface
ubu1010     ether   00:26:bb:12:7a:5e  C          eth0
anya        ether   00:02:cf:aa:68:f0  C          eth0
[root@rhel6 ~]# arp -d anya
[root@rhel6 ~]# arp
Address      HWtype  HWaddress      Flags Mask    Iface
ubu1010     ether   00:26:bb:12:7a:5e  C          eth0
anya        (incomplete)
[root@rhel6 ~]# ping anya
PING anya (192.168.1.1) 56(84) bytes of data.
64 bytes from anya (192.168.1.1): icmp_seq=1 ttl=254 time=10.2 ms
...
[root@rhel6 ~]# arp
Address      HWtype  HWaddress      Flags Mask    Iface
ubu1010     ether   00:26:bb:12:7a:5e  C          eth0
anya        ether   00:02:cf:aa:68:f0  C          eth0
```

19.7. route

Puteți vedea tabelul rutării locale a computerului cu comanda **/sbin/route** (și de asemeni cu **netstat -r**).

```
root@RHEL4b ~]# netstat -r
Kernel IP routing table
Destination Gateway Genmask Flags MS S Window irtt Iface
192.168.1.0 * 255.255.255.0 U 0 0 0 0 eth0
[root@RHEL4b ~]# route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.1.0 * 255.255.255.0 U 0 0 0 eth0
root@RHEL4b ~]#
```

Se pare că acest computer nu are un **gateway** configurat, așa că vom folosi **route add default gw** pentru a adăuga **gateway-ul default**.

```
[root@RHEL4b ~]# route add default gw 192.168.1.1
[root@RHEL4b ~]# route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.1.0 * 255.255.255.0 U 0 0 0 eth0
default 192.68.1.1 0.0.0.0 UG 0 0 0 eth0
[root@RHEL4b ~]#
```

Doar dacă nu configurați gateway-ul în unul dintre fișierele **/etc** de la începutul acestui capitol, computerul va uita această **gateway** după un reboot.

19.8. ping

Dacă puteți face **ping** către o altă gazdă, atunci **tcp/ip** este configurat.

```
[root@RHEL4b ~]# ping 192.168.1.5
PING 192.168.1.5 (192.168.1.5) 56(84) bytes of data.
64 bytes from 192.168.1.5: icmp_seq=0 ttl=64 time=1004 ms
64 bytes from 192.168.1.5: icmp_seq=1 ttl=64 time=1.19 ms
64 bytes from 192.168.1.5: icmp_seq=2 ttl=64 time=0.494 ms
64 bytes from 192.168.1.5: icmp_seq=3 ttl=64 time=0.419 ms

--- 192.168.1.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 0.419/251.574/1004.186/434.520 ms, pipe 2
[root@RHEL4b ~]#
```

19.9. optional: ethtool

Pentru a afișa sau schimba setările plăcii de rețea, folosiți **ethtool**. Rezultatele depind de capacitățile plăcii de rețea. Acest exemplu arată o rețea care auto-negociază mărimea de bandă.

```
root@laika:~# ethtool eth0
Settings for eth0:
    Supported ports: [ TP ]
    Supported link modes:   10baseT/Half 10baseT/Full
                           100baseT/Half 100baseT/Full
                           1000baseT/Full
```

```
Supports auto-negotiation: Yes
Advertised link modes: 10baseT/Half 10baseT/Full
                       100baseT/Half 100baseT/Full
                       1000baseT/Full
```

```
Advertised auto-negotiation: Yes
Speed: 1000Mb/s
Duplex: Full
Port: Twisted Pair
PHYAD: 0
Transceiver: internal
Auto-negotiation: on
Supports Wake-on: pumbg
Wake-on: g
Current message level: 0x00000033 (51)
Link detected: yes
```

Acest exemplu arată cum să utilizăm ethtool pentru a schimba lățimea de bandă de la 1000 Mbit la 100 Mbit și revers. Observați că trece puțin timp înainte ca nic să fie înapoi la 1000 Mbit.

```
root@laika:~# ethtool eth0 | grep Speed
Speed: 1000Mb/s
root@laika:~# ethtool -s eth0 speed 100
root@laika:~# ethtool eth0 | grep Speed
Speed: 100Mb/s
root@laika:~# ethtool -s eth0 speed 1000
root@laika:~# ethtool eth0 | grep Speed
Speed: 1000Mb/s
```

19.10. practică: configurare interfață de rețea

1. Verificați dacă **dhclient** se execută.
2. Afișați adresa (adresele) ip curente.
3. Afișați fișierul de configurație unde este definită **adresa ip**.
4. Urmăriți **configurarea nic** în carte pentru a schimba adresa ip de la **client dhcp** la **fixă**. Păstrați aceeași **adresă ip** pentru a evita conflicte!
5. Ați configurat și **gateway-ul** corect în întrebarea de mai sus? Dacă nu, atunci faceți asta acum.
6. Verificați dacă aveți un gateway.
7. Verificați dacă puteți să vă conectați la gateway, dacă conexiunea este vie.
8. Schimbați ultimele două cifre a **adresei mac**.
9. Care porturi sînt utilizate de http, pop3, ssh, telnet, nntp și ftp?
Observați că **sctp** a fost omis din captura de ecran.
10. Explicați de ce e-mail-ul și siturile web sînt transmise prin **tcp** și nu **udp**.
11. Afișați **hostname-ul** computerului.
12. Cu care adrese ip a avut contact recent computerul?

19.11. soluție: configurare interfață de rețea

1. Verificați dacă dhclient se execută.

```
paul@debian5:~$ ps fax | grep dhclient
```

2. Afișați adresa (adresele) ip curente.

```
paul@debian5:~$ /sbin/ifconfig | grep 'inet '
inet addr:192.168.1.31      Bcast:192.168.1.255  Mask:255.255.255.0
inet addr:127.0.0.1        Mask:255.0.0.0
```

3. Afișați fișierul de configurație unde este definită **adresa ip**.

```
Ubuntu/Debian: cat /etc/network/interfaces
Redhat/Fedora: cat /etc/sysconfig/network-scripts/ifcfg-eth*
```

4. Urmăriți **configurarea nic** în carte pentru a schimba adresa ip de la **client dhcp** la **fixă**. Păstrați aceeași **adresă ip** pentru a evita conflicte!

```
Ubuntu/Debian:
ifdown eth0
vi /etc/network/interfaces
ifup eth0
```

```
Redhat/Fedora:
ifdown eth0
vi /etc/sysconfig/network-scripts/ifcfg-eth0
ifup eth0
```

5. Ați configurat și gateway-ul corect în întrebarea de mai sus? Dacă nu, atunci faceți asta acum.

6. Verificați dacă aveți un gateway.

```
paul@debian5:~$ /sbin/route
Kernel IP routing table

```

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
192.168.1.0	*	255.255.255.0	U	0	0	0	eth0
default	192.168.1.1	0.0.0.0	UG	0	0	0	eth0

7. Verificați dacă puteți să vă conectați la gateway, dacă conexiunea este vie.

```
paul@debian5:~$ ping -c3 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=254 time=2.28 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=254 time=2.94 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=254 time=2.34 ms

--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 2.283/2.524/2.941/0.296 ms
```

8. Schimbați ultimele două cifre a adresei mac.

```
[root@rhel6 ~]# ifconfig eth0 hw ether 08:00:27:ab:67:XX
```

9. Care porturi sînt utilizate de http, pop3, ssh, telnet, nntp și ftp?

```

root@rhel6 ~# grep '^http ' /etc/services
http      80/tcp      www www-http    # WorldWideWeb HTTP
http      80/tcp      www www-http    # HyperText Transfer Protocol
root@rhel6~# grep '^smtp ' /etc/services
smtp      25/tcp      mail
smtp      25/udp      mail
root@rhel6 ~# grep '^ssh ' /etc/services
ssh       22/tcp      # The Secure Shell (SSH) Protocol
ssh       22/udp      # The Secure Shell (SSH) Protocol
root@rhel6~# grep '^telnet ' /etc/services
telnet    23/tcp
telnet    23/udp
root@rhel6~# grep '^nntp ' /etc/services
nntp      119/tcp      readnews untp   # USENET News Transfer Protocol
nntp      119/udp      readnews untp   # USENET News Transfer Protocol
root@rhel6~# grep '^ftp ' /etc/services
ftp       21/tcp
ftp       21/udp      fsp  fspd

```

Observați că sctp a fost omis din captura de ecran.

10. Explicați de ce e-mail-ul și siturile web sînt transmise prin **tcp** și nu **udp**.

Pentru că tcp este de încredere și udp nu.

11. Afișați **hostname-ul** computerului.

```

paul@debian5:~$ hostname
debian5

```

12. Cu care adrese ip a avut contact recent computerul?

```

root@rhel6 ~# arp -a
? (192.168.1.1) at 00:02:cf:aa:68:f0 [ether] on eth2
? (192.168.1.30) at 00:26:bb:12:7a:5e [ether] on eth2
? (192.168.1.31) at 08:00:27:8e:8a:a8 [ether] on eth2

```

Capitolul 20. interceptare rețea

Conținut

20.1. wireshark.	199
20.2. tcpdump.	200
20.3. practică: interceptare rețea.	202
20.4. soluție: interceptare rețea.	203

Un administrator de rețea bun ar trebui să fie capabil să utilizeze un interceptor ca **wireshark** sau **tcpdump** pentru a depana probleme ale rețelei.

Un student bun va utiliza deseori un interceptor pentru a învăța rețele. Acest capitol vă introduce la **interceptarea rețelei**.

20.1. wireshark

instalare wireshark

Acest exemplu arată cum să instalăm **wireshark** pe distribuții bazate pe **.deb** ca Ubuntu și Debian.

```
aptitude install wireshark
```

Pe distribuții bazate pe **.rpm** puteți folosi **yum** pentru a instala wireshark.

```
yum install wireshark
```

selectarea interfeței

Când deschideți pentru prima dată wireshark, va trebui să selectați o interfață de interceptat. Veți vedea o casuță de dialog similară cu aceasta. Alegeți interfața pe care doriți să o interceptați.



Pe unele distribuții doar utilizatorului root îi este permis să intercepteze rețeaua. S-ar putea să aveți nevoie să folosiți **sudo wireshark**.

începeți interceptarea

În acest exemplu, am interceptat un ping între două computere. Prima casuță din tabel arată că wireshark recunoaște protocolul icmp, și a capturat toate pachetele ping dintre cele două computere.

No. .	Time	Source	Destination	Protocol	Info
1	0.000000	192.168.1.34	192.168.1.1	ICMP	Echo (ping) request
2	0.000389	192.168.1.1	192.168.1.34	ICMP	Echo (ping) reply
3	1.000001	192.168.1.34	192.168.1.1	ICMP	Echo (ping) request
4	1.000378	192.168.1.1	192.168.1.34	ICMP	Echo (ping) reply
5	1.999996	192.168.1.34	192.168.1.1	ICMP	Echo (ping) request
6	2.000391	192.168.1.1	192.168.1.34	ICMP	Echo (ping) reply
7	3.000004	192.168.1.34	192.168.1.1	ICMP	Echo (ping) request
8	3.000380	192.168.1.1	192.168.1.34	ICMP	Echo (ping) reply

privind în interiorul pachetelor

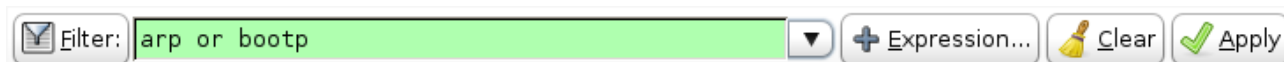
Căsuța din mijloc poate fi mărită. Când selectăm o linie în acest panel, putem vedea biții corespunzători în chenarul din panel-ul de jos.

III		
▶ Frame 1 (98 bytes on wire, 98 bytes captured)		
▼ Ethernet II, Src: Clevo_4e:ae:17 (00:90:f5:4e:ae:17), Dst: Arcadyan_2a:c5:0b (00:12:bf:2a:c5:0b)		
▶ Destination: Arcadyan_2a:c5:0b (00:12:bf:2a:c5:0b)		
▶ Source: Clevo_4e:ae:17 (00:90:f5:4e:ae:17)		
Type: IP (0x0800)		
▶ Internet Protocol, Src: 192.168.1.34 (192.168.1.34), Dst: 192.168.1.1 (192.168.1.1)		
▶ Internet Control Message Protocol		
III		
0000	00 12 bf 2a c5 0b 00 90 f5 4e ae 17 08 00 45 00	...*...N...E.
0010	00 54 00 00 40 00 40 01 b7 35 c0 a8 01 22 c0 a8	.T..@.@. .5..."..
0020	01 01 08 00 4b f2 43 2a 00 99 1f c6 89 49 d1 37	K.C*I.7
0030	03 00 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15	
0040	16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25	 !"#\$\$%
0050	26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35	&'()*+,- ./012345
0060	36 37	67

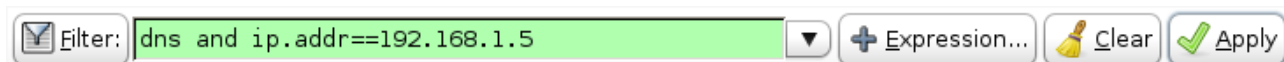
folosiți filtre

Puteți să vă pierdeți în prea multe pachete. O soluție rapidă de a vedea doar pachetele pe care vă interesează este de a aplica filtre. Când tastați **arp** și faceți click apply, veți vedea afișate doar pachetele **arp**.

Puteți combina două protocoale cu un logic **sau** între ele. Exemplul de mai jos arată cum să filtrăm doar pachete **arp** și **bootp** (sau **dhcp**).



Acest exemplu ne arată cum să filtrăm traficul **dns** care conține o anumită **adresă ip**.



20.2. tcpdump

Interceptarea de pe linia de comandă poate fi făcută cu **tcpdump**. Aici sînt niște exemple.

Folosind comanda **tcpdump host \$ip** afișează tot traficul cu o singură gazdă (192.168.1.38 în acest exemplu).

```
root@ubuntu910:~# tcpdump host 192.168.1.38
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 96 bytes
```

Captarea doar a traficului ssh (portul tcp 22) poate fi făcută cu **tcpdump tcp port \$port**. Această captură de ecran este tăiată la 76 de caractere pentru a fi ușor de citit în format pdf.


```

root@deb503:~# tcpdump tcp port 22
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 96 bytes
14:22:20.716313 IP deb503.local.37973 > rhel53.local.ssh: P 666050963:66605
14:22:20.719936 IP rhel53.local.ssh > deb503.local.37973: P 1:49(48) ack 48
14:22:20.720922 IP rhel53.local.ssh > deb503.local.37973: P 49:113(64) ack
14:22:20.721321 IP rhel53.local.ssh > deb503.local.37973: P 113:161(48) ack
14:22:20.721820 IP deb503.local.37973 > rhel53.local.ssh: . ack 161 win 200
14:22:20.722492 IP rhel53.local.ssh > deb503.local.37973: P 161:225(64) ack
14:22:20.760602 IP deb503.local.37973 > rhel53.local.ssh: . ack 225 win 200
14:22:23.108106 IP deb503.local.54424 > ubuntu910.local.ssh: P 467252637:46
14:22:23.116804 IP ubuntu910.local.ssh > deb503.local.54424: P 1:81(80) ack
14:22:23.116844 IP deb503.local.54424 > ubuntu910.local.ssh: . ack 81 win 2
^C
10 packets captured
10 packets received by filter
0 packets dropped by kernel

```

La fel ca mai sus, dar scrie ieşirea într-un fişier cu comanda **tcpdump -w \$filename**

```

root@ubuntu910:~# tcpdump -w sshdump.tcpdump tcp port 22
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 96 bytes
^C
17 packets captured
17 packets received by filter
0 packets dropped by kernel

```

Fişierul creat mai sus poate fi afişat cu comanda **tcpdump -r \$filename**.

```

root@ubuntu910:~# tcpdump -r sshdump.tcpdump

```

Mult mai multe exemple pot fi găsite în pagina de manual a **tpcdump**.

20.3. practică: interceptare rețea

1. Instalați wireshark pe computer (nu în interiorul unei mașini virtuale).
2. Faceți un ping între computer și alt computer.
3. Începeți să interceptați rețeaua.
4. Afișați doar echo al ping-ului în panel-ul de sus folosind un filtru.
5. Acum faceți ping la un nume (ca `www.linux-training.be`) și încercați să interceptați cererea DNS și răspunsul. Care server DNS a fost utilizat? A fost o cerere și un răspuns tcp sau udp?

20.4. soluție: interceptare rețea

1. Instalați wireshark pe computer (nu în interiorul unei mașini virtuale).

Debian/Ubuntu: `aptitude install wireshark`

Red Hat/Mandriva/Fedora: `yum install wireshark`

2. Faceți un ping între computer și alt computer.

`ping $ip_address`

3. Începeți să interceptați rețeaua.

`(sudo) wireshark`

selectați o interfață (probabil `eth0`)

4. Afișați doar echo al ping-ului în panel-ul de sus folosind un filtru.

tastați `'icmp'` (fără ghilimele) în căsuța filtru, apoi faceți click pe `'apply'`

5. Acum faceți ping la un nume (ca `www.linux-training.be`) și încercați să interceptați cererea DNS și răspunsul. Care server DNS a fost utilizat? A fost o cerere și un răspuns tcp sau udp?

Mai întâi porniți interceptorul.

Scrieți `'dns'` în căsuța filtru și faceți click apply.

```
root@ubuntu910:~# ping www.linux-training.be
PING www.linux-training.be (88.151.243.8) 56(84) bytes of data.
64 bytes from fosfor.openminds.be (88.151.243.8): icmp_seq=1 ttl=58 time=14.9 ms
64 bytes from fosfor.openminds.be (88.151.243.8): icmp_seq=2 ttl=58 time=16.0 ms
^C
--- www.linux-training.be ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 14.984/15.539/16.095/0.569 ms
```

Ecranul wireshark ar trebui să arate aproximativ astfel.

File Edit View Go Capture Analyze Statistics Telephony Tools Help					
					
Filter: dns + Expression... Clear Apply					
No..	Time	Source	Destination	Protocol	Info
18	8.710490	192.168.1.34	192.168.1.1	DNS	Standard query A www.linux-training.be
19	8.724596	192.168.1.1	192.168.1.34	DNS	Standard query response A 88.151.243.8

Detaliile în wireshark vor transmite că cererea DNS a fost în interiorul unui pachet udp.

Capitolul 21. legare și cuplare

Conținut

21.1. legare pe Redhat/Fedora.	205
21.2. legare pe Debian/Ubuntu.	206
21.3. cuplare pe Redhat/Fedora.	206
21.4. cuplare pe Debian/Ubuntu.	208
21.5. practică: legare și cuplare.	210
21.6. soluție: legare și cuplare.	211

Uneori un server are nevoie de mai multe **adrese ip** pe aceeași placă de rețea, noi numim asta **legarea (binding)** adreselor ip.

Linux poate de asemeni să activeze mai multe plăci de rețea în spatele aceleiași **adrese ip**, asta se numește **cuplare (bonding)**.

Acest capitol vă va învăța cum să configurați **legarea** și **cuplarea** pe cele mai comune distribuții Linux.

21.1. legare pe Redhat/Fedora

legarea adreselor ip extra

Pentru a lega mai mult decît o singură **adresă ip** pe aceeași interfață, folosiți **ifcfg-eth0:0**, unde ultimul zero poate fi orice altceva. Doar două directive sînt cerute în fișiere.

```
[root@rhel6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0:0
DEVICE="eth0:0"
IPADDR="192.168.1.133"
[root@rhel6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0:1
DEVICE="eth0:0"
IPADDR="192.168.1.142"
```

autorizarea adreselor ip extra

Pentru a activa o interfață de rețea virtuală, folosiți **ifup**, pentru a o dezactiva folosiți **ifdown**.

```
[root@rhel6 ~]# ifup eth0:0
[root@rhel6 ~]# ifconfig | grep 'inet '
    inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:192.168.1.133 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:127.0.0.1 Mask:255.0.0.0
[root@rhel6 ~]# ifup eth0:1
[root@rhel6 ~]# ifconfig | grep 'inet '
    inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:192.168.1.133 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:192.168.1.142 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:127.0.0.1 Mask:255.0.0.0
```

verificarea adreselor ip extra

Folosiți **ping** de pe un alt computer pentru a verifica activarea, sau folosiți **ifconfig** ca în acest screenshot.

```
[root@rhel6 ~]# ifconfig
eth0    Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
        inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
        inet6 addr: fe80::a00:27ff:fedd:d5c/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
        RX packets:1259 errors:0 dropped:0 overruns:0 frame:0
        TX packets:545 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:115260 (112.5 KiB) TX bytes:84293 (82.3 KiB)

eth0:0  Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
        inet addr:192.168.1.133 Bcast:192.168.1.255 Mask:255.255.255.0
        UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1

eth0:1  Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
        inet addr:192.168.1.142 Bcast:192.168.1.255 Mask:255.255.255.0
        UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
```

21.2. legare pe Debian/Ubuntu

legarea adreselor ip extra

Configurarea adreselor ip multiple pe aceeași placă de rețea se face în **/etc/network/interfaces** adăugând dispozitive **eth0:x**.

Adăugarea **netmask** (măștii de rețea) este obligatorie.

```
debian5:~# cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).
```

```
# The loopback network interface
auto lo
iface lo inet loopback
```

```
# The primary network interface
iface eth0 inet static
address 192.168.1.34
network 192.168.1.0
netmask 255.255.255.0
gateway 192.168.1.1
auto eth0
```

```
auto eth0:0
iface eth0:0 inet static
address 192.168.1.233
netmask 255.255.255.0
```

```
auto eth0:1
iface eth0:1 inet static
address 192.168.1.242
netmask 255.255.255.0
```

autorizarea adreselor ip extra

Folosiți **ifup** pentru a autoriza adresele extra.

```
debian5:~# ifup eth0:0
debian5:~# ifup eth0:1
```

verificarea adreselor ip extra

Folosiți **ping** de pe un alt computer pentru a verifica activarea, sau folosiți **ifconfig** ca în această captură de ecran.

```
debian5:~#ifconfig | grep 'inet '
    inet addr:192.168.1.34 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:192.168.1.233 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:192.168.1.242 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:127.0.0.1 Mask:255.0.0.0
```

21.3. cuplare pe Redhat/Fedora

Începem cu **ifconfig -a** pentru a obține o listă a tuturor plăcilor de rețea de pe sistemul nostru.

```
[root@rhel6 network-scripts]# ifconfig -a | grep Ethernet
eth0      Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
eth1      Link encap:Ethernet HWaddr 08:00:27:DA:C1:49
eth2      Link encap:Ethernet HWaddr 08:00:27:40:03:3B
```

În acest demo vom decide să cuplăm **eth1** și **eth2**.

Vom numi cuplarea **bond0** și vom adăuga această intrare în **modprobe** astfel încât kernel-ul să poată încărca **modulul cuplaj** când aducem la viață interfața.

```
[root@rhel6 network-scripts]# cat /etc/modprobe.d/bonding.conf
alias bond0 bonding
```

Apoi creăm **/etc/sysconfig/network-scripts/ifcfg-bond0** pentru a configura interfața noastră **bond0**.

```
[root@rhel6 network-scripts]# pwd
/etc/sysconfig/network-scripts
[root@rhel6 network-scripts]# cat ifcfg-bond0
DEVICE=bond0
IPADDR=192.168.1.199
NETMASK=255.255.255.0
ONBOOT=yes
BOOTPROTO=none
USERCTL=no
```

Apoi creăm două fișiere, una pentru fiecare placă de rețea pe care o vom folosi ca slave în **bond0**.

```
[root@rhel6 network-scripts]# cat ifcfg-eth1
DEVICE=eth1
BOOTPROTO=none
ONBOOT=yes
MASTER=bond0
SLAVE=yes
USERCTL=no
[root@rhel6 network-scripts]# cat ifcfg-eth2
DEVICE=eth2
BOOTPROTO=none
ONBOOT=yes
MASTER=bond0
SLAVE=yes
USERCTL=no
```

La sfârșit aducem interfața la viață cu **ifup bond0**.

```
[root@rhel6 network-scripts]# ifup bond0
[root@rhel6 network-scripts]# ifconfig bond0
bond0      Link encap:Ethernet HWaddr 08:00:27:DA:C1:49
            inet addr:192.168.1.199 Bcast:192.168.1.255 Mask:255.255.255.0
            inet6 addr: fe80::a00:27ff:feda:c149/64 Scope:Link
            UP BROADCAST RUNNING MASTER MULTICAST MTU:1500 Metric:1
            RX packets:251 errors:0 dropped:0 overruns:0 frame:0
            TX packets:21 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:0
            RX bytes:39852 (38.9 KiB) TX bytes:1070 (1.0 KiB)
```

Cuplarea ar trebui să fie de asemeni vizibilă în **/proc/net/bonding**.

```
[root@rhel6 network-scripts]# cat /proc/net/bonding/bond0
Ethernet Channel Bonding Driver: v3.5.0 (November 4, 2008)
```

```
Bonding Mode: load balancing (round-robin)
MII Status: up
MII Polling Interval (ms): 0
Up Delay (ms): 0
Down Delay (ms): 0
```

```
Slave Interface: eth1
MII Status: up
Link Failure Count: 0
Permanent HW addr: 08:00:27:da:c1:49
Slave Interface: eth2
MII Status: up
Link Failure Count: 0
Permanent HW addr: 08:00:27:40:03:3b
```

21.4. cuplare pe Debian/Ubuntu

Începem cu **ifconfig -a** pentru a obține o listă a tuturor plăcilor de rețea de pe sistemul nostru.

```
debian5:~# ifconfig -a | grep Ethernet
eth0      Link encap:Ethernet HWaddr 08:00:27:bb:18:a4
eth1      Link encap:Ethernet HWaddr 08:00:27:63:9a:95
eth2      Link encap:Ethernet HWaddr 08:00:27:27:a4:92
```

În acest demo vom decide să cuplăm **eth1** și **eth2**.

Va trebui de asemeni să instalăm pachetul **ifenslave**.

```
debian5:~# aptitude search ifenslave
p ifenslave          - Attach and detach slave interfaces to a bonding device
p ifenslave-2.6      - Attach and detach slave interfaces to a bonding device
debian5:~# aptitude install ifenslave
Reading package lists... Done
. . .
```

Apoi aducem la zi fișierul **/etc/network/interfaces** cu informații despre interfața **bond0**.

```
debian5:~# tail -7 /etc/network/interfaces
iface bond0 inet static
address 192.168.1.42
netmask 255.255.255.0
gateway 192.168.1.1
slaves eth1 eth2
bond-mode active-backup
bond_primary eth1
```

Pe versiuni mai vechi de Debian/Ubuntu trebuia să tastați **modprobe bonding**, dar asta nu mai e cerut deloc. Folosiți **ifup** pentru a aduce la viață interfața, apoi testați dacă funcționează.


```
debian5:~# ifup bond0
debian5:~# ifconfig bond0
bond0    Link encap:Ethernet HWaddr 08:00:27:63:9a:95
         inet addr:192.168.1.42 Bcast:192.168.1.255 Mask:255.255.255.0
         inet6 addr: fe80::a00:27ff:fe63:9a95/64 Scope:Link
         UP BROADCAST RUNNING MASTER MULTICAST MTU:1500 Metric:1
         RX packets:212 errors:0 dropped:0 overruns:0 frame:0
         TX packets:39 errors:0 dropped:0 overruns:0 carrier:0
         collisions:0 txqueuelen:0
         RX bytes:31978 (31.2 KiB) TX bytes:6709 (6.5 KiB)
```

Cuplarea ar trebui să fie de asemeni vizibilă în **/proc/net/bonding**.

```
debian5:~# cat /proc/net/bonding/bond0
Ethernet Channel Bonding Driver: v3.2.5 (March 21, 2008)
```

```
Bonding Mode: fault-tolerance (active-backup)
Primary Slave: eth1
Currently Active Slave: eth1
MII Status: up
MII Polling Interval (ms): 0
Up Delay (ms): 0
Down Delay (ms): 0
```

```
Slave Interface: eth1
MII Status: up
Link Failure Count: 0
Permanent HW addr: 08:00:27:63:9a:95
```

```
Slave Interface: eth2
MII Status: up
Link Failure Count: 0
Permanent HW addr: 08:00:27:27:a4:92
```

21.5. practică: legare și cuplare

1. Adăugați o **adresă ip** extra la una dintre plăcile de rețea. Testați dacă funcționează (puneți pe un coleg să facă ssh la ea)!
2. Folosiți **ifdown** pentru a anula această **adresă ip** extra.
3. Fiți sigur că colegul a reușit de asemeni să **lege** o adresă ip extra înainte de a continua.
4. Adăugați o altă placă de rețea (sau două) la mașina virtuală și folosiți teoria pentru a **cupla** două plăci de rețea.

21.6. soluție: legare și cuplare

1. Adăugați o **adresă ip** extra la una dintre plăcile de rețea. Testați dacă funcționează (puneți pe un coleg să facă ssh la ea)!

Redhat/Fedora:

adăugați un fișier `/etc/sysconfig/network-scripts/ifcfg-ethX:X` cum este arătat în teorie.

Debian/Ubuntu:

măriți fișierul `/etc/network/interfaces` cum este arătat în teorie.

2. Folosiți **ifdown** pentru a anula această **adresă ip** extra.

```
ifdown eth0:0
```

3. Fiți sigur că colegul a reușit de asemeni să lege o adresă ip extra înainte de a continua.

```
ping $extra_ip_neighbour
```

```
sau
```

```
ssh $extra_ip_neighbour
```

4. Adăugați o altă placă de rețea (sau două) la mașina virtuală și folosiți teoria pentru a **cupla** două plăci de rețea.

Redhat/Fedora:

adăugați fișierele `ifcfg-ethX` și `ifcfg-bondX` în `/etc/sysconfig/network-scripts` așa cum este arătat în teorie și nu uitați `modprobe.conf`

Debian/Ubuntu:

extindeți fișierul `/etc/network/interfaces` așa cum e arătat în teorie și nu uitați să instalați pachetul `ifenslave`.

Capitolul 22. client ssh și server

Conținut

22.1.	despre ssh.	213
22.2.	intrare pe un server remote.	214
22.3.	execuția unei comenzi pe remote.	214
22.4.	scp.	215
22.5.	setare ssh fără parolă.	215
22.6.	destinație X prin ssh.	217
22.7.	depanare ssh.	217
22.8.	sshd.	218
22.9.	chei sshd.	218
22.10.	ssh-agent.	218
22.11.	practică: ssh.	219
22.12.	soluție: ssh.	220

secure shell sau **ssh** este o colecție de utilitare care folosesc un protocol sigur pentru comunicare cu computere Linux remote.

Acest capitol dă o trecere în revistă a celor mai comune comenzi în legătură cu utilizarea serverului **sshd** și a clientului **ssh**.

22.1. despre ssh

shell securizat

Evitați folosirea **telnet**, **rlogin** și **rsh** pentru a vă conecta remote cu serverele. Aceste protocoale mai vechi nu criptează sesiunea de accesare a sistemului, ceea ce înseamnă că utilizatorul id și parola pot fi interceptate cu utilitate ca **wireshark** sau **tcpdump**. Pentru a vă conecta sigur cu serverele, folosiți **ssh**.

Protocolul ssh este sigur în două moduri. În primul rând conexiunea este **criptată** și în al doilea rând conexiunea este **autenticată** în ambele sensuri.

O conexiune ssh începe întotdeauna cu un handshake criptografic, urmat de **criptarea** stratului de transport folosind un cifru simetric. Cu alte cuvinte, tunelul este criptat înainte de a începe să tastați ceva.

Apoi **autentificarea** are loc (folosind parola/user id a utilizatorului sau chei publice/private) și comunicarea poate începe într-o conexiune criptată.

Protocolul ssh își va aminti serverele cu care s-a conectat (și vă va avertiza în caz că ceva suspicios a avut loc).

Pachetul **openssh** este menținut de către comunitatea **OpenBSD** și este distribuit în multe sisteme de operare (ar putea fi chiar cel mai popular pachet software din lume).

/etc/ssh/

Configurarea clientului **ssh** și a serverului ssh se face în directorul **/etc/ssh**. În următoarele secțiuni vom discuta cele mai multe fișiere găsite în **/etc/ssh**.

versiunile de protocol ssh

Protocolul **ssh** are două versiuni (1 și 2). Evitați să folosiți versiunea 1 oriunde, deoarece ea conține unele vulnerabilități cunoscute. Puteți verifica versiunea de protocol prin **/etc/ssh/ssh_config** pentru partea client și **/etc/ssh/sshd_config** pentru daemonul openssh-server.

```
paul@ubu1204:/etc/ssh$ grep Protocol ssh_config
# Protocol 2,1
paul@ubu1204:/etc/ssh$ grep Protocol sshd_config
Protocol 2
```

chei publice și private

Protocolul **ssh** folosește sistemul bine cunoscut al **cheilor publice și private**. Explicația de mai jos este succintă, mai multă informație poate fi găsită pe wikipedia.

http://en.wikipedia.org/wiki/Public-key_cryptography

Imaginați-vă pe Alice și Bob, doi oameni cărora le place să comunice unul cu altul. Folosind **chei publice și private** ei pot comunica **criptat** cu **autentificare**.

Când Alice vrea să-i trimită un mesaj criptat lui Bob, ea folosește **cheia publică** a lui Bob. Bob împarte **cheia lui publică** cu Alice, dar își păstrează **cheia lui privată**! Deoarece Bob este singurul care are cheia lui **privată**, Alice este sigură că Bob este singurul care poate citi mesajul criptat. Când Bob vrea să verifice dacă mesajul a venit de la Alice, Bob folosește **cheia publică** a lui Alice pentru a verifica dacă Alice a semnat mesajul cu cheia ei

privată. Pentru că Alice este singura care are **cheia privată** a lui Alice, Bob este sigur că mesajul a venit de la Alice.

algoritme rsa și dsa

Acest capitol nu explică implementarea tehnică a algoritmilor criptografice, aici explicăm cum să utilizăm utilitățile ssh cu **rsa** și **dsa**. Mai multe informații despre aceste algoritme pot fi găsite aici:

[http://en.wikipedia.org/wiki/RSA_\(algorithm\)](http://en.wikipedia.org/wiki/RSA_(algorithm))

http://en.wikipedia.org/wiki/Digital_Signature_Algorithm

22.2. intrare pe un server remote

Următorul screenshot ne arată cum să utilizăm **ssh** pentru a intra pe un computer remote rulând Linux. Utilizatorul local e numit **paul** și el este intrat în sistem ca utilizatorul **admin42** pe sistemul remote.

```
paul@ubu1204:~$ ssh admin42@192.168.1.30
The authenticity of host '192.168.1.30 (192.168.1.30)' can't be established.
RSA key fingerprint is b5:fb:3c:53:50:b4:ab:81:f3:cd:2e:bb:ba:44:d3:75.
Are you sure you want to continue connecting (yes/no)?
```

Precum vedeți, utilizatorul **paul** este reprezentat cu o amprentă de autentificare **rsa** de la sistemul remote. Utilizatorul poate accepta asta tastând **yes**. Vom vedea mai târziu că o intrare va fi adăugată în fișierul **~/.ssh_known_hosts**.

```
paul@ubu1204:~$ ssh admin42@192.168.1.30
The authenticity of host '192.168.1.30 (192.168.1.30)' can't be established.
RSA key fingerprint is b5:fb:3c:53:50:b4:ab:81:f3:cd:2e:bb:ba:44:d3:75.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.1.30' (RSA) to the list of known hosts.
admin42@192.168.1.30's password:
Welcome to Ubuntu 12.04 LTS (GNU/Linux 3.2.0-26-generic-pae i686)
```

* Documentation: <https://help.ubuntu.com/>

```
1 package can be updated.
0 updates are security updates.
```

```
Last login: Wed Jun 6 19:25:57 2012 from 172.28.0.131
admin42@ubuserver:~$
```

Utilizatorul poate să iasă de pe serverul remote tastând **exit** sau folosind **Ctrl-d**.

```
admin42@ubuserver:~$ exit
logout
Connection to 192.168.1.30 closed.
paul@ubu1204:~$
```

22.3. execuția unei comenzi pe remote

Această captură de ecran arată cum să executăm comanda **pwd** pe serverul remote. Nu există nici o nevoie de a **ieși** de pe server manual.

```
paul@ubu1204:~$ ssh admin42@192.168.1.30 pwd
admin42@192.168.1.30's password:
/home/admin42
paul@ubu1204:~$
```

22.4. scp

Comanda **scp** funcționează la fel ca **cp**, dar permite ca sursa și destinația copiată să fie cu **ssh**. Aici este un exemplu unde copiem fișierul **/etc/hosts** de pe serverul remote în directorul home al utilizatorului paul.

```
paul@ubu1204:~$ scp admin42@192.168.1.30:/etc/hosts /home/paul/serverhosts
admin42@192.168.1.30's password:
hosts                                100%      809          0.8KB/s   00:00
```

Aici e un exemplu în revers, unde copiem un fișier local la un server remote.

```
paul@ubu1204:~$ scp ~/serverhosts admin42@192.168.1.30:/etc/hosts.new
admin42@192.168.1.30's password:
serverhosts                          100%      809          0.8KB/s   00:00
```

22.5. setare ssh fără parolă

Pentru a seta autentificare ssh fără parolă prin intermediul cheilor publice/private, folosiți **ssh-keygen** pentru a genera o pereche de chei fără parolă, și apoi copiați cheia publică pe serverul destinație. Să facem asta pas cu pas.

În exemplul care urmează, vom seta ssh fără parolă între Alice și Bob. Alice are un cont pe un server Red Hat Enterprise Linux, Bob folosește Ubuntu pe laptopul lui. Bob vrea să-i dea acces lui Alice folosind ssh și sistemul de cheie publică și privată. Asta înseamnă că chiar dacă Bob își schimbă parola pe laptopul lui, Alice încă va avea acces.

ssh-keygen

Exemplul de mai jos arată cum Alice folosește **ssh-keygen** pentru a genera o pereche de chei. Alice nu a tastat o parolă.

```
[alice@RHEL5 ~]$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/alice/.ssh/id_rsa):
Created directory '/home/alice/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alice/.ssh/id_rsa.
Your public key has been saved in /home/alice/.ssh/id_rsa.pub.
The key fingerprint is:
9b:ac:ac:56:c2:98:e5:d9:18:c4:2a:51:72:bb:45:eb alice@RHEL5
[alice@RHEL5 ~]$
```

Puteți folosi **ssh-keygen -t dsa** în aceeași modalitate.

~/.ssh

În timp ce **ssh-keygen** generează o cheie publică și o cheie privată, va crea la fel un director ascuns **.ssh** cu permisiunile de rigoare. Dacă creați directorul **.ssh** manual, atunci va trebui să-l faceți cu **chmod 700**! Altfel ssh va refuza să

utilizeze cheile (cheile private cu drepturi de citire pentru ceilalți nu sînt sigure!).

Precum vedeți, directorul **.ssh** este securizat în directorul home a lui Alice.

```
[alice@RHEL5 ~]$ ls -ld .ssh
drwx----- 2 alice alice 4096 May 1 07:38 .ssh
[alice@RHEL5 ~]$
```

Bob utilizează Ubuntu acasă. El decide să creeze manual directorul **.ssh**, astfel că are nevoie să îl securizeze manual.

```
bob@laika:~$ mkdir .ssh
bob@laika:~$ ls -ld .ssh
drwxr-xr-x 2 bob bob 4096 2008-05-14 16:53 .ssh
bob@laika:~$ chmod 700 .ssh/
bob@laika:~$
```

id_rsa și id_rsa.pub

Comanda **ssh-keygen** generează două chei în **.ssh**. Cheia publică este numită **~/.ssh/id_rsa.pub**. Cheia privată este numită **~/.ssh/id_rsa**.

```
[alice@RHEL5 ~]$ ls -l .ssh/
total 16
-rw----- 1 alice alice 1671 May 1 07:38 id_rsa
-rw-r--r-- 1 alice alice  393 May 1 07:38 id_rsa.pub
```

Fișierele vor fi numite **id_dsa** și **id_dsa.pub** cînd folosim **dsa** în loc de **rsa**.

copierea cheii publice către celălalt computer

Pentru a copia cheia publică de la serverul lui Alice pe laptopul lui Bob, Alice decide să folosească **scp**.

```
[alice@RHEL5 .ssh]$ scp id_rsa.pub bob@192.168.48.92:~/.ssh/authorized_keys
bob@192.168.48.92's password:
id_rsa.pub                                100% 393 0.4KB/s 00:00
```

Fiți atenți cînd copiați cea de-a doua cheie! Nu suprascrieți prima cheie, în schimb adăugați cheia în același fișier **~/.ssh/authorized_keys**!

```
cat id_rsa.pub >> ~/.ssh/authorized_keys
```

Alice ar putea să folosească **ssh-copy-id** ca în acest exemplu.

```
ssh-copy-id -i .ssh/id_rsa.pub bob@192.168.48.92
```

authorized_keys

În directorul **~/.ssh**, puteți crea un fișier numit **authorized_keys**. Acest fișier poate conține una sau mai multe chei publice de la oamenii în care aveți încredere. Acei oameni de încredere pot utiliza cheile lor private pentru a demonstra identitatea lor și să obțină acces la contul dumneavoastră prin **ssh** (fără parolă). Acest exemplu arată fișierul **authorized_keys** a lui Bob care conține cheia publică a lui Alice.


```
bob@laika:~$ cat .ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEApcQ9xzyLzJes1sR+hPyqW2vyzt1D4zTLqk\
MDWBR4mMFuUZD/O583I3Lg/Q+JIq0RSksNzaL/BNLDou1jMpBe2Dmf/u22u4KmqlJBfDhe\
yTmGSBzeNYCYRSMq78CT9l9a+y6x/shucwhaILsy8A2XfJ9VCgkVtu7XlWFDL2cum08/0\
mRFwVrfc/uPsAn5XkkTsc14g21mQbnp9wJC40pGSJXXMuF0k8MgCb5ieSnpKFniAKM+tEo\
/vjDGSi3F/bxu691jscrU0VUDIo0So98HUfEf7jKBRikxGAC7I4HLA+/zX730IvRFAb2hv\
tUhn6RHRbTUUJjbSGiYeFTLDFcTQ== alice@RHEL5
```

ssh fără parolă

Alice poate acum să utilizeze ssh fără parolă ca să se conecteze la laptopul lui Bob. În combinație cu capabilitatea **ssh** de a rula comenzi pe gazda remote, asta poate fi folositor în conducte printre mașini diferite.

```
[alice@RHEL5 ~]$ ssh bob@192.168.48.92 "ls -l .ssh"
total 4
-rw-r--r-- 1 bob bob 393 2008-05-14 17:03 authorized_keys
[alice@RHEL5 ~]$
```

22.6. destinație X prin ssh

O altă autorizare populară a **ssh** este numită **destinație X11** și este implementată cu **ssh-X**.

Mai jos este un exemplu a **destinației X**: utilizatorul paul intră în sistem ca utilizatorul greet pe computerul ei pentru a începe aplicația grafică mozilla-thunderbird. Deși aplicația va rula pe computerul remote de la greet, ea va fi afișată pe ecran ca atașată local la computerul lui paul.

```
paul@debian5:~/PDF$ ssh -X greet@greet.dyndns.org -p 55555
Warning: Permanently added the RSA host key for IP address \
'81.240.174.161' to the list of known hosts.
Password:
Linux raika 2.6.8-2-686 #1 Tue Aug 16 13:22:48 UTC 2005 i686 GNU/Linux

Last login: Thu Jan 18 12:35:56 2007
greet@raika:~$ ps fax | grep thun
greet@raika:~$ mozilla-thunderbird &
[1] 30336
```

22.7. depanare ssh

Folosiți **ssh -v** pentru a obține informație debug despre încercările de conexiune ssh.

```
paul@debian5:~$ ssh -v bert@192.168.1.192
OpenSSH_4.3p2 Debian-8ubuntu1, OpenSSL 0.9.8c 05 Sep 2006
debug1: Reading configuration data /home/paul/.ssh/config
debug1: Reading configuration data /etc/ssh/ssh_config
debug1: Applying options for *
debug1: Connecting to 192.168.1.192 [192.168.1.192] port 22.
debug1: Connection established.
debug1: identity file /home/paul/.ssh/identity type -1
debug1: identity file /home/paul/.ssh/id_rsa type 1
debug1: identity file /home/paul/.ssh/id_dsa type -1
debug1: Remote protocol version 1.99, remote software version OpenSSH_3
debug1: match: OpenSSH_3.9p1 pat OpenSSH_3.*
debug1: Enabling compatibility mode for protocol 2.0
. . .
```

22.8. sshd

Serverul ssh este numit **sshd** și este dat de pachetul **openssh-server**.

```
root@ubu1204~# dpkg -l openssh-server | tail -1
ii openssh-server      1:5.9p1-5ubuntu1    secure shell (SSH) server, . . .
```

22.9. chei sshd

Cheile publice folosite de serverul sshd sînt localizate în **/etc/ssh** și au drepturi de citire pentru toți. Cheile private sînt cu drepturi de citire doar pentru root.

```
root@ubu1204~# ls -l /etc/ssh/ssh_host_*
-rw----- 1 root root 668 Jun 7 2011 /etc/ssh/ssh_host_dsa_key
-rw-r--r-- 1 root root 598 Jun 7 2011 /etc/ssh/ssh_host_dsa_key.pub
-rw----- 1 root root 1679 Jun 7 2011 /etc/ssh/ssh_host_rsa_key
-rw-r--r-- 1 root root 390 Jun 7 2011 /etc/ssh/ssh_host_rsa_key.pub
```

22.10. ssh-agent

Cînd generăm chei cu **ssh-keygen**, avem opțiunea de a tasta o parolă pentru a proteja accesul la chei. Pentru a evita să tastăm tot timpul această parolă, puteți adăuga chei în **ssh-agent** folosind **ssh-add**.

Cele mai multe distribuții Linux vor începe **ssh-agent** în mod automat cînd intrăm pe sistem.

```
root@ubu1204~# ps -ef | grep ssh-agent
paul      2405 2365 0 08:13 ?                00:00:00 /usr/bin/ssh-agent...
```

Această captură de ecran prescurtată arată cum să utilizăm **ssh-add** pentru a lista cheile care sînt curent adăugate la **ssh-agent**.

```
paul@debian5:~$ ssh-add -L
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAvgI+Vx5UrisusZPl8da8URHGsxG7yivv3/\
. . .
wMGqa48Kelwom8Tgb4Sgcwpp/V0/ldA5m+BGCw== paul@deb503
```

22.11. practică: ssh

0. Asigurați-vă că aveți acces la **două computere Linux**, sau lucrați împreună cu un partener pentru acest exercițiu. Pentru această practică, vom numi una dintre mașini server.

1. Instalați **sshd** pe server.

2. Verificați în fișierele de configurare ssh să fie permis doar protocolul cu versiunea 2.

3. Folosiți **ssh** pentru a intra pe server, listați directorul curent și apoi ieșiți din server.

4. Folosiți **scp** pentru a copia un fișier de pe computer pe server.

5. Folosiți **scp** pentru a copia un fișier de pe server pe computer.

6. (opțional, funcționează doar când aveți o instalare grafică Linux). Instalați pachetul xeyes pe server și folosiți ssh pentru a executa xeyes pe server, dar afișați-l în client.

7. (opțional, la fel ca mai sus). Creați un bookmark în firefox, apoi închideți firefox pe client și pe server. Folosiți **ssh -X** pentru a executa firefox pe ecran, dar pe computerul colegului. Ați văzut bookmark-ul colegului?

8. Folosiți **ssh-keygen** pentru a crea o pereche de chei fără parolă. Setati ssh fără parolă între dvs. și coleg (sau între client și server).

9. Verificați dacă sînt corecte permisiunile fișierelor cheilor de pe server; permisiuni cu drepturi de citire pentru toți pentru cheile publice și doar acces root pentru cheile private.

10. Verificați dacă **ssh-agent** se execută.

11. (opțional). Protejați perechile de chei cu o **parolă**, apoi adăugați aceste chei la **ssh-agent** și testați conexiunea fără parolă ssh pe server.

22.12. soluție: ssh

0. Asigurați-vă că aveți acces la **două computere Linux**, sau lucrați împreună cu un partener pentru acest exercițiu. Pentru această practică, vom numi una dintre mașini server.

1. Instalați **sshd** pe server.

```
apt-get install openssh-server (pe Ubuntu/Debian)
yum -y install openssh-server (pe Centos/Fedora/Red Hat)
```

2. Verificați în fișierele de configurare ssh să fie permis doar protocolul cu versiunea 2.

```
grep Protocol /etc/ssh/ssh*_config
```

3. Folosiți **ssh** pentru a intra pe server, listați directorul curent și apoi ieșiți din server.

```
user@client$ ssh user@server-ip-address
user@server$ pwd
/home/user
user@server$ exit
```

4. Folosiți **scp** pentru a copia un fișier de pe computer pe server.

```
scp localfile user@server:~
```

5. Folosiți **scp** pentru a copia un fișier de pe server pe computer.

```
scp user@server:~/serverfile
```

6. (opțional, funcționează doar când aveți o instalare grafică Linux). Instalați pachetul xeyes pe server și folosiți ssh pentru a executa xeyes pe server, dar afișați-l în client.

```
pe server:
apt-get install xeyes
pe client:
ssh -X user@server-ip
xeyes
```

7. (opțional, la fel ca mai sus). Creați un bookmark în firefox, apoi închideți firefox pe client și pe server. Folosiți **ssh -X** pentru a executa firefox pe ecran, dar pe computerul colegului. Ați văzut bookmark-ul colegului?

8. Folosiți **ssh-keygen** pentru a crea o pereche de chei fără parolă. Setați ssh fără parolă între dvs. și coleg (sau între client și server).

Vedeți soluția în carte la capitolul "setarea ssh fără parolă".

9. Verificați dacă sînt corecte permisiunile fișierelor cheilor de pe server; permisiuni cu drepturi de citire pentru toți pentru cheile publice și doar acces root pentru cheile private.

```
ls -l /etc/ssh/ssh_host_*
```

10. Verificați dacă **ssh-agent** se execută.

```
ps fax | grep ssh-agent
```

11. (optional). Protejați perechile de chei cu o **parolă**, apoi adăugați aceste chei la **ssh-agent** și testați conexiunea fără parolă ssh pe server.

```
man ssh-keygen
```

```
man ssh-agent
```

```
man ssh-add
```

Capitolul 23. introducere în nfs

Conținut

23.1. versiunile protocol nfs.	223
23.2. rpcinfo.	223
23.3. configurare server.	224
23.4. /etc/exports.	224
23.5. exportfs.	225
23.6. configurare client.	225
23.7. practică: sistem de fișier în rețea.	227

Network File System (sau pe scurt **nfs**) ne permite încă din anii 1980 să partajăm un director cu alte computere pe rețea.

În acest capitol vedem cum să setăm un server **nfs** și un computer client **nfs**.

23.1. versiunile protocol nfs

Versiunile vechi 2 și 3 sînt fixate (**udp**) prin default (dar ele pot utiliza **tcp**).

Versiunea mai recentă **nfs versiunea 4** aduce un protocol fixat cu mai bună performanță și securitate mai puternică.

Versiunea NFS 4 a fost definită în **rfc 3010** în 2000 și **rfc 3530** în 2003 și cere **tcp** (portul 2049). Suportă de asemeni autentificare de utilizator **Kerberos** ca o opțiune cînd se montează un partaj. Versiunile NFS 2 și 3 autentifică doar gazda.

23.2. rpcinfo

Clienții se conectează la server folosind **rpc** (pe Linux asta poate fi aranjat de către daemonul **portmap**). Priviți la **rpcinfo** pentru a verifica dacă **nfs** și serviciile puse în legătură cu el se execută.

```
root@RHELv4u2:~# /etc/init.d/portmap status
portmap (pid 1920) is running...
root@RHELv4u2:~# rpcinfo -p
program      vers      proto     port
100000        2         tcp       111      portmapper
100000        2         udp       111      portmapper
100024        1         udp       32768    status
100024        1         tcp       32769    status
root@RHELv4u2:~# service nfs start
Starting NFS services:                [OK]
Starting NFS quotas:                  [OK]
Starting NFS daemon:                  [OK]
Starting NFS mountd:                  [OK]
```

Aceiași comandă **rpcinfo** cînd este deschis **nfs**.

```

root@RHELv4u2:~# rpcinfo -p
program      vers      proto     port
100000        2         tcp       111      portmapper
100000        2         tcp       111      portmapper
100024        1         udp       32768    status
100024        1         tcp       32769    status
100011        1         udp       985      rquotad
100011        2         udp       985      rquotad
100011        1         tcp       988      rquotad
100011        2         tcp       988      rquotad
100003        1         udp       2049     nfs
100003        3         udp       2049     nfs
100003        4         udp       2049     nfs
100003        1         tcp       2049     nfs
100003        3         tcp       2049     nfs
100003        4         tcp       2049     nfs
100021        1         udp       32770    nlockmgr
100021        3         udp       32770    nlockmgr
100021        4         udp       32770    nlockmgr
100021        1         tcp       32789    nlockmgr
100021        3         tcp       32789    nlockmgr
100021        4         tcp       32789    nlockmgr
100005        1         udp       1004     mountd
100005        1         tcp       1007     mountd
100005        2         udp       1004     mountd
100005        2         tcp       1007     mountd
100005        3         udp       1004     mountd
100005        3         tcp       1007     mountd
root@RHELv4u2:~#

```

23.3. configurare server

nfs este configurat în **/etc/exports**. Ați putea (prin **ldap**?) să sincronizați userid între computere dacă folosiți mult **nfs**.

Opțiunea **rootsquash** va schimba contul de utilizator UID 0 la **UID nobody** (sau similar). Opțiunea **sync** va scrie datele care se pot scrie pe disk înainte de a completa cererea client.

23.4. /etc/exports

Aici este un exemplu **/etc/exports** pentru a explica sintaxa:


```
paul@laika:~$ cat /etc/exports
# Everyone can read this share
/mnt/data/iso *(ro)

# Only the computers named pasha and barry can readwrite this one
/var/www pasha(rw) barry(rw)

# same, but without root squashing for barry
/var/ftp pasha(rw) barry(rw,no_root_squash)

# everyone from the netsec.local domain gets access
/var/backup *.netsec.local(rw)

# ro for one network, rw for the other
/var/upload 192.168.1.0/24(ro) 192.168.5.0/24(rw)
```

Încarnări mai recente a **nfs** cer opțiunea **subtree_check** să fie setată în mod explicit (sau unset cu **no_subtree_check**). Fișierul **/etc/exports** apoi arată astfel:

```
root@debian6 ~# cat /etc/exports
# Everyone can read this share
/srv/iso *(ro,no_subtree_check)

# Only the computers named pasha and barry can readwrite this one
/var/www pasha(rw,no_subtree_check) barry(rw,no_subtree_check)

# same, but without root squashing for barry
/var/ftp pasha(rw,no_subtree_check) barry(rw,no_root_squash,no_subtree_check)
```

23.5. exportfs

Nu trebuie să reporniți serverul **nfs** pentru a începe să exportați exporturile noi create. Puteți folosi comanda **exportfs -va** pentru a face asta. Va scrie directoarele exportate în **/var/lib/nfs/etab**, unde sînt imediat aplicate.

```
root@debian6 ~# exportfs -va
exporting pasha:/var/ftp
exporting barry:/var/ftp
exporting pasha:/var/www
exporting barry:/var/www
exporting */:/srv/iso
```

23.6. configurare client

Am văzut comanda **mount** și fișierul **/etc/fstab** înainte.

```
root@RHELv4u2:~# mount -t nfs barry:/mnt/data/iso /home/project55/
root@RHELv4u2:~# cat /etc/fstab | grep nfs
barry:/mnt/data/iso /home/iso nfs defaults 0 0
root@RHELv4u2:~#
```

Aici este un alt exemplu simplu. Să presupunem că oamenii din **project55** ne spun că au nevoie de cîteva imagini CD-ROM, și noi le avem deja disponibile pe un server **nfs**. Am putea tasta următoarea comandă pentru a monta acest spațiu de stocare pe punctul lor de montare **/home/project55**.

```
root@RHELv4u2:~# mount -t nfs 192.168.1.40:/mnt/data/iso /home/project55/
root@RHELv4u2:~# ls -lh /home/project55/
total 3.6G
drwxr-xr-x 2 1000 1000 4.0K Jan 16 17:55 RHELv4u1
drwxr-xr-x 2 1000 1000 4.0K Jan 16 14:14 RHELv4u2
drwxr-xr-x 2 1000 1000 4.0K Jan 16 14:54 RHELv4u3
drwxr-xr-x 2 1000 1000 4.0K Jan 16 11:09 RHELv4u4
-rw-r--r-- 1 root root 1.6G Oct 13 15:22 sled10-vmwarews5-vm.zip
root@RHELv4u2:~#
```

23.7. practică: sistem de fișier în rețea

1. Creați două directoare cu câteva fișiere. Folosiți **nfs** pentru a partaja unul dintre ele cu drepturi doar de citire, celălalt trebuie să aibă doar drepturi de scriere. Puneți pe coleg să se conecteze la ele pentru a le testa.
2. Investigați utilizatorul proprietar al fișierelor create de coleg.
3. Protejați un partaj prin adresă ip sau hostname, astfel încât doar colegul să se poată conecta.

Capitolul 24. introducere în networking

Conținut

24.1. introducere la iptables.	229
24.2. practică: iptables.	231
24.3. soluție: iptables.	232
24.4. xinetd și inetd.	233
24.5. practică: inetd și xinetd.	235
24.6. sistem fișier în rețea.	236
24.7. practică: sistem fișier în rețea.	238

24.1. introducere la iptables

iptables firewall

Kernel-ul Linux are un firewall înglobat numit **iptables**. Pentru a opri firewall-ul **iptables** pe Red Hat, folosiți comanda **service**.

```
root@RHELv4u4:~# service iptables stop
Flushing firewall rules:          [ OK ]
Setting chains to policy ACCEPT: filter [ OK ]
Unloading iptables modules:      [ OK ]
root@RHELv4u4:~#
```

Modalitatea ușoară de a configura iptables, este să folosiți un utilitar grafic precum **kmyfirewall** KDE sau **Utilitarul de Configurare de Nivel de Securitate**. O puteți găsi pe ultima în meniul grafic, undeva în System Tools – Security, sau puteți să o porniți tastînd **system-config-securitylevel** în bash. Aceste utilitare permit unele configurări firewall de bază. Puteți decide dacă vreți să porniți sau să opriți firewall-ul, și ce porturi standard tipice sînt permise cînd firewall-ul este activ. Puteți chiar să adăugați unele porturi la comandă. Cînd terminați, configurația este scrisă în **/etc/sysconfig/iptables** pe Red Hat.

```
root@RHELv4u4:~# cat /etc/sysconfig/iptables
# Firewall configuration written by system-config-securitylevel
# Manual customization of this file is not recommended.
*filter
:INPUT ACCEPT [0:0]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
:RH-Firewall-1-INPUT - [0:0]
-A INPUT -j RH-Firewall-1-INPUT
-A FORWARD -j RH-Firewall-1-INPUT
-A RH-Firewall-1-INPUT -i lo -j ACCEPT
-A RH-Firewall-1-INPUT -p icmp --icmp-type any -j ACCEPT
-A RH-Firewall-1-INPUT -p 50 -j ACCEPT
-A RH-Firewall-1-INPUT -p 51 -j ACCEPT
-A RH-Firewall-1-INPUT -p udp --dport 5353 -d 224.0.0.251 -j ACCEPT
-A RH-Firewall-1-INPUT -p udp -m udp --dport 631 -j ACCEPT
-A RH-Firewall-1-INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
-A RH-F...NPUT -m state --state NEW -m tcp -p tcp --dport 22 -j ACCEPT
-A RH-F...NPUT -m state --state NEW -m tcp -p tcp --dport 80 -j ACCEPT
-A RH-F...NPUT -m state --state NEW -m tcp -p tcp --dport 21 -j ACCEPT
-A RH-F...NPUT -m state --state NEW -m tcp -p tcp --dport 25 -j ACCEPT
-A RH-Firewall-1-INPUT -j REJECT --reject-with icmp-host-prohibited
COMMIT
root@RHELv4u4:~#
```

Pentru a porni serviciul, tastați comanda **service iptables start**. Puteți configura iptables să înceapă în timpul butării cu **chkconfig**.

```
root@RHELv4u4:~# service iptables start
Applying iptables firewall rules: [OK]
root@RHELv4u4:~# chkconfig iptables on
root@RHELv4u4:~#
```

Una dintre autorizările frumoase ale iptables este că ea afișează informația **status** cînd este interogată cu comanda **service iptables status**.

```

root@RHELv4u4:~# service iptables status
Table: filter
Chain INPUT (policy ACCEPT)
target      prot opt source                destination
RH-Firewall-1-INPUT all -- 0.0.0.0/0             0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target      prot opt source                destination
RH-Firewall-1-INPUT all -- 0.0.0.0/0             0.0.0.0/0

Chain OUTPUT (policy ACCEPT)
target      prot opt source                destination

Chain RH-Firewall-1-INPUT (2 references)
target prot opt  source      destination
ACCEPT all  --  0.0.0.0/0   0.0.0.0/0
ACCEPT icmp --  0.0.0.0/0   0.0.0.0/0   icmp type 255
ACCEPT esp  --  0.0.0.0/0   0.0.0.0/0
ACCEPT ah   --  0.0.0.0/0   0.0.0.0/0
ACCEPT udp  --  0.0.0.0/0   224.0.0.251 udp dpt:5353
ACCEPT udp  --  0.0.0.0/0   0.0.0.0/0   udp dpt:631
ACCEPT all  --  0.0.0.0/0   0.0.0.0/0   state RELATED,ESTABLISHED
ACCEPT tcp  --  0.0.0.0/0   0.0.0.0/0   state NEW tcp dpt:22
ACCEPT tcp  --  0.0.0.0/0   0.0.0.0/0   state NEW tcp dpt:80
ACCEPT tcp  --  0.0.0.0/0   0.0.0.0/0   state NEW tcp dpt:21
ACCEPT tcp  --  0.0.0.0/0   0.0.0.0/0   state NEW tcp dpt:25
REJECT all  --  0.0.0.0/0   0.0.0.0/0   reject-with icmp-host-prohibited

```

```
root@RHELv4u4:~#
```

Stăpînirea configurației firewall cere o cunoaștere decentă a tcp/ip. Tutoriale iptables bune pot fi găsite online la <http://iptables-tutorial.frozentux.net/iptables-tutorial.html> și aici la <http://tldp.org/HOWTO/IP-Masquerade-HOWTO/>.

24.2. practică: iptables

1. Verificați dacă firewall-ul se execută.
2. Închideți firewall-ul care se execută.

24.3. soluție: iptables

1. Verificați dacă firewall-ul se execută.

```
root@rhel55 ~# service iptables status | head
Table: filter
Chain INPUT (policy ACCEPT)
num  target          prot opt source                destination
1    RH-Firewall-1-INPUT  all  --  0.0.0.0/0              0.0.0.0/0

Chain FORWARD (policy ACCEPT)
num  target          prot opt source                destination
1    RH-Firewall-1-INPUT  all  --  0.0.0.0/0              0.0.0.0/0

Chain OUTPUT (policy ACCEPT)
```

2. Închideți firewall-ul care se execută.

```
root@rhel55 ~# service iptables stop
Flushing firewall rules:          [ OK ]
Setting chains to policy ACCEPT: filter [ OK ]
Unloading iptables modules:      [ OK ]
root@rhel55 ~# service iptables status
Firewall is stopped.
```


24.4. xinetd și inetd

superdaemonul

Cînd resursele ca memoria RAM erau limitate, unui super-server îi era încredințat să asculte pe toate soclurile și să înceapă daemonul potrivit doar cînd era necesar. Servicii ca **swat**, **telnet** și **ftp** sînt în mod tipic servite de un astfel de super-server. Superdaemonul **xinetd** este mai recent decît **inetd**. Vom discuta configurația pentru ambii daemoni.

Distribuțiile Linux recente ca RHEL5 și Ubuntu 10.04 nu activează **inetd** sau **xinetd** prin default, ci doar dacă o aplicație o solicită.

inetd sau xinetd

Mai întîi verificați dacă computerul rulează **inetd** sau **xinetd**. Acest Debian 4.0 Etch execută **inetd**.

```
root@barry:~# ps fax | grep inet
3870 ?          Ss      0:00 /usr/sbin/inetd
```

Acest Red Hat Enterprise Linux 4 update 4 rulează **xinetd**.

```
[root@RHEL4b ~]# ps fax | grep inet
3003 ?          Ss      0:00 xinetd -stayalive -pidfile /var/run/xinetd.pid
```

Ambii daemoni au aceeași funcționalitate (ascultă pe multe porturi, încep alți daemoni cînd aceștia sînt necesari), dar ei au fișiere diferite de configurație.

xinetd superdaemon

Daemonul **xinetd** este deseori numit un superdaemon pentru că ascultă multe conexiuni intrare, și dă start altor daemoni cînd este necesar. Cînd este primită o cerere de conexiune, **xinetd** va căuta mai întîi învelișurile TCP (/etc/hosts.allow și /etc/hosts.deny) și apoi va da controlul conexiunii altui daemon. Acest superdaemon este configurat în **/etc/xinetd.conf** și fișierele în directorul **/etc/xinetd.d**. Să ne uităm mai întîi la /etc/xinetd.conf.

```
paul@RHELv4u2:~$ cat /etc/xinetd.conf
#
# Simple configuration file for xinetd
#
# Some defaults, and include /etc/xinetd.d/

defaults
{
    instances                = 60
    log_type                  = SYSLOG authpriv
    log_on_success            = HOST PID
    log_on_failure            = HOST
    cps                       = 25 30
}

includedir /etc/xinetd.d

paul@RHELv4u2:~$
```

Potivit setărilor din acest fișier, xinetd poate să mînuiască 60 de cereri de client odată. Folosește facilitatea **authpriv** pentru a jurnaliza adresele ip

gazdă și pid-ul daemonilor estinși cu succes. Când un serviciu (cunoscut ca protocol legat unui daemon) primește mai mult de 25 cps (conexiuni per secundă), ține cererile pentru 30 de secunde.

Directorul **/etc/xinetd.d** conține mai multe fișiere de configurație specifică. Să ne uităm de asemeni la unul dintre ele.

```
paul@RHELv4u2:~$ ls /etc/xinetd.d
amanda          chargen-udp    echo          klogin        rexec         talk
amandaidx       cups-lpd       echo-udp      krb5-telnet   rlogin        telnet
amidxtape       daytime       eklogin       kshell        rsh           tftp
auth            daytime-udp    finger        ktalk         rsync         time
chargen         dbuskdb-cdb    gssftp        ntalk         swat          time-udp
paul@RHELv4u2:~$ cat /etc/xinetd.d/swat
# default: off
# description: SWAT is the Samba Web Admin Tool. Use swat \
#              to configure your Samba server. To use SWAT, \
#              connect to port 901 with your favorite web browser.
service swat
{
port                = 901
socket_type         = stream
wait                = no
only_from           = 127.0.0.1
user                = root
server              = /usr/sbin/swat
log_on_failure      += USERID
disable             = yes
}
paul@RHELv4u2:~$
```

Serviciile ar trebui listate în fișierul **/etc/services**. Serviciul port determină portul, și trebuie să fie la fel ca portul specificat în **/etc/services**.

socket_type ar trebui setat pe **stream** pentru serviciile tcp (și dgram pentru udp). **log_on_failure+=** concatenează userid pentru a jurnaliza mesaje formatare în **/etc/xinetd.conf**. Ultima setare **disable** poate fi setată la yes sau no. A seta asta cu **no** înseamnă că serviciul este pornit!

Verificați paginile de manual al xinetd și xinetd.conf pentru mai multe opțiuni de configurare.

inetd superdaemon

Acest superdaemon are doar un singur fișier de configurare în **/etc/inetd.conf**. Fiecare protocol sau daemon pe care îl ascultă, primește o linie în acest fișier.

```
root@barry:~# grep ftp /etc/inetd.conf
tftp dgram udp wait nobody /usr/sbin/tcpd /usr/sbin/in.tftpd /boot/tftp
root@barry:~#
```

Puteți deautoriza un serviciu în **inetd.conf** de deasupra punând un # la începutul acelei linii. Aici este un exemplu a deautorizării interfeței web VMware (care ascultă pe portul tcp 902).

```
paul@laika:~$ grep vmware /etc/inetd.conf
#902 stream tcp nowait root /usr/sbin/vmware-authd vmware-authd
```

24.5. practică: inetd și xinetd

1. Verificați pe toate sistemele dacă ele folosesc xinetd sau inetd.
2. Priviți fișierele de configurare.
3. (Dacă telnet est instalabil, atunci înlocuiți swat, în aceste întrebări, cu telnet) Este swat instalat? Dacă nu, atunci instalați swat și priviți la schimbările din configurarea (x)inetd. Este swat oprit sau deschis?
4. Opriți swat, testați-l. Deschideți swat, testați-l.

24.6. sistem fișier în rețea (nfs)

versiuni protocol

Versiunile vechi **nfs** 2 și 3 sînt udp prin default, dar ele pot folosi tcp. Clienții se conectează la server folosind **rpc** (pe Linux asta e controlat de daemonul **portmap**). Priviți la **rpcinfo** pentru a verifica dacă **nfs** și serviciile puse în legătură cu el rulează.

```
root@RHELv4u2:~# /etc/init.d/portmap status
portmap (pid 1920) is running...
root@RHELv4u2:~# rpcinfo -p
program vers proto  port
100000    2    tcp    111  portmapper
100000    2    udp    111  portmapper
100024    1    udp   32768  status
100024    1    tcp   32769  status
root@RHELv4u2:~# service nfs start
Starting NFS services:          [ OK ]
Starting NFS quotas:           [ OK ]
Starting NFS daemon:           [ OK ]
Starting NFS mountd:           [ OK ]
```

Aceeași comandă **rpcinfo** cînd este deschis **nfs**.

```
root@RHELv4u2:~# rpcinfo -p
program  vers  proto  port
100000    2    tcp    111  portmapper
100000    2    udp    111  portmapper
100024    1    udp   32768  status
100024    1    tcp   32769  status
100011    1    udp    985  rquotad
100011    2    udp    985  rquotad
100011    1    tcp    988  rquotad
100011    2    tcp    988  rquotad
100003    2    udp   2049  nfs
100003    3    udp   2049  nfs
100003    4    udp   2049  nfs
100003    2    tcp   2049  nfs
100003    3    tcp   2049  nfs
100003    4    tcp   2049  nfs
100021    1    udp   32770  nlockmgr
100021    3    udp   32770  nlockmgr
100021    4    udp   32770  nlockmgr
100021    1    tcp   32789  nlockmgr
100021    3    tcp   32789  nlockmgr
100021    4    tcp   32789  nlockmgr
100005    1    udp   1004  mountd
100005    1    tcp   1007  mountd
100005    2    udp   1004  mountd
100005    2    tcp   1007  mountd
100005    3    udp   1004  mountd
100005    3    tcp   1007  mountd
root@RHELv4u2:~#
```

nfs versiunea 4 cere tcp (portul 2049) și suportă ca opțiune autentificare de utilizator **Kerberos**. Autentificarea **nfs** are loc doar când montăm ceea ce partajăm. Versiunile 2 și 3 **nfs** autentifică doar gazda.

configurare server

nfs este configurat în **/etc/exports**. Aici este un exemplu din **/etc/exports** pentru a explica sintaxa. Aveți nevoie de un mijloc (domeniu NIS sau LDAP) pentru a sincroniza user id printre computere când folosiți mult **nfs**. Opțiunea **rootsquash** va schimba contul utilizator UID 0 la UID nfsnobody. Opțiunea **sync** va scrie pe disk înainte de a completa cererea client.

```
paul@laika:~$ cat /etc/exports
# Everyone can read this share
/mnt/data/iso *(ro)

# Only the computers barry and pasha can readwrite this one
/var/www pasha(rw) barry(rw)

# same, but without root squashing for barry
/var/ftp pasha(rw) barry(rw,no_root_squash)

# everyone from the netsec.lan domain gets access
/var/backup *.netsec.lan(rw)

# ro for one network, rw for the other
/var/upload 192.168.1.0/24(ro) 192.168.5.0/24(rw)
```

Nu trebuie să reporniți serverul **nfs** pentru a începe să exportați noile exporturi create. Puteți folosi comanda **exportfs -va** pentru a face asta. Va scrie directoarele exportate în **/var/lib/nfs/etab**, unde ele sînt imediat aplicate.

configurare client

Am văzut comanda **mount** și fișierul **/etc/fstab** înainte.

```
root@RHELv4u2:~# mount -t nfs barry:/mnt/data/iso /home/project55/
root@RHELv4u2:~# cat /etc/fstab | grep nfs
barry:/mnt/data/iso /home/iso nfs defaults 0 0
root@RHELv4u2:~#
```

Aici este un alt exemplu simplu. Să presupunem că oamenii din **project55** vă transmit că au nevoie doar de două imagini CD-ROM, și deja le aveți disponibile pe un server **nfs**. Puteți tasta următoarea comandă pentru a monta acest spațiu de stocare pe punctul lor de montare **/home/project55**.

```
root@RHELv4u2:~# mount -t nfs 192.168.1.40:/mnt/data/iso /home/project55/
root@RHELv4u2:~# ls -lh /home/project55/
total 3.6G
drwxr-xr-x 2 1000 1000 4.0K Jan 16 17:55 RHELv4u1
drwxr-xr-x 2 1000 1000 4.0K Jan 16 14:14 RHELv4u2
drwxr-xr-x 2 1000 1000 4.0K Jan 16 14:54 RHELv4u3
drwxr-xr-x 2 1000 1000 4.0K Jan 16 11:09 RHELv4u4
-rw-r--r-- 1 root root 1.6G Oct 13 15:22 sled10-vmwarews5-vm.zip
root@RHELv4u2:~#
```

24.7. practică: sistem fișier în rețea

1. Creați două directoare cu câteva fișiere. Folosiți **nfs** pentru a partaja unul dintre ele cu drepturi doar de citire, celelalte trebuie să fie cu drepturi de scriere. Puneți pe coleg să se conecteze la ele pentru a le testa.
2. Investigați proprietarul owner a fișierelor create de coleg.
3. Protejați un partaj de adresă ip sau un hostname, astfel încât doar colegul să se poată conecta.

Partea VI. management kernel

Capitolul 25. kernel-ul Linux

Conținut

25.1. despre kernel-ul Linux.	.241
25.2. sursă kernel Linux.	.243
25.3. fișierele de butare kernel.	.246
25.4. module kernel Linux.	.247
25.5. compilarea unui kernel.	.251
25.6. compilarea unui modul.	.253

25.1. despre kernel-ul Linux

versiuni kernel

În 1991 Linus Torvalds a scris (prima versiune a) kernel-ului Linux. L-a pus online, și alți oameni au început să contribuie cu cod. Mai mult de 4000 de oameni au contribuit la codul sursă pentru ultima lansare a kernel-ului (versiunea 2.6.27 în noiembrie 2008).

Versiuni majore a kernel-ului Linux obișnuiau să iasă în numere pare și impare. Versiunile **2.0**, **2.2**, **2.4** și **2.6** sînt considerate versiuni kernel stabile. Versiunile **2.1**, **2.3** și **2.5** erau instabile (citiți în dezvoltare). De la ieșirea 2.6.0 în ianuarie 2004, toată dezvoltarea a fost făcută în arborele 2.6. Nu există în mod curent versiunea 2.7.x și după spusele lui Linus chiar și schema veche pară/stabilă versus veche/dezvoltare este abandonată pentru totdeauna.

uname -r

Pentru a vedea versiunea curentă a kernel-ului Linux, tastați comanda **uname -r** cum se vede mai jos.

Acest prim exemplu arată versiunea majoră Linux **2.6** și versiunea minoră **24**. Restul **-22-generic** este specific distribuției (Ubuntu în acest caz).

```
paul@laika:~$ uname -r
2.6.24-22-generic
```

Aceași comandă pe Red Hat Enterprise Linux arată un kernel mai vechi (2.6.18) cu **-92.1.17.el5** fiind specifică distribuției.

```
[paul@RHEL52 ~]$ uname -r
2.6.18-92.1.17.el5
```

/proc/cmdline

Parametrii care au fost trecuți kernel-ului în timpul butării sînt în **/proc/cmdline**.

```
paul@RHELv4u4:~$ cat /proc/cmdline
ro root=/dev/VolGroup00/LogVol00 rhgb quiet
```

single user mode

Cînd butăm kernel-ul cu parametrul **single**, el se pornește în **single user mode**. Linux poate începe într-un shell bash cu utilizatorul **root** intrat în sistem (fără parolă).

Unele distribuții previn utilizarea acestei posibilități (în timpul compilării kernel).

init=/bin/bash

În mod normal kernel-ul invocă **init** ca fiind primul proces daemon. Adăugînd **init=/bin/bash** parametrilor kernel-ului va invoca în schimb bash (încă odată cu root intrat în sistem fără a da o parolă).

/var/log/messages

Kernel-ul raportează în timpul butării în **syslog** care scrie multe din acțiunile kernel-ului în **/var/log/messages**. Privind acest fișier ne dăm seama cînd a fost pornit kernel-ul, incluzînd toate dispozitivele care au fost detectate în timpul

butării.

```
[root@RHEL53 ~]# grep -A16 "syslogd 1.4.1:" /var/log/messages|cut -b24-
syslogd 1.4.1: restart.
kernel: klogd 1.4.1, log source = /proc/kmsg started.
kernel: Linux version 2.6.18-128.el5 (mockbuild@hs20-bc1-5.build.red...
kernel: BIOS-provided physical RAM map:
kernel: BIOS-e820: 0000000000000000 - 0000000000009f800 (usable)
kernel: BIOS-e820: 0000000000009f800 - 000000000000a0000 (reserved)
kernel: BIOS-e820: 000000000000ca000 - 000000000000cc000 (reserved)
kernel: BIOS-e820: 000000000000dc000 - 00000000000100000 (reserved)
kernel: BIOS-e820: 00000000000100000 - 0000000001fef0000 (usable)
kernel: BIOS-e820: 0000000001fef0000 - 0000000001feff000 (ACPI data)
kernel: BIOS-e820: 0000000001feff000 - 0000000001ff00000 (ACPI NVS)
kernel: BIOS-e820: 0000000001ff00000 - 00000000020000000 (usable)
kernel: BIOS-e820: 00000000fec000000 - 00000000fec100000 (reserved)
kernel: BIOS-e820: 00000000fee000000 - 00000000fee010000 (reserved)
kernel: BIOS-e820: 00000000fffe00000 - 00000000100000000 (reserved)
kernel: 0MB HIGHMEM available.
kernel: 512MB LOWMEM available.
```

Acest exemplu arată cum să folosim **/var/log/messages** pentru a vedea informații kernel despre **/dev/sda**.

```
[root@RHEL53 ~]# grep sda /var/log/messages | cut -b24-
kernel: SCSI device sda: 41943040 512-byte hdwr sectors (21475 MB)
kernel: sda: Write Protect is off

kernel: sda: cache data unavailable
kernel: sda: assuming drive cache: write through
kernel: SCSI device sda: 41943040 512-byte hdwr sectors (21475 MB)
kernel: sda: Write Protect is off
kernel: sda: cache data unavailable
kernel: sda: assuming drive cache: write through
kernel: sda: sda1 sda2
kernel: sd 0:0:0:0: Attached scsi disk sda
kernel: EXT3 FS on sda1, internal journal
```

dmesg

Comanda **dmesg** listează toate mesajele bootup kernel (de la ultima butare).

```
[root@RHEL53 ~]# dmesg | head
Linux version 2.6.18-128.el5 (mockbuild@hs20-bc1-5.build.redhat.com)
BIOS-provided physical RAM map:
BIOS-e820: 0000000000000000 - 0000000000009f800 (usable)
BIOS-e820: 0000000000009f800 - 000000000000a0000 (reserved)
BIOS-e820: 000000000000ca000 - 000000000000cc000 (reserved)
BIOS-e820: 000000000000dc000 - 00000000000100000 (reserved)
BIOS-e820: 00000000000100000 - 0000000001fef0000 (usable)
BIOS-e820: 0000000001fef0000 - 0000000001feff000 (ACPI data)
BIOS-e820: 0000000001feff000 - 0000000001ff00000 (ACPI NVS)
BIOS-e820: 0000000001ff00000 - 00000000020000000 (usable)
```

Astfel pentru a găsi informații despre **/dev/sda**, folosind **dmesg** va prezenta doar mesaje kernel de la ultima butare.

```
[root@RHEL53 ~]# dmesg | grep sda
SCSI device sda: 41943040 512-byte hdwr sectors (21475 MB)
sda: Write Protect is off
sda: Mode Sense: 5d 00 00 00
sda: cache data unavailable
sda: assuming drive cache: write through
SCSI device sda: 41943040 512-byte hdwr sectors (21475 MB)
sda: Write Protect is off
sda: Mode Sense: 5d 00 00 00
sda: cache data unavailable
sda: assuming drive cache: write through
sda: sda1 sda2
sd 0:0:0:0: Attached scsi disk sda
EXT3 FS on sda1, internal journal
```

25.2. sursă kernel Linux

ftp.kernel.org

Rezidența sursei kernel-ului Linux este **ftp.kernel.org**. Conține toate lansările oficiale a sursei codului kernel-ului Linux din 1991. Prevede download-uri gratuite în http, ftp și rsync a tuturor acestor lansări, precum și schimbările și patch-urile. Mai multe informații pot fi obținute pe website-ul **www.kernel.org**.

Oricine poate să utilizeze anonim un client ftp pentru a accesa **ftp.kernel.org**

```
paul@laika:~$ ftp ftp.kernel.org
Connected to pub3.kernel.org.
220 Welcome to ftp.kernel.org.
Name (ftp.kernel.org:paul): anonymous
331 Please specify the password.
Password:
230- Welcome to the
230-
230- LINUX KERNEL ARCHIVES
230- ftp.kernel.org
```

Toate versiunile de kernel Linux sînt localizate în directorul pub/linux/kernel.

```
ftp> ls pub/linux/kernel/v*
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
drwxrwsr-x  2 536      536      4096 Mar 20  2003 v1.0
drwxrwsr-x  2 536      536     20480 Mar 20  2003 v1.1
drwxrwsr-x  2 536      536      8192 Mar 20  2003 v1.2
drwxrwsr-x  2 536      536     40960 Mar 20  2003 v1.3
drwxrwsr-x  3 536      536     16384 Feb 08  2004 v2.0
drwxrwsr-x  2 536      536     53248 Mar 20  2003 v2.1
drwxrwsr-x  3 536      536     12288 Mar 24  2004 v2.2
drwxrwsr-x  2 536      536     24576 Mar 20  2003 v2.3
drwxrwsr-x  5 536      536     28672 Dec 02  8:14 v2.4
drwxrwsr-x  4 536      536     32768 Jul 14  2003 v2.5
drwxrwsr-x  7 536      536    110592 Dec 05 22:36 v2.6
226 Directory send OK.
ftp>
```

/usr/src

Pe computerul local, kernel-ul sursă este localizat în **/usr/src**. Observați totuși că structura în interiorul **/usr/src** poate fi diferită depinzând de distribuția pe care o folosiți.

Mai întâi să ne uităm la **/usr/src** pe **Debian**. Acolo apar două versiuni complete a codului sursă Linux. Privind la un fișier specific (**e1000_main.c**) cu **find** relevă locația lui exactă.

```
paul@barry:~$ ls -l /usr/src/
drwxr-xr-x 20 root root  4096 2006-04-04 22:12 linux-source-2.6.15
drwxr-xr-x 19 root root  4096 2006-07-15 17:32 linux-source-2.6.16
paul@barry:~$ find /usr/src -name e1000_main.c
/usr/src/linux-source-2.6.15/drivers/net/e1000/e1000_main.c
/usr/src/linux-source-2.6.16/drivers/net/e1000/e1000_main.c
```

Asta este similar cu **/usr/src** pe **Ubuntu**, cu excepția că există doar un singur kernel aici (și este mai nou).

```
paul@laika:~$ ls -l /usr/src/
drwxr-xr-x 23 root root  4096 2008-11-24 23:28 linux-source-2.6.24
paul@laika:~$ find /usr/src -name "e1000_main.c"
/usr/src/linux-source-2.6.24/drivers/net/e1000/e1000_main.c
```

Acum priviți la **/usr/src** pe **Red Hat Enterprise Linux**.

```
[paul@RHEL52 ~]$ ls -l /usr/src/
drwxr-xr-x 5 root root 4096 Dec 5  19:23 kernels
drwxr-xr-x 7 root root 4096 Oct 11 13:22 redhat
```

Va trebui să intrăm în profunzime să găsim sursa kernel pe Red Hat!

```
[paul@RHEL52 ~]$ cd /usr/src/redhat/BUILD/
[paul@RHEL52 BUILD]$ find . -name "e1000_main.c"
./kernel-2.6.18/linux-2.6.18.i686/drivers/net/e1000/e1000_main.c
```

download sursă kernel

Debian

Instalarea sursei kernel pe Debian este într-adevăr simplă cu **aptitude install linux-source**. Puteți face o căutare pentru toate pachetele linux-source mai întâi, ca în această captură de ecran.

```
root@barry:~# aptitude search linux-source
v   linux-source          -
v   linux-source-2.6       -
id  linux-source-2.6.15    - Linux kernel source for version 2.6.15
i   linux-source-2.6.16    - Linux kernel source for version 2.6.16
p   linux-source-2.6.18    - Linux kernel source for version 2.6.18
p   linux-source-2.6.24    - Linux kernel source for version 2.6.24
```

Și apoi folosiți **aptitude install** pentru a downloada și instala codul sursă de kernel Debian Linux.

```
root@barry:~# aptitude install linux-source-2.6.24
```

Cînd aptitude a terminat, puteți vedea un nou fișier numit **/usr/src/linux-source-<version>.tar.bz2**.

```
root@barry:/usr/src# ls -lh
drwxr-xr-x 20 root root 4.0K 2006-04-04 22:12 linux-source-2.6.15
drwxr-xr-x 19 root root 4.0K 2006-07-15 17:32 linux-source-2.6.16
-rw-r--r-- 1 root root 45M 2008-12-02 10:56 linux-source-2.6.24.tar.bz2
```

Ubuntu

Ubuntu e bazat pe Debian și la fel folosește **aptitude**, astfel că sarcina este similară.

```
root@laika:~# aptitude search linux-source
i   linux-source          - Linux kernel source with Ubuntu patches
v   linux-source-2.6      -
i A linux-source-2.6.24   - Linux kernel source for version 2.6.24
root@laika:~# aptitude install linux-source
```

Și cînd aptitude se termină, avem un fișier **/usr/src/linux-source-<version>.tar.bz**.

```
root@laika:~# ll /usr/src
total 45M
-rw-r--r-- 1 root root 45M 2008-11-24 23:30 linux-source-2.6.24.tar.bz2
```

Red Hat Enterprise Linux

Sursa kernel Red Hat este localizată pe a patra sursă cdrom. Fișierul este numit **kernel-2.6.9-42.EL.src.rpm** (exemplu pentru RHELv4u4). Este de asemni disponibilă la <ftp://ftp.redhat.com/pub/redhat/linux/enterprise/5Server/en/os/SRPMS/> (exemplu pentru RHEL5).

Pentru a downloada sursa kernel pe RHEL, folosiți această comandă wget lungă (pe o singură linie, fără \ între os și SRPMS).

```
wget ftp://ftp.redhat.com/pub/redhat/linux/enterprise/5Server/en/os/\
SRPMS/kernel-`uname -r`.src.rpm
```

Cînd s-a terminat downloadul wget, avem un fișier .rpm de 60 M.

```
[root@RHEL52src]# ll
total 60M
-rw-r--r-- 1 root root 60M Dec 5 20:54 kernel-2.6.18-92.1.17.el5.src.rpm
drwxr-xr-x 5 root root 4.0K Dec 5 19:23 kernels
drwxr-xr-x 7 root root 4.0K Oct 11 13:22 redhat
```

Trebuie să facem încă cîțiva pași înainte să utilizăm codul sursă kernel.

Mai întii, tastăm comanda **rpm -i kernel-2.6.9-42.EL.src.rpm** pentru a instala acest pachet Red Hat.

```
[root@RHEL52src]# ll
total 60M
-rw-r--r-- 1 root root 60M Dec 5 20:54 kernel-2.6.18-92.1.17.el5.src.rpm
drwxr-xr-x 5 root root 4.0K Dec 5 19:23 kernels
drwxr-xr-x 7 root root 4.0K Oct 11 13:22 redhat
[root@RHEL52src]# rpm -i kernel-2.6.18-92.1.17.el5.src.rpm
```

Apoi ne mutăm în directorul SPECS și construim un **rpmbuild**.

```
[root@RHEL52 ~]# cd /usr/src/redhat/SPECS
[root@RHEL52 SPECS]# rpmbuild -bp -vv --target=i686 kernel-2.6.spec
```

Comanda **rpmbuild** pune codul sursă kernel Linux RHEL în **/usr/src/redhat/BUILD/kernel-<version>/**.

```
[root@RHEL52 kernel-2.6.18]# pwd
/usr/src/redhat/BUILD/kernel-2.6.18
[root@RHEL52 kernel-2.6.18]# ll
total 20K
drwxr-xr-x  2 root root 4.0K Dec  6 2007  config
-rw-r--r--  1 root root 3.1K Dec  5 20:58  Config.mk
drwxr-xr-x 20 root root 4.0K Dec  5 20:58  linux-2.6.18.i686
drwxr-xr-x 19 root root 4.0K Sep 20 2006  vanilla
drwxr-xr-x  8 root root 4.0K Dec  6 2007  xen
```

25.3. fișierele de butare kernel

vmlinuz

Fișierul **vmlinuz** din **/boot** este kernel-ul compresat.

```
paul@barry:~$ ls -lh /boot | grep vmlinuz
-rw-r--r-- 1 root root 1.2M 2006-03-06 16:22 vmlinuz-2.6.15-1-486
-rw-r--r-- 1 root root 1.1M 2006-03-06 16:30 vmlinuz-2.6.15-1-686
-rw-r--r-- 1 root root 1.3M 2008-02-11 00:00 vmlinuz-2.6.18-6-686
paul@barry:~$
```

initrd

Kernel-ul folosește **initrd** (un **initial RAM disk**) la butare. **initrd** este montat înainte de încărcarea kernel-ului, și poate conține drivere adăugate și module. Este o **arhivă compresată cpio**, astfel că ne putem uita la conținut în această modalitate.

```
root@RHELv4u4:/boot# mkdir /mnt/initrd
root@RHELv4u4:/boot# cp initrd-2.6.9-42.0.3.EL.img TMPinitrd.gz
root@RHELv4u4:/boot# gunzip TMPinitrd.gz
root@RHELv4u4:/boot# file TMPinitrd
TMPinitrd: ASCII cpio archive (SVR4 with no CRC)
root@RHELv4u4:/boot# cd /mnt/initrd/
root@RHELv4u4:/mnt/initrd# cpio -i | /boot/TMPinitrd
4985 blocks
root@RHELv4u4:/mnt/initrd# ls -l
total 76
drwxr-xr-x 2 root root 4096 Feb  5 08:36 bin
drwxr-xr-x 2 root root 4096 Feb  5 08:36 dev
drwxr-xr-x 4 root root 4096 Feb  5 08:36 etc
-rwxr-xr-x 1 root root 1607 Feb  5 08:36 init
drwxr-xr-x 2 root root 4096 Feb  5 08:36 lib
drwxr-xr-x 2 root root 4096 Feb  5 08:36 loopfs
drwxr-xr-x 2 root root 4096 Feb  5 08:36 proc
lrwxrwxrwx 1 root root    3 Feb  5 08:36 sbin -> bin
drwxr-xr-x 2 root root 4096 Feb  5 08:36 sys
drwxr-xr-x 2 root root 4096 Feb  5 08:36 sysroot
```

System.map

System.map conține tabelul simbol și se schimbă cu fiecare compilare kernel.

Tabelul simbol este de asemeni prezent în **/proc/kallsyms** (kernel-urile de dinainte de versiunea 2.6 numeau acest fișier **/proc/ksyms**).

```
root@RHELv4u4:/boot# head System.map-`uname -r`
00000400 A __kernel_vsyscall
0000041a A SYSENTER_RETURN_OFFSET
00000420 A __kernel_sigreturn
00000440 A __kernel_rt_sigreturn
c0100000 A _text
c0100000 T startup_32
c01000c6 t checkCPUtype
c0100147 t is486
c010014e t is386
c010019f t L6
root@RHELv4u4:/boot# head /proc/kallsyms
c0100228 t _stext
c0100228 t calibrate_delay_direct
c0100228 t stext
c0100337 t calibrate_delay
c01004db t rest_init
c0100580 t do_pre_smp_initcalls
c0100585 t run_init_process
c01005ac t init
c0100789 t early_param_test
c01007ad t early_setup_test
root@RHELv4u4:/boot#
```

.config

Ultimul fișier copiat în directorul **/boot** este configurația kernel folosită pentru compilare. Acest fișier nu este necesar în directorul **/boot**, dar se obișnuiește să se pună o copie acolo. Vă permite să recompilați un kernel, începînd de la aceeași configurație ca unul existent și funcționabil.

25.4. module kernel Linux

despre module kernel

Kernel-ul Linux este un kernel monolitic cu module încărcabile. Aceste module conțin părți ale kernel-ului folosite în mod tipic pentru drivere de dispozitive, sisteme de fișier și protocoale network. De foarte multe ori modulele kernel necesare sînt încărcate automat și dinamic fără interacțiune din partea administratorului.

/lib/modules

Modulele sînt stocate în directorul **/lib/modules/<versiune-kernel>**. Există un director separat pentru fiecare kernel care a fost compilat pentru sistem.

```
paul@laika:~$ ll /lib/modules/
total 12K
drwxr-xr-x 7 root root 4.0K 2008-11-10 14:32 2.6.24-16-generic
drwxr-xr-x 8 root root 4.0K 2008-12-06 15:39 2.6.24-21-generic
drwxr-xr-x 8 root root 4.0K 2008-12-05 12:58 2.6.24-22-generic
```

<module>.ko

Fișierul care conține modulele se termină de obicei în **.ko**. Această captură de ecran arată locația fișierelor modulului isdn.

```
paul@laika:~$ find /lib/modules -name isdn.ko
/lib/modules/2.6.24-21-generic/kernel/drivers/isdn/i4l/isdn.ko
/lib/modules/2.6.24-22-generic/kernel/drivers/isdn/i4l/isdn.ko
/lib/modules/2.6.24-16-generic/kernel/drivers/isdn/i4l/isdn.ko
```

lsmod

Pentru a vedea o listă a modulelor încărcate curent, folosiți **lsmod**. Vedeți numele fiecărui modul încărcat, mărimea, utilizarea, și numele altor module care-l folosește.

```
[root@RHEL52 ~]# lsmod | head -5
Module                Size      Used by
autofs4               24517      2
hidp                  23105      2
rfcomm                42457      0
l2cap                 29505     10 hidp,rfcomm
```

/proc/modules

/proc/modules listează toate modulele încărcate de kernel. Ieșirea ar fi prea lungă pentru a fi afișată aici, așa că să încercăm **grep** pentru modulul **vm**.

Vedem că și **vmmon** și **vmnet** sînt încărcate. Puteți afișa aceeași informație cu **lsmod**. În realitate **lsmod** doar citește și reformatează ieșirea din **/proc/modules**.

```
paul@laika:~$ cat /proc/modules | grep vm
vmnet 36896 13 - Live 0xffffffff88b21000 (P)
vmmon 194540 0 - Live 0xffffffff88af0000 (P)
paul@laika:~$ lsmod | grep vm
vmnet                36896   13
vmmon                194540   0
paul@laika:~$
```

dependențe modul

Unele module depind de altele. În exemplul următor, puteți vedea că modulul **nfsd** este folosit de **exportfs**, **lockd** și **sunrpc**.

```
paul@laika:~$ cat /proc/modules | grep nfsd
nfsd 267432 17 - Live 0xffffffff88a40000
exportfs 7808 1 nfsd, Live 0xffffffff88a3d000
lockd 73520 3 nfs,nfsd, Live 0xffffffff88a2a000
sunrpc 185032 12 nfs,nfsd,lockd, Live 0xffffffff889fb000
paul@laika:~$ lsmod | grep nfsd
nfsd                267432  17
exportfs             7808      1 nfsd
lockd                73520      3 nfs,nfsd
sunrpc              185032  12 nfs,nfsd,lockd
paul@laika:~$
```


insmod

Modulele kernel pot fi încărcate manual cu comanda **insmod**. Aceasta este o modalitate foarte simplă (și învechită) de a încărca module. Acest screenshot arată **insmod** încărcând modulul fat (pentru susținerea sistemului de fișier fat).

```
root@barry:/lib/modules/2.6.17-2-686# pwd
/lib/modules/2.6.17-2-686
root@barry:/lib/modules/2.6.17-2-686# lsmod | grep fat
root@barry:/lib/modules/2.6.17-2-686# insmod kernel/fs/fat/fat.ko
root@barry:/lib/modules/2.6.17-2-686# lsmod | grep fat
fat                46588                0
```

insmod nu detectează dependențele, astfel eșuează să încarce modulul isdn (pentru că modulul isdn depinde de modulul slhc).

```
[root@RHEL52 drivers]# pwd
/lib/modules/2.6.18-92.1.18.el5/kernel/drivers
[root@RHEL52 kernel]# insmod isdn/i4l/isdn.ko
insmod: error inserting 'isdn/i4l/isdn.ko': -1 Unknown symbol in module
```

modinfo

După cum vedeți în captura de ecran a **modinfo** de mai jos, modulul isdn depinde de modulul slhc.

```
[root@RHEL52 drivers]# modinfo isdn/i4l/isdn.ko | head -6
filename: isdn/i4l/isdn.ko
license: GPL
author: Fritz Elfert
description: ISDN4Linux: link layer
srcversion: 99650346E708173496F6739
depends: slhc
```

modprobe

Marele avantaj al **modprobe** față de **insmod** este că modprobe va încărca toate modulele necesare, în timp ce insmod cere încărcarea manuală a dependențelor. Un alt avantaj este că nu trebuie să scrieți calea absolută spre numele fișier.

Acest screenshot arată cum modprobe încarcă modulul isdn, încărcând automatic slhc în background.

```
[root@RHEL52kernel]# lsmod | grep isdn
[root@RHEL52kernel]# modprobe isdn
[root@RHEL52kernel]# lsmod | grep isdn
isdn                122433    0
slhc                 10561    1 isdn
[root@RHEL52kernel]#
```

/lib/modules/<kernel>/modules.dep

Dependențele modul sînt stocate în **modules.dep**.

```
[root@RHEL52 2.6.18-92.1.18.el5]# pwd
/lib/modules/2.6.18-92.1.18.el5
[root@RHEL52 2.6.18-92.1.18.el5]# head -3 modules.dep
/lib/modules/2.6.18-92.1.18.el5/kernel/drivers/net/tokenring/3c359.ko:
/lib/modules/2.6.18-92.1.18.el5/kernel/drivers/net/pcmcia/3c574_cs.ko:
/lib/modules/2.6.18-92.1.18.el5/kernel/drivers/net/pcmcia/3c589_cs.ko:
```

depmod

Fișierul **modules.dep** poate fi adus la zi (recreat) cu comanda **depmod**. În această captură de ecran nu a fost adăugat nici un modul, astfel **depmod** generează același fișier.

```
root@barry:/lib/modules/2.6.17-2-686# ls -l modules.dep
-rw-r--r-- 1 root root 310676 2008-03-01 16:32 modules.dep
root@barry:/lib/modules/2.6.17-2-686# depmod
root@barry:/lib/modules/2.6.17-2-686# ls -l modules.dep
-rw-r--r-- 1 root root 310676 2008-12-07 13:54 modules.dep
```

rmmod

Similară cu **insmod**, comanda **rmmod** este foarte rar folosită.

```
[root@RHELv4u3 ~]# modprobe isdn
[root@RHELv4u3 ~]# rmmod slhc
ERROR: Module slhc is in use by isdn
[root@RHELv4u3 ~]# rmmod isdn
[root@RHELv4u3 ~]# rmmod slhc
[root@RHELv4u3 ~]# lsmod | grep isdn
[root@RHELv4u3 ~]#
```

modprobe -r

Contrar comenzii **rmmod**, **modprobe** va șterge automat modulele care nu sînt necesare.

```
[root@RHELv4u3~]# modprobe isdn
[root@RHELv4u3~]# lsmod | grep isdn
isdn          133537  0
slhc          7233    1 isdn
[root@RHELv4u3~]# modprobe -r isdn
[root@RHELv4u3~]# lsmod | grep isdn
[root@RHELv4u3~]# lsmod | grep slhc
[root@RHELv4u3~]#
```

/etc/modprobe.conf

Fișierul **/etc/modprobe.conf** și directorul **/etc/modprobe.d** pot conține aliasuri (folosite de oameni) și opțiuni (pentru module dependente) pentru **modprobe**.

```
[root@RHEL52 ~]# cat /etc/modprobe.conf
alias scsi_hostadapter mptbase
alias scsi_hostadapter1 mptspi
alias scsi_hostadapter2 ata_piix
alias eth0 pcnet32
alias eth2 pcnet32
alias eth1 pcnet32
```

25.5. compilarea unui kernel

extraversiune

Intrați în `/usr/src/redhat/BUILD/kernel-2.6.9/linux-2.6.9/` și schimbați **extraversiunea** în Makefile.

```
[root@RHEL52 linux-2.6.18.i686]# pwd
/usr/src/redhat/BUILD/kernel-2.6.18/linux-2.6.18.i686
[root@RHEL52 linux-2.6.18.i686]# vi Makefile
[root@RHEL52 linux-2.6.18.i686]# head -4 Makefile
VERSION = 2
PATCHLEVEL = 6
SUBLEVEL = 18
EXTRAVERSION = -paul2008
```

make mrproper

Acum curățați sursa de oricare altă instalare de mai înainte cu **make mrproper**. Dacă acest lucru este primul după ce downloadați codul sursă, atunci aceasta nu mai este necesar.

```
[root@RHEL52 linux-2.6.18.i686]# make mrproper
CLEAN scripts/basic
CLEAN scripts/kconfig
CLEAN include/config
CLEAN .config .config.old
```

.config

Acum copiați un fișier care funcționează **.config** din `/boot` în directorul nostru kernel. Acest fișier conține configurarea care a fost folosită pentru kernel-ul care se execută acum. El determină dacă modulele sînt incluse în compilare sau nu.

```
[root@RHEL52 linux-2.6.18.i686]# cp /boot/config-2.6.18-92.1.18.el5 .config
```

make menuconfig

Acum executați **make menuconfig** (sau graficul **make xconfig**). Acest utilitar vă permite să selectați dacă compilați lucruri ca un modul (m), ca parte a kernel-ului (*), sau deloc (mărime kernel mai mică). Dacă scoateți prea multe, kernel-ul nu va funcționa. Configurația va fi stocată în fișierul ascuns **.config**.

```
[root@RHEL52 linux-2.6.18.i686]# make menuconfig
```

make clean

Tastați un **make clean** pentru a pregăti kernel-ul pentru compilare. **make clean** va șterge cele mai multe fișiere generate, dar păstrează configurarea kernel. Executarea unui **make mrproper** în acest punct va distruge fișierul **.config** pe care l-ați construit cu **make menuconfig**.

```
[root@RHEL52 linux-2.6.18.i686]# make clean
```

make bzImage

Și apoi executați **make bzImage**, stați comod și relaxați-vă în timp ce kernel-ul

se compilează. Puteți folosi **time make bzImage** pentru a ști cât timp cere să se compileze, așa că data viitoare puteți face o plimbare scurtă.

```
[root@RHEL52 linux-2.6.18.i686]# time make bzImage
HOSTCC scripts/basic/fixdep
HOSTCC scripts/basic/docproc
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/kxgettext.o
...
```

Această comandă va termina prin a vă informa unde este localizat fișierul **bzImage** (și cu **time info** dacă ați specificat de asemeni comanda **time**).

Kernel: arch/i386/boot/bzImage is ready (#1)

```
real 13m59.573s
user 1m22.631s
sys 11m51.034s
[root@RHEL52 linux-2.6.18.i686]#
```

Puteți copia deja această imagine în **/boot** cu **cp arch/i386/boot/bzImage /boot/vmlinuz-<kernel-version>**.

make modules

Acum rulați **make modules**. Poate lua 20 pînă la 50 de minute să compilați toate modulele.

```
[root@RHEL52 linux-2.6.18.i686]# time make modules
CHK      include/linux/version.h
CHK      include/linux/utsrelease.h
CC [M] arch/i386/kernel/msr.o
CC [M] arch/i386/kernel/cpuid.o
CC [M] arch/i386/kernel/microcode.o
```

make modules_install

Pentru a copia modulele compilate în **/lib/modules** executați doar **make modules_install** (ia aproape 20 de secunde). Aici e un screenshot de dinaintea comenzii.

```
[root@RHEL52 linux-2.6.18.i686]# ls -l /lib/modules/
total 20
drwxr-xr-x 6 root root 4096 Oct 15 13:09 2.6.18-92.1.13.el5
drwxr-xr-x 6 root root 4096 Nov 11 08:51 2.6.18-92.1.17.el5
drwxr-xr-x 6 root root 4096 Dec  6 07:11 2.6.18-92.1.18.el5
[root@RHEL52 linux-2.6.18.i686]# make modules_install
```

Și aici este același director după. Observați că **make modules_install** a creat un nou director pentru noul kernel.

```
[root@RHEL52 linux-2.6.18.i686]# ls -l /lib/modules/
total 24
drwxr-xr-x 6 root root 4096 Oct 15 13:09 2.6.18-92.1.13.el5
drwxr-xr-x 6 root root 4096 Nov 11 08:51 2.6.18-92.1.17.el5
drwxr-xr-x 6 root root 4096 Dec  6 07:11 2.6.18-92.1.18.el5
drwxr-xr-x 3 root root 4096 Dec  6 08:50 2.6.18-paul2008
```

/boot

Trebuie totuși să copiem kernel-ul, System.map, și fișierul nostru de configurare în /boot. Strict vorbind fișierul .config nu este obligatoriu, dar ar putea să vă fie de folos în viitoarele compilări a kernel-ului.

```
[root@RHEL52 ]# pwd
/usr/src/redhat/BUILD/kernel-2.6.18/linux-2.6.18.i686
[root@RHEL52 ]# cp System.map /boot/System.map-2.6.18-paul2008
[root@RHEL52 ]# cp .config /boot/config-2.6.18-paul2008
[root@RHEL52 ]# cp arch/i386/boot/bzImage /boot/vmlinuz-2.6.18-paul2008
```

mkinitrd

Kernel-ul deseori folosește un fișier initrd la bootup. Putem folosi **mkinitrd** pentru a genera acest fișier. Asigurați-vă că folosiți numele corect al kernel-ului!

```
[root@RHEL52 ]# pwd
/usr/src/redhat/BUILD/kernel-2.6.18/linux-2.6.18.i686
[root@RHEL52 ]# mkinitrd /boot/initrd-2.6.18-paul2008 2.6.18-paul2008
```

bootloader

Compilarea este acum terminată, nu uitați să creați o stanză adițională în grub sau lilo.

25.6. compilarea unui modul

hello.c

Un mic program C care va fi modulul nostru.

```
[root@rhel4a kernel_module]# cat hello.c
#include <linux/module.h>
#include <section>

int init_module(void)
{
    printk(KERN_INFO "Start Hello World...\n");
    return 0;
}

void cleanup_module(void)
{
    printk(KERN_INFO "End Hello World... \n");
}
```

Makefile

Fișierul make pentru acest modul.

```
[root@rhel4a kernel_module]# cat Makefile
obj-m += hello.o
all:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Acestea sînt doar cele două fişiere care ne trebuie.

```
[root@rhel4a kernel_module]# ll
total 16
-rw-rw-r-- 1 paul paul 250 Feb 15 19:14 hello.c
-rw-rw-r-- 1 paul paul 153 Feb 15 19:15 Makefile
```

make

Executarea comenzii **make**.

```
[root@rhel4a kernel_module]# make
make -C /lib/modules/2.6.9-paul-2/build M=~/.kernel_module modules
make[1]: Entering dir... `/usr/src/redhat/BUILD/kernel-2.6.9/linux-2.6.9'
CC [M] /home/paul/kernel_module/hello.o
Building modules, stage 2.
MODPOST
CC      /home/paul/kernel_module/hello.mod.o
LD [M] /home/paul/kernel_module/hello.ko
make[1]: Leaving dir... `/usr/src/redhat/BUILD/kernel-2.6.9/linux-2.6.9'
[root@rhel4a kernel_module]#
```

Acum avem mai multe fişiere.

```
[root@rhel4a kernel_module]# ll
total 172
-rw-rw-r-- 1 paul paul 250 Feb 15 19:14 hello.c
-rw-r--r-- 1 root root 64475 Feb 15 19:15 hello.ko
-rw-r--r-- 1 root root 632 Feb 15 19:15 hello.mod.c
-rw-r--r-- 1 root root 37036 Feb 15 19:15 hello.mod.o
-rw-r--r-- 1 root root 28396 Feb 15 19:15 hello.o
-rw-rw-r-- 1 paul paul 153 Feb 15 19:15 Makefile
[root@rhel4a kernel_module]#
```

hello.ko

Folosiţi **modinfo** pentru a verifica dacă este într-adevăr un modul.

```
[root@rhel4a kernel_module]# modinfo hello.ko
filename:      hello.ko
vermagic:      2.6.9-paul-2 SMP 686 REGPARM 4KSTACKS gcc-3.4
depends:
```

Bine, așa că putem să încercăm modulul nostru hello.

```
[root@rhel4a kernel_module]# lsmod | grep hello
[root@rhel4a kernel_module]# insmod ./hello.ko
[root@rhel4a kernel_module]# lsmod | grep hello
hello          5504 0
[root@rhel4a kernel_module]# tail -1 /var/log/messages
Feb 15 19:16:07 rhel4a kernel: Start Hello World...
[root@rhel4a kernel_module]# rmmod hello
[root@rhel4a kernel_module]#
```

La sfîrșit **/var/log/messages** are o mică surpriză.

```
[root@rhel4a kernel_module]# tail -2 /var/log/messages
Feb 15 19:16:07 rhel4a kernel: Start Hello World...
Feb 15 19:16:35 rhel4a kernel: End Hello World...
[root@rhel4a kernel_module]#
```

Capitolul 26. management biblioteci

Conținut

26.1. introducere.	257
26.2. /lib și /usr/lib.	257
26.3. ldd.	257
26.4. ltrace.	257
26.5. dpkg -S și debsums.	258
26.6. rpm -qf și rpm -V.	258
26.7. urmărire cu strace.	259

26.1. introducere

Cu **biblioteci** vorbim aici despre biblioteci legate dinamic (cunoscute și ca obiecte partajate). Acestea sînt executabile care conțin funcții și nu sînt pornite ele însele ca programe, dar sînt invocate de către alte executabile.

Cîteva programe pot folosi aceeași bibliotecă. Numele fișierului bibliotecii de obicei începe cu **lib**, urmat de numele real al bibliotecii, apoi caracterele **.so** și în final un număr de versiune.

26.2. /lib și /usr/lib

Cînd priviți la directorul **/lib** sau **/usr/lib**, veți vedea o mulțime de legături simbolice. Cele mai multe **biblioteci** au un număr de versiune detaliat în numele lor, dar primesc o legătură simbolică din numele de fișier care conține doar numărul versiunii majore.

```
root@rhel53 ~# ls -l /lib/libext*
lrwxrwxrwx 1 root root 16 Feb 18 16:36 /lib/libext2fs.so.2 -> libext2fs.so.2.4
-rwxr-xr-x 1 root root 113K Jun 30 2009 /lib/libext2fs.so.2.4
```

26.3. ldd

Multe programe au dependențe la instalarea anumitor biblioteci. Puteți afișa aceste dependențe cu **ldd**.

Acest exemplu arată dependențele comenzii **su**.

```
paul@RHEL5 ~$ ldd /bin/su
linux-gate.so.1 => (0x003f7000)
libpam.so.0 => /lib/libpam.so.0 (0x00d5c000)
libpam_misc.so.0 => /lib/libpam_misc.so.0 (0x0073c000)
libcrypt.so.1 => /lib/libcrypt.so.1 (0x00aa4000)
libdl.so.2 => /lib/libdl.so.2 (0x00800000)
libc.so.6 => /lib/libc.so.6 (0x00ec1000)
libaudit.so.0 => /lib/libaudit.so.0 (0x0049f000)
/lib/ld-linux.so.2 (0x4769c000)
```

26.4. ltrace

Programul **ltrace** permite să vedem toate invocările făcute funcțiilor bibliotecii de către un program. Exemplul de mai jos folosește opțiunea **-c** pentru a obține doar o enumerare sumară (pot exista multe invocări), și opțiunea **-l** pentru a afișa invocări într-un singur fișier de bibliotecă. Toate acestea pentru a vedea ce invocări sînt făcute cînd executăm **su** - **serena** ca root.

```
root@deb503:~# ltrace -c -l /lib/libpam.so.0 su - serena
serena@deb503:~$ exit
logout
```

% time	seconds	usecs/call	calls	function
70.31	0.014117	14117	1	pam_start
12.36	0.002482	2482	1	pam_open_session
5.17	0.001039	1039	1	pam_acct_mgmt
4.36	0.000876	876	1	pam_end
3.36	0.000675	675	1	pam_close_session
3.22	0.000646	646	1	pam_authenticate
0.48	0.000096	48	2	pam_set_item
0.27	0.000054	54	1	pam_setcred
0.25	0.000050	50	1	pam_getenvlist
0.22	0.000044	44	1	pam_get_item
100.00	0.20079		11	total

26.5. dpkg -S și debsums

Găsiți pe Debian/Ubuntu cărui pachet aparține o bibliotecă.

```
paul@deb503:/lib$ dpkg -S libext2fs.so.2.4
e2fslibs: /lib/libext2fs.so.2.4
```

Puteți verifica integritatea tuturor fișierelor în acest pachet folosind **debsums**.

```
paul@deb503:~$ debsums e2fslibs
/usr/share/doc/e2fslibs/changelog.Debian.gz      OK
/usr/share/doc/e2fslibs/copyright                OK
/lib/libe2p.so.2.3                               OK
/lib/libext2fs.so.2.4                            OK
```

Dacă o bibliotecă este spartă, atunci reinstalați-o cu **aptitude reinstall \$package**.

```
root@deb503:~# aptitude reinstall e2fslibs
Reading package lists... Done
Building dependency tree
Reading state information... Done
Reading extended state information
Initializing package states... Done
Reading task descriptions... Done
The following packages will be REINSTALLED:
e2fslibs
. . .
```

26.6. rpm -qf și rpm -V

Găsiți pe Red Hat/Fedora cărui pachet aparține o bibliotecă.

```
paul@RHEL5 ~$ rpm -qf /lib/libext2fs.so.2.4
e2fsprogs-libs-1.39-8.el5
```

Puteți apoi folosi **rpm -V** pentru a verifica toate fișierele în acest pachet. În exemplul de mai jos ieșirea arată că timbrul **Size** și **Time** al fișierului s-au schimbat de la instalare.

```
root@rhel53 ~# rpm -V e2fsprogs-libs
prelink: /lib/libext2fs.so.2.4: prelinked file size differs
S.?....T      /lib/libext2fs.so.2.4
```

Atunci puteți folosi **yum reinstall \$package** pentru a suprascrie biblioteca existentă cu o versiune originală.

```
root@rhel53 lib# yum reinstall e2fsprogs-libs
Loaded plugins: rhnplugin, security
Setting up Reinstall Process
Resolving Dependencies
--> Running transaction check
---> Package e2fsprogs-libs.i386 0:1.39-23.el5 set to be erased
---> Package e2fsprogs-libs.i386 0:1.39-23.el5 set to be updated
--> Finished Dependency Resolution
...
```

Verificarea pachetului raportează acum că nu sînt probleme cu biblioteca.

```
root@rhel53 lib# rpm -V e2fsprogs-libs
root@rhel53 lib#
```

26.7. urmărire cu strace

Urmărirea mai detaliată a tuturor invocărilor funcției poate fi făcută cu **strace**. Începem prin crearea unui fișier cu drepturi doar de citire.

```
root@deb503:~# echo hello > 42.txt
root@deb503:~# chmod 400 42.txt
root@deb503:~# ls -l 42.txt
-r----- 1 root root 6 2011-09-26 12:03 42.txt
```

Deschidem fișierul cu **vi**, dar includem comanda **strace** cu un fișier ieșire pentru urmărire înainte de **vi**. Asta va crea un fișier cu toate invocările funcției făcute de **vi**.

```
root@deb503:~# strace -o strace.txt vi 42.txt
```

Fișierul are drepturi doar de citire, însă încă putem schimba conținutul, și folosim directiva **:w!** pentru a scrie în acest fișier. Apoi închidem **vi** și privim la fișierul-jurnal.

```
root@deb503:~# grep chmod strace.txt
chmod("42.txt", 0100600)      = -1 ENOENT (No such file or directory)
chmod("42.txt", 0100400)      = 0
root@deb503:~# ls -l 42.txt
-r----- 1 root root 12 2011-09-26 12:04 42.txt
```

Observați că **vi** a schimbat permisiunile de fișier de două ori. Fișierul-jurnal este prea lung pentru a arăta o captură de ecran completă în această carte.

```
root@deb503:~# wc -l strace.txt
941 strace.txt
```

Partea VII. management backup

Capitolul 27. backup

Conținut

27.1. despre dispozitive bandă.	.262
27.2. compresie.	263
27.3. tar.	263
27.4. tipuri de backup.	.265
27.5. dump și restore.	265
27.6. cpio.	.266
27.7. dd.	.266
27.8. split.	267
27.9. practică: backup.	.269

27.1. despre dispozitive bandă

Nu uitați că numele unui dispozitiv strict vorbind nu are nici un înțeles pentru că kernel-ul va utiliza numărul major și minor pentru a găsi hardware! Vedeți pagina de manual a **mknod** și fișierul `devices.txt` în sursa kernel linux pentru mai multe informații.

benzi SCSI

Pe lista oficială a dispozitivelor Linux (<http://www.lanana.org/docs/device-list/>) găsim nume pentru benzi SCSI (major 9 char). Dispozitivele bandă SCSI sînt localizate sub **/dev/st** și sînt numărate începînd cu 0 pentru primul dispozitiv bandă.

```
/dev/st0    First tape device
/dev/st1    Second tape device
/dev/st2    Third tape device
```

Pentru a preveni **derularea automată a benzilor**, puneți-le prefix cu litera **n**.

```
/dev/nst0   First no rewind tape device
/dev/nst1   Second no rewind tape device
/dev/nst2   Third no rewind tape device
```

Prin default, benzile SCSI pe Linux vor folosi cea mai mare compresie hardware care este suportată de dispozitivul bandă. Pentru a micșora nivelul comprimării, adăugați una dintre literele **l** (low), **m** (mediu) sau **a** (auto) pe numele benzii.

```
/dev/st0l   First low compression tape device
/dev/st0m   First medium compression tape device
/dev/nst2m  Third no rewind medium compression tape device
```

benzi IDE

În lista oficială a dispozitivelor Linux (<http://www.lanana.org/docs/device-list/>) găsim numele benzilor IDE (major char 37). Dispozitivele bandă IDE sînt localizate sub **/dev/ht** și sînt numărate începînd cu 0 pentru primul dispozitiv bandă. Nici o derulare înapoi sau compresie nu este similară cu benzile SCSI.

```
/dev/ht0    First IDE tape device
/dev/nht0    Second no rewind IDE tape device
/dev/ht0m    First medium compression IDE tape device
```

mt

Pentru a face managementul benzilor, folosiți **mt** (Magnetic Tape). Unele exemple.

Pentru a primi informație despre statusul unei benzi.

```
mt -f /dev/st0 status
```

Pentru a da înapoi o bandă ...

```
mt -f /dev/st0 rewind
```

Pentru a da înapoi și a scoate o bandă ...

```
mt -f /dev/st0 eject
```

Pentru a șterge o bandă ...

```
mt -f /dev/st0 erase
```

27.2. compresie

Poate fi benefic să comprimăm fișierele înainte de backup. Cele mai populare două utilitare pentru compresia fișierelor regulate pe Linux sînt **gzip/gunzip** și **bzip2/bunzip2**. Mai jos puteți vedea gzip în acțiune, observați că acesta adaugă extensia **.gz** fișierului.

```
paul@RHELv4u4:~/test$ ls -l allfiles.tx*
-rw-rw-r-- 1 paul paul 8813553 Feb 27 05:38 allfiles.txt
paul@RHELv4u4:~/test$ gzip allfiles.txt
paul@RHELv4u4:~/test$ ls -l allfiles.tx*
-rw-rw-r-- 1 paul paul 931863 Feb 27 05:38 allfiles.txt.gz
paul@RHELv4u4:~/test$ gunzip allfiles.txt.gz
paul@RHELv4u4:~/test$ ls -l allfiles.tx*
-rw-rw-r-- 1 paul paul 8813553 Feb 27 05:38 allfiles.txt
paul@RHELv4u4:~/test$
```

În general, gzip este mult mai rapid decît bzip2, dar ultimul comprimă mult mai bine. Să comparăm.

```
paul@RHELv4u4:~/test$ cp allfiles.txt bllfiles.txt
```

```
paul@RHELv4u4:~/test$ time gzip allfiles.txt
```

```
real 0m0.050s
user 0m0.041s
sys 0m0.009s
paul@RHELv4u4:~/test$ time bzip2 bllfiles.txt
```

```
real 0m5.968s
user 0m5.794s
sys 0m0.076s
```

```
paul@RHELv4u4:~/test$ ls -l ?llfiles.tx*
-rw-rw-r-- 1 paul paul 931863 Feb 27 05:38 allfiles.txt.gz
-rw-rw-r-- 1 paul paul 708871 May 12 10:52 bllfiles.txt.bz2
paul@RHELv4u4:~/test$
```

27.3. tar

Utilitarul **tar** își ia numele de la **Tape Archive**. Acest utilitar va primi și va trimite fișiere unei destinații (în mod tipic o bandă sau un fișier regulat). Opțiunea **c** este folosită pentru a crea o arhivă tar (sau fișier tar), opțiunea **f** pentru a numi/crea **tarfile**. Exemplul de mai jos face un backup a /etc în fișierul /backup/etc.tar.

```
root@RHELv4u4:~# tar cf /backup/etc.tar /etc
root@RHELv4u4:~# ls -l /backup/etc.tar
-rw-r--r-- 1 root root 47800320 May 12 11:47 /backup/etc.tar
root@RHELv4u4:~#
```

Compresia poate fi obținută fără conducte pentru că tar folosește construcția **z** pentru a comprima cu gzip, și construcția **j** pentru a comprima cu bzip2.

```

root@RHELv4u4:~# tar czf /backup/etc.tar.gz /etc
root@RHELv4u4:~# tar cjf /backup/etc.tar.bz2 /etc
root@RHELv4u4:~# ls -l /backup/etc.ta*
-rw-r--r-- 1 root root 47800320 May 12 11:47 /backup/etc.tar
-rw-r--r-- 1 root root 6077340 May 12 11:48 /backup/etc.tar.bz2
-rw-r--r-- 1 root root 8496607 May 12 11:47 /backup/etc.tar.gz
root@RHELv4u4:~#

```

Opțiunea **t** este folosită pentru a **lista conținutul unui fișier tar**. Modul **verbose** este autorizat cu **v** (de asemeni folositor când vreți să vedeți fișierele fiind arhivate în timpul arhivării).

```

root@RHELv4u4:~# tar tvf /backup/etc.tar
drwxr-xr-x root/root          0 2007-05-12 09:38:21 etc/
-rw-r--r-- root/root       2657 2004-09-27 10:15:03 etc/warnquota.conf
-rw-r--r-- root/root      13136 2006-11-03 17:34:50 etc/mime.types
drwxr-xr-x root/root          0 2004-11-03 13:35:50 etc/sound/
...

```

Pentru a **lista un fișier specific într-o arhivă tar**, folosiți opțiunea **t**, adăugat cu numele fișierului (fără **/**).

```

root@RHELv4u4:~# tar tvf /backup/etc.tar etc/resolv.conf
-rw-r--r-- root/root          77 2007-05-12 08:31:32 etc/resolv.conf
root@RHELv4u4:~#

```

Folosiți construcția **x** pentru a **restaura o arhivă tar**, sau un singur fișier din arhivă. Țineți minte că prin default tar va restaura fișierul în directorul curent.

```

root@RHELv4u4:~# tar xvf /backup/etc.tar etc/resolv.conf
etc/resolv.conf
root@RHELv4u4:~# ls -l /etc/resolv.conf
-rw-r--r-- 2 root root 40 May 12 12:05 /etc/resolv.conf
root@RHELv4u4:~# ls -l etc/resolv.conf
-rw-r--r-- 1 root root 77 May 12 08:31 etc/resolv.conf
root@RHELv4u4:~#

```

Puteți **păstra permisiunile fișierului** cu construcția **p**. Și puteți exclude directoare sau fișiere cu **--exclude**.

```

root ~# tar cpzf /backup/etc_with_perms.tgz /etc
root ~# tar cpzf /backup/etc_no_sysconf.tgz /etc --exclude /etc/sysconfig
root ~# ls -l /backup/etc_*
-rw-r--r-- 1 root root 8434293 May 12 12:48 /backup/etc_no_sysconf.tgz
-rw-r--r-- 1 root root 8496591 May 12 12:48 /backup/etc_with_perms.tgz
root ~#

```

Puteți de asemeni crea un fișier text cu numele fișierelor și directoarelor de arhivat, și apoi să furnizați acest fișier lui tar cu construcția **-T**.

```

root@RHELv4u4:~# find /etc -name *.conf > files_to_archive.txt
root@RHELv4u4:~# find /home -name *.pdf >> files_to_archive.txt
root@RHELv4u4:~# tar cpzf /backup/backup.tgz -T files_to_archive.txt

```

Utilitarul tar poate primi nume de fișier din comanda find, cu ajutorul xargs.

```

find /etc -type f -name "*.conf" | xargs tar czf /backup/confs.tar.gz

```


Puteți de asemeni folosi tar pentru a copia un director, asta e mai eficient decât utilizând cp -r.

```
(cd /etc; tar -cf - . ) | (cd /backup/copy_of_etc/; tar -xpf - )
```

Un alt exemplu tar, acesta copie securizat un director din rețea.

```
(cd /etc;tar -cf - . )|(ssh user@srv 'cd /backup/cp_of_etc/; tar -xf - ')
```

tar poate fi folosit împreună cu gzip și copie un fișier de la un server remote prin ssh.

```
cat backup.tar | gzip | ssh bashuser@192.168.1.105 "cat - > backup.tgz"
```

Comprimați backup-ul tar când este în rețea, dar lăsați-l decomprimat la destinație.

```
cat backup.tar | gzip | ssh user@192.168.1.105 "gunzip|cat - > backup.tar"
```

La fel ca mai sus, dar lăsăm ssh să se ocupe de compresie.

```
cat backup.tar | ssh -C bashuser@192.168.1.105 "cat - > backup.tar"
```

27.4. tipuri de backup

Linux folosește backup-uri **incrementale multinivel** folosind nivele distincte. Un backup deplin este un backup la nivel 0. Un nivel mai sus x backup va include toate schimbările de la ultimul nivel backup x-1.

Să presupunem că faceți un backup deplin luni (nivel 0) și un backup nivel 1 marțea, atunci backup-ul de marți va conține toate schimbările făcute de luni. Facerea unui backup nivel 2 miercurea va conține toate schimbările de marți (ultimul nivel 2-1). Un backup nivel 3 joia va conține toate schimbările de miercuri (ultimul nivel 3-1). Un alt backup nivel 3 vinerea va conține de asemeni toate schimbările de miercuri. Un backup nivel 2 duminica va lua toate schimbările de la ultimul nivel 1 de marți.

27.5. dump și restore

În timp ce **dump** este similar cu tar, este de asemeni foarte diferit, pentru că țintește la sistemul fișier. Pe când tar primește o listă a fișierelor pentru a le face backup, dump va găsi fișiere prin el pentru a face backup examinând ext2. Fișiere găsite de dump vor fi copiate pe o bandă sau într-un fișier regular. Dacă ținta nu este destul de mare pentru a menține dump (end-of-media), se împarte în volume multiple.

Restaurarea fișierelor care le-au fost făcute backup se face cu comanda **restore**. În exemplul de mai jos vom lua un backup deplin nivel 0 a două partiții pe o bandă SCSI. No rewind este obligatoriu pentru a pune volumele în spatele fiecăreia pe bandă.

```
dump 0f /dev/nst0 /boot
dump 0f /dev/nst0 /
```

Listarea fișierelor într-o arhivă dump se face cu **dump -t**, și puteți compara fișierele cu **dump -C**.

Puteți omite fișiere dintr-un dump schimbând atributul dump cu comanda **chattr**. Atributul d pe ext va transmite dump să sară fișierul, chiar și în timpul unui backup deplin. În exemplul următor, /etc/hosts este exclus din arhivele dump.

```
chattr +d /etc/hosts
```

Pentru a restaura întregul sistem de fișier cu **restore**, folosiți opțiunea **-r**. Asta poate fi folositor pentru a schimba mărimea sau mărimea bloc a unui sistem fișier. Ar trebui să aveți un sistem de fișier montat curat și să puteți face cd în el. Precum arată acest exemplu.

```
mke2fs /dev/hda3
mount /dev/hda3 /mnt/data
cd /mnt/data
restore rf /dev/nst0
```

Pentru a extrage doar un singur fișier sau director din dump, folosiți opțiunea **-x**.

```
restore -xf /dev/st0 /etc
```

27.6. cpio

Diferit față de tar și dump e **cpio** (**C**opy **I**nput and **O**utput). Poate fi utilizat să primească nume de fișiere, dar copie fișiere reale. Asta îl face un însoțitor bun cu find! Unele exemple mai jos.

find trimite nume de fișiere lui cpio, care pune fișierele într-o arhivă.

```
find /etc -depth -print | cpio -oaV -O archive.cpio
```

La fel, dar compresează cu gzip.

```
find /etc -depth -print | cpio -oaV | gzip -c > archive.cpio.gz
```

Acum creăm o conductă prin ssh (face backup la fișiere într-un fișier compresat pe o altă mașină).

```
find /etc -depth -print|cpio -oaV|gzip -c|ssh server "cat - > etc.cpio.gz"
```

```
find trimite numele de fișiere lui cpio | cpio trimite fișiere la ssh | ssh
trimite fișiere lui cpio 'cpio extrage fișiere'
```

```
find /etc -depth -print | cpio -oaV | ssh user@host 'cpio -imVd'
```

Aceași comandă dar inversată: copie un director de pe gazda îndepărtată pe mașina locală.

```
ssh user@host "find path -depth -print | cpio -oaV" | cpio -imVd
```

27.7. dd

despre dd

Unii oameni folosesc **dd** pentru a crea backup-uri. Acest lucru poate fi foarte puternic, dar backup-urile cu dd pot fi restaurate doar pe partiții sau dispozitive foarte similare. Există oricum o mulțime de lucruri folositoare posibile cu dd. Unele exemple.

crearea unei imagini CDROM

Cea mai ușoară modalitate de a crea un **fișier .ISO** de la orice CD. Întrerupătorul **if** înseamnă Input File, **of** este Output File. Orice utilitar bun poate face o copie a CD-ului cu acest fișier .ISO.

```
dd if=/dev/cdrom of=/path/to/cdrom.ISO
```

crearea unei imagini floppy

Poate un pic veche, dar doar dacă e cazul: facem un fișier imagine dintr-un floppy de 1.44MB. bs este definit ca blocksize, și count conține numărul block-urilor pentru a fi copiate.

```
dd if=/dev/floppy of=/path/to/floppy.img bs=1024 count=1440
```

copiați master boot record

Folosiți dd pentru a copia **MBR (Master Boot Record)** a unui hard-disk /dev/hda într-un fișier.

```
dd if=/dev/hda of=/MBR.img bs=512 count=1
```

copiați fișiere

Acest exemplu arată cum dd poate copia fișiere. Copiați fișierul summer.txt în copy_of_summer.txt.

```
dd if=~/summer.txt of=~/copy_of_summer.txt
```

disk-uri imagine sau partiții

Și cine are nevoie de ghost când dd poate crea o imagine (comprimată) a unei partiții.

```
dd if=/dev/hdb2 of=/image_of_hdb2.IMG  
dd if=/dev/hdb2 | gzip > /image_of_hdb2.IMG.gz
```

creați fișiere de o anumită mărime

dd poate fi utilizat pentru a crea un fișier de orice mărime. Primul exemplu crează un fișier de 1 megabit, al doilea un fișier de 1 megabait.

```
dd if=/dev/zero of=file1MB count=1024 bs=1024  
dd if=/dev/zero of=file1MB count=1000 bs=1024
```

exemplu CDROM server

Și există desigur combinații nesfârșite cu ssh și bzip2. Acest exemplu pune un backup bzip2 a unui cdrom pe un server remote.

```
dd if=/dev/cdrom |bzip2|ssh user@host "cat - > /backups/cd/cdrom.iso.bz2"
```

27.8. split

Comanda **split** este folosită pentru a împărți fișierele în fișiere mai mici. Asta poate fi folositor pentru a potrivi fișierul în instanțe multiple a unui mediu prea mic pentru a conține fișierul întreg. În exemplul de mai jos, un fișier de mărimea de 5000 biți este împărțit în trei fișiere mai mici, cu maximum de 2000 biți fiecare.

```
paul@laika:~/test$ ls -l
total 8
-rw-r--r-- 1 paul paul 5000 2007-09-09 20:46 bigfile1
paul@laika:~/test$ split -b 2000 bigfile1 splitfile.
paul@laika:~/test$ ls -l
total 20
-rw-r--r-- 1 paul paul 5000 2007-09-09 20:46 bigfile1
-rw-r--r-- 1 paul paul 2000 2007-09-09 20:47 splitfile.aa
-rw-r--r-- 1 paul paul 2000 2007-09-09 20:47 splitfile.ab
-rw-r--r-- 1 paul paul 1000 2007-09-09 20:47 splitfile.ac
```

27.9. practică: backup

!! Fiți atenți cu opțiunile tar și poziția fișierului backup, greșelile pot să vă distrugă sistemul!!

1. Creați un director (sau o partiție dacă doriți) pentru backup-uri. Legați-l (sau montați-o) în /mnt/backup.

2a. Folosiți tar pentru a face backup la /etc în /mnt/backup/etc_date.tgz, backup-ul trebuie să fie gzip. (Înlocuiți data cu data curentă).

2b. Folosiți tar pentru a face backup la /bin în /mnt/backup/bin_date.tar.bz2, backup-ul trebuie să fie bzip2.

2c. Alegeți un fișier în /etc și /bin și verificați cu tar dacă fișierului îi este într-adevăr făcut backup.

2d. Extrageți cele două fișiere în directorul home.

3a. Creați un backup unui director pentru coleg, faceți-l accesibil în /mnt/NumeleColegului.

3b. Combinați ssh și tar pentru a pune un backup al /boot pe computerul colegului în /mnt/NumeleDumneavoastră.

4a. Combinați find și cpio pentru a crea o arhivă cpio a /etc.

4b. Alegeți un fișier din /etc și restaurați-l din arhiva cpio în directorul home.

5. Folosiți dd și ssh pentru a pune un backup a master boot record pe computerul colegilor.

6. (Pe un computer real) Creați și montați o imagine ISO a unui cdrom Ubuntu.

7. Combinați dd și gzip pentru a crea o imagine 'ghost' a uneia dintre partiții pe o altă partiție.

8. Folosiți dd pentru a crea un fișier de 4 megabiți în ~/testsplit și numiți-l biggest. Apoi împărțiți acest fișier în două părți mai mici de 2 megabiți.

```
mkdir testsplit
```

```
dd if=/dev/zero of=~/testsplit/biggest count=5000 bs=1024
```

```
split -b 2000000 biggest parts
```

Partea VIII. Apendice

Apendice A. Licență (în limba română)

Licența GNU Pentru Documentația Liberă

Versiunea 1.3, 3 Noiembrie 2008

Drepturi de autor 2000, 2001, 2002, 2007, 2008 Fundația Soft-ului Liber, Inc.

Oricui îi este permisă copierea și distribuirea de copii identice ale acestui document, dar fără modificarea lui.

0. PREAMBUL

Scopul acestei Licențe este de a conferi „gratuitate” unui manual, colecții de texte, sau altui document funcțional și folositor, în sensul libertății: de a asigura tuturor permisiunea de copiere și redistribuire, cu sau fără modificări, în scopuri comerciale și necomerciale. Ca scop secundar, această Licență rezervă autorului și editorului dreptul de a fi creditați pentru munca lor, atât timp cât nu sunt responsabili pentru modificările efectuate de către alții.

Această Licență conferă un fel de „obligații”, ceea ce înseamnă că lucrările derivate dintr-un document trebuie să fie și ele libere, la rîndul lor. Această Licență este inspirată de Licența Publică Generală GNU, care este o licență similară, concepută pentru a acoperi softul liber.

Am creat această Licență pentru a fi de folos manualelor pentru softul liber, deoarece un soft liber necesită o documentație liberă: un program trebuie însoțit de manuale care oferă aceeași libertate de folosire ca și softul. Această Licență nu este limitată, însă, la manualele pentru soft; ea poate fi folosită pentru textul oricărei lucrări, indiferent de subiect sau de modul de publicare. Această Licență este recomandată în principal pentru lucrări care servesc drept referință sau au fost scrise în scop de instruire.

1. APLICABILITATE ȘI DEFINIȚII

Această Licență se aplică oricărei lucrări sau manual, în orice mediu, care conține o notă, inclusă de către deținătorul dreptului de autor, care permite distribuția în termenii acestei Licențe. Această notă conferă dreptul universal, fără indemnizație și nelimitat ca durată de a folosi lucrarea în condițiile de față. Termenul Document, de mai jos, se referă la un astfel de manual sau lucrare. Orice membru din public este un beneficiar al acestei Licențe și va fi desemnat prin termenul dumneavoastră. Se consideră, în mod automat, că ați acceptat termenii acestei Licențe, în urma copierii, modificării sau distribuirii unei lucrări într-un mod care necesită permisiunea autorului, în condițiile legii drepturilor de autor.

O „Versiune Modificată” a Documentului este orice lucrare conținând Documentul sau o porțiune de-a lui, copiată identic sau cu modificări și/sau tradusă într-o altă limbă.

O „Secțiune Secundară” este o anexă cu titlu, sau o secțiune menționată în cuprins care are ca scop exclusiv descrierea relației editorilor sau a autorilor Documentului cu subiectul Documentului (sau cu aspecte conexe) și care nu conține referiri directe la subiectul Documentului. (Astfel, dacă Documentul este în parte manual de matematică, o Secțiune Secundară nu poate conține deloc explicații matematice.) Poate exista doar o conexiune istorică cu subiectul și cu problemele înrudite cu subiectul, ori pot fi prezentate puncte de vedere legale, comerciale, filozofice, etice sau politice legate de acesta.

„Secțiunile Neschimbabile” sînt anumite Secțiuni Secundare ale căror titluri sînt specificate ca fiind acele titluri de Secțiuni Neschimbabile din nota ce permite distribuția Documentului sub acoperirea acestei Licențe. Dacă o secțiune nu este conformă cu definiția de mai sus a unei Secțiuni Secundare atunci ea nu poate fi desemnată ca fiind Neschimbabilă. Documentul poate să nu conțină Secțiuni Neschimbabile. Dacă Documentul nu specifică vreo Secțiune Neschimbabilă atunci se consideră că nu există nici una.

„Textele De Copertă” sînt anumite pasaje scurte de text care sunt listate ca Texte Pentru Coperta I sau ca Texte Pentru Coperta IV în nota care specifică distribuirea Documentului sub acoperirea acestei Licențe. Un Text Pentru Coperta I poate avea cel mult 5 cuvinte, iar un Text Pentru Coperta IV poate avea cel mult 25 de cuvinte.

O copie „Transparentă” a Documentului este o copie în format electronic, reprezentată într-un format ale cărui specificații sînt disponibile publicului, fiind ușor de modificat cu ajutorul unui editor de text generic sau (pentru imagini compuse din pixeli) cu un editor grafic generic ori (pentru desene) cu un editor larg răspîndit de grafică vectorială, și care poate fi folosit ca intrare în procesoarele de text sau de transformare automată în diverse formate adecvate ca intrare pentru procesoarele de text. O copie făcută într-un format de fișier Transparent dar care, prin prezența sau absența anumitor elemente specifice formatului, descurajează sau împiedică modificările ulterioare, nu reprezintă o copie Transparentă. Un format de imagine nu este Transparent dacă este folosit pentru a reprezenta o cantitate substanțială de text. O copie care nu este „Transparentă” se numește „Opacă”.

Exemple de formate compatibile cu copiile Transparente: textul ASCII fără marcaje, formatul de intrare Texinfo, formatele de intrare LaTeX, SGML și XML folosind un DTD public, HTML simplu și standard, fișierele PostScript și PDF modificabile. Exemple de formate Transparente pentru imagine: PNG, XCF și JPG. Formatele Opace includ formate de text ce pot fi citite și editate doar de procesoare de text proprietare, SGML și XML pentru care DTD-ul și/sau uneltele de procesare nu sunt disponibile, HTML generat automat, documentele PostScript și PDF produse de diverse procesoare de text doar în scopul printării/afișării.

„Pagina de Titlu” înseamnă, pentru o carte tipărită, pagina cu titlul și paginile următoare, necesare pentru a prezenta, lizibil, materialul care trebuie tipărit, conform acestei Licențe, pe Pagina de Titlu. Pentru lucrări care nu au o pagină cu titlu propriu-zisă, „Pagina de Titlu” este textul aflat lîngă principala apariție a titlului lucrării, precedînd începutul corpului Documentului.

„Editorul” reprezintă orice persoană sau entitate care distribuie copii ale documentului pentru public.

O secțiune „Numită XYZ” este o subunitate a Documentului, al cărei titlu este, fie XYZ, fie conține XYZ în paranteze, după textul care traduce XYZ în altă limbă. (Aici XYZ înlocuiește nume specifice ce vor fi menționate mai jos, ca de exemplu „Mulțumiri”, „Dedicații”, „Giruri” sau „Istorie”.) Pentru a „Păstra Titlul” unei astfel de secțiuni atunci cînd modificați Documentul înseamnă că va rămîne o secțiune „Numită XYZ”, conform acestei definiții.

Documentul poate include Limitări de Responsabilitate atașate notificării care afirmă că această Licență se aplică Documentului. Acestea se consideră a fi incluse prin referință în această Licență, dar numai cu privire la limitările de responsabilitate: orice alte implicații pe care aceste Limitări de Responsabilitate le-ar putea avea sînt nule și nu au nici un efect asupra înțelesului acestei Licențe.

2. COPII IDENTICE

Puteți copia și distribui Documentul pe orice mediu, fie comercial sau necomercial, atît timp cît această Licență, notificările de drepturi de autor și notificarea de licență care spune că această Licență se aplică acestui Document, sînt reproduse în toate copiile, și atîta timp cît nu adăugați nici un fel de altă condiție în afară de cele prezente în această Licență. Nu aveți dreptul să luați măsuri tehnice de a obstrucționa sau controla citirea sau recopierea copiilor pe care le faceți sau le distribuiți. Aveți totuși dreptul să acceptați compensații în schimbul copiilor. Dacă distribuiți un număr suficient de mare de copii, atunci trebuie să respectați și condițiile din secțiunea 3.

Aveți, de asemenea, dreptul să împrumutați copii în aceleași condiții ca cele de mai sus, și aveți dreptul să afișați copii.

3. COPIEREA ÎN CANTITĂȚI MARI

Dacă publicați copii tipărite (sau copii în medii care folosesc de obicei coperti tipărite) ale Documentului, în număr mai mare de 100 și dacă notificarea de licență a Documentului cere Texte de Copertă, trebuie să includeți copiile pe coperti care să conțină, clar și lizibil, toate aceste Texte de Copertă: Textele Pentru Coperta I pe coperta I și Texte Pentru Coperta IV pe coperta IV. Ambele coperti trebuie de asemenea să vă identifice în mod clar și lizibil ca editor al respectivelor copii. Coperta I trebuie să prezinte titlul în întregime, cu toate cuvintele din titlu la fel de vizibile și proeminente. Puteți adăuga alte materiale pe copertă în plus. Copierea cu modificările limitate la coperti, atîta timp cît satisfac aceste condiții, pot fi tratate în toate celelalte aspecte ca și copii identice.

Dacă textele necesare pentru oricare dintre coperti sunt prea voluminoase pentru a încăpea în mod lizibil, trebuie să puneți primele rînduri (atîtea cît încap în mod rezonabil) pe coperta efectivă și să continuați cu restul pe pagini adiacente.

Dacă publicați sau distribuiți copii Opace ale Documentului în număr mai mare de 100, trebuie ori să includeți cîte o copie Transparentă în format electronic împreună cu fiecare copie Opacă, ori să specificați în sau împreună cu fiecare copie Opacă o locație din rețeaua electronică la care publicul general care folosește rețeaua să aibă acces pentru a descărca, folosind un protocol standard public, copii complete, Transparente ale documentului, fără adăugarea oricărui material adițional. Dacă folosiți a doua opțiune trebuie să faceți demersuri rezonabil de prudente ca atunci cînd începeți distribuirea copiilor Opace să vă asigurați că această copie Transparentă va rămîne accesibilă, în acest fel, la locația respectivă timp de cel puțin un an după distribuția ultimei copii Opace (în mod direct sau prin agenți ori distribuitori) a respectivei ediții pentru public.

Se cere, dar nu în mod necesar, să contactați autorii Documentului cu o perioadă bună înainte de a distribui orice cantitate mare de copii, pentru a le da ocazia să vă pună la dispoziție o versiune actualizată a Documentului.

4. MODIFICĂRI

Puteți copia și distribui o Versiune Modificată a Documentului în condițiile secțiunilor 2 și 3 de mai sus, cu condiția de a acoperi Versiunea Modificată sub exact această Licență, cu Versiunea Modificată ținînd locul Documentului, astfel licențind distribuirea și modificările Versiunii Modificate oricui intră în posesia unei copii ale acesteia. În plus, trebuie să faceți următoarele lucruri în Versiunea Modificată:

* A. Folosiți în Pagina de Titlu (și pe coperti, dacă există) un titlu diferit

de cel al Documentului, și de versiunile sale anterioare (care trebuie, dacă există, să fie listate în secțiunea de Istorie a Documentului). Puteți folosi același titlu ca o versiune anterioară dacă editorul original al acelei copii vă dă permisiunea.

* B. Listați pe Pagina de Titlu, ca autori, una sau mai multe dintre persoanele sau entitățile responsabile în calitate de autori pentru modificările Versiunii Modificate, împreună cu cel puțin cinci dintre autorii principali ai Documentului (toți autorii principali, dacă are mai puțin de cinci), în afară de cazul că aceștia vă eliberează de această obligație.

* C. Includeți pe Pagina de Titlu numele editorului Versiunii Modificate în calitate de editor.

* D. Păstrați toate notificările de drepturi de autor ale Documentului.

* E. Adăugați o notificare de drepturi de autori relevantă pentru modificările dumneavoastră adiacent celorlalte notificări de drepturi de autor.

* F. Includeți, imediat după notificările de drepturi de autor, o notificare de licență dînd permisiune publică de a folosi Versiunea Modificată în condițiile acestei Licențe, sub forma prezentată în Apendicele de mai jos.

* G. Păstrați în acea notificare de licență lista integrală a Secțiunilor Neschimbabile și Textele de Copertă necesare, date în notificarea de licență a Documentului.

* H. Includeți o copie nealterată a acestei Licențe.

* I. Păstrați secțiunea Numită „Istorie”, păstrați-i Titlul și adăugați-i un element care să indice măcar titlul, anul, noii autori și editorul Versiunii Modificate așa cum este dat pe Pagina de Titlu. Dacă nu există o secțiune numită „Istorie” în Document, creați una în care indicați titlul, anul, autorii și editorul Documentului așa cum este dat pe Pagina de Titlu al acestuia și apoi adăugați un element care să descrie Versiunea Modificată așa cum a fost cerut în fraza precedentă.

* J. Păstrați locația de rețea, dacă există, dată în Document pentru acces public la o copie Transparentă a Documentului, cît și locațiile de rețea date în Document pentru versiunile mai vechi pe care s-a bazat acesta. Acestea pot fi incluse în secțiunea numită „Istorie”. Puteți omite locația de rețea a unei lucrări care a fost publicată cu cel puțin patru ani înainte de Documentul în sine, sau dacă editorul original al versiunii la care se referă vă dă permisiunea.

* K. Pentru orice secțiune numită „Mulțumiri” sau „Dedicații” păstrați Titlul secțiunii și păstrați în secțiunile respective toată substanța și tonul mulțumirilor și/sau dedicațiilor fiecărui contributor.

* L. Păstrați toate Secțiunile Neschimbabile ale Documentului, nealterate ca text și ca titluri. Numerotarea secțiunilor sau echivalentul numerotării nu sunt considerate ca făcînd parte din titlurile secțiunilor.

* M. Ștergeți orice secțiune Numită „Giruri”. O astfel de secțiune nu poate fi inclusă în Versiunea Modificată.

* N. Nu modificați titlul nici unei secțiuni existente pentru a fi numită „Giruri” sau pentru a intra în conflict cu vreo Secțiune Neschimbabilă.

* O. Păstrați toate Limitările de Responsabilitate.

Dacă Versiunea Modificată include secțiuni noi incluse în titlu sau anexe care se califică drept Secțiuni Secundare și nu conțin material copiat din Document, aveți dreptul la alegerea dumneavoastră să numiți unele sau toate acestea ca fiind Neschimbabile. Pentru a face aceasta, adăugați-le titlurile la lista de Secțiuni Neschimbabile în notificarea de licență a Versiunii Modificate. Aceste titluri trebuie să fie distincte față de toate celelalte titluri de secțiune. Puteți adăuga o secțiune Numită „Giruri” doar dacă aceasta conține numai girurile a diverse entități asupra Versiunii Modificate – de exemplu recenzii sau faptul că textul a fost aprobat de o organizație ca fiind o definiție autoritară a unui standard.

Puteți adăuga un pasaj de cel mult cinci cuvinte ca Text Pentru Coperta I și un pasaj de cel mult 25 de cuvinte ca Text Pentru Coperta IV la sfîrșitul Textelor De Copertă în Versiunea Modificată. Numai un singur pasaj poate fi adăugat la

Textul Pentru Coperta I și unul la Textul Pentru Coperta IV de către (sau prin aranjament cu) orice entitate. Dacă Documentul conține deja texte de copertă pentru coperta respectivă, adăugat în prealabil de dumneavoastră sau prin aranjament cu aceeași entitate în numele căreia acționați, atunci nu puteți adăuga un altul, însă puteți să-l înlocuiți pe cel vechi numai cu permisiunea explicită a editorului anterior care l-a adăugat pe cel vechi.

Autorul (autorii) și editorul (editorii) Documentului nu vă dau prin această Licență permisiunea de a le folosi numele pentru publicitate sau pentru a pretinde sau implica vreo girare a oricărei Versiuni Modificate.

5. COMBINAREA DOCUMENTELOR

Puteți combina Documentul cu alte documente acoperite de această Licență sub termenii definiți în secțiunea 4 de mai sus pentru versiuni modificate, cu condiția să includeți în versiunea combinată toate Secțiunile Neschimbabile ale tuturor documentelor originale, nemodificate, și să le listați pe toate ca Secțiuni Neschimbabile ale versiunii combinate în notificarea de licență, cât și să păstrați toate Limitările de Responsabilitate.

Versiunea modificată nu trebuie să conțină decît o singură copie a acestei Licențe, iar duplicatele identice ale Secțiunilor Neschimbabile pot fi înlocuite cu o singură copie. Dacă există Secțiuni Neschimbabile cu nume identice și conținut diferit, schimbați-le numele adăugând la sfîrșitul titlului, în paranteză, ori numele autorului sau al editorului original al acelei secțiuni dacă acesta este cunoscut, ori un număr unic. Faceți aceleași modificări respective titlurilor secțiunilor în lista de Secțiuni Neschimbabile din notificarea de licență a versiunii combinate.

În versiunea combinată trebuie să combinați și toate secțiunile numite „Istorie” din diversele documente originale, creînd o secțiune unică numită „Istorie”; la fel trebuie să combinați și toate secțiunile numite „Mulțumiri” cît și cele numite „Dedicații”. Trebuie să ștergeți toate secțiunile numite „Giruri”.

6. COLECȚII DE DOCUMENTE

Puteți crea o colecție formată din Document și alte documente acoperite de această Licență și să înlocuiți copiile individuale ale acestei Licențe din diversele documente cu o singură copie care să fie inclusă în colecție cu condiția să urmați regulile acestei Licențe pentru copii identice pentru fiecare document în toate celelalte privințe.

Puteți să extrageți un document dintr-o astfel de colecție și să-l distribuiți individual sub această Licență cu condiția de a include o copie a acestei Licențe în documentul extras și să urmați condițiile acestei Licențe în toate celelalte privințe în legătură cu copiile identice ale aceluși document.

7. AGREGAREA CU LUCRĂRI INDEPENDENTE

O compilație a Documentului sau a unui derivat al său cu orice document sau lucrare separată independentă, în sau pe un volum de stocare sau distribuie se numește „agregat” dacă drepturile de autor rezultate în urma compilării nu sînt folosite pentru a limita drepturile legale ale utilizatorilor compilației mai mult decît permit lucrările individuale. Cînd Documentul este inclus într-un agregat, această Licență nu se aplică celorlalte lucrări din agregat care nu sînt ele însele rezultate derivate ale Documentului.

Dacă cerințele legate de Textele de Copertă din secțiunea 3 se aplică acestor copii ale Documentului, atunci dacă Documentul este mai puțin de jumătate din întregul agregat atunci Textele de Copertă ale Documentului pot fi puse pe coperti care să separe Documentul în cadrul agregatului, sau pe un echivalent

electronic al acestora, dacă Documentul se prezintă în format electronic. Altfel ele trebuie să apară pe coperțile tipărite care îmbracă întreg agregatul.

8. TRADUCERE

Traducerea este considerată o formă de modificare, drept care puteți distribui traduceri ale Documentului sub cerințele secțiunii 4. Înlocuirea Secțiunilor Neschimbabile cu traduceri ale acestora necesită permisiune specială din partea celor care dețin drepturile de autor, însă puteți include traduceri ale unora dintre sau tuturor Secțiunilor Neschimbabile împreună cu variantele originale ale acestora. Puteți include o traducere a acestei Licențe cât și toate notificările de licență din Document, cât și Limitările de Responsabilitate atâta timp cât includeți și versiunea originală în engleză a acestei Licențe, plus versiunile originale ale respectivelor notificări de licență și limitări de responsabilitate. În cazul apariției oricăror discrepanțe între versiunea tradusă și versiunea originală a acestei Licențe, a vreunei notificări de licență sau a vreunei limitări de responsabilitate, versiunea originală are prioritate.

Dacă vreo secțiune din Document este Numită „Mulțumiri”, „Dedicații” sau „Istorie” cerința (din secțiunea 4) de a-i păstra Titlul (secțiunea 1) va necesita în mod normal schimbarea titlului în sine.

9. REZILIERE

Nu puteți copia, modifica, sublicenția sau distribui Documentul decât în condițiile specificate explicit în această Licență. Orice copiere, modificare sau redistribuire a Documentului în vreo altă condiție este nulă și vă va anula în mod automat drepturile conferite de această Licență.

Cu toate acestea, dacă încetați orice încălcare a acestei Licențe, licența din partea titularului dreptului de autor este reinstaurată (a) cu titlu provizoriu, cu excepția cazului când titularul dreptului de autor încetează în mod explicit și în cele din urmă licența, și (b) permanent, în cazul în care titularul dreptului de autor nu vă anunță încălcarea, prin mijloace rezonabile, în termen de 60 de zile de la încetare.

În plus, licența de la titularul particular al dreptului de autor este repusă permanent în cazul în care titularul dreptului de autor vă anunță de încălcare prin mijloace rezonabile, și este prima dată când ați primit o notificare de încălcare a acestei Licențe (pentru orice lucrare), din partea titularului dreptului de autor, și ați încetat încălcarea cu 30 de zile înainte de primirea notificării.

Încetarea drepturilor dumneavoastră, în conformitate cu această secțiune, nu încetează licențele părților care au primit copii sau drepturi de la dumneavoastră sub această Licență. Dacă drepturile dumneavoastră au fost terminate și nu s-au repus permanent, primirea unei copii ale aceluiași material nu vă dă nici un drept să-l folosiți.

10. VERSIUNI VIITOARE ALE ACESTEI LICENȚE

Fundația Free Software Foundation poate publica, din când în când, versiuni noi, revizuite ale acestei Licențe GNU pentru Documentația liberă. Aceste noi versiuni vor păstra spiritul acestei versiuni dar pot diferi în privința detaliilor, cu scopul de a se adresa unor noi probleme reale sau potențiale. A se vedea <http://www.gnu.org/copyleft/>.

Fiecărei versiuni ale acestei Licențe îi este asociat un număr de versiune distinct. Dacă Documentul specifică un anumit număr de versiune „sau orice versiune ulterioară” al acestei Licențe, aveți de ales între a vă conforma

termenilor și condițiilor ori ale versiunii specificate explicit sau ale oricărei variante ulterioare publicate (nu ca variantă preliminară) de către Free Software Foundation. Dacă Documentul nu specifică un număr de versiune al acestei Licențe atunci puteți alege orice versiune publicată (nu ca variantă preliminară) de către Free Software Foundation.

11. RELICENȚIEREA

„Site-ul de Colaborare Masivă a Multiautorilor” (sau „MMC Site”) înseamnă orice server www care publică lucrări posibil de a fi supuse drepturilor de autor și, de asemenea, oferă facilități proeminente pentru oricine editează aceste lucrări. Un server public wiki este un exemplu în care oricine poate edita lucrări scrise. Un „Site de Colaborare Masivă a Multiautorilor” (sau „MMC Site”) în conținutul său înseamnă o mulțime de lucrări susceptibile de a fi supuse licențelor supuse site-ului MMC.

„CC-BZ-SA” înseamnă licență a Creative Commons Attribution-Share Alike 3.0 publicată de Corporația Creative Commons, o corporație nonprofit cu sediul principal la San Francisco, California, și de asemenea viitoarele variante de „obligații” a acestei licențe publicată de aceeași organizație.

„Încorporarea” înseamnă publicarea ori republicarea unui Document, în întregime sau în parte, ca parte a unui alt document.

Un MMC este „eligibil pentru reautorizare” dacă este licențiat sub această Licență, și în cazul în care toate lucrările care au fost publicate mai întâi în această Licență în altă parte decât acest MMC, și, ulterior, au fost încorporate în totalitate sau în parte în MMC, (1) nu a avut texte de copertă sau secțiuni invariante, și (2) au fost astfel incluse până la 1 noiembrie 2008.

Operatorul unui site MMC poate republica un MMC conținut în site sub CC-BY-SA în același loc, în orice moment înainte de 1 august 2009, cu condiția ca MMC să fie eligibil pentru reautorizare.

Apendice B. Licență (în limba engleză)

GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document "free" in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of "copyleft", which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The "Document", below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as "you". You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A "Modified Version" of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A "Secondary Section" is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document's overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The "Invariant Sections" are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The "Cover Texts" are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A "Transparent" copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not "Transparent" is called "Opaque".

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The "Title Page" means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, "Title Page" means the text near the most prominent appearance of the work's title, preceding the beginning of the body of the text.

The "publisher" means any person or entity that distributes copies of the Document to the public.

A section "Entitled XYZ" means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as "Acknowledgements", "Dedications", "Endorsements", or "History".) To "Preserve the Title" of such a section when you modify the Document means that it remains a section "Entitled XYZ" according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may

publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- * A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.

- * B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.

- * C. State on the Title page the name of the publisher of the Modified Version, as the publisher.

- * D. Preserve all the copyright notices of the Document.

- * E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.

- * F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this

License, in the form shown in the Addendum below.

- * G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.

- * H. Include an unaltered copy of this License.

- * I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.

- * J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.

- * K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.

- * L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.

- * M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.

- * N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.

- * O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements".

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an "aggregate" if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled "Acknowledgements", "Dedications", or "History", the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License "or any later version" applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

"Massive Multiauthor Collaboration Site" (or "MMC Site") means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A "Massive Multiauthor Collaboration" (or "MMC") contained in the site means any set of copyrightable works thus published on the MMC site.

"CC-BY-SA" means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

"Incorporate" means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is "eligible for relicensing" if it is licensed under this License, and if all works that were first published under this License somewhere other than

this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.